

Physlib Monthly Report

1–31 January 2025

Contributors: Joseph Tooby-Smith, kuotsanhsu, Pietro Monticone

Reviewers: Joseph Tooby-Smith

Generated 14 August 2026 from commit `6aab0ba`.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 January 2025 on branch `master` of [leanprover-community/physlib](#), between commits `abfda8f` and `6aab0ba`. Over that period 3 contributors landed 145 commits, changing 19,567 lines across 158 files and adding 925 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	HepLean.Lorentz.SL2C.SelfAdjoint	3
2.2	HepLean.Mathematics.Fin.Involutions	4
2.3	HepLean.Mathematics.List	8
2.4	HepLean.Mathematics.List.InsertIdx	10
2.5	HepLean.Mathematics.List.InsertionSort	13
2.6	HepLean.Mathematics.SchurTriangulation	20
2.7	HepLean.Meta.Basic	22
2.8	HepLean.Meta.Remark.Basic	22
2.9	HepLean.Meta.Remark.Properties	23
2.10	HepLean.Meta.TODO.Basic	23
2.11	HepLean.PerturbationTheory.Algebras.CrAnAlgebra.Basic	24
2.12	HepLean.PerturbationTheory.Algebras.CrAnAlgebra.Grading	27
2.13	HepLean.PerturbationTheory.Algebras.CrAnAlgebra.NormalOrder	30
2.14	HepLean.PerturbationTheory.Algebras.CrAnAlgebra.NormTimeOrder	34
2.15	HepLean.PerturbationTheory.Algebras.CrAnAlgebra.SuperCommute	34
2.16	HepLean.PerturbationTheory.Algebras.CrAnAlgebra.TimeOrder	41
2.17	HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.Basic	44
2.18	HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.NormalOrder	49
2.19	HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.StaticWickTheorem	55
2.20	HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.SuperCommute	56

2.21	HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.TimeContraction . .	62
2.22	HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.TimeOrder	62
2.23	HepLean.PerturbationTheory.CreateAnnihilate	64
2.24	HepLean.PerturbationTheory.FieldSpecification.Basic	65
2.25	HepLean.PerturbationTheory.FieldSpecification.CrAnSection	67
2.26	HepLean.PerturbationTheory.FieldSpecification.CrAnStates	73
2.27	HepLean.PerturbationTheory.FieldSpecification.Examples	75
2.28	HepLean.PerturbationTheory.FieldSpecification.Filters	75
2.29	HepLean.PerturbationTheory.FieldSpecification.NormalOrder	77
2.30	HepLean.PerturbationTheory.FieldSpecification.TimeOrder	84
2.31	HepLean.PerturbationTheory.FieldStatistics.Basic	92
2.32	HepLean.PerturbationTheory.FieldStatistics.ExchangeSign	95
2.33	HepLean.PerturbationTheory.FieldStatistics.OfFinset	96
2.34	HepLean.PerturbationTheory.Koszul.KoszulSign	97
2.35	HepLean.PerturbationTheory.Koszul.KoszulSignInsert	100
2.36	HepLean.PerturbationTheory.WickContraction.Basic	101
2.37	HepLean.PerturbationTheory.WickContraction.Erase	107
2.38	HepLean.PerturbationTheory.WickContraction.ExtractEquiv	108
2.39	HepLean.PerturbationTheory.WickContraction.InsertAndContract	109
2.40	HepLean.PerturbationTheory.WickContraction.InsertAndContractNat . . .	113
2.41	HepLean.PerturbationTheory.WickContraction.Involutions	117
2.42	HepLean.PerturbationTheory.WickContraction.IsFull	118
2.43	HepLean.PerturbationTheory.WickContraction.Sign	119
2.44	HepLean.PerturbationTheory.WickContraction.StaticContract	124
2.45	HepLean.PerturbationTheory.WickContraction.TimeContract	125
2.46	HepLean.PerturbationTheory.WickContraction.Uncontracted	127
2.47	HepLean.PerturbationTheory.WickContraction.UncontractedList	128
2.48	HepLean.PerturbationTheory.WicksTheorem	133
2.49	HepLean.Tensors.Tree.NodeIdentities.ProdContr	135
2.50	scripts.check_file_imports	136
2.51	scripts.MetaPrograms.free_simps	136
2.52	scripts.MetaPrograms.notes	136
2.53	scripts.MetaPrograms.TODO_to_ym1	137

1 Introduction

During Jan 2025, 3 contributors submitted 145 commits that changed 19,567 lines (+15,850 / -3,717) across 158 files spanning 24 top-level areas of the repository. Of those, 52 files were newly added and 15 were removed outright. This report catalogues 925 newly added Lean declarations: 715 lemmas, 162 defs, 30 instances, 6 structures, 5 abbrevs, 4 theorems, and 3 inductives. We're delighted to recognize the following first-time contributor this month: [kuotsanhsu](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Joseph Tooby-Smith	33,472	102
kuotsanhsu	1,632	6
Pietro Monticone	695	37

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	5

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
HepLean/PerturbationTheory	53	+38	-14	+12,192	-2,961
HepLean/Mathematics	10	+4	-1	+1,963	-108
HepLean/Lorentz	23	+1	—	+506	-244
HepLean/Tensors	18	—	—	+233	-218
scripts/MetaPrograms	3	+1	—	+236	-19
HepLean/Meta	10	+3	—	+206	-25
docs/_data	1	+1	—	+190	-0
docs/CuratedNotes	1	+1	—	+73	-0
HepLean/AnomalyCancellation	16	—	—	+30	-42
docs	4	+2	—	+57	-10
HepLean.lean	1	—	—	+46	-15
HepLean/StandardModel	5	—	—	+22	-21
lake-manifest.json	1	—	—	+21	-21
docs/assets	1	+1	—	+20	-0
HepLean/BeyondTheStandardModel	1	—	—	+10	-9
Archive/Papers	1	—	—	+6	-6
scripts	1	—	—	+12	-0
lakefile.toml	1	—	—	+7	-2
HepLean/FlavorPhysics	1	—	—	+3	-5

HepLean/SpaceTime	2	—	—	+4	-3
README.md	1	—	—	+3	-4
.github/workflows	1	—	—	+3	-3
docs/_layouts	1	—	—	+6	-0
lean-toolchain	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
lemma	715
def	162
instance	30
structure	6
abbrev	5
theorem	4
inductive	3

2 Details by file

2.1 HepLean.Lorentz.SL2C.SelfAdjoint

[HepLean/Lorentz/SL2C/SelfAdjoint.lean](#) on GitHub.

abbrev $\mathbb{H}_2$ [\[source\]](#)

A convenient abbreviation for the type of self-adjoint complex 2-by-2 matrices.

```
noncomputable abbrev  $\mathbb{H}_2$ 
```

def toSelfAdjointMap' [\[source\]](#)

Definitionally equal to Lorentz.SL2C.toSelfAdjointMap but dropping the requirement that M be special linear.

```
noncomputable def toSelfAdjointMap' (M :  $\mathbb{C}^{2 \times 2}$ ) :  $\mathbb{H}_2 \rightarrow_l [\mathbb{R}] \mathbb{H}_2$  where
```

lemma toSelfAdjointMap_det_one' [\[source\]](#)

```
lemma toSelfAdjointMap_det_one' {M :  $\mathbb{C}^{2 \times 2}$ } (hM : M.IsUpperTriangular) (detM : M.det = 1) :  
  LinearMap.det (toSelfAdjointMap' M) = 1
```

def toSelfAdjointEquiv [\[source\]](#)

This promotes Lorentz.SL2C.toSelfAdjointMap M and its definitional equivalence, Lorentz.SL2C.toSelfAdjointMap' M , to a linear equivalence by recognising the linear inverse to be Lorentz.SL2C.toSelfAdjointMap M^{-1} , i.e., Lorentz.SL2C.toSelfAdjointMap' M^{-1} .

```
noncomputable def toSelfAdjointEquiv (M :  $\mathbb{C}^{2 \times 2}$ ) [Invertible M] :  $\mathbb{H}_2 \approx_l [\mathbb{R}] \mathbb{H}_2$  where
```

lemma toSelfAdjointMap_mul[\[source\]](#)

```
lemma toSelfAdjointMap_mul (M N :  $\mathbb{C}^{2 \times 2}$ ) :
  toSelfAdjointMap' (M * N) = toSelfAdjointMap' M  $\circ_l$  toSelfAdjointMap' N
```

lemma toSelfAdjointMap_similar_det[\[source\]](#)

```
lemma toSelfAdjointMap_similar_det (M N :  $\mathbb{C}^{2 \times 2}$ ) [Invertible M] :
  LinearMap.det (toSelfAdjointMap' (M * N * M-1)) = LinearMap.det (toSelfAdjointMap' N)
```

2.2 HepLean.Mathematics.Fin.Involutions

[HepLean/Mathematics/Fin/Involutions.lean](#) on GitHub.

def involutionCons[\[source\]](#)

There is an equivalence between involutions of $\text{Fin } n.\text{succ}$ and involutions of $\text{Fin } n$ and an optional valid choice of an element in $\text{Fin } n$ (which is where 0 in $\text{Fin } n.\text{succ}$ will be sent).

```
def involutionCons (n : ℕ) : {f : Fin n.succ → Fin n.succ // Function.Involutive f} ≈
  (f : {f : Fin n → Fin n // Function.Involutive f}) × {i : Option (Fin n) //
    ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)} where
```

lemma involutionCons_ext[\[source\]](#)

```
lemma involutionCons_ext {n : ℕ} {f1 f2 : (f : {f : Fin n → Fin n // Function.Involutive f}) ×
  {i : Option (Fin n) // ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)}}
  (h1 : f1.1 = f2.1) (h2 : f1.2 = Equiv.subtypeEquivRight (by rw [h1]; simp) f2.2) : f1 = f2
```

def involutionAddEquiv[\[source\]](#)

Given an involution of $\text{Fin } n$, the optional choice of an element in $\text{Fin } n$ which maps to itself is equivalent to the optional choice of an element in Fin ($\text{Finset.univ.filter fun i => f.1 i = i}$).card.

```
def involutionAddEquiv {n : ℕ} (f : {f : Fin n → Fin n // Function.Involutive f}) :
  {i : Option (Fin n) // ∀ (h : i.isSome), f.1 (Option.get i h) = (Option.get i h)} ≈
  Option (Fin (Finset.univ.filter fun i => f.1 i = i).card)
```

lemma involutionAddEquiv_none_image_zero[\[source\]](#)

```
lemma involutionAddEquiv_none_image_zero {n : ℕ} :
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  → involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2 = none
  → f.1 {0, Nat.zero_lt_succ n} = {0, Nat.zero_lt_succ n}
```

lemma involutionAddEquiv_cast[\[source\]](#)

```
lemma involutionAddEquiv_cast {n : ℕ} {f1 f2 : {f : Fin n → Fin n // Function.Involutive f}}
  (hf : f1 = f2) :
  involutionAddEquiv f1 = (Equiv.subtypeEquivRight (by rw [hf]; simp)).trans
    ((involutionAddEquiv f2).trans (Equiv.optionCongr (finCongr (by rw [hf]))))
```

lemma involutionAddEquiv_cast'[\[source\]](#)

```

lemma involutionAddEquiv_cast' {m : ℕ} {f1 f2 : {f : Fin m → Fin m // Function.Involutive f}}
  {N : ℕ} (hf : f1 = f2) (n : Option (Fin N))
  (hn1 : N = (Finset.filter (fun i => f1.1 i = i) Finset.univ).card)
  (hn2 : N = (Finset.filter (fun i => f2.1 i = i) Finset.univ).card) :
  HEq ((involutionAddEquiv f1).symm (Option.map (finCongr hn1) n))
  ((involutionAddEquiv f2).symm (Option.map (finCongr hn2) n))

```

lemma involutionAddEquiv_none_succ[\[source\]](#)

```

lemma involutionAddEquiv_none_succ {n : ℕ}
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  (h : involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2 = none)
  (x : Fin n) : f.1 x.succ = x.succ ↔ (involutionCons n f).1.1 x = x

```

lemma involutionAddEquiv_isSome_image_zero[\[source\]](#)

```

lemma involutionAddEquiv_isSome_image_zero {n : ℕ} :
  {f : {f : Fin n.succ → Fin n.succ // Function.Involutive f}}
  → (involutionAddEquiv (involutionCons n f).1 (involutionCons n f).2).isSome
  → ¬ f.1 ⟨0, Nat.zero_lt_succ n⟩ = ⟨0, Nat.zero_lt_succ n⟩

```

def involutionNoFixedEquivSum[\[source\]](#)

Fixed point free involutions of $\text{Fin } n.\text{succ}$ can be separated based on where they sent 0 .

```

def involutionNoFixedEquivSum {n : ℕ} :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f}
  ∧ ∀ i, f i ≠ i} ≈ Σ (k : Fin n), {f : Fin n.succ → Fin n.succ // Function.Involutive f
  ∧ (∀ i, f i ≠ i) ∧ f 0 = k.succ} where

```

def involutionNoFixedZeroSetEquivEquiv[\[source\]](#)

The condition on fixed point free involutions of $\text{Fin } n.\text{succ}$ for a fixed value of $f\ 0$, can be modified by conjugation with an equivalence.

```

def involutionNoFixedZeroSetEquivEquiv {n : ℕ}
  (k : Fin n) (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧ (∀ i, f i ≠ i) ∧ f 0 = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive (e.symm ∘ f ∘ e) ∧
  (∀ i, (e.symm ∘ f ∘ e) i ≠ i) ∧ (e.symm ∘ f ∘ e) 0 = k.succ} where

```

def involutionNoFixedZeroSetEquivSetEquiv[\[source\]](#)

The condition on fixed point free involutions of $\text{Fin } n.\text{succ}$ for a fixed value of $f\ 0$ given an equivalence e , can be modified so that only the condition on $f\ 0$ is up-to the equivalence e .

```

def involutionNoFixedZeroSetEquivSetEquiv {n : ℕ} (k : Fin n)
  (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive (e.symm ∘ f ∘ e) ∧
  (∀ i, (e.symm ∘ f ∘ e) i ≠ i) ∧ (e.symm ∘ f ∘ e) 0 = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧

```

$$(\forall i, f i \neq i) \wedge (e.\text{symm} \circ f \circ e) \theta = k.\text{succ}$$
def involutionNoFixedZeroSetEquivEquiv'
[\[source\]](#)

Fixed point free involutions of $\text{Fin } n.\text{succ}$ fixing $(e.\text{symm} \circ f \circ e) = k.\text{succ}$ for a given e are equivalent to fixing $f (e \theta) = e k.\text{succ}$.

```
def involutionNoFixedZeroSetEquivEquiv' {n : ℕ} (k : Fin n) (e : Fin n.succ ≈ Fin n.succ) :
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ (e.symm ∘ f ∘ e) θ = k.succ} ≈
  {f : Fin n.succ → Fin n.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f (e θ) = e k.succ}
```

def involutionNoFixedZeroSetEquivSetOne
[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ.succ}$ with $f \theta = k.\text{succ}$ are equivalent to fixed point involutions with $f \theta = 1$.

```
def involutionNoFixedZeroSetEquivSetOne {n : ℕ} (k : Fin n.succ) :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f θ = k.succ}
  ≈ {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f θ = 1}
```

def involutionNoFixedSetOne
[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ.succ}$ fixing $f \theta = 1$ are equivalent to fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedSetOne {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f θ = 1} ≈ {f : Fin n → Fin n // Function.Involutive f ∧
    (∀ i, f i ≠ i)} where
```

def involutionNoFixedZeroSetEquiv
[\[source\]](#)

Fixed point involutions of $\text{Fin } n.\text{succ.succ}$ for fixed $f \theta = k.\text{succ}$ are equivalent to fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedZeroSetEquiv {n : ℕ} (k : Fin n.succ) :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧
    (∀ i, f i ≠ i) ∧ f θ = k.succ}
  ≈ {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

def involutionNoFixedEquivSumSame
[\[source\]](#)

The type of fixed point free involutions of $\text{Fin } n.\text{succ.succ}$ is equivalent to the sum of $\text{Fin } n.\text{succ}$ copies of fixed point involutions of $\text{Fin } n$.

```
def involutionNoFixedEquivSumSame {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  ≈ Σ (_ : Fin n.succ),
    {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

def involutionNoFixedZeroEquivProd[\[source\]](#)

Ever fixed-point free involutions of $\text{Fin } n.\text{succ}.\text{succ}$ can be decomposed into a element of $\text{Fin } n.\text{succ}$ (where 0 is sent) and a fixed-point free involution of $\text{Fin } n$.

```
def involutionNoFixedZeroEquivProd {n : ℕ} :
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  ≈ Fin n.succ ×
  {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

instance Fintype { f // Function.Involutive f ∧ ∀ (i : Fin n), f i ≠ i }[\[source\]](#)

The type of fixed-point free involutions of $\text{Fin } n$ is finite.

```
instance {n : ℕ} : Fintype { f // Function.Involutive f ∧ ∀ (i : Fin n), f i ≠ i }
```

lemma involutionNoFixed_card_succ[\[source\]](#)

```
lemma involutionNoFixed_card_succ {n : ℕ} :
  Fintype.card
  {f : Fin n.succ.succ → Fin n.succ.succ // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = n.succ *
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)}
```

lemma involutionNoFixed_card_mul_two[\[source\]](#)

```
lemma involutionNoFixed_card_mul_two : (n : ℕ) →
  Fintype.card {f : Fin (2 * n) → Fin (2 * n) // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = (2 * n - 1)!!
| 0 => rfl
| Nat.succ n => by
  erw [involutionNoFixed_card_succ]
  erw [involutionNoFixed_card_mul_two n]
  exact Eq.symm (Nat.doubleFactorial_add_one (Nat.mul 2 n))
```

lemma involutionNoFixed_card_mul_two_plus_one[\[source\]](#)

```
lemma involutionNoFixed_card_mul_two_plus_one : (n : ℕ) →
  Fintype.card {f : Fin (2 * n + 1) → Fin (2 * n + 1) // Function.Involutive f ∧ (∀ i, f i ≠ i)}
  = 0
| 0 => rfl
| Nat.succ n => by
  erw [involutionNoFixed_card_succ]
  erw [involutionNoFixed_card_mul_two_plus_one n]
  exact rfl
```

lemma involutionNoFixed_card_even[\[source\]](#)

```
lemma involutionNoFixed_card_even : (n : ℕ) → (he : Even n) →
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)} = (n - 1)!!
```

lemma involutionNoFixed_card_odd[\[source\]](#)


```
lemma involutionNoFixed_card_odd : (n : ℕ) → (ho : Odd n) →
  Fintype.card {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)} = 0
```

2.3 HepLean.Mathematics.List

[HepLean/Mathematics/List.lean](#) on GitHub.

lemma insertionSort_length

[\[source\]](#)

```
lemma insertionSort_length {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (l : List I) :
  (List.insertionSort le1 l).length = l.length
```

lemma lt_orderedInsertPos_rel_fin

[\[source\]](#)

```
lemma lt_orderedInsertPos_rel_fin {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
  (r0 : I) (r : List I) (n : Fin (List.orderedInsert le1 r0 r).length)
  (hn : n < (orderedInsertPos le1 r r0)) : ¬ le1 r0 ((List.orderedInsert le1 r0 r).get n)
```

lemma orderedInsertEquiv_monotone_fin_succ

[\[source\]](#)

```
lemma orderedInsertEquiv_monotone_fin_succ {I : Type}
  (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
  (r0 : I) (n m : Fin r.length)
  (hx : orderedInsertEquiv le1 r r0 n.succ < orderedInsertEquiv le1 r r0 m.succ) :
  n < m
```

lemma insertionSortEquiv_order

[\[source\]](#)

```
lemma insertionSortEquiv_order {α : Type} {r : α → α → Prop} [DecidableRel r] :
  (l : List α) → (i : Fin l.length) → (j : Fin l.length) → (hij : i < j)
  → (hij' : insertionSortEquiv r l j < insertionSortEquiv r l i) →
  ¬ r l[i] l[j]
| [], i, _, _, _ => Fin.elim0 i
| a :: as, ⟨0, hi⟩, ⟨j + 1, hj⟩, hij, hij' => by
  simp only [List.length_cons, Fin.zero_eta, Fin.getElem_fin, Fin.val_zero,
    List.getElem_cons_zero, List.getElem_cons_succ]
  nth_rewrite 2 [insertionSortEquiv] at hij'
  simp only [List.insertionSort.eq_2, List.length_cons, Nat.succ_eq_add_one, Fin.zero_eta,
    Equiv.trans_apply, equivCons_zero] at hij'
  have hx
```

lemma eraseIdx_length'

[\[source\]](#)

```
lemma eraseIdx_length' {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length = l.length - 1
```

lemma eraseIdx_length_succ

[\[source\]](#)

```
lemma eraseIdx_length_succ {I : Type} (l : List I) (i : Fin l.length) :
  (List.eraseIdx l i).length.succ = l.length
```

def insertionSortMinPos[\[source\]](#)

Given a list $i :: l$ the left-most minimal position a of $i :: l$ wrt r . That is the first position of l such that for every element $(i :: l)[b]$ before that position $r ((i :: l)[b]) ((i :: l)[a])$ is not true. The use of $i :: l$ here rather than just l is to ensure that such a position exists. .

```
def insertionSortMinPos {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  Fin (i :: l).length
```

def insertionSortMin[\[source\]](#)

The element of $i :: l$ at insertionSortMinPos.

```
def insertionSortMin {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  α
```

lemma insertionSortMin_eq_insertionSort_head[\[source\]](#)

```
lemma insertionSortMin_eq_insertionSort_head {α : Type} (r : α → α → Prop) [DecidableRel r]
  (i : α) (l : List α) :
  insertionSortMin r i l = (List.insertionSort r (i :: l)).head (by
  refine List.ne_nil_of_length_pos ?_
  rw [insertionSort_length]
  exact Nat.zero_lt_succ l.length)
```

def insertionSortDropMinPos[\[source\]](#)

The list remaining after dropping the element at the position determined by insertionSortMinPos.

```
def insertionSortDropMinPos {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α)
  :
  List α
```

lemma insertionSort_eq_insertionSortMin_cons[\[source\]](#)

```
lemma insertionSort_eq_insertionSortMin_cons {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (i : α) (l : List α) :
  List.insertionSort r (i :: l) =
  (insertionSortMin r i l) :: List.insertionSort r (insertionSortDropMinPos r i l)
```

lemma optionEraseZ_some_length[\[source\]](#)

```
@[simp]
lemma optionEraseZ_some_length {I : Type} (l : List I) (a : I) (i : (Fin l.length)) :
  (optionEraseZ l a (some i)).length = l.length - 1
```

lemma optionEraseZ_ext[\[source\]](#)

```
lemma optionEraseZ_ext {I : Type} {l l' : List I} {a a' : I} {i : Option (Fin l.length)}
  {i' : Option (Fin l'.length)} (hl : l = l') (ha : a = a')
  (hi : Option.map (Fin.cast (by rw [hl])) i = i') :
  optionEraseZ l a i = optionEraseZ l' a' i'
```

lemma mem_take_finrange[\[source\]](#)

```

lemma mem_take_finrange : (n m : ℕ) → (a : Fin n) → a ∈ List.take m (List.finRange n) ↔ a.val <
  m
| 0, m, a => Fin.elim0 a
| n+1, 0, a => by
  simp
| n +1, m + 1, ⟨0, h⟩ => by
  simp [List.finRange_succ]
| n +1, m + 1, ⟨i + 1, h⟩ => by
  simp only [List.finRange_succ, List.take_succ_cons, List.mem_cons, Fin.ext_iff, Fin.
    val_zero,
    AddLeftCancelMonoid.add_eq_zero, one_ne_zero, and_false, false_or, add_lt_add_iff_right]
  rw [← List.map_take]
  rw [← List.mem_map]
  apply Iff.intro
  · intro h
    obtain ⟨a, ha⟩

```

2.4 HepLean.Mathematics.List.InsertIdx

[HepLean/Mathematics/List/InsertIdx.lean](#) on GitHub.

lemma insertIdx_map[\[source\]](#)

```

lemma insertIdx_map {I J : Type} (f : I → J) : (i : ℕ) → (r : List I) → (r0 : I) →
  (List.insertIdx i r0 r).map f = List.insertIdx i (f r0) (r.map f)
| 0, [], r0 => by simp
| n+1, [], r0 => by simp
| 0, a::as, r0 => by simp
| n+1, a::as, r0 => by
  simp only [List.insertIdx_succ_cons, List.map_cons, List.cons.injEq, true_and]
  exact insertIdx_map f n as r0

```

lemma eraseIdx_sorted[\[source\]](#)

```

lemma eraseIdx_sorted {I : Type} (le : I → I → Prop) :
  (r : List I) → (n : ℕ) →
  List.Sorted le r → List.Sorted le (r.eraseIdx n)
| [], _, _ => by simp
| a::as, 0, h => by
  simp only [List.eraseIdx]
  simp only [List.sorted_cons] at h
  exact h.2
| a::as, n+1, h => by
  simp only [List.eraseIdx_cons_succ, List.sorted_cons]
  simp only [List.sorted_cons] at h
  refine And.intro ?_ (eraseIdx_sorted le as n h.2)
  intro b hb
  refine h.1 _ ?_
  exact List.mem_of_mem_eraseIdx hb

```

lemma mem_eraseIdx_nodup[\[source\]](#)

```

lemma mem_eraseIdx_nodup {I : Type} (i : I) :
  (l : List I) → (n : ℕ) → (hn : n < l.length) → (h : List.Nodup l) →

```

```

i ∈ l.eraseIdx n ↔ i ∈ l ∧ i ≠ l[n]
| [], _, _, _ => by simp
| a1 :: as, 0, _, h => by
  simp only [List.eraseIdx_zero, List.tail_cons, List.mem_cons, List.getElem_cons_zero, ne_eq]
  by_cases hi : i = a1
  · subst hi
    simp only [List.nodup_cons] at h
    simp [h]
  · simp [hi]
| a1 :: as, n+1, hn, h => by
  simp only [List.eraseIdx_cons_succ, List.mem_cons, List.getElem_cons_succ, ne_eq]
  simp only [List.nodup_cons] at h
  rw [mem_eraseIdx_nodup i as n (Nat.succ_lt_succ_iff.mp hn) h.2]
  simp_all only [ne_eq]
  obtain ⟨left, right⟩

```

lemma insertIdx_eq_take_drop[\[source\]](#)

```

lemma insertIdx_eq_take_drop {I : Type} (i : I) : (r : List I) → (n : Fin r.length.succ) →
  List.insertIdx n i r = List.take n r ++ i :: r.drop n
| [], 0 => by simp
| a :: as, 0 => by
  simp
| a :: as, ⟨n + 1, h⟩ => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.drop_succ_cons, List.
    cons_append,
    List.cons.injEq, true_and]
  exact insertIdx_eq_take_drop i as ⟨n, Nat.succ_lt_succ_iff.mp h⟩

```

lemma insertIdx_length_fin[\[source\]](#)

```

@[simp]
lemma insertIdx_length_fin {I : Type} (i : I) :
  (r : List I) → (n : Fin r.length.succ) →
  (List.insertIdx n i r).length = r.length.succ
| [], 0 => by simp
| a :: as, 0 => by simp
| a :: as, ⟨n + 1, h⟩ => by
  simp only [List.insertIdx_succ_cons, List.length_cons, Nat.succ_eq_add_one, add_left_inj]
  exact insertIdx_length_fin i as ⟨n, Nat.succ_lt_succ_iff.mp h⟩

```

lemma insertIdx_getElem_fin[\[source\]](#)

```

@[simp]
lemma insertIdx_getElem_fin {I : Type} (i : I) :
  (r : List I) → (k : Fin r.length.succ) → (m : Fin r.length) →
  (List.insertIdx k i r)[(k.succAbove m).val] = r[m.val]
| [], 0, m => by exact Fin.elim0 m
| a :: as, 0, m => by simp
| a :: as, ⟨n + 1, h⟩, ⟨0, h0⟩ => by
  simp [Fin.succAbove, Fin.lt_def]
| a :: as, ⟨n + 1, h⟩, ⟨m+1, hm⟩ => by
  simp only [List.insertIdx_succ_cons, List.length_cons, Nat.succ_eq_add_one,
    List.getElem_cons_succ]
  conv_rhs => rw [← insertIdx_getElem_fin i as ⟨n, Nat.succ_lt_succ_iff.mp h⟩]

```

```

    (m, Nat.lt_of_succ_lt_succ hm)]
simp only [Fin.succAbove, Fin.castSucc_mk, Fin.lt_def, add_lt_add_iff_right, Fin.succ_mk,
  Nat.succ_eq_add_one]
split
· simp_all only [List.getElem_cons_succ]
· simp_all only [List.getElem_cons_succ]

```

lemma insertIdx_eraseIdx_fin[\[source\]](#)

```

lemma insertIdx_eraseIdx_fin {I : Type} :
  (r : List I) → (k : Fin r.length) →
  (List.eraseIdx r k).insertIdx k r[k] = r
| [], k => by exact Fin.elim0 k
| a :: as, ⟨0, h⟩ => by simp
| a :: as, ⟨n + 1, h⟩ => by
  simp only [List.length_cons, Fin.getElem_fin, List.getElem_cons_succ, List.
    eraseIdx_cons_succ,
    List.insertIdx_succ_cons, List.cons.injEq, true_and]
  exact insertIdx_eraseIdx_fin as ⟨n, Nat.lt_of_succ_lt_succ h⟩

```

lemma insertIdx_length_fst_append[\[source\]](#)

```

lemma insertIdx_length_fst_append {I : Type} (φ : I) : (φs φs' : List I) →
  List.insertIdx φs.length φ (φs ++ φs') = (φs ++ φ :: φs')
| [], φs' => by simp
| φ' :: φs, φs' => by
  simp only [List.length_cons, List.cons_append, List.insertIdx_succ_cons, List.cons.injEq,
    true_and]
  exact insertIdx_length_fst_append φ φs φs'

```

lemma get_eq_insertIdx_succAbove[\[source\]](#)

```

lemma get_eq_insertIdx_succAbove {I : Type} (i : I) (r : List I) (k : Fin r.length.succ) :
  r.get = (List.insertIdx k i r).get ◦
  (finCongr (insertIdx_length_fin i r k).symm) ◦ k.succAbove

```

lemma take_insert_same[\[source\]](#)

```

lemma take_insert_same {I : Type} (i : I) :
  (n : ℕ) → (r : List I) →
  List.take n (List.insertIdx n i r) = List.take n r
| 0, _ => by simp
| _+1, [] => by simp
| n+1, a::as => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
  exact take_insert_same i n as

```

lemma take_eraseIdx_same[\[source\]](#)

```

lemma take_eraseIdx_same {I : Type} :
  (n : ℕ) → (r : List I) →
  List.take n (List.eraseIdx r n) = List.take n r
| 0, _ => by simp
| _+1, [] => by simp

```

```
| n+1, a::as => by
  simp only [List.eraseIdx_cons_succ, List.take_succ_cons, List.cons.injEq, true_and]
  exact take_eraseIdx_same n as
```

lemma drop_eraseIdx_succ[\[source\]](#)

```
lemma drop_eraseIdx_succ {I : Type} :
  (n : ℕ) → (r : List I) → (hn : n < r.length) →
  r[n] :: List.drop n (List.eraseIdx r n) = List.drop n r
| 0, _, _ => by
  simp only [List.eraseIdx_zero, List.drop_tail, zero_add, List.drop_one, List.drop_zero]
  rw [@List.getElem_zero]
  exact List.head_cons_tail _ _
| n+1, [], hn => by simp at hn
| n+1, a::as, hn => by
  simp only [List.getElem_cons_succ, List.eraseIdx_cons_succ, List.drop_succ_cons]
  refine drop_eraseIdx_succ n as _
```

lemma take_insert_gt[\[source\]](#)

```
lemma take_insert_gt {I : Type} (i : I) :
  (n m : ℕ) → (h : n < m) → (r : List I) →
  List.take n (List.insertIdx m i r) = List.take n r
| 0, 0, _, _ => by simp
| 0, m + 1, _, _ => by simp
| n+1, m + 1, _, [] => by simp
| n+1, m + 1, h, a::as => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
  refine take_insert_gt i n m (Nat.succ_lt_succ_iff.mp h) as
```

lemma take_insert_let[\[source\]](#)

```
lemma take_insert_let {I : Type} (i : I) :
  (n m : ℕ) → (h : m ≤ n) → (r : List I) → (hm : m ≤ r.length) →
  (List.take (n + 1) (List.insertIdx m i r)).Perm (i :: List.take n r)
| 0, 0, h, _, _ => by simp
| m + 1, 0, h, r, _ => by simp
| n + 1, m + 1, h, [], hm => by
  simp at hm
| n + 1, m + 1, h, a::as, hm => by
  simp only [List.insertIdx_succ_cons, List.take_succ_cons]
  have hp : (i :: a :: List.take n as).Perm (a :: i :: List.take n as)
```

2.5 HepLean.Mathematics.List.InsertionSort

[HepLean/Mathematics/List/InsertionSort.lean](#) on GitHub.

lemma insertionSortMin_lt_length_succ[\[source\]](#)

```
lemma insertionSortMin_lt_length_succ {α : Type} (r : α → α → Prop) [DecidableRel r]
  (i : α) (l : List α) :
  insertionSortMinPos r i l < (insertionSortDropMinPos r i l).length.succ
```

def insertionSortMinPosFin[\[source\]](#)

Given a list $i :: l$ the left-most minimal position a of $i :: l$ wrt r as an element of $\text{Fin } (\text{insertionSortDropMinPos } r \ i \ l).length.succ$.

```
def insertionSortMinPosFin {α : Type} (r : α → α → Prop) [DecidableRel r] (i : α) (l : List α) :
  Fin (insertionSortDropMinPos r i l).length.succ
```

lemma insertionSortMin_lt_mem_insertionSortDropMinPos[\[source\]](#)

```
lemma insertionSortMin_lt_mem_insertionSortDropMinPos {α : Type} (r : α → α → Prop) [
  DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) (l : List α)
  (i : Fin (insertionSortDropMinPos r a l).length) :
  r (insertionSortMin r a l) ((insertionSortDropMinPos r a l)[i])
```

lemma insertionSortMinPos_insertionSortEquiv[\[source\]](#)

```
lemma insertionSortMinPos_insertionSortEquiv {α : Type} (r : α → α → Prop) [DecidableRel r]
  (a : α) (l : List α) :
  insertionSortEquiv r (a :: l) (insertionSortMinPos r a l) =
  ⟨0, by simp [List.orderedInsert_length]⟩
```

lemma insertionSortEquiv_gt_zero_of_ne_insertionSortMinPos[\[source\]](#)

```
lemma insertionSortEquiv_gt_zero_of_ne_insertionSortMinPos {α : Type} (r : α → α → Prop)
  [DecidableRel r] (a : α) (l : List α) (k : Fin (a :: l).length)
  (hk : k ≠ insertionSortMinPos r a l) :
  ⟨0, by simp [List.orderedInsert_length]⟩ < insertionSortEquiv r (a :: l) k
```

lemma insertionSortMin_lt_mem_insertionSortDropMinPos_of_lt[\[source\]](#)

```
lemma insertionSortMin_lt_mem_insertionSortDropMinPos_of_lt {α : Type} (r : α → α → Prop)
  [DecidableRel r] (a : α) (l : List α)
  (i : Fin (insertionSortDropMinPos r a l).length)
  (h : (insertionSortMinPosFin r a l).succAbove i < insertionSortMinPosFin r a l) :
  ~ r ((insertionSortDropMinPos r a l)[i]) (insertionSortMin r a l)
```

lemma insertionSort_insertionSort[\[source\]](#)

```
lemma insertionSort_insertionSort {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (ll : List α) :
  List.insertionSort r (List.insertionSort r ll) = List.insertionSort r ll
```

lemma orderedInsert_commute[\[source\]](#)

```
lemma orderedInsert_commute {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a b : α) (hr : ~ r a b) : (l : List α) →
  List.orderedInsert r a (List.orderedInsert r b l) =
  List.orderedInsert r b (List.orderedInsert r a l)
| [] => by
  have hrb : r b a
```

lemma insertionSort_orderedInsert_append[\[source\]](#)

```

lemma insertionSort_orderedInsert_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
  List.insertionSort r (List.orderedInsert r a l1 ++ l2) = List.insertionSort r (a :: l1 ++
  l2)
| [], l2 => by
  simp
| b :: l1, l2 => by
  conv_lhs => simp
  by_cases h : r a b
  · simp [h]
  conv_lhs => simp [h]
  rw [insertionSort_orderedInsert_append r a l1 l2]
  simp only [List.insertionSort, List.append_eq]
  rw [orderedInsert_commute r a b h]

```

lemma insertionSort_insertionSort_append[\[source\]](#)

```

lemma insertionSort_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] : (l1 l2 : List α) →
  List.insertionSort r (List.insertionSort r l1 ++ l2) = List.insertionSort r (l1 ++ l2)
| [], l2 => by
  simp
| a :: l1, l2 => by
  conv_lhs => simp
  rw [insertionSort_orderedInsert_append]
  simp only [List.insertionSort, List.append_eq]
  rw [insertionSort_insertionSort_append r l1 l2]

```

lemma insertionSort_append_insertionSort_append[\[source\]](#)

```

lemma insertionSort_append_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] : (l1 l2 l3 : List α) →
  List.insertionSort r (l1 ++ List.insertionSort r l2 ++ l3) =
  List.insertionSort r (l1 ++ l2 ++ l3)
| [], l2, l3 => by
  simp only [List.nil_append]
  exact insertionSort_insertionSort_append r l2 l3
| a :: l1, l2, l3 => by
  simp only [List.insertionSort, List.append_eq]
  rw [insertionSort_append_insertionSort_append r l1 l2 l3]

```

lemma orderedInsert_length[\[source\]](#)

```

@[simp]
lemma orderedInsert_length {α : Type} (r : α → α → Prop) [DecidableRel r] (a : α) (l : List α)
  :
  (List.orderedInsert r a l).length = (a :: l).length

```

lemma takeWhile_orderedInsert[\[source\]](#)

```

lemma takeWhile_orderedInsert {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r]
  (a b : α) (hr : ¬ r a b) : (l : List α) →

```



```

(List.takeWhile (fun c => !decide (r a c)) (List.orderedInsert r b l)).length =
(List.takeWhile (fun c => !decide (r a c)) l).length + 1
| [] => by
  simp [hr]
| c :: l => by
  simp only [List.orderedInsert]
  by_cases h : r b c
  · simp only [h, ↓reduceIte]
    rw [List.takeWhile_cons_of_pos]
    simp only [List.length_cons]
    simp [hr]
  · simp only [h, ↓reduceIte]
    have hrba : r b a

```

lemma takeWhile_orderedInsert'[\[source\]](#)

```

lemma takeWhile_orderedInsert' {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r]
  (a b : α) (hr : ¬ r a b) : (l : List α) →
  (List.takeWhile (fun c => !decide (r b c)) (List.orderedInsert r a l)).length =
  (List.takeWhile (fun c => !decide (r b c)) l).length
| [] => by
  simp only [List.orderedInsert, List.takeWhile_nil, List.length_nil, List.length_eq_zero,
    List.takeWhile_eq_nil_iff, List.length_singleton, zero_lt_one, Fin.zero_eta, Fin.isValue,
    List.get_eq_getElem, Fin.val_eq_zero, List.getElem_cons_zero, Bool.not_eq_eq_eq_not,
    Bool.not_true, decide_eq_false_iff_not, Decidable.not_not, forall_const]
  have ht

```

lemma insertionSortEquiv_commute[\[source\]](#)

```

lemma insertionSortEquiv_commute {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a b : α) (hr : ¬ r a b) (n : ℕ) : (l : List α) →
  (hn : n + 2 < (a :: b :: l).length) →
  insertionSortEquiv r (a :: b :: l) (n + 2, hn) = (finCongr (by simp))
  (insertionSortEquiv r (b :: a :: l) (n + 2, hn))

```

lemma insertionSortEquiv_orderedInsert_append[\[source\]](#)

```

lemma insertionSortEquiv_orderedInsert_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a a2 : α) : (l1 l2 : List α) →
  (insertionSortEquiv r (List.orderedInsert r a l1 ++ a2 :: l2) (l1.length + 1, by
    simp))
  = (finCongr (by
    simp only [List.insertionSort, List.append_eq, orderedInsert_length, List.length_cons,
      List.length_insertionSort, List.length_append]
    omega))
  ((insertionSortEquiv r (a :: l1 ++ a2 :: l2) (l1.length + 1, by simp)))
| [], l2 => by
  simp
| b :: l1, l2 => by
  by_cases h : r a b
  · have h1 : (List.orderedInsert r a (b :: l1) ++ a2 :: l2) = (a :: b :: l1 ++ a2 :: l2)

```

lemma insertionSortEquiv_insertionSort_append[\[source\]](#)

```

lemma insertionSortEquiv_insertionSort_append {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) : (l1 l2 : List α) →
  (insertionSortEquiv r (List.insertionSort r l1 ++ a :: l2) (l1.length, by simp))
= finCongr (by simp) (insertionSortEquiv r (l1 ++ a :: l2) (l1.length, by simp))
| [], l2 => by
  simp only [List.insertionSort, List.nil_append, List.length_cons, List.length_nil, Fin.
    zero_eta,
    finCongr_refl, Equiv.refl_apply]
| b :: l1, l2 => by
  simp only [List.insertionSort, List.length_cons, List.cons_append, finCongr_apply]
have hl

```

lemma orderedInsert_filter_of_pos

[\[source\]](#)

```

lemma orderedInsert_filter_of_pos {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) (p : α → Prop) [DecidablePred p] (h : p a) : (l : List α) →
  (hl : l.Sorted r) →
  List.filter p (List.orderedInsert r a l) = List.orderedInsert r a (List.filter p l)
| [], hl => by
  simp only [List.orderedInsert, List.filter_eq_self, List.mem_singleton, decide_eq_true_eq,
    forall_eq]
  exact h
| b :: l, hl => by
  simp only [List.orderedInsert]
  by_cases hpb : p b <=> by_cases hab : r a b
  · simp only [hab, ↓reduceIte, hpb, decide_true, List.filter_cons_of_pos,
    List.orderedInsert.eq_2]
    rw [List.filter_cons_of_pos (by simp [h])]
    rw [List.filter_cons_of_pos (by simp [hpb])]
  · simp only [hab, ↓reduceIte]
    rw [List.filter_cons_of_pos (by simp [hpb])]
    rw [List.filter_cons_of_pos (by simp [hpb])]
    simp only [List.orderedInsert, hab, ↓reduceIte, List.cons.injEq, true_and]
    simp only [List.sorted_cons] at hl
    exact orderedInsert_filter_of_pos r a p h l hl.2
  · simp only [hab, ↓reduceIte]
    rw [List.filter_cons_of_pos (by simp [h]),
      List.filter_cons_of_neg (by simp [hpb])]
    rw [List.orderedInsert_eq_take_drop]
    have hl : List.takeWhile (fun b => decide ¬r a b)
      (List.filter (fun b => decide (p b)) l) = []

```

lemma orderedInsert_filter_of_neg

[\[source\]](#)

```

lemma orderedInsert_filter_of_neg {α : Type} (r : α → α → Prop) [DecidableRel r]
  (a : α) (p : α → Prop) [DecidablePred p] (h : ¬ p a) (l : List α) :
  List.filter p (List.orderedInsert r a l) = (List.filter p l)

```

lemma insertionSort_filter

[\[source\]](#)

```

lemma insertionSort_filter {α : Type} (r : α → α → Prop) [DecidableRel r] [IsTotal α r]
  [IsTrans α r] (p : α → Prop) [DecidablePred p] : (l : List α) →
  List.insertionSort r (List.filter p l) =
  List.filter p (List.insertionSort r l)
| [] => by simp

```

```

| a :: l => by
  simp only [List.insertionSort]
  by_cases h : p a
  · rw [orderedInsert_filter_of_pos]
    rw [List.filter_cons_of_pos]
    simp only [List.insertionSort]
    rw [insertionSort_filter]
    simp_all only [decide_true]
    simp_all only
    exact List.sorted_insertionSort r l
  · rw [orderedInsert_filter_of_neg]
    rw [List.filter_cons_of_neg]
    rw [insertionSort_filter]
    simp_all only [decide_false, Bool.false_eq_true, not_false_eq_true]
    exact h

```

lemma takeWhile_sorted_eq_filter[\[source\]](#)

```

lemma takeWhile_sorted_eq_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Sorted r) →
  List.takeWhile (fun c => ¬ r a c) l = List.filter (fun c => ¬ r a c) l
| [], _ => by simp
| b :: l, hl => by
  simp only [List.sorted_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [decide_not, hb, decide_false, Bool.not_false, List.takeWhile_cons_of_pos,
    List.filter_cons_of_pos, List.cons.injEq, true_and]
    simp using takeWhile_sorted_eq_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_not, decide_true, Bool.not_true, Bool.
    false_eq_true,
    not_false_eq_true, List.takeWhile_cons_of_neg, List.filter_cons_of_neg, List.nil_eq,
    List.filter_eq_nil_iff, Bool.not_eq_eq_eq_not, decide_eq_false_iff_not]
    intro c hc
    apply IsTrans.trans a b c hb
    exact hl.1 c hc

```

lemma dropWhile_sorted_eq_filter[\[source\]](#)

```

lemma dropWhile_sorted_eq_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Sorted r) →
  List.dropWhile (fun c => ¬ r a c) l = List.filter (fun c => r a c) l
| [], _ => by simp
| b :: l, hl => by
  simp only [List.sorted_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [decide_not, hb, decide_false, Bool.not_false, List.dropWhile_cons_of_pos,
    Bool.false_eq_true, not_false_eq_true, List.filter_cons_of_neg]
    simp using dropWhile_sorted_eq_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_not, decide_true, Bool.not_true, Bool.
    false_eq_true,
    not_false_eq_true, List.dropWhile_cons_of_neg, List.filter_cons_of_pos, List.cons.injEq,
    true_and]
    symm
    rw [List.filter_eq_self]
    intro c hc
    simp only [decide_eq_true_eq]
    apply IsTrans.trans a b c hb

```

```
exact hl.1 c hc
```

lemma dropWhile_sorted_eq_filter_filter

[\[source\]](#)

```
lemma dropWhile_sorted_eq_filter_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTrans α r] (a : α) : (l : List α) → (hl : l.Sorted r) →
  List.filter (fun c => r a c) l =
  List.filter (fun c => r a c ∧ r c a) l ++ List.filter (fun c => r a c ∧ ¬ r c a) l
| [], _ => by
  simp
| b :: l, hl => by
  simp only [List.sorted_cons] at hl
  by_cases hb : ¬ r a b
  · simp only [hb, decide_false, Bool.false_eq_true, not_false_eq_true, List.
    filter_cons_of_neg,
    Bool.decide_and, Bool.false_and, decide_not]
    simp using dropWhile_sorted_eq_filter_filter r a l hl.2
  · simp_all only [Decidable.not_not, decide_true, List.filter_cons_of_pos, Bool.decide_and,
    decide_not]
    by_cases hba : r b a
    · simp only [hba, decide_true, Bool.not_true, Bool.and_false, Bool.false_eq_true,
      not_false_eq_true, List.filter_cons_of_neg]
      rw [List.filter_cons_of_pos]
      rw [dropWhile_sorted_eq_filter_filter]
      simp only [Bool.decide_and, decide_not, List.cons_append]
      exact hl.2
      simp_all
    · simp only [hba, decide_false, Bool.and_false, Bool.false_eq_true, not_false_eq_true,
      List.filter_cons_of_neg]
      have hl : List.filter (fun c => decide (r a c) && decide (r c a)) l = []
```

lemma filter_rel_eq_insertionSort

[\[source\]](#)

```
lemma filter_rel_eq_insertionSort {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) : (l : List α) →
  List.filter (fun c => r a c ∧ r c a) (l.insertionSort r) =
  List.filter (fun c => r a c ∧ r c a) l
| [] => by simp
| b :: l => by
  simp only [List.insertionSort]
  by_cases h : r a b ∧ r b a
  · have hl
```

lemma insertionSort_of_eq_list

[\[source\]](#)

```
lemma insertionSort_of_eq_list {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) (l1 l l2 : List α)
  (h : ∀ b ∈ l, r a b ∧ r b a) :
  List.insertionSort r (l1 ++ l ++ l2) =
  (List.takeWhile (fun c => ¬ r a c) ((l1 ++ l2).insertionSort r))
  ++ (List.filter (fun c => r a c ∧ r c a) l1)
  ++ l
  ++ (List.filter (fun c => r a c ∧ r c a) l2)
  ++ (List.filter (fun c => r a c ∧ ¬ r c a) ((l1 ++ l2).insertionSort r))
```

lemma insertionSort_of_takeWhile_filter[\[source\]](#)

```

lemma insertionSort_of_takeWhile_filter {α : Type} (r : α → α → Prop) [DecidableRel r]
  [IsTotal α r] [IsTrans α r] (a : α) (l1 l2 : List α) :
  List.insertionSort r (l1 ++ l2) =
  (List.takeWhile (fun c => ¬ r a c) ((l1 ++ l2).insertionSort r))
  ++ (List.filter (fun c => r a c ∧ r c a) l1)
  ++ (List.filter (fun c => r a c ∧ r c a) l2)
  ++ (List.filter (fun c => r a c ∧ ¬ r c a) ((l1 ++ l2).insertionSort r))

```

2.6 HepLean.Mathematics.SchurTriangulation

[HepLean/Mathematics/SchurTriangulation.lean](#) on GitHub.

def sumEquivSigmaCond[\[source\]](#)

An alternative form of `Equiv.sumEquivSigmaBool` where `Bool.casesOn` is replaced by `cond`.

```

def sumEquivSigmaCond : Fin m ⊕ Fin n ≈ Σ b, cond b (Fin m) (Fin n)

```

def finAddEquivSigmaCond[\[source\]](#)

The composition of `finSumFinEquiv` and `Equiv.sumEquivSigmaCond` used by `LinearMap.SchurTriangulationA`

```

def finAddEquivSigmaCond : Fin (m + n) ≈ Σ b, cond b (Fin m) (Fin n)

```

lemma finAddEquivSigmaCond_true[\[source\]](#)

```

lemma finAddEquivSigmaCond_true (h : i < m) : finAddEquivSigmaCond i = {true, i, h}

```

lemma finAddEquivSigmaCond_false[\[source\]](#)

```

lemma finAddEquivSigmaCond_false (h : ¬ i < m) : finAddEquivSigmaCond i = {false, i.subNat' h}

```

instance Fintype (cond b m n)[\[source\]](#)

The type family parameterized by `Bool` is finite if each type variant is finite.

```

instance [M : Fintype m] [N : Fintype n] (b : Bool) : Fintype (cond b m n)

```

instance DecidableEq (Σ b, cond b m n) | {true, _}, {false, _} | {false, _}, {tr... [\[source\]](#)

The type family parameterized by `Bool` has decidable equality if each type variant is decidable.

```

instance [DecidableEq m] [DecidableEq n] : DecidableEq (Σ b, cond b m n)
| {true, _}, {false, _}
| {false, _}, {true, _} => isFalse nofun
| {false, i}, {false, j}
| {true, i}, {true, j} =>
  if h : i = j then isTrue (Sigma.eq rfl h) else isFalse fun | rfl => h rfl

```

abbrev IsUpperTriangular[\[source\]](#)

The property of a matrix being upper triangular. See also `Matrix.det_of_upperTriangular`.

```
abbrev IsUpperTriangular [LT n] [CommRing R] (A : Matrix n n R)
```

abbrev UpperTriangular[\[source\]](#)

The subtype of upper triangular matrices.

```
abbrev UpperTriangular (n R) [LT n] [CommRing R]
```

lemma toMatrixOrthonormal_apply_apply[\[source\]](#)

```
lemma toMatrixOrthonormal_apply_apply (b : OrthonormalBasis n ℚ E) (f : Module.End ℚ E)
  (i j : n) :
  toMatrixOrthonormal b f i j = ⟨⟨b i, f (b j)⟩⟩_ℚ
```

lemma toMatrixOrthonormal_reindex[\[source\]](#)

```
lemma toMatrixOrthonormal_reindex [Fintype m] [DecidableEq m]
  (b : OrthonormalBasis m ℚ E) (e : m ≈ n) (f : Module.End ℚ E) :
  toMatrixOrthonormal (b.reindex e) f = Matrix.reindex e e (toMatrixOrthonormal b f)
```

structure SchurTriangulationAux[\[source\]](#)

***Don't use this definition directly.** Instead, use `Matrix.schurTriangulationBasis`, `Matrix.schurTriangulationUnitary`, and `Matrix.schurTriangulation`. See also `LinearMap.SchurTriangulation` and `Matrix.schurTriangulationAux`.*

```
structure SchurTriangulationAux (f : Module.End ℚ E) where
  dim : ℕ
  hdim : Module.finrank ℚ E = dim
  basis : OrthonormalBasis (Fin dim) ℚ E
  upperTriangular : (toMatrix basis.toBasis basis.toBasis f).IsUpperTriangular
```

def of[\[source\]](#)

***Don't use this definition directly.** This is the key algorithm behind `Matrix.schur_triangulation`.*

```
protected noncomputable def SchurTriangulationAux.of
  [NormedAddCommGroup E] [InnerProductSpace ℚ E] [FiniteDimensional ℚ E] (f : Module.End ℚ E)
  :
  SchurTriangulationAux f
```

def schurTriangulationAux[\[source\]](#)

***Don't use this definition directly.** Instead, use `Matrix.schurTriangulationBasis`, `Matrix.schurTriangulationUnitary`, and `Matrix.schurTriangulation` for which this is their simultaneous definition. This is `LinearMap.SchurTriangulationAux` adapted for matrices in the Euclidean space.*

```
noncomputable def schurTriangulationAux :
  OrthonormalBasis n ℚ (EuclideanSpace ℚ n) × UpperTriangular n ℚ
```

def schurTriangulationBasis[\[source\]](#)

The change of basis that induces the upper triangular form $A.\text{schurTriangulation}$ of a matrix A over an algebraically closed field.

```
noncomputable def schurTriangulationBasis : OrthonormalBasis n  $\mathbb{K}$  (EuclideanSpace  $\mathbb{K}$  n)
```

def schurTriangulationUnitary[\[source\]](#)

The unitary matrix that induces the upper triangular form $A.\text{schurTriangulation}$ to which a matrix A over an algebraically closed field is unitarily similar.

```
noncomputable def schurTriangulationUnitary : unitaryGroup n  $\mathbb{K}$  where
```

def schurTriangulation[\[source\]](#)

The upper triangular form induced by $A.\text{schurTriangulationUnitary}$ to which a matrix A over an algebraically closed field is unitarily similar.

```
noncomputable def schurTriangulation : UpperTriangular n  $\mathbb{K}$ 
```

lemma schur_triangulation[\[source\]](#)

***Schur triangulation**, **Schur decomposition** for matrices over an algebraically closed field. In particular, a complex matrix can be converted to upper-triangular form by a change of basis. In other words, any complex matrix is unitarily similar to an upper triangular matrix.*

```
lemma schur_triangulation :  
  A = A.schurTriangulationUnitary * A.schurTriangulation * star A.schurTriangulationUnitary
```

2.7 HepLean.Meta.Basic

[HepLean/Meta/Basic.lean](#) on GitHub.

def getDocString[\[source\]](#)

The doc string from a name.

```
def Name.getDocString (c : Name) : MetaM String
```

2.8 HepLean.Meta.Remark.Basic

[HepLean/Meta/Remark/Basic.lean](#) on GitHub.

structure RemarkInfo[\[source\]](#)

The information from a remark ... command. To be used in a note file

```
structure RemarkInfo where  
  content : String  
  fileName : Name  
  line : Nat  
  name : Name  
  nameSpace : Name
```

def elabRemark[\[source\]](#)*Elaborator for the note ... command*

@[command_elab remark_syntax]

def elabRemark : Lean.Elab.Command.CommandElab**2.9 HepLean.Meta.Remark.Properties**[HepLean/Meta/Remark/Properties.lean](#) on GitHub.**def allRemarkInfo**[\[source\]](#)*All remarks in the enviroment.***def** Name.allRemarkInfo : MetaM (List RemarkInfo)**def toFullName**[\[source\]](#)*The full name of a remark (name and namespace).***def** RemarkInfo.toFullName (r : RemarkInfo) : Name**def IsRemark**[\[source\]](#)*A Bool which is true if a name correponds to a remark.***def** RemarkInfo.IsRemark (n : Name) : MetaM Bool**def getRemarkInfo**[\[source\]](#)*Gets the remarkInfo from a name corresponding to a remark..***def** RemarkInfo.getRemarkInfo (n : Name) : MetaM RemarkInfo**2.10 HepLean.Meta.TODO.Basic**[HepLean/Meta/TODO/Basic.lean](#) on GitHub.**structure todoInfo**[\[source\]](#)*The information from a TODO ... command.***structure** todoInfo **where**

content : String

fileName : Name

line : Nat

def elabTODO[\[source\]](#)*Elaborator for the TODO ... command*

@[command_elab todo_comment]

def elabTODO : Lean.Elab.Command.CommandElab

2.11 HepLean.PerturbationTheory.Algebras.CrAnAlgebra.Basic

[HepLean/PerturbationTheory/Algebras/CrAnAlgebra/Basic.lean](#) on GitHub.

abbrev CrAnAlgebra

[\[source\]](#)

The creation and annihilation free-algebra. The free algebra generated by CrAnStates, that is a position based states or asymptotic states with a specification of whether the state is a creation or annihilation state. As a module CrAnAlgebra is spanned by lists of CrAnStates.

```
abbrev CrAnAlgebra (F : FieldSpecification) : Type
```

def ofCrAnState

[\[source\]](#)

Maps a creation and annihilation state to the creation and annihilation free-algebra.

```
def ofCrAnState (φ : F.CrAnStates) : CrAnAlgebra F
```

def ofCrAnList

[\[source\]](#)

Maps a list creation and annihilation state to the creation and annihilation free-algebra by taking their product.

```
def ofCrAnList (φs : List F.CrAnStates) : CrAnAlgebra F
```

lemma ofCrAnList_nil

[\[source\]](#)

`@[simp]`

```
lemma ofCrAnList_nil : ofCrAnList ([]) : List F.CrAnStates = 1
```

lemma ofCrAnList_cons

[\[source\]](#)

```
lemma ofCrAnList_cons (φ : F.CrAnStates) (φs : List F.CrAnStates) :  
  ofCrAnList (φ :: φs) = ofCrAnState φ * ofCrAnList φs
```

lemma ofCrAnList_append

[\[source\]](#)

```
lemma ofCrAnList_append (φs φs' : List F.CrAnStates) :  
  ofCrAnList (φs ++ φs') = ofCrAnList φs * ofCrAnList φs'
```

lemma ofCrAnList_singleton

[\[source\]](#)

```
lemma ofCrAnList_singleton (φ : F.CrAnStates) :  
  ofCrAnList [φ] = ofCrAnState φ
```

def ofState

[\[source\]](#)

Maps a state to the sum of creation and annihilation operators in creation and annihilation free-algebra.

```
def ofState (φ : F.States) : CrAnAlgebra F
```

def ofStateList[\[source\]](#)

*Maps a list of states to the creation and annihilation free-algebra by taking the product of their sums of creation and annihilation operators. Roughly $[\phi 1, \phi 2]$ gets sent to $(\phi 1^c + \phi 1^a) * (\phi 2^c + \phi 2^a)$ etc.*

```
def ofStateList (ϕs : List ℱ.States) : CrAnAlgebra ℱ
```

instance Coe (List ℱ.States) (CrAnAlgebra ℱ)[\[source\]](#)

Coercion from List ℱ.States to CrAnAlgebra ℱ through ofStateList.

```
instance : Coe (List ℱ.States) (CrAnAlgebra ℱ)
```

lemma ofStateList_nil[\[source\]](#)

```
@[simp]
```

```
lemma ofStateList_nil : ofStateList ([] : List ℱ.States) = 1
```

lemma ofStateList_cons[\[source\]](#)

```
lemma ofStateList_cons (ϕ : ℱ.States) (ϕs : List ℱ.States) :  
  ofStateList (ϕ :: ϕs) = ofState ϕ * ofStateList ϕs
```

lemma ofStateList_singleton[\[source\]](#)

```
lemma ofStateList_singleton (ϕ : ℱ.States) :  
  ofStateList [ϕ] = ofState ϕ
```

lemma ofStateList_append[\[source\]](#)

```
lemma ofStateList_append (ϕs ϕs' : List ℱ.States) :  
  ofStateList (ϕs ++ ϕs') = ofStateList ϕs * ofStateList ϕs'
```

lemma ofStateList_sum[\[source\]](#)

```
lemma ofStateList_sum (ϕs : List ℱ.States) :  
  ofStateList ϕs = ∑ (s : CrAnSection ϕs), ofCrAnList s.1
```

def crPartF[\[source\]](#)

The algebra map taking an element of the free-state algebra to the part of it in the creation and annihilation free algebra spanned by creation operators.

```
def crPartF : ℱ.States → ℱ.CrAnAlgebra
```

lemma crPartF_negAsymp[\[source\]](#)

```
@[simp]
```

```
lemma crPartF_negAsymp (ϕ : ℱ.IncomingAsymptotic) :  
  crPartF (States.inAsymp ϕ) = ofCrAnState (States.inAsymp ϕ, ())
```

lemma crPartF_position[\[source\]](#)

```
@[simp]
lemma crPartF_position ( $\varphi$  :  $\mathcal{F}$ .PositionStates) :
  crPartF (States.position  $\varphi$ ) =
    ofCrAnState (States.position  $\varphi$ , CreateAnnihilate.create)
```

lemma crPartF_posAsymp[\[source\]](#)

```
@[simp]
lemma crPartF_posAsymp ( $\varphi$  :  $\mathcal{F}$ .OutgoingAsymptotic) :
  crPartF (States.outAsymp  $\varphi$ ) = 0
```

def anPartF[\[source\]](#)

The algebra map taking an element of the free-state algebra to the part of it in the creation and annihilation free algebra spanned by annihilation operators.

```
def anPartF :  $\mathcal{F}$ .States  $\rightarrow$   $\mathcal{F}$ .CrAnAlgebra
```

lemma anPartF_negAsymp[\[source\]](#)

```
@[simp]
lemma anPartF_negAsymp ( $\varphi$  :  $\mathcal{F}$ .IncomingAsymptotic) :
  anPartF (States.inAsymp  $\varphi$ ) = 0
```

lemma anPartF_position[\[source\]](#)

```
@[simp]
lemma anPartF_position ( $\varphi$  :  $\mathcal{F}$ .PositionStates) :
  anPartF (States.position  $\varphi$ ) =
    ofCrAnState (States.position  $\varphi$ , CreateAnnihilate.annihilate)
```

lemma anPartF_posAsymp[\[source\]](#)

```
@[simp]
lemma anPartF_posAsymp ( $\varphi$  :  $\mathcal{F}$ .OutgoingAsymptotic) :
  anPartF (States.outAsymp  $\varphi$ ) = ofCrAnState (States.outAsymp  $\varphi$ , ())
```

lemma ofState_eq_crPartF_add_anPartF[\[source\]](#)

```
lemma ofState_eq_crPartF_add_anPartF ( $\varphi$  :  $\mathcal{F}$ .States) :
  ofState  $\varphi$  = crPartF  $\varphi$  + anPartF  $\varphi$ 
```

def ofCrAnListBasis[\[source\]](#)

The basis of the free creation and annihilation algebra formed by lists of CrAnStates.

```
noncomputable def ofCrAnListBasis : Basis (List  $\mathcal{F}$ .CrAnStates)  $\mathbb{C}$  (CrAnAlgebra  $\mathcal{F}$ ) where
```

lemma ofListBasis_eq_ofList[\[source\]](#)

```
@[simp]
lemma ofListBasis_eq_ofList (φs : List ℱ.CrAnStates) :
  ofCrAnListBasis φs = ofCrAnList φs
```

def mulLinearMap[\[source\]](#)

The bi-linear map associated with multiplication in CrAnAlgebra.

```
noncomputable def mulLinearMap : CrAnAlgebra ℱ →ℓ[ℂ] CrAnAlgebra ℱ →ℓ[ℂ] CrAnAlgebra ℱ where
```

lemma mulLinearMap_apply[\[source\]](#)

```
lemma mulLinearMap_apply (a b : CrAnAlgebra ℱ) :
  mulLinearMap a b = a * b
```

def smulLinearMap[\[source\]](#)

The linear map associated with scalar-multiplication in CrAnAlgebra.

```
noncomputable def smulLinearMap (c : ℂ) : CrAnAlgebra ℱ →ℓ[ℂ] CrAnAlgebra ℱ where
```

2.12 HepLean.PerturbationTheory.Algebras.CrAnAlgebra.Grading

[HepLean/PerturbationTheory/Algebras/CrAnAlgebra/Grading.lean](#) on GitHub.

def statisticSubmodule[\[source\]](#)

The submodule of CrAnAlgebra spanned by lists of field statistic f.

```
def statisticSubmodule (f : FieldStatistic) : Submodule ℂ ℱ.CrAnAlgebra
```

lemma ofCrAnList_mem_statisticSubmodule_of[\[source\]](#)

```
lemma ofCrAnList_mem_statisticSubmodule_of (φs : List ℱ.CrAnStates) (f : FieldStatistic)
  (h : (ℱ |>s φs) = f) :
  ofCrAnList φs ∈ statisticSubmodule f
```

lemma ofCrAnList_bosonic_or_fermionic[\[source\]](#)

```
lemma ofCrAnList_bosonic_or_fermionic (φs : List ℱ.CrAnStates) :
  ofCrAnList φs ∈ statisticSubmodule bosonic ∨ ofCrAnList φs ∈ statisticSubmodule fermionic
```

lemma ofCrAnState_bosonic_or_fermionic[\[source\]](#)

```
lemma ofCrAnState_bosonic_or_fermionic (φ : ℱ.CrAnStates) :
  ofCrAnState φ ∈ statisticSubmodule bosonic ∨ ofCrAnState φ ∈ statisticSubmodule fermionic
```

def bosonicProj[\[source\]](#)

The projection of an element of CrAnAlgebra onto it's bosonic part.

```
def bosonicProj :  $\mathcal{F}$ .CrAnAlgebra  $\rightarrow_l$  [C] statisticSubmodule ( $\mathcal{F} := \mathcal{F}$ ) bosonic
```

lemma bosonicProj_ofCrAnList

[\[source\]](#)

```
lemma bosonicProj_ofCrAnList ( $\phi s$  : List  $\mathcal{F}$ .CrAnStates) :
  bosonicProj (ofCrAnList  $\phi s$ ) = if  $h$  : ( $\mathcal{F} \mid>_s \phi s$ ) = bosonic then
    (ofCrAnList  $\phi s$ , Submodule.mem_span.mpr fun _  $a \Rightarrow a$  ( $\phi s$ , {rfl,  $h$ })) else 0
```

lemma bosonicProj_of_mem_bosonic

[\[source\]](#)

```
lemma bosonicProj_of_mem_bosonic ( $a$  :  $\mathcal{F}$ .CrAnAlgebra) ( $h$  :  $a \in$  statisticSubmodule bosonic) :
  bosonicProj  $a$  = ( $a$ ,  $h$ )
```

lemma bosonicProj_of_mem_fermionic

[\[source\]](#)

```
lemma bosonicProj_of_mem_fermionic ( $a$  :  $\mathcal{F}$ .CrAnAlgebra) ( $h$  :  $a \in$  statisticSubmodule fermionic) :
  bosonicProj  $a$  = 0
```

lemma bosonicProj_of_bonosic_part

[\[source\]](#)

```
@[simp]
lemma bosonicProj_of_bonosic_part
  ( $a$  : DirectSum FieldStatistic (fun  $i \Rightarrow$  (statisticSubmodule ( $\mathcal{F} := \mathcal{F}$ )  $i$ ))) :
  bosonicProj ( $a$  bosonic) =  $a$  bosonic
```

lemma bosonicProj_of_fermionic_part

[\[source\]](#)

```
@[simp]
lemma bosonicProj_of_fermionic_part
  ( $a$  : DirectSum FieldStatistic (fun  $i \Rightarrow$  (statisticSubmodule ( $\mathcal{F} := \mathcal{F}$ )  $i$ ))) :
  bosonicProj ( $a$  fermionic).1 = 0
```

def fermionicProj

[\[source\]](#)

The projection of an element of CrAnAlgebra onto it's fermionic part.

```
def fermionicProj :  $\mathcal{F}$ .CrAnAlgebra  $\rightarrow_l$  [C] statisticSubmodule ( $\mathcal{F} := \mathcal{F}$ ) fermionic
```

lemma fermionicProj_ofCrAnList

[\[source\]](#)

```
lemma fermionicProj_ofCrAnList ( $\phi s$  : List  $\mathcal{F}$ .CrAnStates) :
  fermionicProj (ofCrAnList  $\phi s$ ) = if  $h$  : ( $\mathcal{F} \mid>_s \phi s$ ) = fermionic then
    (ofCrAnList  $\phi s$ , Submodule.mem_span.mpr fun _  $a \Rightarrow a$  ( $\phi s$ , {rfl,  $h$ })) else 0
```

lemma fermionicProj_ofCrAnList_if_bosonic

[\[source\]](#)

```
lemma fermionicProj_ofCrAnList_if_bosonic ( $\phi s$  : List  $\mathcal{F}$ .CrAnStates) :
  fermionicProj (ofCrAnList  $\phi s$ ) = if  $h$  : ( $\mathcal{F} \mid>_s \phi s$ ) = bosonic then
    0 else (ofCrAnList  $\phi s$ , Submodule.mem_span.mpr fun _  $a \Rightarrow a$  ( $\phi s$ , {rfl,
      by simp using  $h$ }))
```

lemma fermionicProj_of_mem_fermionic[\[source\]](#)

```

lemma fermionicProj_of_mem_fermionic (a :  $\mathcal{F}$ .CrAnAlgebra) (h : a ∈ statisticSubmodule fermionic) :
  fermionicProj a = ⟨a, h⟩

```

lemma fermionicProj_of_mem_bosonic[\[source\]](#)

```

lemma fermionicProj_of_mem_bosonic (a :  $\mathcal{F}$ .CrAnAlgebra) (h : a ∈ statisticSubmodule bosonic) :
  fermionicProj a = 0

```

lemma fermionicProj_of_bosonic_part[\[source\]](#)

```

@[simp]
lemma fermionicProj_of_bosonic_part
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  fermionicProj (a bosonic).1 = 0

```

lemma fermionicProj_of_fermionic_part[\[source\]](#)

```

@[simp]
lemma fermionicProj_of_fermionic_part
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  fermionicProj (a fermionic) = a fermionic

```

lemma bosonicProj_add_fermionicProj[\[source\]](#)

```

lemma bosonicProj_add_fermionicProj (a :  $\mathcal{F}$ .CrAnAlgebra) :
  a.bosonicProj + (a.fermionicProj).1 = a

```

lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic[\[source\]](#)

```

lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  DirectSum.coeAddMonoidHom statisticSubmodule a = a.1 bosonic + a.1 fermionic

```

lemma directSum_eq_bosonic_plus_fermionic[\[source\]](#)

```

lemma directSum_eq_bosonic_plus_fermionic
  (a : DirectSum FieldStatistic (fun i => (statisticSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  a = (DirectSum.of (fun i => 1(statisticSubmodule i)) bosonic) (a bosonic) +
    (DirectSum.of (fun i => 1(statisticSubmodule i)) fermionic) (a fermionic)

```

instance crAnAlgebraGrade[\[source\]](#)

The instance of a graded algebra on CrAnAlgebra.

```

instance crAnAlgebraGrade : GradedAlgebra (A :=  $\mathcal{F}$ .CrAnAlgebra) statisticSubmodule where

```

lemma eq_zero_of_bosonic_and_fermionic[\[source\]](#)

```

lemma eq_zero_of_bosonic_and_fermionic {a :  $\mathcal{F}$ .CrAnAlgebra}
  (hb : a ∈ statisticSubmodule bosonic) (hf : a ∈ statisticSubmodule fermionic) : a = 0

```

lemma bosonicProj_mul[\[source\]](#)

```

lemma bosonicProj_mul (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  (a * b).bosonicProj.1 = a.bosonicProj.1 * b.bosonicProj.1
  + a.fermionicProj.1 * b.fermionicProj.1

```

lemma fermionicProj_mul[\[source\]](#)

```

lemma fermionicProj_mul (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  (a * b).fermionicProj.1 = a.bosonicProj.1 * b.fermionicProj.1
  + a.fermionicProj.1 * b.bosonicProj.1

```

2.13 HepLean.PerturbationTheory.Algebras.CrAnAlgebra.NormalOrder

[HepLean/PerturbationTheory/Algebras/CrAnAlgebra/NormalOrder.lean](#) on GitHub.

def normalOrderF[\[source\]](#)

The linear map on the free creation and annihilation algebra defined as the map taking a list of CrAnStates to the normal-ordered list of states multiplied by the sign corresponding to the number of fermionic-fermionic exchanges done in ordering.

```

def normalOrderF : CrAnAlgebra  $\mathcal{F}$   $\rightarrow$   $![\mathbb{C}]$  CrAnAlgebra  $\mathcal{F}$ 

```

lemma normalOrderF_ofCrAnList[\[source\]](#)

```

lemma normalOrderF_ofCrAnList ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) :
   $\mathcal{N}^f$ (ofCrAnList  $\phi$ s) = normalOrderSign  $\phi$ s • ofCrAnList (normalOrderList  $\phi$ s)

```

lemma ofCrAnList_eq_normalOrderF[\[source\]](#)

```

lemma ofCrAnList_eq_normalOrderF ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) :
  ofCrAnList (normalOrderList  $\phi$ s) = normalOrderSign  $\phi$ s •  $\mathcal{N}^f$ (ofCrAnList  $\phi$ s)

```

lemma normalOrderF_one[\[source\]](#)

```

lemma normalOrderF_one : normalOrderF ( $\mathcal{F}$  :=  $\mathcal{F}$ ) 1 = 1

```

lemma normalOrderF_ofCrAnList_cons_create[\[source\]](#)

```

lemma normalOrderF_ofCrAnList_cons_create ( $\phi$  :  $\mathcal{F}$ .CrAnStates)
  (h $\phi$  :  $\mathcal{F}$   $|>^c$   $\phi$  = CreateAnnihilate.create) ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) :
   $\mathcal{N}^f$ (ofCrAnList ( $\phi$  ::  $\phi$ s)) = ofCrAnState  $\phi$  *  $\mathcal{N}^f$ (ofCrAnList  $\phi$ s)

```

lemma normalOrderF_create_mul[\[source\]](#)

```

lemma normalOrderF_create_mul ( $\phi$  :  $\mathcal{F}$ .CrAnStates)
  (h $\phi$  :  $\mathcal{F}$   $|>^c$   $\phi$  = CreateAnnihilate.create) (a : CrAnAlgebra  $\mathcal{F}$ ) :
   $\mathcal{N}^f$ (ofCrAnState  $\phi$  * a) = ofCrAnState  $\phi$  *  $\mathcal{N}^f$ (a)

```

lemma normalOrderF_ofCrAnList_append_annihilate[\[source\]](#)

```

lemma normalOrderF_ofCrAnList_append_annihilate (φ :  $\mathcal{F}$ .CrAnStates)
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.annihilate}$ ) (ψs : List  $\mathcal{F}$ .CrAnStates) :
   $\mathcal{N}^f(\text{ofCrAnList } (\psi s ++ [\varphi])) = \mathcal{N}^f(\text{ofCrAnList } \psi s) * \text{ofCrAnState } \varphi$ 

```

lemma normalOrderF_mul_annihilate[\[source\]](#)

```

lemma normalOrderF_mul_annihilate (φ :  $\mathcal{F}$ .CrAnStates)
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.annihilate}$ )
  (a : CrAnAlgebra  $\mathcal{F}$ ) :  $\mathcal{N}^f(a * \text{ofCrAnState } \varphi) = \mathcal{N}^f(a) * \text{ofCrAnState } \varphi$ 

```

lemma normalOrderF_crPartF_mul[\[source\]](#)

```

lemma normalOrderF_crPartF_mul (φ :  $\mathcal{F}$ .States) (a : CrAnAlgebra  $\mathcal{F}$ ) :
   $\mathcal{N}^f(\text{crPartF } \varphi * a) =$ 
   $\text{crPartF } \varphi * \mathcal{N}^f(a)$ 

```

lemma normalOrderF_mul_anPartF[\[source\]](#)

```

lemma normalOrderF_mul_anPartF (φ :  $\mathcal{F}$ .States) (a : CrAnAlgebra  $\mathcal{F}$ ) :
   $\mathcal{N}^f(a * \text{anPartF } \varphi) =$ 
   $\mathcal{N}^f(a) * \text{anPartF } \varphi$ 

```

lemma normalOrderF_swap_create_annihlate_ofCrAnList_ofCrAnList[\[source\]](#)

```

lemma normalOrderF_swap_create_annihlate_ofCrAnList_ofCrAnList (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid >^c \varphi c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (ψs ψs' : List  $\mathcal{F}$ .CrAnStates) :
   $\mathcal{N}^f(\text{ofCrAnList } \psi s' * \text{ofCrAnState } \varphi c * \text{ofCrAnState } \varphi a * \text{ofCrAnList } \psi s) = \mathcal{S}(\mathcal{F} \mid >_s \varphi c, \mathcal{F} \mid >_s \varphi a) \cdot$ 
   $\mathcal{N}^f(\text{ofCrAnList } \psi s' * \text{ofCrAnState } \varphi a * \text{ofCrAnState } \varphi c * \text{ofCrAnList } \psi s)$ 

```

lemma normalOrderF_swap_create_annihlate_ofCrAnList[\[source\]](#)

```

lemma normalOrderF_swap_create_annihlate_ofCrAnList (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid >^c \varphi c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (ψs : List  $\mathcal{F}$ .CrAnStates) (a :  $\mathcal{F}$ .CrAnAlgebra) :
   $\mathcal{N}^f(\text{ofCrAnList } \psi s * \text{ofCrAnState } \varphi c * \text{ofCrAnState } \varphi a * a) = \mathcal{S}(\mathcal{F} \mid >_s \varphi c, \mathcal{F} \mid >_s \varphi a) \cdot$ 
   $\mathcal{N}^f(\text{ofCrAnList } \psi s * \text{ofCrAnState } \varphi a * \text{ofCrAnState } \varphi c * a)$ 

```

lemma normalOrderF_swap_create_annihlate[\[source\]](#)

```

lemma normalOrderF_swap_create_annihlate (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid >^c \varphi c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (a b :  $\mathcal{F}$ .CrAnAlgebra) :
   $\mathcal{N}^f(a * \text{ofCrAnState } \varphi c * \text{ofCrAnState } \varphi a * b) = \mathcal{S}(\mathcal{F} \mid >_s \varphi c, \mathcal{F} \mid >_s \varphi a) \cdot$ 
   $\mathcal{N}^f(a * \text{ofCrAnState } \varphi a * \text{ofCrAnState } \varphi c * b)$ 

```

lemma normalOrderF_superCommuteF_create_annihilate[\[source\]](#)

```

lemma normalOrderF_superCommuteF_create_annihilate (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid >^c \varphi c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )

```



```
(a b :  $\mathcal{F}$ .CrAnAlgebra) :
 $\mathcal{N}^f(a * [\text{ofCrAnState } \varphi_c, \text{ofCrAnState } \varphi_a]_{sc} * b) = 0$ 
```

lemma normalOrderF_superCommuteF_annihilate_create

[\[source\]](#)

```
lemma normalOrderF_superCommuteF_annihilate_create ( $\varphi_c \varphi_a : \mathcal{F}$ .CrAnStates)
(h $\varphi_c : \mathcal{F} \mid >^c \varphi_c = \text{CreateAnnihilate.create}$ ) (h $\varphi_a : \mathcal{F} \mid >^c \varphi_a = \text{CreateAnnihilate.annihilate}$ )
(a b :  $\mathcal{F}$ .CrAnAlgebra) :
 $\mathcal{N}^f(a * [\text{ofCrAnState } \varphi_a, \text{ofCrAnState } \varphi_c]_{sc} * b) = 0$ 
```

lemma normalOrderF_swap_crPartF_anPartF

[\[source\]](#)

```
lemma normalOrderF_swap_crPartF_anPartF ( $\varphi \varphi' : \mathcal{F}$ .States) (a b : CrAnAlgebra  $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * (\text{crPartF } \varphi) * (\text{anPartF } \varphi') * b) =$ 
 $S(\mathcal{F} \mid >_s \varphi, \mathcal{F} \mid >_s \varphi') \cdot$ 
 $\mathcal{N}^f(a * (\text{anPartF } \varphi') * (\text{crPartF } \varphi) * b)$ 
```

lemma normalOrderF_swap_anPartF_crPartF

[\[source\]](#)

```
lemma normalOrderF_swap_anPartF_crPartF ( $\varphi \varphi' : \mathcal{F}$ .States) (a b : CrAnAlgebra  $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * (\text{anPartF } \varphi) * (\text{crPartF } \varphi') * b) =$ 
 $S(\mathcal{F} \mid >_s \varphi, \mathcal{F} \mid >_s \varphi') \cdot \mathcal{N}^f(a * (\text{crPartF } \varphi') * (\text{anPartF } \varphi) * b)$ 
```

lemma normalOrderF_superCommuteF_crPartF_anPartF

[\[source\]](#)

```
lemma normalOrderF_superCommuteF_crPartF_anPartF ( $\varphi \varphi' : \mathcal{F}$ .States) (a b : CrAnAlgebra  $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * \text{superCommuteF}$ 
 $(\text{crPartF } \varphi) (\text{anPartF } \varphi') * b) = 0$ 
```

lemma normalOrderF_superCommuteF_anPartF_crPartF

[\[source\]](#)

```
lemma normalOrderF_superCommuteF_anPartF_crPartF ( $\varphi \varphi' : \mathcal{F}$ .States) (a b : CrAnAlgebra  $\mathcal{F}$ ) :
 $\mathcal{N}^f(a * \text{superCommuteF}$ 
 $(\text{anPartF } \varphi) (\text{crPartF } \varphi') * b) = 0$ 
```

lemma normalOrderF_crPartF_mul_crPartF

[\[source\]](#)

```
@[simp]
lemma normalOrderF_crPartF_mul_crPartF ( $\varphi \varphi' : \mathcal{F}$ .States) :
 $\mathcal{N}^f(\text{crPartF } \varphi * \text{crPartF } \varphi') =$ 
 $\text{crPartF } \varphi * \text{crPartF } \varphi'$ 
```

lemma normalOrderF_anPartF_mul_anPartF

[\[source\]](#)

```
@[simp]
lemma normalOrderF_anPartF_mul_anPartF ( $\varphi \varphi' : \mathcal{F}$ .States) :
 $\mathcal{N}^f(\text{anPartF } \varphi * \text{anPartF } \varphi') =$ 
 $\text{anPartF } \varphi * \text{anPartF } \varphi'$ 
```

lemma normalOrderF_crPartF_mul_anPartF[\[source\]](#)

```
@[simp]
lemma normalOrderF_crPartF_mul_anPartF ( $\phi \phi' : \mathcal{F}.\text{States}$ ) :
   $\mathcal{N}^f(\text{crPartF } \phi * \text{anPartF } \phi') =$ 
   $\text{crPartF } \phi * \text{anPartF } \phi'$ 
```

lemma normalOrderF_anPartF_mul_crPartF[\[source\]](#)

```
@[simp]
lemma normalOrderF_anPartF_mul_crPartF ( $\phi \phi' : \mathcal{F}.\text{States}$ ) :
   $\mathcal{N}^f(\text{anPartF } \phi * \text{crPartF } \phi') =$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi') \cdot$ 
   $(\text{crPartF } \phi' * \text{anPartF } \phi)$ 
```

lemma normalOrderF_ofState_mul_ofState[\[source\]](#)

```
lemma normalOrderF_ofState_mul_ofState ( $\phi \phi' : \mathcal{F}.\text{States}$ ) :
   $\mathcal{N}^f(\text{ofState } \phi * \text{ofState } \phi') =$ 
   $\text{crPartF } \phi * \text{crPartF } \phi' +$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi') \cdot$ 
   $(\text{crPartF } \phi' * \text{anPartF } \phi) +$ 
   $\text{crPartF } \phi * \text{anPartF } \phi' +$ 
   $\text{anPartF } \phi * \text{anPartF } \phi'$ 
```

lemma normalOrderF_superCommuteF_ofCrAnList_create_create_ofCrAnList[\[source\]](#)

```
lemma normalOrderF_superCommuteF_ofCrAnList_create_create_ofCrAnList
  ( $\phi c \phi c' : \mathcal{F}.\text{CrAnStates}$ ) ( $h\phi c : \mathcal{F} \mid_{>c} \phi c = \text{CreateAnnihilate.create}$ )
  ( $h\phi c' : \mathcal{F} \mid_{>c} \phi c' = \text{CreateAnnihilate.create}$ ) ( $\phi s \phi s' : \text{List } \mathcal{F}.\text{CrAnStates}$ ) :
   $(\mathcal{N}^f(\text{ofCrAnList } \phi s * [\text{ofCrAnState } \phi c, \text{ofCrAnState } \phi c']_{sca} * \text{ofCrAnList } \phi s')) =$ 
   $\text{normalOrderSign } (\phi s ++ \phi c' :: \phi c :: \phi s') \cdot$ 
   $(\text{ofCrAnList } (\text{createFilter } \phi s) * [\text{ofCrAnState } \phi c, \text{ofCrAnState } \phi c']_{sca} * \text{ofCrAnList } (\text{createFilter } \phi s') * \text{ofCrAnList } (\text{annihilateFilter } (\phi s ++ \phi s')))$ 
```

lemma normalOrderF_superCommuteF_ofCrAnList_annihilate_annihilate_ofCrAnList[\[source\]](#)

```
lemma normalOrderF_superCommuteF_ofCrAnList_annihilate_annihilate_ofCrAnList
  ( $\phi a \phi a' : \mathcal{F}.\text{CrAnStates}$ )
  ( $h\phi a : \mathcal{F} \mid_{>c} \phi a = \text{CreateAnnihilate.annihilate}$ )
  ( $h\phi a' : \mathcal{F} \mid_{>c} \phi a' = \text{CreateAnnihilate.annihilate}$ )
  ( $\phi s \phi s' : \text{List } \mathcal{F}.\text{CrAnStates}$ ) :
   $\mathcal{N}^f(\text{ofCrAnList } \phi s * [\text{ofCrAnState } \phi a, \text{ofCrAnState } \phi a']_{sca} * \text{ofCrAnList } \phi s') =$ 
   $\text{normalOrderSign } (\phi s ++ \phi a' :: \phi a :: \phi s') \cdot$ 
   $(\text{ofCrAnList } (\text{createFilter } (\phi s ++ \phi s'))$ 
   $* \text{ofCrAnList } (\text{annihilateFilter } \phi s) * [\text{ofCrAnState } \phi a, \text{ofCrAnState } \phi a']_{sca}$ 
   $* \text{ofCrAnList } (\text{annihilateFilter } \phi s'))$ 
```

lemma ofCrAnList_superCommuteF_normalOrderF_ofCrAnList[\[source\]](#)

```
lemma ofCrAnList_superCommuteF_normalOrderF_ofCrAnList ( $\phi s \phi s' : \text{List } \mathcal{F}.\text{CrAnStates}$ ) :
   $[\text{ofCrAnList } \phi s, \mathcal{N}^f(\text{ofCrAnList } \phi s')]_{sca} =$ 
   $\text{ofCrAnList } \phi s * \mathcal{N}^f(\text{ofCrAnList } \phi s') -$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi s, \mathcal{F} \mid_{>s} \phi s') \cdot \mathcal{N}^f(\text{ofCrAnList } \phi s') * \text{ofCrAnList } \phi s$ 
```

lemma ofCrAnList_superCommuteF_normalOrderF_ofStateList[\[source\]](#)

```

lemma ofCrAnList_superCommuteF_normalOrderF_ofStateList (φs : List  $\mathcal{F}$ .CrAnStates)
  (φs' : List  $\mathcal{F}$ .States) : [ofCrAnList φs,  $\mathcal{N}^f$ (ofStateList φs')]_{sc} =
  ofCrAnList φs *  $\mathcal{N}^f$ (ofStateList φs') -
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi s') \cdot \mathcal{N}^f$ (ofStateList φs') * ofCrAnList φs

```

lemma ofCrAnList_mul_normalOrderF_ofStateList_eq_superCommuteF[\[source\]](#)

```

lemma ofCrAnList_mul_normalOrderF_ofStateList_eq_superCommuteF (φs : List  $\mathcal{F}$ .CrAnStates)
  (φs' : List  $\mathcal{F}$ .States) :
  ofCrAnList φs *  $\mathcal{N}^f$ (ofStateList φs') =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi s') \cdot \mathcal{N}^f$ (ofStateList φs') * ofCrAnList φs
  + [ofCrAnList φs,  $\mathcal{N}^f$ (ofStateList φs')]_{sc}

```

lemma ofCrAnState_mul_normalOrderF_ofStateList_eq_superCommuteF[\[source\]](#)

```

lemma ofCrAnState_mul_normalOrderF_ofStateList_eq_superCommuteF (φ :  $\mathcal{F}$ .CrAnStates)
  (φs' : List  $\mathcal{F}$ .States) : ofCrAnState φ *  $\mathcal{N}^f$ (ofStateList φs') =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi s') \cdot \mathcal{N}^f$ (ofStateList φs') * ofCrAnState φ
  + [ofCrAnState φ,  $\mathcal{N}^f$ (ofStateList φs')]_{sc}

```

lemma anPartF_mul_normalOrderF_ofStateList_eq_superCommuteF[\[source\]](#)

```

lemma anPartF_mul_normalOrderF_ofStateList_eq_superCommuteF (φ :  $\mathcal{F}$ .States)
  (φs' : List  $\mathcal{F}$ .States) :
  anPartF φ *  $\mathcal{N}^f$ (ofStateList φs') =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi s') \cdot \mathcal{N}^f$ (ofStateList φs' * anPartF φ)
  + [anPartF φ,  $\mathcal{N}^f$ (ofStateList φs')]_{sc}

```

2.14 HepLean.PerturbationTheory.Algebras.CrAnAlgebra.NormTimeOrder

[HepLean/PerturbationTheory/Algebras/CrAnAlgebra/NormTimeOrder.lean](#) on GitHub.

def normTimeOrder[\[source\]](#)

The normal-time ordering on CrAnAlgebra.

```

def normTimeOrder : CrAnAlgebra  $\mathcal{F} \rightarrow_l[\mathbb{C}]$  CrAnAlgebra  $\mathcal{F}$ 

```

lemma normTimeOrder_ofCrAnList[\[source\]](#)

```

lemma normTimeOrder_ofCrAnList (φs : List  $\mathcal{F}$ .CrAnStates) :
   $\mathcal{TN}^f$ (ofCrAnList φs) = normTimeOrderSign φs * ofCrAnList (normTimeOrderList φs)

```

2.15 HepLean.PerturbationTheory.Algebras.CrAnAlgebra.SuperCommute

[HepLean/PerturbationTheory/Algebras/CrAnAlgebra/SuperCommute.lean](#) on GitHub.

def superCommuteF[\[source\]](#)

The super commutator on the creation and annihilation algebra. For two bosonic operators or a bosonic and fermionic operator this corresponds to the usual commutator whilst for two fermionic operators this corresponds to the anti-commutator.

```
noncomputable def superCommutateF :  $\mathcal{F}.$ CrAnAlgebra  $\rightarrow_l[\mathbb{C}]$   $\mathcal{F}.$ CrAnAlgebra  $\rightarrow_l[\mathbb{C}]$   $\mathcal{F}.$ CrAnAlgebra
```

lemma superCommutateF_ofCrAnList_ofCrAnList

[\[source\]](#)

```
lemma superCommutateF_ofCrAnList_ofCrAnList ( $\phi$ s  $\phi$ s' : List  $\mathcal{F}.$ CrAnStates) :  
  [ofCrAnList  $\phi$ s, ofCrAnList  $\phi$ s']sca =  
  ofCrAnList ( $\phi$ s ++  $\phi$ s') -  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi$ s,  $\mathcal{F} \mid_{>s} \phi$ s') • ofCrAnList ( $\phi$ s' ++  $\phi$ s)
```

lemma superCommutateF_ofCrAnState_ofCrAnState

[\[source\]](#)

```
lemma superCommutateF_ofCrAnState_ofCrAnState ( $\phi$   $\phi$ ' :  $\mathcal{F}.$ CrAnStates) :  
  [ofCrAnState  $\phi$ , ofCrAnState  $\phi$ ']sca =  
  ofCrAnState  $\phi$  * ofCrAnState  $\phi$ ' -  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • ofCrAnState  $\phi$ ' * ofCrAnState  $\phi$ 
```

lemma superCommutateF_ofCrAnList_ofStatesList

[\[source\]](#)

```
lemma superCommutateF_ofCrAnList_ofStatesList ( $\phi$ cas : List  $\mathcal{F}.$ CrAnStates) ( $\phi$ s : List  $\mathcal{F}.$ States) :  
  [ofCrAnList  $\phi$ cas, ofStateList  $\phi$ s]sca = ofCrAnList  $\phi$ cas * ofStateList  $\phi$ s -  
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi$ cas,  $\mathcal{F} \mid_{>s} \phi$ s) • ofStateList  $\phi$ s * ofCrAnList  $\phi$ cas
```

lemma superCommutateF_ofStateList_ofStatesList

[\[source\]](#)

```
lemma superCommutateF_ofStateList_ofStatesList ( $\phi$  : List  $\mathcal{F}.$ States) ( $\phi$ s : List  $\mathcal{F}.$ States) :  
  [ofStateList  $\phi$ , ofStateList  $\phi$ s]sca = ofStateList  $\phi$  * ofStateList  $\phi$ s -  
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi$ s) • ofStateList  $\phi$ s * ofStateList  $\phi$ 
```

lemma superCommutateF_ofState_ofStatesList

[\[source\]](#)

```
lemma superCommutateF_ofState_ofStatesList ( $\phi$  :  $\mathcal{F}.$ States) ( $\phi$ s : List  $\mathcal{F}.$ States) :  
  [ofState  $\phi$ , ofStateList  $\phi$ s]sca = ofState  $\phi$  * ofStateList  $\phi$ s -  
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi$ s) • ofStateList  $\phi$ s * ofState  $\phi$ 
```

lemma superCommutateF_ofStateList_ofState

[\[source\]](#)

```
lemma superCommutateF_ofStateList_ofState ( $\phi$ s : List  $\mathcal{F}.$ States) ( $\phi$  :  $\mathcal{F}.$ States) :  
  [ofStateList  $\phi$ s, ofState  $\phi$ ]sca = ofStateList  $\phi$ s * ofState  $\phi$  -  
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi$ s,  $\mathcal{F} \mid_{>s} \phi$ ) • ofState  $\phi$  * ofStateList  $\phi$ s
```

lemma superCommutateF_anPartF_crPartF

[\[source\]](#)

```
lemma superCommutateF_anPartF_crPartF ( $\phi$   $\phi$ ' :  $\mathcal{F}.$ States) :  
  [anPartF  $\phi$ , crPartF  $\phi$ ']sca = anPartF  $\phi$  * crPartF  $\phi$ ' -  
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • crPartF  $\phi$ ' * anPartF  $\phi$ 
```

lemma superCommutateF_crPartF_anPartF

[\[source\]](#)

```
lemma superCommutateF_crPartF_anPartF ( $\phi$   $\phi$ ' :  $\mathcal{F}.$ States) :  
  [crPartF  $\phi$ , anPartF  $\phi$ ']sca = crPartF  $\phi$  * anPartF  $\phi$ ' -  
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • anPartF  $\phi$ ' * crPartF  $\phi$ 
```

lemma superCommuteF_crPartF_crPartF[\[source\]](#)

```
lemma superCommuteF_crPartF_crPartF (φ φ' :  $\mathcal{F}$ .States) :
  [crPartF φ, crPartF φ']sca = crPartF φ * crPartF φ' -
   $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot \text{crPartF } \varphi' * \text{crPartF } \varphi$ 
```

lemma superCommuteF_anPartF_anPartF[\[source\]](#)

```
lemma superCommuteF_anPartF_anPartF (φ φ' :  $\mathcal{F}$ .States) :
  [anPartF φ, anPartF φ']sca =
  anPartF φ * anPartF φ' -  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot \text{anPartF } \varphi' * \text{anPartF } \varphi$ 
```

lemma superCommuteF_crPartF_ofStateList[\[source\]](#)

```
lemma superCommuteF_crPartF_ofStateList (φ :  $\mathcal{F}$ .States) (ψs : List  $\mathcal{F}$ .States) :
  [crPartF φ, ofStateList ψs]sca =
  crPartF φ * ofStateList ψs -  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi_s) \cdot \text{ofStateList } \psi_s * \text{crPartF } \varphi$ 
```

lemma superCommuteF_anPartF_ofStateList[\[source\]](#)

```
lemma superCommuteF_anPartF_ofStateList (φ :  $\mathcal{F}$ .States) (ψs : List  $\mathcal{F}$ .States) :
  [anPartF φ, ofStateList ψs]sca =
  anPartF φ * ofStateList ψs -  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi_s) \cdot \text{ofStateList } \psi_s * \text{anPartF } \varphi$ 
```

lemma superCommuteF_crPartF_ofState[\[source\]](#)

```
lemma superCommuteF_crPartF_ofState (φ φ' :  $\mathcal{F}$ .States) :
  [crPartF φ, ofState φ']sca =
  crPartF φ * ofState φ' -
   $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot \text{ofState } \varphi' * \text{crPartF } \varphi$ 
```

lemma superCommuteF_anPartF_ofState[\[source\]](#)

```
lemma superCommuteF_anPartF_ofState (φ φ' :  $\mathcal{F}$ .States) :
  [anPartF φ, ofState φ']sca =
  anPartF φ * ofState φ' -
   $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot \text{ofState } \varphi' * \text{anPartF } \varphi$ 
```

lemma ofCrAnList_mul_ofCrAnList_eq_superCommuteF[\[source\]](#)

```
lemma ofCrAnList_mul_ofCrAnList_eq_superCommuteF (ψs ψs' : List  $\mathcal{F}$ .CrAnStates) :
  ofCrAnList ψs * ofCrAnList ψs' =  $\mathcal{S}(\mathcal{F} \mid>_s \psi_s, \mathcal{F} \mid>_s \psi_s') \cdot \text{ofCrAnList } \psi_s' * \text{ofCrAnList } \psi_s$ 
  + [ofCrAnList ψs, ofCrAnList ψs']sca
```

lemma ofCrAnState_mul_ofCrAnList_eq_superCommuteF[\[source\]](#)

```
lemma ofCrAnState_mul_ofCrAnList_eq_superCommuteF (φ :  $\mathcal{F}$ .CrAnStates) (ψs' : List  $\mathcal{F}$ .CrAnStates) :
  :
  ofCrAnState φ * ofCrAnList ψs' =  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi_s') \cdot \text{ofCrAnList } \psi_s' * \text{ofCrAnState } \varphi$ 
  + [ofCrAnState φ, ofCrAnList ψs']sca
```

lemma ofStateList_mul_ofStateList_eq_superCommuterF[\[source\]](#)

```

lemma ofStateList_mul_ofStateList_eq_superCommuterF (φs φs' : List  $\mathcal{F}$ .States) :
  ofStateList φs * ofStateList φs' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_s')$  • ofStateList φs' * ofStateList
  φs
  + [ofStateList φs, ofStateList φs']sca

```

lemma ofState_mul_ofStateList_eq_superCommuterF[\[source\]](#)

```

lemma ofState_mul_ofStateList_eq_superCommuterF (φ :  $\mathcal{F}$ .States) (φs' : List  $\mathcal{F}$ .States) :
  ofState φ * ofStateList φs' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi_s')$  • ofStateList φs' * ofState φ
  + [ofState φ, ofStateList φs']sca

```

lemma ofStateList_mul_ofState_eq_superCommuterF[\[source\]](#)

```

lemma ofStateList_mul_ofState_eq_superCommuterF (φs : List  $\mathcal{F}$ .States) (φ :  $\mathcal{F}$ .States) :
  ofStateList φs * ofState φ =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi)$  • ofState φ * ofStateList φs
  + [ofStateList φs, ofState φ]sca

```

lemma crPartF_mul_anPartF_eq_superCommuterF[\[source\]](#)

```

lemma crPartF_mul_anPartF_eq_superCommuterF (φ φ' :  $\mathcal{F}$ .States) :
  crPartF φ * anPartF φ' =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • anPartF φ' * crPartF φ +
  [crPartF φ, anPartF φ']sca

```

lemma anPartF_mul_crPartF_eq_superCommuterF[\[source\]](#)

```

lemma anPartF_mul_crPartF_eq_superCommuterF (φ φ' :  $\mathcal{F}$ .States) :
  anPartF φ * crPartF φ' =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  •
  crPartF φ' * anPartF φ +
  [anPartF φ, crPartF φ']sca

```

lemma crPartF_mul_crPartF_eq_superCommuterF[\[source\]](#)

```

lemma crPartF_mul_crPartF_eq_superCommuterF (φ φ' :  $\mathcal{F}$ .States) :
  crPartF φ * crPartF φ' =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • crPartF φ' * crPartF φ +
  [crPartF φ, crPartF φ']sca

```

lemma anPartF_mul_anPartF_eq_superCommuterF[\[source\]](#)

```

lemma anPartF_mul_anPartF_eq_superCommuterF (φ φ' :  $\mathcal{F}$ .States) :
  anPartF φ * anPartF φ' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • anPartF φ' * anPartF φ +
  [anPartF φ, anPartF φ']sca

```

lemma ofCrAnList_mul_ofStateList_eq_superCommuterF[\[source\]](#)

```

lemma ofCrAnList_mul_ofStateList_eq_superCommuterF (φs : List  $\mathcal{F}$ .CrAnStates) (φs' : List  $\mathcal{F}$ .
  States) :
  ofCrAnList φs * ofStateList φs' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_s')$  • ofStateList φs' * ofCrAnList
  φs

```

```
+ [ofCrAnList φs, ofStateList φs']sca
```

lemma superCommuteF_ofCrAnList_ofCrAnList_symm

[\[source\]](#)

```
lemma superCommuteF_ofCrAnList_ofCrAnList_symm (φs φs' : List ℱ.CrAnStates) :
  [ofCrAnList φs, ofCrAnList φs']sca =
  (- S(ℱ |>s φs, ℱ |>s φs')) • [ofCrAnList φs', ofCrAnList φs]sca
```

lemma superCommuteF_ofCrAnState_ofCrAnState_symm

[\[source\]](#)

```
lemma superCommuteF_ofCrAnState_ofCrAnState_symm (φ φ' : ℱ.CrAnStates) :
  [ofCrAnState φ, ofCrAnState φ']sca =
  (- S(ℱ |>s φ, ℱ |>s φ')) • [ofCrAnState φ', ofCrAnState φ]sca
```

lemma superCommuteF_ofCrAnList_ofCrAnList_cons

[\[source\]](#)

```
lemma superCommuteF_ofCrAnList_ofCrAnList_cons (φ : ℱ.CrAnStates) (φs φs' : List ℱ.CrAnStates)
  :
  [ofCrAnList φs, ofCrAnList (φ :: φs')]sca =
  [ofCrAnList φs, ofCrAnState φ]sca * ofCrAnList φs' +
  S(ℱ |>s φs, ℱ |>s φ)
  • ofCrAnState φ * [ofCrAnList φs, ofCrAnList φs']sca
```

lemma superCommuteF_ofCrAnList_ofStateList_cons

[\[source\]](#)

```
lemma superCommuteF_ofCrAnList_ofStateList_cons (φ : ℱ.States) (φs : List ℱ.CrAnStates)
  (φs' : List ℱ.States) : [ofCrAnList φs, ofStateList (φ :: φs')]sca =
  [ofCrAnList φs, ofState φ]sca * ofStateList φs' +
  S(ℱ |>s φs, ℱ |>s φ) • ofState φ * [ofCrAnList φs, ofStateList φs']sca
```

lemma superCommuteF_ofCrAnList_ofCrAnList_eq_sum

[\[source\]](#)

Within the creation and annihilation algebra, we have that $[\varphi^{ca}s, \varphi^{ca}_0 \dots \varphi^{ca}_n]_s ca = \sum i, s_i \cdot \varphi^{ca}s_0 \dots \varphi^{ca}_{i-1} * [\varphi^{ca}s, \varphi^{ca}_i]_s ca * \varphi^{ca}_{i+1} \dots \varphi^{ca}_n$ where s_i is the exchange sign for $\varphi^{ca}s$ and $\varphi^{ca}s_0 \dots \varphi^{ca}_{i-1}$.

```
lemma superCommuteF_ofCrAnList_ofCrAnList_eq_sum (φs : List ℱ.CrAnStates) :
  (φs' : List ℱ.CrAnStates) → [ofCrAnList φs, ofCrAnList φs']sca =
  ∑ (n : Fin φs'.length), S(ℱ |>s φs, ℱ |>s φs'.take n) •
  ofCrAnList (φs'.take n) * [ofCrAnList φs, ofCrAnState (φs'.get n)]sca *
  ofCrAnList (φs'.drop (n + 1))
| [] => by
  simp [← ofCrAnList_nil, superCommuteF_ofCrAnList_ofCrAnList]
| φ :: φs' => by
  rw [superCommuteF_ofCrAnList_ofCrAnList_cons, superCommuteF_ofCrAnList_ofCrAnList_eq_sum φs
    φs']
  conv_rhs => erw [Fin.sum_univ_succ]
  congr 1
  • simp
  • simp [Finset.mul_sum, smul_smul, ofCrAnList_cons, mul_assoc,
    FieldStatistic.ofList_cons_eq_mul, mul_comm]
```

lemma superCommuteF_ofCrAnList_ofStateList_eq_sum[\[source\]](#)

```

lemma superCommuteF_ofCrAnList_ofStateList_eq_sum (φs : List  $\mathcal{F}$ .CrAnStates) : (φs' : List  $\mathcal{F}$ .
  States) →
  [ofCrAnList φs, ofStateList φs']sca =
  ∑ (n : Fin φs'.length),  $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s'.take\ n) \cdot$ 
    ofStateList (φs'.take n) * [ofCrAnList φs, ofState (φs'.get n)]sca *
    ofStateList (φs'.drop (n + 1))
| [] => by
  simp only [superCommuteF_ofCrAnList_ofStatesList, instCommGroup, ofList_empty,
    exchangeSign_bosonic, one_smul, List.length_nil, Finset.univ_eq_empty, List.take_nil,
    List.get_eq_getElem, List.drop_nil, Finset.sum_empty]
  simp
| φ :: φs' => by
  rw [superCommuteF_ofCrAnList_ofStateList_cons,
    superCommuteF_ofCrAnList_ofStateList_eq_sum φs φs']
  conv_rhs => erw [Fin.sum_univ_succ]
  congr 1
  · simp
  · simp [Finset.mul_sum, smul_smul, ofStateList_cons, mul_assoc,
    FieldStatistic.ofList_cons_eq_mul, mul_comm]

```

lemma summerCommute_jacobi_ofCrAnList[\[source\]](#)

```

lemma summerCommute_jacobi_ofCrAnList (φs1 φs2 φs3 : List  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnList φs1, [ofCrAnList φs2, ofCrAnList φs3]]sca =
   $\mathcal{S}(\mathcal{F} \mid>_s \phi s1, \mathcal{F} \mid>_s \phi s3) \cdot$ 
  (-  $\mathcal{S}(\mathcal{F} \mid>_s \phi s2, \mathcal{F} \mid>_s \phi s3) \cdot$  [ofCrAnList φs3, [ofCrAnList φs1, ofCrAnList φs2]]sca -
   $\mathcal{S}(\mathcal{F} \mid>_s \phi s1, \mathcal{F} \mid>_s \phi s2) \cdot$  [ofCrAnList φs2, [ofCrAnList φs3, ofCrAnList φs1]]sca)

```

lemma superCommuteF_grade[\[source\]](#)

```

lemma superCommuteF_grade {a b :  $\mathcal{F}$ .CrAnAlgebra} {f1 f2 : FieldStatistic}
  (ha : a ∈ statisticSubmodule f1) (hb : b ∈ statisticSubmodule f2) :
  [a, b]sca ∈ statisticSubmodule (f1 + f2)

```

lemma superCommuteF_bosonic_bosonic[\[source\]](#)

```

lemma superCommuteF_bosonic_bosonic {a b :  $\mathcal{F}$ .CrAnAlgebra}
  (ha : a ∈ statisticSubmodule bosonic) (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sca = a * b - b * a

```

lemma superCommuteF_bosonic_fermionic[\[source\]](#)

```

lemma superCommuteF_bosonic_fermionic {a b :  $\mathcal{F}$ .CrAnAlgebra}
  (ha : a ∈ statisticSubmodule bosonic) (hb : b ∈ statisticSubmodule fermionic) :
  [a, b]sca = a * b - b * a

```

lemma superCommuteF_fermionic_bonsonic[\[source\]](#)

```

lemma superCommuteF_fermionic_bonsonic {a b :  $\mathcal{F}$ .CrAnAlgebra}
  (ha : a ∈ statisticSubmodule fermionic) (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sca = a * b - b * a

```


lemma superCommuteF_bonsonic[\[source\]](#)

```
lemma superCommuteF_bonsonic {a b :  $\mathcal{F}$ .CrAnAlgebra} (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sca = a * b - b * a
```

lemma bosonic_superCommuteF[\[source\]](#)

```
lemma bosonic_superCommuteF {a b :  $\mathcal{F}$ .CrAnAlgebra} (ha : a ∈ statisticSubmodule bosonic) :
  [a, b]sca = a * b - b * a
```

lemma superCommuteF_bonsonic_symm[\[source\]](#)

```
lemma superCommuteF_bonsonic_symm {a b :  $\mathcal{F}$ .CrAnAlgebra} (hb : b ∈ statisticSubmodule bosonic) :
  [a, b]sca = - [b, a]sca
```

lemma bonsonic_superCommuteF_symm[\[source\]](#)

```
lemma bonsonic_superCommuteF_symm {a b :  $\mathcal{F}$ .CrAnAlgebra} (ha : a ∈ statisticSubmodule bosonic) :
  [a, b]sca = - [b, a]sca
```

lemma superCommuteF_fermionic_fermionic[\[source\]](#)

```
lemma superCommuteF_fermionic_fermionic {a b :  $\mathcal{F}$ .CrAnAlgebra}
  (ha : a ∈ statisticSubmodule fermionic) (hb : b ∈ statisticSubmodule fermionic) :
  [a, b]sca = a * b + b * a
```

lemma superCommuteF_fermionic_fermionic_symm[\[source\]](#)

```
lemma superCommuteF_fermionic_fermionic_symm {a b :  $\mathcal{F}$ .CrAnAlgebra}
  (ha : a ∈ statisticSubmodule fermionic) (hb : b ∈ statisticSubmodule fermionic) :
  [a, b]sca = [b, a]sca
```

lemma superCommuteF_expand_bosonicProj_fermionicProj[\[source\]](#)

```
lemma superCommuteF_expand_bosonicProj_fermionicProj (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  [a, b]sca = bosonicProj a * bosonicProj b - bosonicProj b * bosonicProj a +
  bosonicProj a * fermionicProj b - fermionicProj b * bosonicProj a +
  fermionicProj a * bosonicProj b - bosonicProj b * fermionicProj a +
  fermionicProj a * fermionicProj b + fermionicProj b * fermionicProj a
```

lemma superCommuteF_ofCrAnList_ofCrAnList_bosonic_or_fermionic[\[source\]](#)

```
lemma superCommuteF_ofCrAnList_ofCrAnList_bosonic_or_fermionic (φs φs' : List  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnList φs, ofCrAnList φs']sca ∈ statisticSubmodule bosonic v
  [ofCrAnList φs, ofCrAnList φs']sca ∈ statisticSubmodule fermionic
```

lemma superCommuteF_ofCrAnState_ofCrAnState_bosonic_or_fermionic[\[source\]](#)

```
lemma superCommuteF_ofCrAnState_ofCrAnState_bosonic_or_fermionic (φ φ' :  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnState φ, ofCrAnState φ']sca ∈ statisticSubmodule bosonic v
```

```
[ofCrAnState φ, ofCrAnState φ']sca ∈ statisticSubmodule fermionic
```

lemma superCommuteF_superCommuteF_ofCrAnState_bosonic_or_fermionic

[\[source\]](#)

```
lemma superCommuteF_superCommuteF_ofCrAnState_bosonic_or_fermionic (φ1 φ2 φ3 :  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnState φ1, [ofCrAnState φ2, ofCrAnState φ3]sca]sca ∈ statisticSubmodule bosonic v
  [ofCrAnState φ1, [ofCrAnState φ2, ofCrAnState φ3]sca]sca ∈ statisticSubmodule fermionic
```

lemma superCommuteF_bosonic_ofCrAnList_eq_sum

[\[source\]](#)

```
lemma superCommuteF_bosonic_ofCrAnList_eq_sum (a :  $\mathcal{F}$ .CrAnAlgebra) (φs : List  $\mathcal{F}$ .CrAnStates)
  (ha : a ∈ statisticSubmodule bosonic) :
  [a, ofCrAnList φs]sca =  $\sum$  (n : Fin φs.length),
    ofCrAnList (φs.take n) * [a, ofCrAnState (φs.get n)]sca *
    ofCrAnList (φs.drop (n + 1))
```

lemma superCommuteF_fermionic_ofCrAnList_eq_sum

[\[source\]](#)

```
lemma superCommuteF_fermionic_ofCrAnList_eq_sum (a :  $\mathcal{F}$ .CrAnAlgebra) (φs : List  $\mathcal{F}$ .CrAnStates)
  (ha : a ∈ statisticSubmodule fermionic) :
  [a, ofCrAnList φs]sca =  $\sum$  (n : Fin φs.length),  $S(\text{fermionic}, \mathcal{F} \mid_{>s} \phi.s.\text{take } n) \cdot$ 
    ofCrAnList (φs.take n) * [a, ofCrAnState (φs.get n)]sca *
    ofCrAnList (φs.drop (n + 1))
```

lemma statistic_neq_of_superCommuteF_fermionic

[\[source\]](#)

```
lemma statistic_neq_of_superCommuteF_fermionic {φs φs' : List  $\mathcal{F}$ .CrAnStates}
  (h : [ofCrAnList φs, ofCrAnList φs']sca ∈ statisticSubmodule fermionic) :
  ( $\mathcal{F} \mid_{>s} \phi.s$ )  $\neq$  ( $\mathcal{F} \mid_{>s} \phi.s'$ )  $\vee$  [ofCrAnList φs, ofCrAnList φs']sca = 0
```

2.16 HepLean.PerturbationTheory.Algebras.CrAnAlgebra.TimeOrder

[HepLean/PerturbationTheory/Algebras/CrAnAlgebra/TimeOrder.lean](#) on GitHub.

def timeOrderF

[\[source\]](#)

Time ordering for the CrAnAlgebra.

```
def timeOrderF : CrAnAlgebra  $\mathcal{F} \rightarrow_l[\mathbb{C}]$  CrAnAlgebra  $\mathcal{F}$ 
```

lemma timeOrderF_ofCrAnList

[\[source\]](#)

```
lemma timeOrderF_ofCrAnList (φs : List  $\mathcal{F}$ .CrAnStates) :
   $\mathcal{T}^f(\text{ofCrAnList } \phi.s) = \text{crAnTimeOrderSign } \phi.s \cdot \text{ofCrAnList } (\text{crAnTimeOrderList } \phi.s)$ 
```

lemma timeOrderF_timeOrderF_mid

[\[source\]](#)

```
lemma timeOrderF_timeOrderF_mid (a b c :  $\mathcal{F}$ .CrAnAlgebra) :  $\mathcal{T}^f(a * b * c) = \mathcal{T}^f(a * \mathcal{T}^f(b) * c)$ 
```

lemma timeOrderF_timeOrderF_right[\[source\]](#)

```
lemma timeOrderF_timeOrderF_right (a b :  $\mathcal{F}$ .CrAnAlgebra) :  $\mathcal{T}^f(a * b) = \mathcal{T}^f(a * \mathcal{T}^f(b))$ 
```

lemma timeOrderF_timeOrderF_left[\[source\]](#)

```
lemma timeOrderF_timeOrderF_left (a b :  $\mathcal{F}$ .CrAnAlgebra) :  $\mathcal{T}^f(a * b) = \mathcal{T}^f(\mathcal{T}^f(a) * b)$ 
```

lemma timeOrderF_ofStateList[\[source\]](#)

```
lemma timeOrderF_ofStateList ( $\phi$ s : List  $\mathcal{F}$ .States) :  
   $\mathcal{T}^f(\text{ofStateList } \phi\text{s}) = \text{timeOrderSign } \phi\text{s} \cdot \text{ofStateList } (\text{timeOrderList } \phi\text{s})$ 
```

lemma timeOrderF_ofStateList_nil[\[source\]](#)

```
lemma timeOrderF_ofStateList_nil : timeOrderF ( $\mathcal{F} := \mathcal{F}$ ) (ofStateList []) = 1
```

lemma timeOrderF_ofStateList_singleton[\[source\]](#)

```
@[simp]  
lemma timeOrderF_ofStateList_singleton ( $\phi$  :  $\mathcal{F}$ .States) :  $\mathcal{T}^f(\text{ofStateList } [\phi]) = \text{ofStateList } [\phi]$ 
```

lemma timeOrderF_ofState_ofState_ordered[\[source\]](#)

```
lemma timeOrderF_ofState_ofState_ordered { $\phi \psi$  :  $\mathcal{F}$ .States} (h : timeOrderRel  $\phi \psi$ ) :  
   $\mathcal{T}^f(\text{ofState } \phi * \text{ofState } \psi) = \text{ofState } \phi * \text{ofState } \psi$ 
```

lemma timeOrderF_ofState_ofState_not_ordered[\[source\]](#)

```
lemma timeOrderF_ofState_ofState_not_ordered { $\phi \psi$  :  $\mathcal{F}$ .States} (h :  $\neg \text{timeOrderRel } \phi \psi$ ) :  
   $\mathcal{T}^f(\text{ofState } \phi * \text{ofState } \psi) = S(\mathcal{F} \mid_{>_s} \phi, \mathcal{F} \mid_{>_s} \psi) \cdot \text{ofState } \psi * \text{ofState } \phi$ 
```

lemma timeOrderF_ofState_ofState_not_ordered_eq_timeOrderF[\[source\]](#)

```
lemma timeOrderF_ofState_ofState_not_ordered_eq_timeOrderF { $\phi \psi$  :  $\mathcal{F}$ .States}  
  (h :  $\neg \text{timeOrderRel } \phi \psi$ ) :  
   $\mathcal{T}^f(\text{ofState } \phi * \text{ofState } \psi) = S(\mathcal{F} \mid_{>_s} \phi, \mathcal{F} \mid_{>_s} \psi) \cdot \mathcal{T}^f(\text{ofState } \psi * \text{ofState } \phi)$ 
```

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel[\[source\]](#)

```
lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel  
  { $\phi \psi$  :  $\mathcal{F}$ .CrAnStates} (h :  $\neg \text{crAnTimeOrderRel } \phi \psi$ ) :  
   $\mathcal{T}^f([\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca}) = 0$ 
```

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel_right [\[source\]](#)

```
lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel_right  
  { $\phi \psi$  :  $\mathcal{F}$ .CrAnStates} (h :  $\neg \text{crAnTimeOrderRel } \phi \psi$ ) (a :  $\mathcal{F}$ .CrAnAlgebra) :  
   $\mathcal{T}^f(a * [\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca}) = 0$ 
```

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel_left [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel_left
 { $\phi \psi : \mathcal{F}.\text{CrAnStates}$ } (h : $\neg \text{crAnTimeOrderRel } \phi \psi$) (a : $\mathcal{F}.\text{CrAnAlgebra}$) :
 $\mathcal{T}^f([\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca} * a) = 0$

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel_mid [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_not_crAnTimeOrderRel_mid
 { $\phi \psi : \mathcal{F}.\text{CrAnStates}$ } (h : $\neg \text{crAnTimeOrderRel } \phi \psi$) (a b : $\mathcal{F}.\text{CrAnAlgebra}$) :
 $\mathcal{T}^f(a * [\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca} * b) = 0$

lemma timeOrderF_superCommuteF_superCommuteF_ofCrAnState_not_crAnTimeOrderRel [\[source\]](#)

lemma timeOrderF_superCommuteF_superCommuteF_ofCrAnState_not_crAnTimeOrderRel
 { $\phi_1 \phi_2 : \mathcal{F}.\text{CrAnStates}$ } (h : $\neg \text{crAnTimeOrderRel } \phi_1 \phi_2$) (a : $\mathcal{F}.\text{CrAnAlgebra}$) :
 $\mathcal{T}^f([a, [\text{ofCrAnState } \phi_1, \text{ofCrAnState } \phi_2]_{sca}]_{sca}) = 0$

lemma timeOrderF_superCommuteF_ofCrAnState_superCommuteF_not_crAnTimeOrderRel [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnState_superCommuteF_not_crAnTimeOrderRel
 { $\phi_1 \phi_2 \phi_3 : \mathcal{F}.\text{CrAnStates}$ } (h12 : $\neg \text{crAnTimeOrderRel } \phi_1 \phi_2$)
 (h13 : $\neg \text{crAnTimeOrderRel } \phi_1 \phi_3$) :
 $\mathcal{T}^f([\text{ofCrAnState } \phi_1, [\text{ofCrAnState } \phi_2, \text{ofCrAnState } \phi_3]_{sca}]_{sca}) = 0$

lemma timeOrderF_superCommuteF_ofCrAnState_superCommuteF_not_crAnTimeOrderRel' [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnState_superCommuteF_not_crAnTimeOrderRel'
 { $\phi_1 \phi_2 \phi_3 : \mathcal{F}.\text{CrAnStates}$ } (h12 : $\neg \text{crAnTimeOrderRel } \phi_2 \phi_1$)
 (h13 : $\neg \text{crAnTimeOrderRel } \phi_3 \phi_1$) :
 $\mathcal{T}^f([\text{ofCrAnState } \phi_1, [\text{ofCrAnState } \phi_2, \text{ofCrAnState } \phi_3]_{sca}]_{sca}) = 0$

lemma timeOrderF_superCommuteF_ofCrAnState_superCommuteF_all_not_crAnTimeOrderRel [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnState_superCommuteF_all_not_crAnTimeOrderRel
 ($\phi_1 \phi_2 \phi_3 : \mathcal{F}.\text{CrAnStates}$) (h : $\neg$
 ($\text{crAnTimeOrderRel } \phi_1 \phi_2 \wedge \text{crAnTimeOrderRel } \phi_1 \phi_3 \wedge$
 $\text{crAnTimeOrderRel } \phi_2 \phi_1 \wedge \text{crAnTimeOrderRel } \phi_2 \phi_3 \wedge$
 $\text{crAnTimeOrderRel } \phi_3 \phi_1 \wedge \text{crAnTimeOrderRel } \phi_3 \phi_2$)) :
 $\mathcal{T}^f([\text{ofCrAnState } \phi_1, [\text{ofCrAnState } \phi_2, \text{ofCrAnState } \phi_3]_{sca}]_{sca}) = 0$

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_eq_time [\[source\]](#)

lemma timeOrderF_superCommuteF_ofCrAnState_ofCrAnState_eq_time
 { $\phi \psi : \mathcal{F}.\text{CrAnStates}$ } (h1 : $\text{crAnTimeOrderRel } \phi \psi$) (h2 : $\text{crAnTimeOrderRel } \psi \phi$) :
 $\mathcal{T}^f([\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca}) = [\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca}$

lemma timeOrderF_eq_maxTimeField_mul [\[source\]](#)

*In the state algebra time, ordering obeys $T(\phi \circ \phi_1 \dots \phi_n) = s * \phi_i * T(\phi \circ \phi_1 \dots \phi_{i-1} \phi_{i+1} \dots \phi_n)$ where ϕ_i is the state which has maximum time and s is the exchange sign of ϕ_i and $\phi \circ \phi_1 \dots \phi_{i-1}$.*

```

lemma timeOrderF_eq_maxTimeField_mul (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States) :
   $\mathcal{T}^f(\text{ofStateList } (\phi :: \phi s)) =$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \text{maxTimeField } \phi \ \phi s, \mathcal{F} \mid_{>s} (\phi :: \phi s).take \ (\text{maxTimeFieldPos } \phi \ \phi s)) \cdot$ 
   $\text{ofState } (\text{maxTimeField } \phi \ \phi s) * \mathcal{T}^f(\text{ofStateList } (\text{eraseMaxTimeField } \phi \ \phi s))$ 

```

lemma timeOrderF_eq_maxTimeField_mul_finset

[source]

*In the state algebra time, ordering obeys $T(\phi \phi_1 \dots \phi_n) = s * \phi_i * T(\phi \phi_1 \dots \phi_{i-1} \phi_{i+1} \dots \phi_n)$ where ϕ_i is the state which has maximum time and s is the exchange sign of ϕ_i and $\phi \phi_1 \dots \phi_{i-1}$. Here s is written using finite sets.*

```

lemma timeOrderF_eq_maxTimeField_mul_finset (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States) :
   $\mathcal{T}^f(\text{ofStateList } (\phi :: \phi s)) = \mathcal{S}(\mathcal{F} \mid_{>s} \text{maxTimeField } \phi \ \phi s, \mathcal{F} \mid_{>s} ((\text{eraseMaxTimeField } \phi \ \phi s).$ 
   $\text{get},$ 
   $(\text{Finset.filter } (\text{fun } x \Rightarrow$ 
   $(\text{maxTimeFieldPosFin } \phi \ \phi s).succAbove \ x < \text{maxTimeFieldPosFin } \phi \ \phi s) \text{ Finset.univ}))) \cdot$ 
   $\text{ofState } (\text{maxTimeField } \phi \ \phi s) * \mathcal{T}^f(\text{ofStateList } (\text{eraseMaxTimeField } \phi \ \phi s))$ 

```

2.17 HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.Basic

[HepLean/PerturbationTheory/Algebras/FieldOpAlgebra/Basic.lean](#) on GitHub.

def fieldOpIdealSet

[source]

The set contains the super-commutators equal to zero in the operator algebra. This contains e.g. the super-commutator of two creation operators.

```

def fieldOpIdealSet : Set (CrAnAlgebra  $\mathcal{F}$ )

```

abbrev FieldOpAlgebra

[source]

The algebra spanned by cr and an parts of fields, with appropriate super-commutators set to zero.

```

abbrev FieldOpAlgebra : Type

```

instance Setoid (CrAnAlgebra $\mathcal{F}$)

[source]

The instance of a setoid on CrAnAlgebra from the ideal TwoSidedIdeal.

```

instance : Setoid (CrAnAlgebra  $\mathcal{F}$ )

```

lemma equiv_iff_sub_mem_ideal

[source]

```

lemma equiv_iff_sub_mem_ideal (x y : CrAnAlgebra  $\mathcal{F}$ ) :
   $x \approx y \leftrightarrow x - y \in \text{TwoSidedIdeal.span } \mathcal{F}.fieldOpIdealSet$ 

```

def ι

[source]

The projection of CrAnAlgebra down to FieldOpAlgebra as an algebra map.

```

def ι : CrAnAlgebra  $\mathcal{F} \rightarrow_a [\mathbb{C}] \text{FieldOpAlgebra } \mathcal{F}$  where

```

lemma $\iota_{\text{surjective}}$ [\[source\]](#)

```
lemma  $\iota_{\text{surjective}}$  : Function.Surjective (@ $\iota$   $\mathcal{F}$ )
```

lemma ι_{apply} [\[source\]](#)

```
lemma  $\iota_{\text{apply}}$  (x : CrAnAlgebra  $\mathcal{F}$ ) :  $\iota$  x = Quotient.mk _ x
```

lemma $\iota_{\text{of_mem_fieldOpIdealSet}}$ [\[source\]](#)

```
lemma  $\iota_{\text{of\_mem\_fieldOpIdealSet}}$  (x : CrAnAlgebra  $\mathcal{F}$ ) (hx : x  $\in$   $\mathcal{F}.\text{fieldOpIdealSet}$ ) :  
   $\iota$  x = 0
```

lemma $\iota_{\text{superCommuteF_of_create_create}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_of\_create\_create}}$  ( $\phi$   $\phi'$  :  $\mathcal{F}.\text{CrAnStates}$ ) ( $h\phi$  :  $\mathcal{F} \mid >^c \phi = .\text{create}$ )  
  ( $h\phi'$  :  $\mathcal{F} \mid >^c \phi' = .\text{create}$ ) :  $\iota$  [ofCrAnState  $\phi$ , ofCrAnState  $\phi'$ ] $_s$ ca = 0
```

lemma $\iota_{\text{superCommuteF_of_annihilate_annihilate}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_of\_annihilate\_annihilate}}$  ( $\phi$   $\phi'$  :  $\mathcal{F}.\text{CrAnStates}$ )  
  ( $h\phi$  :  $\mathcal{F} \mid >^c \phi = .\text{annihilate}$ ) ( $h\phi'$  :  $\mathcal{F} \mid >^c \phi' = .\text{annihilate}$ ) :  
   $\iota$  [ofCrAnState  $\phi$ , ofCrAnState  $\phi'$ ] $_s$ ca = 0
```

lemma $\iota_{\text{superCommuteF_of_diff_statistic}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_of\_diff\_statistic}}$  { $\phi$   $\psi$  :  $\mathcal{F}.\text{CrAnStates}$ }  
  (h : ( $\mathcal{F} \mid >_s \phi$ )  $\neq$  ( $\mathcal{F} \mid >_s \psi$ )) :  $\iota$  [ofCrAnState  $\phi$ , ofCrAnState  $\psi$ ] $_s$ ca = 0
```

lemma $\iota_{\text{superCommuteF_zero_of_fermionic}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_zero\_of\_fermionic}}$  ( $\phi$   $\psi$  :  $\mathcal{F}.\text{CrAnStates}$ )  
  (h : [ofCrAnState  $\phi$ , ofCrAnState  $\psi$ ] $_s$ ca  $\in$  statisticSubmodule fermionic) :  
   $\iota$  [ofCrAnState  $\phi$ , ofCrAnState  $\psi$ ] $_s$ ca = 0
```

lemma $\iota_{\text{superCommuteF_ofCrAnState_ofCrAnState_bosonic_or_zero}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_ofCrAnState\_ofCrAnState\_bosonic\_or\_zero}}$  ( $\phi$   $\psi$  :  $\mathcal{F}.\text{CrAnStates}$ ) :  
  [ofCrAnState  $\phi$ , ofCrAnState  $\psi$ ] $_s$ ca  $\in$  statisticSubmodule bosonic  $\vee$   
   $\iota$  [ofCrAnState  $\phi$ , ofCrAnState  $\psi$ ] $_s$ ca = 0
```

lemma $\iota_{\text{superCommuteF_ofCrAnState_superCommuteF_ofCrAnState_ofCrAnState}}$ [\[source\]](#)

```
@[simp]  
lemma  $\iota_{\text{superCommuteF\_ofCrAnState\_superCommuteF\_ofCrAnState\_ofCrAnState}}$  ( $\phi_1$   $\phi_2$   $\phi_3$  :  $\mathcal{F}.$   
  CrAnStates) :  
   $\iota$  [ofCrAnState  $\phi_1$ , [ofCrAnState  $\phi_2$ , ofCrAnState  $\phi_3$ ] $_s$ ca] $_s$ ca = 0
```

lemma $\iota_{\text{superCommuteF_superCommuteF_ofCrAnState_ofCrAnState_ofCrAnState}}$ [\[source\]](#)

```

lemma  $\iota_{\text{superCommuteF\_superCommuteF\_ofCrAnState\_ofCrAnState\_ofCrAnState}}$  ( $\phi_1 \phi_2 \phi_3 : \mathcal{F}.$ 
  CrAnStates) :
   $\iota_{[[\text{ofCrAnState } \phi_1, \text{ofCrAnState } \phi_2]_{\text{sc}}, \text{ofCrAnState } \phi_3]_{\text{sc}}} = 0$ 

```

lemma $\iota_{\text{superCommuteF_superCommuteF_ofCrAnState_ofCrAnState_ofCrAnList}}$ [\[source\]](#)

```

lemma  $\iota_{\text{superCommuteF\_superCommuteF\_ofCrAnState\_ofCrAnState\_ofCrAnList}}$  ( $\phi_1 \phi_2 : \mathcal{F}.$ CrAnStates)
  ( $\phi_s : \text{List } \mathcal{F}.$ CrAnStates) :
   $\iota_{[[\text{ofCrAnState } \phi_1, \text{ofCrAnState } \phi_2]_{\text{sc}}, \text{ofCrAnList } \phi_s]_{\text{sc}}} = 0$ 

```

lemma $\iota_{\text{superCommuteF_superCommuteF_ofCrAnState_ofCrAnState_crAnAlgebra}}$ [\[source\]](#)

```

@[simp]
lemma  $\iota_{\text{superCommuteF\_superCommuteF\_ofCrAnState\_ofCrAnState\_crAnAlgebra}}$  ( $\phi_1 \phi_2 : \mathcal{F}.$ CrAnStates)
  ( $a : \mathcal{F}.$ CrAnAlgebra) :  $\iota_{[[\text{ofCrAnState } \phi_1, \text{ofCrAnState } \phi_2]_{\text{sc}}, a]_{\text{sc}}} = 0$ 

```

lemma $\iota_{\text{commute_crAnAlgebra_superCommuteF_ofCrAnState_ofCrAnState}}$ [\[source\]](#)

```

lemma  $\iota_{\text{commute\_crAnAlgebra\_superCommuteF\_ofCrAnState\_ofCrAnState}}$  ( $\phi_1 \phi_2 : \mathcal{F}.$ CrAnStates)
  ( $a : \mathcal{F}.$ CrAnAlgebra) :  $\iota a * \iota_{[\text{ofCrAnState } \phi_1, \text{ofCrAnState } \phi_2]_{\text{sc}}} -$ 
   $\iota_{[\text{ofCrAnState } \phi_1, \text{ofCrAnState } \phi_2]_{\text{sc}}} * \iota a = 0$ 

```

lemma $\iota_{\text{superCommuteF_ofCrAnState_ofCrAnState_mem_center}}$ [\[source\]](#)

```

lemma  $\iota_{\text{superCommuteF\_ofCrAnState\_ofCrAnState\_mem\_center}}$  ( $\phi \psi : \mathcal{F}.$ CrAnStates) :
   $\iota_{[\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{\text{sc}}} \in \text{Subalgebra.center } \mathbb{C} \mathcal{F}.$ FieldOpAlgebra

```

lemma $\iota_{\text{eq_zero_iff_mem_ideal}}$ [\[source\]](#)

```

lemma  $\iota_{\text{eq\_zero\_iff\_mem\_ideal}}$  ( $x : \text{CrAnAlgebra } \mathcal{F}$ ) :
   $\iota x = 0 \leftrightarrow x \in \text{TwoSidedIdeal.span } \mathcal{F}.$ fieldOpIdealSet

```

lemma $\text{bosonicProj_mem_fieldOpIdealSet_or_zero}$ [\[source\]](#)

```

lemma  $\text{bosonicProj\_mem\_fieldOpIdealSet\_or\_zero}$  ( $x : \text{CrAnAlgebra } \mathcal{F}$ ) ( $hx : x \in \mathcal{F}.$ fieldOpIdealSet)
  :
   $x.\text{bosonicProj}.1 \in \mathcal{F}.$ fieldOpIdealSet  $\vee x.\text{bosonicProj} = 0$ 

```

lemma $\text{fermionicProj_mem_fieldOpIdealSet_or_zero}$ [\[source\]](#)

```

lemma  $\text{fermionicProj\_mem\_fieldOpIdealSet\_or\_zero}$  ( $x : \text{CrAnAlgebra } \mathcal{F}$ ) ( $hx : x \in \mathcal{F}.$ 
  fieldOpIdealSet) :
   $x.\text{fermionicProj}.1 \in \mathcal{F}.$ fieldOpIdealSet  $\vee x.\text{fermionicProj} = 0$ 

```

lemma $\text{bosonicProj_mem_ideal}$ [\[source\]](#)

```

lemma  $\text{bosonicProj\_mem\_ideal}$  ( $x : \text{CrAnAlgebra } \mathcal{F}$ ) ( $hx : x \in \text{TwoSidedIdeal.span } \mathcal{F}.$ fieldOpIdealSet
  ) :
   $x.\text{bosonicProj}.1 \in \text{TwoSidedIdeal.span } \mathcal{F}.$ fieldOpIdealSet

```

lemma fermionicProj_mem_ideal[\[source\]](#)

```
lemma fermionicProj_mem_ideal (x : CrAnAlgebra  $\mathcal{F}$ ) (hx : x ∈ TwoSidedIdeal.span  $\mathcal{F}$ .
  fieldOpIdealSet) :
  x.fermionicProj.1 ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet
```

lemma ι _eq_zero_iff_ ι _bosonicProj_fermonicProj_zero[\[source\]](#)

```
lemma  $\iota$ _eq_zero_iff_ $\iota$ _bosonicProj_fermonicProj_zero (x : CrAnAlgebra  $\mathcal{F}$ ) :
   $\iota$  x = 0  $\leftrightarrow$   $\iota$  x.bosonicProj.1 = 0  $\wedge$   $\iota$  x.fermionicProj.1 = 0
```

def ofFieldOp[\[source\]](#)

An element of FieldOpAlgebra from a States.

```
def ofFieldOp ( $\phi$  :  $\mathcal{F}$ .States) :  $\mathcal{F}$ .FieldOpAlgebra
```

lemma ofFieldOp_eq_ ι _ofState[\[source\]](#)

```
lemma ofFieldOp_eq_ $\iota$ _ofState ( $\phi$  :  $\mathcal{F}$ .States) : ofFieldOp  $\phi$  =  $\iota$  (ofState  $\phi$ )
```

def ofFieldOpList[\[source\]](#)

An element of FieldOpAlgebra from a list of States.

```
def ofFieldOpList ( $\phi$ s : List  $\mathcal{F}$ .States) :  $\mathcal{F}$ .FieldOpAlgebra
```

lemma ofFieldOpList_eq_ ι _ofStateList[\[source\]](#)

```
lemma ofFieldOpList_eq_ $\iota$ _ofStateList ( $\phi$ s : List  $\mathcal{F}$ .States) :
  ofFieldOpList  $\phi$ s =  $\iota$  (ofStateList  $\phi$ s)
```

lemma ofFieldOpList_append[\[source\]](#)

```
lemma ofFieldOpList_append ( $\phi$ s  $\psi$ s : List  $\mathcal{F}$ .States) :
  ofFieldOpList ( $\phi$ s ++  $\psi$ s) = ofFieldOpList  $\phi$ s * ofFieldOpList  $\psi$ s
```

lemma ofFieldOpList_cons[\[source\]](#)

```
lemma ofFieldOpList_cons ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States) :
  ofFieldOpList ( $\phi$  ::  $\phi$ s) = ofFieldOp  $\phi$  * ofFieldOpList  $\phi$ s
```

lemma ofFieldOpList_singleton[\[source\]](#)

```
lemma ofFieldOpList_singleton ( $\phi$  :  $\mathcal{F}$ .States) :
  ofFieldOpList [ $\phi$ ] = ofFieldOp  $\phi$ 
```

def ofCrAnFieldOp[\[source\]](#)

An element of FieldOpAlgebra from a CrAnStates.

```
def ofCrAnFieldOp ( $\phi$  :  $\mathcal{F}$ .CrAnStates) :  $\mathcal{F}$ .FieldOpAlgebra
```


lemma ofCrAnFieldOp_eq_ι_ofCrAnState[\[source\]](#)

```
lemma ofCrAnFieldOp_eq_ι_ofCrAnState (φ :  $\mathcal{F}$ .CrAnStates) :
  ofCrAnFieldOp φ = ι (ofCrAnState φ)
```

lemma ofFieldOp_eq_sum[\[source\]](#)

```
lemma ofFieldOp_eq_sum (φ :  $\mathcal{F}$ .States) :
  ofFieldOp φ = ( $\sum$  i :  $\mathcal{F}$ .statesToCrAnType φ, ofCrAnFieldOp (φ, i))
```

def ofCrAnFieldOpList[\[source\]](#)

An element of FieldOpAlgebra from a list of CrAnStates.

```
def ofCrAnFieldOpList (φs : List  $\mathcal{F}$ .CrAnStates) :  $\mathcal{F}$ .FieldOpAlgebra
```

lemma ofCrAnFieldOpList_eq_ι_ofCrAnList[\[source\]](#)

```
lemma ofCrAnFieldOpList_eq_ι_ofCrAnList (φs : List  $\mathcal{F}$ .CrAnStates) :
  ofCrAnFieldOpList φs = ι (ofCrAnList φs)
```

lemma ofCrAnFieldOpList_append[\[source\]](#)

```
lemma ofCrAnFieldOpList_append (φs ψs : List  $\mathcal{F}$ .CrAnStates) :
  ofCrAnFieldOpList (φs ++ ψs) = ofCrAnFieldOpList φs * ofCrAnFieldOpList ψs
```

lemma ofCrAnFieldOpList_singleton[\[source\]](#)

```
lemma ofCrAnFieldOpList_singleton (φ :  $\mathcal{F}$ .CrAnStates) :
  ofCrAnFieldOpList [φ] = ofCrAnFieldOp φ
```

lemma ofFieldOpList_eq_sum[\[source\]](#)

```
lemma ofFieldOpList_eq_sum (φs : List  $\mathcal{F}$ .States) :
  ofFieldOpList φs =  $\sum$  s : CrAnSection φs, ofCrAnFieldOpList s.1
```

def anPart[\[source\]](#)

The annihilation part of a state.

```
def anPart (φ :  $\mathcal{F}$ .States) :  $\mathcal{F}$ .FieldOpAlgebra
```

lemma anPart_eq_ι_anPartF[\[source\]](#)

```
lemma anPart_eq_ι_anPartF (φ :  $\mathcal{F}$ .States) : anPart φ = ι (anPartF φ)
```

lemma anPart_negAsymp[\[source\]](#)

```
@[simp]
lemma anPart_negAsymp (φ :  $\mathcal{F}$ .IncomingAsymptotic) :
  anPart (States.inAsymp φ) = 0
```

lemma anPart_position[\[source\]](#)

```
@[simp]
lemma anPart_position ( $\phi$  :  $\mathcal{F}$ .PositionStates) :
  anPart (States.position  $\phi$ ) =
    ofCrAnFieldOp (States.position  $\phi$ , CreateAnnihilate.annihilate)
```

lemma anPart_posAsymp[\[source\]](#)

```
@[simp]
lemma anPart_posAsymp ( $\phi$  :  $\mathcal{F}$ .OutgoingAsymptotic) :
  anPart (States.outAsymp  $\phi$ ) = ofCrAnFieldOp (States.outAsymp  $\phi$ , ())
```

def crPart[\[source\]](#)

The creation part of a state.

```
def crPart ( $\phi$  :  $\mathcal{F}$ .States) :  $\mathcal{F}$ .FieldOpAlgebra
```

lemma crPart_eq_ι_crPartF[\[source\]](#)

```
lemma crPart_eq_ι_crPartF ( $\phi$  :  $\mathcal{F}$ .States) : crPart  $\phi$  =  $\iota$  (crPartF  $\phi$ )
```

lemma crPart_negAsymp[\[source\]](#)

```
@[simp]
lemma crPart_negAsymp ( $\phi$  :  $\mathcal{F}$ .IncomingAsymptotic) :
  crPart (States.inAsymp  $\phi$ ) = ofCrAnFieldOp (States.inAsymp  $\phi$ , ())
```

lemma crPart_position[\[source\]](#)

```
@[simp]
lemma crPart_position ( $\phi$  :  $\mathcal{F}$ .PositionStates) :
  crPart (States.position  $\phi$ ) =
    ofCrAnFieldOp (States.position  $\phi$ , CreateAnnihilate.create)
```

lemma crPart_posAsymp[\[source\]](#)

```
@[simp]
lemma crPart_posAsymp ( $\phi$  :  $\mathcal{F}$ .OutgoingAsymptotic) :
  crPart (States.outAsymp  $\phi$ ) = 0
```

lemma ofFieldOp_eq_crPart_add_anPart[\[source\]](#)

```
lemma ofFieldOp_eq_crPart_add_anPart ( $\phi$  :  $\mathcal{F}$ .States) :
  ofFieldOp  $\phi$  = crPart  $\phi$  + anPart  $\phi$ 
```

2.18 HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.NormalOrder

[HepLean/PerturbationTheory/Algebras/FieldOpAlgebra/NormalOrder.lean](#) on GitHub.

lemma `ι_normalOrderF_superCommuteF_ofCrAnList_ofCrAnList_eq_zero`

[\[source\]](#)

```
lemma ι_normalOrderF_superCommuteF_ofCrAnList_ofCrAnList_eq_zero
  (φa φa' :  $\mathcal{F}$ .CrAnStates) (φs φs' : List  $\mathcal{F}$ .CrAnStates) :
  ι  $\mathcal{N}^f$ (ofCrAnList φs * [ofCrAnState φa, ofCrAnState φa']sca * ofCrAnList φs') = 0
```

lemma `ι_normalOrderF_superCommuteF_ofCrAnList_eq_zero`

[\[source\]](#)

```
lemma ι_normalOrderF_superCommuteF_ofCrAnList_eq_zero
  (φa φa' :  $\mathcal{F}$ .CrAnStates) (φs : List  $\mathcal{F}$ .CrAnStates)
  (a :  $\mathcal{F}$ .CrAnAlgebra) : ι  $\mathcal{N}^f$ (ofCrAnList φs * [ofCrAnState φa, ofCrAnState φa']sca * a) = 0
```

lemma `ι_normalOrderF_superCommuteF_ofCrAnState_eq_zero_mul`

[\[source\]](#)

```
lemma ι_normalOrderF_superCommuteF_ofCrAnState_eq_zero_mul (φa φa' :  $\mathcal{F}$ .CrAnStates)
  (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  ι  $\mathcal{N}^f$ (a * [ofCrAnState φa, ofCrAnState φa']sca * b) = 0
```

lemma `ι_normalOrderF_superCommuteF_ofCrAnState_ofCrAnList_eq_zero_mul`

[\[source\]](#)

```
lemma ι_normalOrderF_superCommuteF_ofCrAnState_ofCrAnList_eq_zero_mul (φa :  $\mathcal{F}$ .CrAnStates)
  (φs : List  $\mathcal{F}$ .CrAnStates)
  (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  ι  $\mathcal{N}^f$ (a * [ofCrAnState φa, ofCrAnList φs]sca * b) = 0
```

lemma `ι_normalOrderF_superCommuteF_ofCrAnList_ofCrAnState_eq_zero_mul`

[\[source\]](#)

```
lemma ι_normalOrderF_superCommuteF_ofCrAnList_ofCrAnState_eq_zero_mul (φa :  $\mathcal{F}$ .CrAnStates)
  (φs : List  $\mathcal{F}$ .CrAnStates) (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  ι  $\mathcal{N}^f$ (a * [ofCrAnList φs, ofCrAnState φa]sca * b) = 0
```

lemma `ι_normalOrderF_superCommuteF_ofCrAnList_ofCrAnList_eq_zero_mul`

[\[source\]](#)

```
lemma ι_normalOrderF_superCommuteF_ofCrAnList_ofCrAnList_eq_zero_mul
  (φs φs' : List  $\mathcal{F}$ .CrAnStates) (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  ι  $\mathcal{N}^f$ (a * [ofCrAnList φs, ofCrAnList φs']sca * b) = 0
```

lemma `ι_normalOrderF_superCommuteF_ofCrAnList_eq_zero_mul`

[\[source\]](#)

```
lemma ι_normalOrderF_superCommuteF_ofCrAnList_eq_zero_mul
  (φs : List  $\mathcal{F}$ .CrAnStates)
  (a b c :  $\mathcal{F}$ .CrAnAlgebra) :
  ι  $\mathcal{N}^f$ (a * [ofCrAnList φs, c]sca * b) = 0
```

lemma `ι_normalOrderF_superCommuteF_eq_zero_mul`

[\[source\]](#)

```
@[simp]
lemma ι_normalOrderF_superCommuteF_eq_zero_mul
  (a b c d :  $\mathcal{F}$ .CrAnAlgebra) : ι  $\mathcal{N}^f$ (a * [d, c]sca * b) = 0
```

lemma $\mathfrak{I}_{\text{normalOrder_superCommuteF_eq_zero_mul_right}}$ [\[source\]](#)

@[simp]

lemma $\mathfrak{I}_{\text{normalOrder_superCommuteF_eq_zero_mul_right}}$ (b c d : $\mathcal{F}.\text{CrAnAlgebra}$) :
 $\mathfrak{I} \mathcal{N}^f([d, c]_s c a * b) = 0$

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero_mul_left}}$ [\[source\]](#)

@[simp]

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero_mul_left}}$ (a c d : $\mathcal{F}.\text{CrAnAlgebra}$) :
 $\mathfrak{I} \mathcal{N}^f(a * [d, c]_s c a) = 0$

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero_mul_mul_right}}$ [\[source\]](#)

@[simp]

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero_mul_mul_right}}$ (a b1 b2 c d : $\mathcal{F}.\text{CrAnAlgebra}$) :
 $\mathfrak{I} \mathcal{N}^f(a * [d, c]_s c a * b1 * b2) = 0$

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero}}$ [\[source\]](#)

@[simp]

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero}}$ (c d : $\mathcal{F}.\text{CrAnAlgebra}$) : $\mathfrak{I} \mathcal{N}^f([d, c]_s c a) = 0$

lemma $\mathfrak{I}_{\text{normalOrderF_zero_of_mem_ideal}}$ [\[source\]](#)

lemma $\mathfrak{I}_{\text{normalOrderF_zero_of_mem_ideal}}$ (a : $\mathcal{F}.\text{CrAnAlgebra}$)

(h : a ∈ TwoSidedIdeal.span $\mathcal{F}.\text{fieldOpIdealSet}$) : $\mathfrak{I} \mathcal{N}^f(a) = 0$

lemma $\mathfrak{I}_{\text{normalOrderF_eq_of_equiv}}$ [\[source\]](#)

lemma $\mathfrak{I}_{\text{normalOrderF_eq_of_equiv}}$ (a b : $\mathcal{F}.\text{CrAnAlgebra}$) (h : a ≈ b) :
 $\mathfrak{I} \mathcal{N}^f(a) = \mathfrak{I} \mathcal{N}^f(b)$

def normalOrder[\[source\]](#)

Normal ordering on FieldOpAlgebra.

noncomputable def normalOrder : FieldOpAlgebra $\mathcal{F} \rightarrow_l [C]$ FieldOpAlgebra $\mathcal{F}$ **where**

lemma normalOrder_eq_ $\mathfrak{I}_{\text{normalOrderF}}$ [\[source\]](#)

lemma normalOrder_eq_ $\mathfrak{I}_{\text{normalOrderF}}$ (a : $\mathcal{F}.\text{CrAnAlgebra}$) :
 $\mathcal{N}(\mathfrak{I} a) = \mathfrak{I} \mathcal{N}^f(a)$

lemma normalOrder_ofCrAnFieldOpList[\[source\]](#)

lemma normalOrder_ofCrAnFieldOpList (φs : List $\mathcal{F}.\text{CrAnStates}$) :
 $\mathcal{N}(\text{ofCrAnFieldOpList } \phi s) = \text{normalOrderSign } \phi s \bullet \text{ofCrAnFieldOpList } (\text{normalOrderList } \phi s)$

lemma normalOrder_one_eq_one[\[source\]](#)

@[simp]

lemma normalOrder_one_eq_one : normalOrder ($\mathcal{F} := \mathcal{F}$) 1 = 1**lemma ofCrAnFieldOpList_eq_normalOrder**[\[source\]](#)**lemma** ofCrAnFieldOpList_eq_normalOrder (ϕs : List $\mathcal{F}$.CrAnStates) :
ofCrAnFieldOpList (normalOrderList ϕs) = normalOrderSign ϕs • $\mathcal{N}$ (ofCrAnFieldOpList ϕs)**lemma normalOrder_mul_anPart**[\[source\]](#)**lemma** normalOrder_mul_anPart (ϕ : $\mathcal{F}$.States) (a : $\mathcal{F}$.FieldOpAlgebra) :
 $\mathcal{N}(a * \text{anPart } \phi) = \mathcal{N}(a) * \text{anPart } \phi$ **lemma crPart_mul_normalOrder**[\[source\]](#)**lemma** crPart_mul_normalOrder (ϕ : $\mathcal{F}$.States) (a : $\mathcal{F}$.FieldOpAlgebra) :
 $\mathcal{N}(\text{crPart } \phi * a) = \text{crPart } \phi * \mathcal{N}(a)$ **lemma normalOrder_superCommute_eq_zero**[\[source\]](#)

@[simp]

lemma normalOrder_superCommute_eq_zero ($a \ b$: $\mathcal{F}$.FieldOpAlgebra) :
 $\mathcal{N}([a, b]_s) = 0$ **lemma normalOrder_superCommute_left_eq_zero**[\[source\]](#)

@[simp]

lemma normalOrder_superCommute_left_eq_zero ($a \ b \ c$: $\mathcal{F}$.FieldOpAlgebra) :
 $\mathcal{N}([a, b]_s * c) = 0$ **lemma normalOrder_superCommute_right_eq_zero**[\[source\]](#)

@[simp]

lemma normalOrder_superCommute_right_eq_zero ($a \ b \ c$: $\mathcal{F}$.FieldOpAlgebra) :
 $\mathcal{N}(c * [a, b]_s) = 0$ **lemma normalOrder_superCommute_mid_eq_zero**[\[source\]](#)

@[simp]

lemma normalOrder_superCommute_mid_eq_zero ($a \ b \ c \ d$: $\mathcal{F}$.FieldOpAlgebra) :
 $\mathcal{N}(a * [c, d]_s * b) = 0$ **lemma normalOrder_ofFieldOp_ofFieldOp_swap**[\[source\]](#)**lemma** normalOrder_ofFieldOp_ofFieldOp_swap ($\phi \ \phi'$: $\mathcal{F}$.States) :
 $\mathcal{N}(\text{ofFieldOp } \phi * \text{ofFieldOp } \phi') = \mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi') \bullet \mathcal{N}(\text{ofFieldOp } \phi' * \text{ofFieldOp } \phi)$

lemma normalOrder_ofCrAnFieldOp_ofCrAnFieldOpList[\[source\]](#)

```

lemma normalOrder_ofCrAnFieldOp_ofCrAnFieldOpList (φ :  $\mathcal{F}$ .CrAnStates)
  (ψs : List  $\mathcal{F}$ .CrAnStates) :  $\mathcal{N}(\text{ofCrAnFieldOp } \varphi * \text{ofCrAnFieldOpList } \psi\text{s}) =$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \psi\text{s}) \cdot \mathcal{N}(\text{ofCrAnFieldOpList } \psi\text{s} * \text{ofCrAnFieldOp } \varphi)$ 

```

lemma normalOrder_ofCrAnFieldOp_ofFieldOpList_swap[\[source\]](#)

```

lemma normalOrder_ofCrAnFieldOp_ofFieldOpList_swap (φ :  $\mathcal{F}$ .CrAnStates) (φ' : List  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{ofCrAnFieldOp } \varphi * \text{ofFieldOpList } \varphi') = \mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \cdot$ 
   $\mathcal{N}(\text{ofFieldOpList } \varphi' * \text{ofCrAnFieldOp } \varphi)$ 

```

lemma normalOrder_anPart_ofFieldOpList_swap[\[source\]](#)

```

lemma normalOrder_anPart_ofFieldOpList_swap (φ :  $\mathcal{F}$ .States) (φ' : List  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{anPart } \varphi * \text{ofFieldOpList } \varphi') = \mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \cdot \mathcal{N}(\text{ofFieldOpList } \varphi' * \text{anPart } \varphi)$ 

```

lemma normalOrder_ofFieldOpList_anPart_swap[\[source\]](#)

```

lemma normalOrder_ofFieldOpList_anPart_swap (φ :  $\mathcal{F}$ .States) (φ' : List  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{ofFieldOpList } \varphi' * \text{anPart } \varphi) = \mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \cdot \mathcal{N}(\text{anPart } \varphi * \text{ofFieldOpList } \varphi')$ 

```

lemma normalOrder_ofFieldOpList_mul_anPart_swap[\[source\]](#)

```

lemma normalOrder_ofFieldOpList_mul_anPart_swap (φ :  $\mathcal{F}$ .States) (ψs : List  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{ofFieldOpList } \psi\text{s}) * \text{anPart } \varphi = \mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \psi\text{s}) \cdot \mathcal{N}(\text{anPart } \varphi * \text{ofFieldOpList } \psi\text{s})$ 

```

lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute (φ :  $\mathcal{F}$ .States)
  (ψs' : List  $\mathcal{F}$ .States) :  $\text{anPart } \varphi * \mathcal{N}(\text{ofFieldOpList } \psi\text{s}') =$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \psi\text{s}') \cdot \mathcal{N}(\text{ofFieldOpList } \psi\text{s}' * \text{anPart } \varphi) +$ 
   $[\text{anPart } \varphi, \mathcal{N}(\text{ofFieldOpList } \psi\text{s}')]_s$ 

```

lemma ofCrAnFieldOp_superCommute_normalOrder_ofCrAnFieldOpList_sum[\[source\]](#)

```

lemma ofCrAnFieldOp_superCommute_normalOrder_ofCrAnFieldOpList_sum (φ :  $\mathcal{F}$ .CrAnStates)
  (ψs : List  $\mathcal{F}$ .CrAnStates) :  $[\text{ofCrAnFieldOp } \varphi, \mathcal{N}(\text{ofCrAnFieldOpList } \psi\text{s})]_s = \sum n : \text{Fin } \psi\text{s}.$ 
  length,
   $\mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} (\psi\text{s}.take\ n)) \cdot [\text{ofCrAnFieldOp } \varphi, \text{ofCrAnFieldOp } \psi\text{s}[n]]_s$ 
  *  $\mathcal{N}(\text{ofCrAnFieldOpList } (\psi\text{s}.eraseIdx\ n))$ 

```

lemma ofCrAnFieldOp_superCommute_normalOrder_ofFieldOpList_sum[\[source\]](#)

```

lemma ofCrAnFieldOp_superCommute_normalOrder_ofFieldOpList_sum (φ :  $\mathcal{F}$ .CrAnStates)
  (ψs : List  $\mathcal{F}$ .States) :
   $[\text{ofCrAnFieldOp } \varphi, \mathcal{N}(\text{ofFieldOpList } \psi\text{s})]_s = \sum n : \text{Fin } \psi\text{s}.length,$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} (\psi\text{s}.take\ n)) \cdot$ 
   $[\text{ofCrAnFieldOp } \varphi, \text{ofFieldOp } \psi\text{s}[n]]_s * \mathcal{N}(\text{ofFieldOpList } (\psi\text{s}.eraseIdx\ n))$ 

```

lemma anPart_superCommute_normalOrder_ofFieldOpList_sum[\[source\]](#)

Within a proto-operator algebra we have that $[anPartF \varphi, \mathcal{N}(\varphi s)] = \sum i, s_i \cdot [anPartF \varphi, \varphi_i]_s * \mathcal{N}(\varphi_{\emptyset \dots \varphi_{i-1} \varphi_{i+1} \dots \varphi_n})$ where s_i is the exchange sign for φ and $\varphi_{\emptyset \dots \varphi_{i-1}}$.

```
lemma anPart_superCommute_normalOrder_ofFieldOpList_sum (φ : F.States) (φs : List F.States) :
  [anPart φ, N(ofFieldOpList φs)]_s = ∑ n : Fin φs.length, S(F |>_s φ, F |>_s (φs.take n)) •
  [anPart φ, ofState φs[n]]_s * N(ofFieldOpList (φs.eraseIdx n))
```

lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute_reorder[\[source\]](#)

Within a proto-operator algebra we have that $anPartF \varphi * \mathcal{N}(\varphi_{\emptyset} \varphi_{1 \dots \varphi_n}) = \mathcal{N}((anPart \varphi) \varphi_{\emptyset} \varphi_{1 \dots \varphi_n}) + [anpart \varphi, \mathcal{N}(\varphi_{\emptyset} \varphi_{1 \dots \varphi_n})]_s$.

```
lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute_reorder (φ : F.States)
  (φs : List F.States) : anPart φ * N(ofFieldOpList φs) =
  N(anPart φ * ofFieldOpList φs) + [anPart φ, N(ofFieldOpList φs)]_s
```

lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_superCommute[\[source\]](#)

Within a proto-operator algebra we have that $\varphi * \mathcal{N}^f(\varphi_{\emptyset} \varphi_{1 \dots \varphi_n}) = \mathcal{N}^f(\varphi \varphi_{\emptyset} \varphi_{1 \dots \varphi_n}) + [anpart \varphi, \mathcal{N}^f(\varphi_{\emptyset} \varphi_{1 \dots \varphi_n})]_s ca$.

```
lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_superCommute (φ : F.States)
  (φs : List F.States) : ofFieldOp φ * N(ofFieldOpList φs) =
  N(ofFieldOp φ * ofFieldOpList φs) + [anPart φ, N(ofFieldOpList φs)]_s
```

def contractStateAtIndex[\[source\]](#)

In the expansion of $ofState \varphi * normalOrderF (ofStateList \varphi s)$ the element of $\mathcal{O}.A$ associated with contracting φ with the (optional) n th element of φs .

```
noncomputable def contractStateAtIndex (φ : F.States) (φs : List F.States)
  (n : Option (Fin φs.length)) : F.FieldOpAlgebra
```

lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_sum[\[source\]](#)

Within a proto-operator algebra, $\varphi * N(\varphi_{\emptyset} \varphi_{1 \dots \varphi_n}) = N(\varphi \varphi_{\emptyset} \varphi_{1 \dots \varphi_n}) + \sum i, (s_i \cdot [anPartF \varphi, \varphi_i]_s) * N(\varphi_{\emptyset} \varphi_{1 \dots \varphi_{i-1} \varphi_{i+1} \dots \varphi_n})$, where s_n is the exchange sign for φ and $\varphi_{\emptyset} \varphi_{1 \dots \varphi_{i-1}}$.

```
lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_sum (φ : F.States)
  (φs : List F.States) : ofFieldOp φ * N(ofFieldOpList φs) =
  ∑ n : Option (Fin φs.length), contractStateAtIndex φ φs n *
  N(ofFieldOpList (HepLean.List.optionEraseZ φs φ n))
```

lemma ofFieldOpList_normalOrder_insert[\[source\]](#)

Within a proto-operator algebra, $N(\varphi \varphi_{\emptyset} \varphi_{1 \dots \varphi_n}) = s \cdot N(\varphi_{\emptyset \dots \varphi_{k-1} \varphi \varphi_{k \dots \varphi_n})$, where s is the exchange sign for φ and $\varphi_{\emptyset \dots \varphi_{k-1}}$.

```
lemma ofFieldOpList_normalOrder_insert (φ : F.States) (φs : List F.States)
  (k : Fin φs.length.succ) : N(ofFieldOpList (φ :: φs)) =
  S(F |>_s φ, F |>_s φs.take k) • N(ofFieldOpList (φs.insertIdx k φ))
```

lemma normalOrder_crPart_mul_crPart[\[source\]](#)

```
@[simp]
lemma normalOrder_crPart_mul_crPart (φ φ' :  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{crPart } \varphi * \text{crPart } \varphi') = \text{crPart } \varphi * \text{crPart } \varphi'$ 
```

lemma normalOrder_anPart_mul_anPart[\[source\]](#)

```
@[simp]
lemma normalOrder_anPart_mul_anPart (φ φ' :  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{anPart } \varphi * \text{anPart } \varphi') = \text{anPart } \varphi * \text{anPart } \varphi'$ 
```

lemma normalOrder_crPart_mul_anPart[\[source\]](#)

```
@[simp]
lemma normalOrder_crPart_mul_anPart (φ φ' :  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{crPart } \varphi * \text{anPart } \varphi') = \text{crPart } \varphi * \text{anPart } \varphi'$ 
```

lemma normalOrder_anPart_mul_crPart[\[source\]](#)

```
@[simp]
lemma normalOrder_anPart_mul_crPart (φ φ' :  $\mathcal{F}$ .States) :
   $\mathcal{N}(\text{anPart } \varphi * \text{crPart } \varphi') = \mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot \text{crPart } \varphi' * \text{anPart } \varphi$ 
```

lemma normalOrder_ofFieldOp_mul_ofFieldOp[\[source\]](#)

```
lemma normalOrder_ofFieldOp_mul_ofFieldOp (φ φ' :  $\mathcal{F}$ .States) :  $\mathcal{N}(\text{ofFieldOp } \varphi * \text{ofFieldOp } \varphi') =$ 
   $\text{crPart } \varphi * \text{crPart } \varphi' + \mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \varphi') \cdot (\text{crPart } \varphi' * \text{anPart } \varphi) +$ 
   $\text{crPart } \varphi * \text{anPart } \varphi' + \text{anPart } \varphi * \text{anPart } \varphi'$ 
```

2.19 HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.StaticWickTheorem

[HepLean/PerturbationTheory/Algebras/FieldOpAlgebra/StaticWickTheorem.lean](#) on GitHub.

lemma static_wick_theorem_nil[\[source\]](#)

```
lemma static_wick_theorem_nil : ofFieldOpList [] =  $\sum (\varphi s \Lambda : \text{WickContraction []}.length),$ 
   $\varphi s \Lambda.\text{sign } (\mathcal{F} := \mathcal{F}) \cdot \varphi s \Lambda.\text{staticContract} * \mathcal{N}(\text{ofFieldOpList } [\varphi s \Lambda])^{u_c}$ 
```

theorem static_wick_theorem[\[source\]](#)

```
theorem static_wick_theorem : (φ s : List  $\mathcal{F}$ .States) →
  ofFieldOpList φ s =  $\sum (\varphi s \Lambda : \text{WickContraction } \varphi s.length),$ 
   $\varphi s \Lambda.\text{sign} \cdot \varphi s \Lambda.\text{staticContract} * \mathcal{N}(\text{ofFieldOpList } [\varphi s \Lambda])^{u_c}$ 
| [] => static_wick_theorem_nil
| φ :: φ s => by
  rw [ofFieldOpList_cons]
  rw [static_wick_theorem φ s]
  rw [show (φ :: φ s) = φ s.insertIdx ((0, Nat.zero_lt_succ φ s.length) : Fin φ s.length.succ) φ
    from rfl]
  conv_rhs => rw [insertLift_sum]
  rw [Finset.mul_sum]
  apply Finset.sum_congr rfl
  intro c _
```



```
trans (sign φs c • ιc.staticContract * (ofFieldOp φ * normalOrder (ofFieldOpList [c]uc)))
• have ht
```

2.20 HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.SuperCommute

[HepLean/PerturbationTheory/Algebras/FieldOpAlgebra/SuperCommute.lean](#) on GitHub.

lemma ι_superCommuteF_eq_zero_of_ι_right_zero

[\[source\]](#)

```
lemma ι_superCommuteF_eq_zero_of_ι_right_zero (a b :  $\mathcal{F}$ .CrAnAlgebra) (h : ι b = 0) :
  ι [a, b]sca = 0
```

lemma ι_superCommuteF_eq_zero_of_ι_left_zero

[\[source\]](#)

```
lemma ι_superCommuteF_eq_zero_of_ι_left_zero (a b :  $\mathcal{F}$ .CrAnAlgebra) (h : ι a = 0) :
  ι [a, b]sca = 0
```

lemma ι_superCommuteF_right_zero_of_mem_ideal

[\[source\]](#)

```
lemma ι_superCommuteF_right_zero_of_mem_ideal (a b :  $\mathcal{F}$ .CrAnAlgebra)
  (h : b ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet) : ι [a, b]sca = 0
```

lemma ι_superCommuteF_eq_of_equiv_right

[\[source\]](#)

```
lemma ι_superCommuteF_eq_of_equiv_right (a b1 b2 :  $\mathcal{F}$ .CrAnAlgebra) (h : b1 ≈ b2) :
  ι [a, b1]sca = ι [a, b2]sca
```

def superCommuteRight

[\[source\]](#)

The super commutator on the FieldOpAlgebra defined as a linear map $[a, _]_s$.

```
noncomputable def superCommuteRight (a :  $\mathcal{F}$ .CrAnAlgebra) :
  FieldOpAlgebra  $\mathcal{F} \rightarrow_{\iota} [\mathbb{C}]$  FieldOpAlgebra  $\mathcal{F}$  where
```

lemma superCommuteRight_apply_ι

[\[source\]](#)

```
lemma superCommuteRight_apply_ι (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  superCommuteRight a (ι b) = ι [a, b]sca
```

lemma superCommuteRight_apply_quot

[\[source\]](#)

```
lemma superCommuteRight_apply_quot (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  superCommuteRight a  $\llbracket b \rrbracket$  = ι [a, b]sca
```

lemma superCommuteRight_eq_of_equiv

[\[source\]](#)

```
lemma superCommuteRight_eq_of_equiv (a1 a2 :  $\mathcal{F}$ .CrAnAlgebra) (h : a1 ≈ a2) :
  superCommuteRight a1 = superCommuteRight a2
```

def superCommute[\[source\]](#)*The super commutor on the FieldOpAlgebra.*

```

noncomputable def superCommute : FieldOpAlgebra  $\mathcal{F}$   $\rightarrow_l$  [ $\mathbb{C}$ ]
  FieldOpAlgebra  $\mathcal{F} \rightarrow_l$  [ $\mathbb{C}$ ] FieldOpAlgebra  $\mathcal{F}$  where

```

lemma superCommute_eq_ι_superCommuteF[\[source\]](#)

```

lemma superCommute_eq_ι_superCommuteF (a b :  $\mathcal{F}$ .CrAnAlgebra) :
  [ι a, ι b]s = ι [a, b]sca

```

lemma superCommute_create_create[\[source\]](#)

```

lemma superCommute_create_create {φ φ' :  $\mathcal{F}$ .CrAnStates}
  (h :  $\mathcal{F} \mid >^c \varphi = \text{.create}$ ) (h' :  $\mathcal{F} \mid >^c \varphi' = \text{.create}$ ) :
  [ofCrAnFieldOp φ, ofCrAnFieldOp φ']s = 0

```

lemma superCommute_annihilate_annihilate[\[source\]](#)

```

lemma superCommute_annihilate_annihilate {φ φ' :  $\mathcal{F}$ .CrAnStates}
  (h :  $\mathcal{F} \mid >^c \varphi = \text{.annihilate}$ ) (h' :  $\mathcal{F} \mid >^c \varphi' = \text{.annihilate}$ ) :
  [ofCrAnFieldOp φ, ofCrAnFieldOp φ']s = 0

```

lemma superCommute_diff_statistic[\[source\]](#)

```

lemma superCommute_diff_statistic {φ φ' :  $\mathcal{F}$ .CrAnStates} (h : ( $\mathcal{F} \mid >_s \varphi \neq \mathcal{F} \mid >_s \varphi'$ ) :
  [ofCrAnFieldOp φ, ofCrAnFieldOp φ']s = 0

```

lemma superCommute_ofCrAnFieldOp_ofFieldOp_diff_stat_zero[\[source\]](#)

```

lemma superCommute_ofCrAnFieldOp_ofFieldOp_diff_stat_zero (φ :  $\mathcal{F}$ .CrAnStates) (ψ :  $\mathcal{F}$ .States)
  (h : ( $\mathcal{F} \mid >_s \varphi \neq \mathcal{F} \mid >_s \psi$ ) : [ofCrAnFieldOp φ, ofFieldOp ψ]s = 0

```

lemma superCommute_anPart_ofState_diff_grade_zero[\[source\]](#)

```

lemma superCommute_anPart_ofState_diff_grade_zero (φ ψ :  $\mathcal{F}$ .States)
  (h : ( $\mathcal{F} \mid >_s \varphi \neq \mathcal{F} \mid >_s \psi$ ) : [anPart φ, ofFieldOp ψ]s = 0

```

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp_mem_center[\[source\]](#)

```

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp_mem_center (φ φ' :  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnFieldOp φ, ofCrAnFieldOp φ']s ∈ Subalgebra.center  $\mathbb{C}$  (FieldOpAlgebra  $\mathcal{F}$ )

```

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp_commute[\[source\]](#)

```

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp_commute (φ φ' :  $\mathcal{F}$ .CrAnStates)
  (a : FieldOpAlgebra  $\mathcal{F}$ ) :
  a * [ofCrAnFieldOp φ, ofCrAnFieldOp φ']s = [ofCrAnFieldOp φ, ofCrAnFieldOp φ']s * a

```

lemma superCommute_ofCrAnFieldOp_ofFieldOp_mem_center[\[source\]](#)

lemma superCommute_ofCrAnFieldOp_ofFieldOp_mem_center ($\phi : \mathcal{F}.\text{CrAnStates}$) ($\phi' : \mathcal{F}.\text{States}$) :
 $[\text{ofCrAnFieldOp } \phi, \text{ofFieldOp } \phi']_s \in \text{Subalgebra.center } \mathbb{C} (\text{FieldOpAlgebra } \mathcal{F})$

lemma superCommute_ofCrAnFieldOp_ofFieldOp_commute[\[source\]](#)

lemma superCommute_ofCrAnFieldOp_ofFieldOp_commute ($\phi : \mathcal{F}.\text{CrAnStates}$) ($\phi' : \mathcal{F}.\text{States}$)
 $(a : \text{FieldOpAlgebra } \mathcal{F}) : a * [\text{ofCrAnFieldOp } \phi, \text{ofFieldOp } \phi']_s =$
 $[\text{ofCrAnFieldOp } \phi, \text{ofFieldOp } \phi']_s * a$

lemma superCommute_anPart_ofFieldOp_mem_center[\[source\]](#)

lemma superCommute_anPart_ofFieldOp_mem_center ($\phi \ \phi' : \mathcal{F}.\text{States}$) :
 $[\text{anPart } \phi, \text{ofFieldOp } \phi']_s \in \text{Subalgebra.center } \mathbb{C} (\text{FieldOpAlgebra } \mathcal{F})$

lemma superCommute_ofCrAnFieldOpList_ofCrAnFieldOpList[\[source\]](#)

lemma superCommute_ofCrAnFieldOpList_ofCrAnFieldOpList ($\phi s \ \phi s' : \text{List } \mathcal{F}.\text{CrAnStates}$) :
 $[\text{ofCrAnFieldOpList } \phi s, \text{ofCrAnFieldOpList } \phi s']_s =$
 $\text{ofCrAnFieldOpList } (\phi s ++ \phi s') - \mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s') \cdot \text{ofCrAnFieldOpList } (\phi s' ++ \phi s)$

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp[\[source\]](#)

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp ($\phi \ \phi' : \mathcal{F}.\text{CrAnStates}$) :
 $[\text{ofCrAnFieldOp } \phi, \text{ofCrAnFieldOp } \phi']_s = \text{ofCrAnFieldOp } \phi * \text{ofCrAnFieldOp } \phi' -$
 $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi') \cdot \text{ofCrAnFieldOp } \phi' * \text{ofCrAnFieldOp } \phi$

lemma superCommute_ofCrAnFieldOpList_ofFieldOpList[\[source\]](#)

lemma superCommute_ofCrAnFieldOpList_ofFieldOpList ($\phi cas : \text{List } \mathcal{F}.\text{CrAnStates}$)
 $(\phi s : \text{List } \mathcal{F}.\text{States}) :$
 $[\text{ofCrAnFieldOpList } \phi cas, \text{ofFieldOpList } \phi s]_s = \text{ofCrAnFieldOpList } \phi cas * \text{ofFieldOpList } \phi s -$
 $\mathcal{S}(\mathcal{F} \mid>_s \phi cas, \mathcal{F} \mid>_s \phi s) \cdot \text{ofFieldOpList } \phi s * \text{ofCrAnFieldOpList } \phi cas$

lemma superCommute_ofFieldOpList_ofFieldOpList[\[source\]](#)

lemma superCommute_ofFieldOpList_ofFieldOpList ($\phi s \ \phi s' : \text{List } \mathcal{F}.\text{States}$) :
 $[\text{ofFieldOpList } \phi s, \text{ofFieldOpList } \phi s']_s = \text{ofFieldOpList } \phi s * \text{ofFieldOpList } \phi s' -$
 $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s') \cdot \text{ofFieldOpList } \phi s' * \text{ofFieldOpList } \phi s$

lemma superCommute_ofFieldOp_ofFieldOpList[\[source\]](#)

lemma superCommute_ofFieldOp_ofFieldOpList ($\phi : \mathcal{F}.\text{States}$) ($\phi s : \text{List } \mathcal{F}.\text{States}$) :
 $[\text{ofFieldOp } \phi, \text{ofFieldOpList } \phi s]_s = \text{ofFieldOp } \phi * \text{ofFieldOpList } \phi s -$
 $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi s) \cdot \text{ofFieldOpList } \phi s * \text{ofFieldOp } \phi$

lemma superCommute_ofFieldOpList_ofFieldOp[\[source\]](#)

lemma superCommute_ofFieldOpList_ofFieldOp ($\phi s : \text{List } \mathcal{F}.\text{States}$) ($\phi : \mathcal{F}.\text{States}$) :
 $[\text{ofFieldOpList } \phi s, \text{ofFieldOp } \phi]_s = \text{ofFieldOpList } \phi s * \text{ofFieldOp } \phi -$
 $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi) \cdot \text{ofFieldOp } \phi * \text{ofFieldOpList } \phi s$

lemma superCommute_anPart_crPart[\[source\]](#)

```

lemma superCommute_anPart_crPart ( $\phi \ \phi' : \mathcal{F}.States$ ) :
  [anPart  $\phi$ , crPart  $\phi'$ ]s = anPart  $\phi$  * crPart  $\phi'$  -
  S( $\mathcal{F} \mid>_s \phi$ ,  $\mathcal{F} \mid>_s \phi'$ ) • crPart  $\phi'$  * anPart  $\phi$ 

```

lemma superCommute_crPart_anPart[\[source\]](#)

```

lemma superCommute_crPart_anPart ( $\phi \ \phi' : \mathcal{F}.States$ ) :
  [crPart  $\phi$ , anPart  $\phi'$ ]s = crPart  $\phi$  * anPart  $\phi'$  -
  S( $\mathcal{F} \mid>_s \phi$ ,  $\mathcal{F} \mid>_s \phi'$ ) • anPart  $\phi'$  * crPart  $\phi$ 

```

lemma superCommute_crPart_crPart[\[source\]](#)

```

@[simp]

```

```

lemma superCommute_crPart_crPart ( $\phi \ \phi' : \mathcal{F}.States$ ) : [crPart  $\phi$ , crPart  $\phi'$ ]s = 0

```

lemma superCommute_anPart_anPart[\[source\]](#)

```

@[simp]

```

```

lemma superCommute_anPart_anPart ( $\phi \ \phi' : \mathcal{F}.States$ ) : [anPart  $\phi$ , anPart  $\phi'$ ]s = 0

```

lemma superCommute_crPart_ofFieldOpList[\[source\]](#)

```

lemma superCommute_crPart_ofFieldOpList ( $\phi : \mathcal{F}.States$ ) ( $\phi s : List \ \mathcal{F}.States$ ) :
  [crPart  $\phi$ , ofFieldOpList  $\phi s$ ]s = crPart  $\phi$  * ofFieldOpList  $\phi s$  -
  S( $\mathcal{F} \mid>_s \phi$ ,  $\mathcal{F} \mid>_s \phi s$ ) • ofFieldOpList  $\phi s$  * crPart  $\phi$ 

```

lemma superCommute_anPart_ofFieldOpList[\[source\]](#)

```

lemma superCommute_anPart_ofFieldOpList ( $\phi : \mathcal{F}.States$ ) ( $\phi s : List \ \mathcal{F}.States$ ) :
  [anPart  $\phi$ , ofFieldOpList  $\phi s$ ]s = anPart  $\phi$  * ofFieldOpList  $\phi s$  -
  S( $\mathcal{F} \mid>_s \phi$ ,  $\mathcal{F} \mid>_s \phi s$ ) • ofFieldOpList  $\phi s$  * anPart  $\phi$ 

```

lemma superCommute_crPart_ofFieldOp[\[source\]](#)

```

lemma superCommute_crPart_ofFieldOp ( $\phi \ \phi' : \mathcal{F}.States$ ) :
  [crPart  $\phi$ , ofFieldOp  $\phi'$ ]s = crPart  $\phi$  * ofFieldOp  $\phi'$  -
  S( $\mathcal{F} \mid>_s \phi$ ,  $\mathcal{F} \mid>_s \phi'$ ) • ofFieldOp  $\phi'$  * crPart  $\phi$ 

```

lemma superCommute_anPart_ofFieldOp[\[source\]](#)

```

lemma superCommute_anPart_ofFieldOp ( $\phi \ \phi' : \mathcal{F}.States$ ) :
  [anPart  $\phi$ , ofFieldOp  $\phi'$ ]s = anPart  $\phi$  * ofFieldOp  $\phi'$  -
  S( $\mathcal{F} \mid>_s \phi$ ,  $\mathcal{F} \mid>_s \phi'$ ) • ofFieldOp  $\phi'$  * anPart  $\phi$ 

```

lemma ofCrAnFieldOpList_mul_ofCrAnFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnFieldOpList_mul_ofCrAnFieldOpList_eq_superCommute ( $\phi s \ \phi s' : List \ \mathcal{F}.CrAnStates$ ) :
  ofCrAnFieldOpList  $\phi s$  * ofCrAnFieldOpList  $\phi s'$  =
  S( $\mathcal{F} \mid>_s \phi s$ ,  $\mathcal{F} \mid>_s \phi s'$ ) • ofCrAnFieldOpList  $\phi s'$  * ofCrAnFieldOpList  $\phi s$ 
  + [ofCrAnFieldOpList  $\phi s$ , ofCrAnFieldOpList  $\phi s'$ ]s

```

lemma ofCrAnFieldOp_mul_ofCrAnFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnFieldOp_mul_ofCrAnFieldOpList_eq_superCommute ( $\phi$  :  $\mathcal{F}.$ CrAnStates)
  ( $\phi s'$  : List  $\mathcal{F}.$ CrAnStates) : ofCrAnFieldOp  $\phi$  * ofCrAnFieldOpList  $\phi s'$  =
   $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi s')$  • ofCrAnFieldOpList  $\phi s'$  * ofCrAnFieldOp  $\phi$ 
  + [ofCrAnFieldOp  $\phi$ , ofCrAnFieldOpList  $\phi s'$ ]s

```

lemma ofFieldOpList_mul_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofFieldOpList_mul_ofFieldOpList_eq_superCommute ( $\phi s \phi s'$  : List  $\mathcal{F}.$ States) :
  ofFieldOpList  $\phi s$  * ofFieldOpList  $\phi s'$  =
   $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s')$  • ofFieldOpList  $\phi s'$  * ofFieldOpList  $\phi s$ 
  + [ofFieldOpList  $\phi s$ , ofFieldOpList  $\phi s'$ ]s

```

lemma ofFieldOp_mul_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofFieldOp_mul_ofFieldOpList_eq_superCommute ( $\phi$  :  $\mathcal{F}.$ States) ( $\phi s'$  : List  $\mathcal{F}.$ States) :
  ofFieldOp  $\phi$  * ofFieldOpList  $\phi s'$  =  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi s')$  • ofFieldOpList  $\phi s'$  * ofFieldOp  $\phi$ 
  + [ofFieldOp  $\phi$ , ofFieldOpList  $\phi s'$ ]s

```

lemma ofFieldOp_mul_ofFieldOp_eq_superCommute[\[source\]](#)

```

lemma ofFieldOp_mul_ofFieldOp_eq_superCommute ( $\phi \phi'$  :  $\mathcal{F}.$ States) :
  ofFieldOp  $\phi$  * ofFieldOp  $\phi'$  =  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$  • ofFieldOp  $\phi'$  * ofFieldOp  $\phi$ 
  + [ofFieldOp  $\phi$ , ofFieldOp  $\phi'$ ]s

```

lemma ofFieldOpList_mul_ofFieldOp_eq_superCommute[\[source\]](#)

```

lemma ofFieldOpList_mul_ofFieldOp_eq_superCommute ( $\phi s$  : List  $\mathcal{F}.$ States) ( $\phi$  :  $\mathcal{F}.$ States) :
  ofFieldOpList  $\phi s$  * ofFieldOp  $\phi$  =  $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi)$  • ofFieldOp  $\phi$  * ofFieldOpList  $\phi s$ 
  + [ofFieldOpList  $\phi s$ , ofFieldOp  $\phi$ ]s

```

lemma ofCrAnFieldOpList_mul_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnFieldOpList_mul_ofFieldOpList_eq_superCommute ( $\phi s$  : List  $\mathcal{F}.$ CrAnStates)
  ( $\phi s'$  : List  $\mathcal{F}.$ States) : ofCrAnFieldOpList  $\phi s$  * ofFieldOpList  $\phi s'$  =
   $\mathcal{S}(\mathcal{F} \mid>_s \phi s, \mathcal{F} \mid>_s \phi s')$  • ofFieldOpList  $\phi s'$  * ofCrAnFieldOpList  $\phi s$ 
  + [ofCrAnFieldOpList  $\phi s$ , ofFieldOpList  $\phi s'$ ]s

```

lemma crPart_mul_anPart_eq_superCommute[\[source\]](#)

```

lemma crPart_mul_anPart_eq_superCommute ( $\phi \phi'$  :  $\mathcal{F}.$ States) :
  crPart  $\phi$  * anPart  $\phi'$  =  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$  • anPart  $\phi'$  * crPart  $\phi$ 
  + [crPart  $\phi$ , anPart  $\phi'$ ]s

```

lemma anPart_mul_crPart_eq_superCommute[\[source\]](#)

```

lemma anPart_mul_crPart_eq_superCommute ( $\phi \phi'$  :  $\mathcal{F}.$ States) :
  anPart  $\phi$  * crPart  $\phi'$  =  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$  • crPart  $\phi'$  * anPart  $\phi$ 
  + [anPart  $\phi$ , crPart  $\phi'$ ]s

```

lemma crPart_mul_crPart_swap[\[source\]](#)

```
lemma crPart_mul_crPart_swap (φ φ' :  $\mathcal{F}$ .States) :
  crPart φ * crPart φ' =  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$  • crPart φ' * crPart φ
```

lemma anPart_mul_anPart_swap[\[source\]](#)

```
lemma anPart_mul_anPart_swap (φ φ' :  $\mathcal{F}$ .States) :
  anPart φ * anPart φ' =  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$  • anPart φ' * anPart φ
```

lemma superCommute_ofCrAnFieldOpList_ofCrAnFieldOpList_symm[\[source\]](#)

```
lemma superCommute_ofCrAnFieldOpList_ofCrAnFieldOpList_symm (φs φs' : List  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnFieldOpList φs, ofCrAnFieldOpList φs']s =
  (-  $\mathcal{S}(\mathcal{F} \mid>_s \phi_s, \mathcal{F} \mid>_s \phi'_s)$ ) • [ofCrAnFieldOpList φs', ofCrAnFieldOpList φs]s
```

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp_symm[\[source\]](#)

```
lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOp_symm (φ φ' :  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnFieldOp φ, ofCrAnFieldOp φ']s =
  (-  $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi')$ ) • [ofCrAnFieldOp φ', ofCrAnFieldOp φ]s
```

lemma superCommute_ofCrAnFieldOpList_ofCrAnFieldOpList_eq_sum[\[source\]](#)

```
lemma superCommute_ofCrAnFieldOpList_ofCrAnFieldOpList_eq_sum (φs φs' : List  $\mathcal{F}$ .CrAnStates) :
  [ofCrAnFieldOpList φs, ofCrAnFieldOpList φs']s =
   $\sum (n : \text{Fin } \phi'_s.\text{length}), \mathcal{S}(\mathcal{F} \mid>_s \phi_s, \mathcal{F} \mid>_s \phi'_s.\text{take } n) \cdot$ 
  ofCrAnFieldOpList (φs'.take n) * [ofCrAnFieldOpList φs, ofCrAnFieldOp (φs'.get n)]s *
  ofCrAnFieldOpList (φs'.drop (n + 1))
```

lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOpList_eq_sum[\[source\]](#)

```
lemma superCommute_ofCrAnFieldOp_ofCrAnFieldOpList_eq_sum (φ :  $\mathcal{F}$ .CrAnStates)
  (φs' : List  $\mathcal{F}$ .CrAnStates) : [ofCrAnFieldOp φ, ofCrAnFieldOpList φs']s =
   $\sum (n : \text{Fin } \phi'_s.\text{length}), \mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi'_s.\text{take } n) \cdot$ 
  [ofCrAnFieldOp φ, ofCrAnFieldOp (φs'.get n)]s * ofCrAnFieldOpList (φs'.eraseIdx n)
```

lemma superCommute_ofCrAnFieldOpList_ofFieldOpList_eq_sum[\[source\]](#)

```
lemma superCommute_ofCrAnFieldOpList_ofFieldOpList_eq_sum (φs : List  $\mathcal{F}$ .CrAnStates)
  (φs' : List  $\mathcal{F}$ .States) : [ofCrAnFieldOpList φs, ofFieldOpList φs']s =
   $\sum (n : \text{Fin } \phi'_s.\text{length}), \mathcal{S}(\mathcal{F} \mid>_s \phi_s, \mathcal{F} \mid>_s \phi'_s.\text{take } n) \cdot$ 
  ofFieldOpList (φs'.take n) * [ofCrAnFieldOpList φs, ofFieldOp (φs'.get n)]s *
  ofFieldOpList (φs'.drop (n + 1))
```

lemma superCommute_ofCrAnFieldOp_ofFieldOpList_eq_sum[\[source\]](#)

```
lemma superCommute_ofCrAnFieldOp_ofFieldOpList_eq_sum (φ :  $\mathcal{F}$ .CrAnStates) (φs' : List  $\mathcal{F}$ .States) :
  :
  [ofCrAnFieldOp φ, ofFieldOpList φs']s =
   $\sum (n : \text{Fin } \phi'_s.\text{length}), \mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi'_s.\text{take } n) \cdot$ 
  [ofCrAnFieldOp φ, ofFieldOp (φs'.get n)]s * ofFieldOpList (φs'.eraseIdx n)
```

2.21 HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.TimeContraction

[HepLean/PerturbationTheory/Algebras/FieldOpAlgebra/TimeContraction.lean](#) on GitHub.

def timeContract [\[source\]](#)

The time contraction of two States as an element of $\mathcal{O}.A$ defined as their time ordering in the state algebra minus their normal ordering in the creation and annihilation algebra, both mapped to $\mathcal{O}.A$.

```
def timeContract (φ ψ : F.States) : F.FieldOpAlgebra
```

lemma timeContract_eq_smul [\[source\]](#)

```
lemma timeContract_eq_smul (φ ψ : F.States) : timeContract φ ψ =
  T(ofFieldOp φ * ofFieldOp ψ) + (-1 : ℂ) • N(ofFieldOp φ * ofFieldOp ψ)
```

lemma timeContract_of_timeOrderRel [\[source\]](#)

```
lemma timeContract_of_timeOrderRel (φ ψ : F.States) (h : timeOrderRel φ ψ) :
  timeContract φ ψ = [anPart φ, ofFieldOp ψ]s
```

lemma timeContract_of_not_timeOrderRel [\[source\]](#)

```
lemma timeContract_of_not_timeOrderRel (φ ψ : F.States) (h : ¬ timeOrderRel φ ψ) :
  timeContract φ ψ = S(F |>s φ, F |>s ψ) • timeContract ψ φ
```

lemma timeContract_mem_center [\[source\]](#)

```
lemma timeContract_mem_center (φ ψ : F.States) :
  timeContract φ ψ ∈ Subalgebra.center ℂ F.FieldOpAlgebra
```

lemma timeContract_zero_of_diff_grade [\[source\]](#)

```
lemma timeContract_zero_of_diff_grade (φ ψ : F.States) (h : (F |>s φ) ≠ (F |>s ψ)) :
  timeContract φ ψ = 0
```

2.22 HepLean.PerturbationTheory.Algebras.FieldOpAlgebra.TimeOrder

[HepLean/PerturbationTheory/Algebras/FieldOpAlgebra/TimeOrder.lean](#) on GitHub.

lemma ι_timeOrderF_superCommuteF_superCommuteF_eq_time_ofCrAnList [\[source\]](#)

```
lemma ι_timeOrderF_superCommuteF_superCommuteF_eq_time_ofCrAnList {φ1 φ2 φ3 : F.CrAnStates}
  (φs1 φs2 : List F.CrAnStates) (h :
    crAnTimeOrderRel φ1 φ2 ∧ crAnTimeOrderRel φ1 φ3 ∧
    crAnTimeOrderRel φ2 φ1 ∧ crAnTimeOrderRel φ2 φ3 ∧
    crAnTimeOrderRel φ3 φ1 ∧ crAnTimeOrderRel φ3 φ2) :
  ι Tf(ofCrAnList φs1 * [ofCrAnState φ1, [ofCrAnState φ2, ofCrAnState φ3]sca]sca *
    ofCrAnList φs2) = 0
```

lemma $\iota_{\text{timeOrderF_superCommutateF_superCommutateF_ofCrAnList}}$ [\[source\]](#)

```

lemma  $\iota_{\text{timeOrderF\_superCommutateF\_superCommutateF\_ofCrAnList}}$  { $\phi_1 \phi_2 \phi_3 : \mathcal{F}.\text{CrAnStates}$ }
  ( $\phi_{s1} \phi_{s2} : \text{List } \mathcal{F}.\text{CrAnStates}$ ) :
   $\iota \mathcal{T}^f(\text{ofCrAnList } \phi_{s1} * [\text{ofCrAnState } \phi_1, [\text{ofCrAnState } \phi_2, \text{ofCrAnState } \phi_3]_{sca}]_{sca} * \text{ofCrAnList } \phi_{s2})$ 
  = 0

```

lemma $\iota_{\text{timeOrderF_superCommutateF_superCommutateF}}$ [\[source\]](#)

```

@[simp]
lemma  $\iota_{\text{timeOrderF\_superCommutateF\_superCommutateF}}$  { $\phi_1 \phi_2 \phi_3 : \mathcal{F}.\text{CrAnStates}$ } (a b :  $\mathcal{F}.\text{CrAnAlgebra}$ ) :
  :
   $\iota \mathcal{T}^f(a * [\text{ofCrAnState } \phi_1, [\text{ofCrAnState } \phi_2, \text{ofCrAnState } \phi_3]_{sca}]_{sca} * b) = 0$ 

```

lemma $\iota_{\text{timeOrderF_superCommutateF_eq_time}}$ [\[source\]](#)

```

lemma  $\iota_{\text{timeOrderF\_superCommutateF\_eq\_time}}$  { $\phi \psi : \mathcal{F}.\text{CrAnStates}$ }
  ( $h\phi\psi : \text{crAnTimeOrderRel } \phi \psi$ ) ( $h\psi\phi : \text{crAnTimeOrderRel } \psi \phi$ ) (a b :  $\mathcal{F}.\text{CrAnAlgebra}$ ) :
   $\iota \mathcal{T}^f(a * [\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca} * b) =$ 
   $\iota ([\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca} * \mathcal{T}^f(a * b))$ 

```

lemma $\iota_{\text{timeOrderF_superCommutateF_neq_time}}$ [\[source\]](#)

```

lemma  $\iota_{\text{timeOrderF\_superCommutateF\_neq\_time}}$  { $\phi \psi : \mathcal{F}.\text{CrAnStates}$ }
  ( $h\phi\psi : \neg (\text{crAnTimeOrderRel } \phi \psi \wedge \text{crAnTimeOrderRel } \psi \phi)$ ) (a b :  $\mathcal{F}.\text{CrAnAlgebra}$ ) :
   $\iota \mathcal{T}^f(a * [\text{ofCrAnState } \phi, \text{ofCrAnState } \psi]_{sca} * b) = 0$ 

```

lemma $\iota_{\text{timeOrderF_zero_of_mem_ideal}}$ [\[source\]](#)

```

lemma  $\iota_{\text{timeOrderF\_zero\_of\_mem\_ideal}}$  (a :  $\mathcal{F}.\text{CrAnAlgebra}$ )
  (h : a ∈  $\text{TwoSidedIdeal.span } \mathcal{F}.\text{fieldOpIdealSet}$ ) :  $\iota \mathcal{T}^f(a) = 0$ 

```

lemma $\iota_{\text{timeOrderF_eq_of_equiv}}$ [\[source\]](#)

```

lemma  $\iota_{\text{timeOrderF\_eq\_of\_equiv}}$  (a b :  $\mathcal{F}.\text{CrAnAlgebra}$ ) (h : a ≈ b) :
   $\iota \mathcal{T}^f(a) = \iota \mathcal{T}^f(b)$ 

```

def timeOrder[\[source\]](#)

Time ordering on FieldOpAlgebra.

```

noncomputable def timeOrder :  $\text{FieldOpAlgebra } \mathcal{F} \rightarrow_l [\mathbb{C}] \text{FieldOpAlgebra } \mathcal{F}$  where

```

lemma timeOrder_eq_ι_timeOrderF[\[source\]](#)

```

lemma timeOrder_eq_ι_timeOrderF (a :  $\mathcal{F}.\text{CrAnAlgebra}$ ) :
   $\mathcal{T}(\iota a) = \iota \mathcal{T}^f(a)$ 

```

lemma timeOrder_ofFieldOp_ofFieldOp_ordered[\[source\]](#)

```

lemma timeOrder_ofFieldOp_ofFieldOp_ordered { $\phi \psi : \mathcal{F}.\text{States}$ } (h : timeOrderRel  $\phi \psi$ ) :
   $\mathcal{T}(\text{ofFieldOp } \phi * \text{ofFieldOp } \psi) = \text{ofFieldOp } \phi * \text{ofFieldOp } \psi$ 

```


lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered[\[source\]](#)

```
lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered {φ ψ : F.States} (h : ¬ timeOrderRel φ ψ) :
  T(ofFieldOp φ * ofFieldOp ψ) = S(F |>_s φ, F |>_s ψ) • ofFieldOp ψ * ofFieldOp φ
```

lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered_eq_timeOrder[\[source\]](#)

```
lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered_eq_timeOrder {φ ψ : F.States}
  (h : ¬ timeOrderRel φ ψ) :
  T(ofFieldOp φ * ofFieldOp ψ) = S(F |>_s φ, F |>_s ψ) • T(ofFieldOp ψ * ofFieldOp φ)
```

lemma timeOrder_ofFieldOpList_nil[\[source\]](#)

```
lemma timeOrder_ofFieldOpList_nil : T(ofFieldOpList (F := F) []) = 1
```

lemma timeOrder_ofFieldOpList_singleton[\[source\]](#)

```
@[simp]
lemma timeOrder_ofFieldOpList_singleton (φ : F.States) :
  T(ofFieldOpList [φ]) = ofFieldOpList [φ]
```

lemma timeOrder_eq_maxTimeField_mul_finset[\[source\]](#)

```
lemma timeOrder_eq_maxTimeField_mul_finset (φ : F.States) (φs : List F.States) :
  T(ofFieldOpList (φ :: φs)) = S(F |>_s maxTimeField φ φs, F |>_s ((eraseMaxTimeField φ φs).
  get,
  (Finset.filter (fun x =>
    (maxTimeFieldPosFin φ φs).succAbove x < maxTimeFieldPosFin φ φs) Finset.univ))) •
  ofFieldOp (maxTimeField φ φs) * T(ofFieldOpList (eraseMaxTimeField φ φs))
```

2.23 HepLean.PerturbationTheory.CreateAnnihilate

[HepLean/PerturbationTheory/CreateAnnihilate.lean](#) on GitHub.

inductive CreateAnnihilate[\[source\]](#)

The type specifying whether an operator is a creation or annihilation operator.

```
inductive CreateAnnihilate where
| create : CreateAnnihilate
| annihilate : CreateAnnihilate
```

instance Fintype CreateAnnihilate[\[source\]](#)

The type CreateAnnihilate is finite.

```
instance : Fintype CreateAnnihilate where
```

lemma eq_create_or_annihilate[\[source\]](#)

```
lemma eq_create_or_annihilate (φ : CreateAnnihilate) : φ = create ∨ φ = annihilate
```

def normalOrder[\[source\]](#)

The normal ordering on creation and annihilation operators. Under this relation, normalOrder a b is false only if a is annihilate and b is create.

```
def normalOrder : CreateAnnihilate → CreateAnnihilate → Prop
| create, _ => True
| annihilate, annihilate => True
| annihilate, create => False
```

instance (φ φ' : CreateAnnihilate) → Decidable (normalOrder φ φ') | create, crea...[\[source\]](#)

The normal ordering on CreateAnnihilate is decidable.

```
instance : (φ φ' : CreateAnnihilate) → Decidable (normalOrder φ φ')
| create, create => isTrue True.intro
| annihilate, annihilate => isTrue True.intro
| create, annihilate => isTrue True.intro
| annihilate, create => isFalse False.elim
```

instance IsTotal CreateAnnihilate normalOrder[\[source\]](#)

Normal ordering is total.

```
instance : IsTotal CreateAnnihilate normalOrder where
```

instance IsTrans CreateAnnihilate normalOrder[\[source\]](#)

Normal ordering is transitive.

```
instance : IsTrans CreateAnnihilate normalOrder where
```

lemma not_normalOrder_annihilate_iff_false[\[source\]](#)

```
@[simp]
lemma not_normalOrder_annihilate_iff_false (a : CreateAnnihilate) :
  (¬ normalOrder a annihilate) ↔ False
```

lemma sum_eq[\[source\]](#)

```
lemma sum_eq {M : Type} [AddCommMonoid M] (f : CreateAnnihilate → M) :
  ∑ i, f i = f create + f annihilate
```

lemma CreateAnnihilate_card_eq_two[\[source\]](#)

```
@[simp]
lemma CreateAnnihilate_card_eq_two : Fintype.card CreateAnnihilate = 2
```

2.24 HepLean.PerturbationTheory.FieldSpecification.Basic

[HepLean/PerturbationTheory/FieldSpecification/Basic.lean](#) on GitHub.

structure FieldSpecification[\[source\]](#)

A field specification is a type, $\mathcal{F}$, elements of which are fields present in a theory, and a map statistics from $\mathcal{F}$ to FieldStatistic which identifies each field as a boson or a fermion.

```
structure FieldSpecification where
  Fields : Type
  statistics : Fields → FieldStatistic
```

def IncomingAsymptotic[\[source\]](#)

An incoming asymptotic state is a field and a momentum.

```
def IncomingAsymptotic : Type
```

def OutgoingAsymptotic[\[source\]](#)

An outgoing asymptotic states is a field and a momentum.

```
def OutgoingAsymptotic : Type
```

def PositionStates[\[source\]](#)

A position state is a field and a space-time position.

```
def PositionStates : Type
```

inductive States[\[source\]](#)

The type States is the inductive type combining the asymptotic states and position states.

```
inductive States ( $\mathcal{F}$  : FieldSpecification) where
| inAsymp :  $\mathcal{F}.$ IncomingAsymptotic →  $\mathcal{F}.$ States
| position :  $\mathcal{F}.$ PositionStates →  $\mathcal{F}.$ States
| outAsymp :  $\mathcal{F}.$ OutgoingAsymptotic →  $\mathcal{F}.$ States
```

def statesToField[\[source\]](#)

Taking a state to its underlying field.

```
def statesToField :  $\mathcal{F}.$ States →  $\mathcal{F}.$ Fields
| States.inAsymp  $\phi$  =>  $\phi.1$ 
| States.position  $\phi$  =>  $\phi.1$ 
| States.outAsymp  $\phi$  =>  $\phi.1$ 
```

def statesIsPosition[\[source\]](#)

The bool on States which is true only for position states.

```
def statesIsPosition :  $\mathcal{F}.$ States → Bool
| States.position _ => true
| _ => false
```

def statesStatistic[\[source\]](#)

The statistics associated to a state.

```
def statesStatistic :  $\mathcal{F}$ .States → FieldStatistic
```

2.25 HepLean.PerturbationTheory.FieldSpecification.CrAnSection

[HepLean/PerturbationTheory/FieldSpecification/CrAnSection.lean](#) on GitHub.

def CrAnSection[\[source\]](#)

The sections in $\mathcal{F}$.CrAnStates over a list $\phi s : \text{List } \mathcal{F}.\text{States}$. In terms of physics, given some fields $\phi_1 \dots \phi_n$, the different ways one can associate each field as a creation or an annihilation operator. E.g. the number of terms $\phi_1^0 \phi_2^1 \dots \phi_n^0 \phi_1^1 \phi_2^1 \dots \phi_n^0$ etc. If some fields are exclusively creation or annihilation operators at this point (e.g. asymptotic states) this is accounted for.

```
def CrAnSection ( $\phi s : \text{List } \mathcal{F}.\text{States}$ ) : Type
```

lemma length_eq[\[source\]](#)

```
@[simp]
```

```
lemma length_eq ( $\psi s : \text{CrAnSection } \phi s$ ) :  $\psi s.1.\text{length} = \phi s.\text{length}$ 
```

def tail[\[source\]](#)

The tail of a section for ϕs .

```
def tail : { $\phi s : \text{List } \mathcal{F}.\text{States}$ } → ( $\psi s : \text{CrAnSection } \phi s$ ) → CrAnSection  $\phi s.\text{tail}$ 
| [], ([], h) => ([], h)
|  $\phi :: \phi s$ , ([], h) => False.elim (by simp at h)
|  $\phi :: \phi s$ , ( $\psi :: \psi s$ , h) => ( $\psi s$ , by rw [List.map_cons, List.cons.injEq] at h; exact h.2)
```

lemma head_state_eq[\[source\]](#)

```
lemma head_state_eq { $\phi : \mathcal{F}.\text{States}$ } : ( $\psi s : \text{CrAnSection } (\phi :: \phi s)$ ) →
  ( $\psi s.1.\text{head (by simp [← List.length_pos_iff_ne_nil])}.1 = \phi$ 
| ([], h) => False.elim (by simp at h)
| ( $\psi :: \psi s$ , h) => by
  simp only [List.map_cons, List.cons.injEq] at h
  exact h.1
```

lemma statistics_eq_state_statistics[\[source\]](#)

```
lemma statistics_eq_state_statistics ( $\psi s : \text{CrAnSection } \phi s$ ) :
  ( $\mathcal{F} \mid>_s \psi s.1$ ) =  $\mathcal{F} \mid>_s \phi s$ 
```

lemma take_statistics_eq_take_state_statistics[\[source\]](#)

```
lemma take_statistics_eq_take_state_statistics ( $\psi s : \text{CrAnSection } \phi s$ ) n :
  ( $\mathcal{F} \mid>_s (\psi s.1.\text{take } n)$ ) =  $\mathcal{F} \mid>_s (\phi s.\text{take } n)$ 
```

def head[\[source\]](#)

The head of a section for $\phi :: \phi s$ as an element in $\mathcal{F}.statesToCreateAnnihilateType$ ϕ .

```
def head : { $\phi$  :  $\mathcal{F}.States$ }  $\rightarrow$  ( $\psi s$  : CrAnSection ( $\phi :: \phi s$ ))  $\rightarrow$ 
   $\mathcal{F}.statesToCrAnType$   $\phi$ 
|  $\phi$ , ([], h)  $\Rightarrow$  False.elim (by simp at h)
|  $\phi$ , ( $\psi :: \psi s$ , h)  $\Rightarrow$   $\mathcal{F}.statesToCreateAnnihilateTypeCongr$  (by
  simp using head_state_eq ( $\psi :: \psi s$ , h))  $\psi.2$ 
```

lemma eq_head_cons_tail[\[source\]](#)

```
lemma eq_head_cons_tail { $\phi$  :  $\mathcal{F}.States$ } { $\psi s$  : CrAnSection ( $\phi :: \phi s$ )} :
   $\psi s.1 = (\phi, head \psi s) :: \psi s.tail.1$ 
```

def cons[\[source\]](#)

The creation of a section from for $\phi : \phi s$ from a section for ϕs and a element of $\mathcal{F}.statesToCreateAnnihilateType$ ϕ .

```
def cons { $\phi$  :  $\mathcal{F}.States$ } ( $\psi$  :  $\mathcal{F}.statesToCrAnType$   $\phi$ ) ( $\psi s$  : CrAnSection  $\phi s$ ) :
  CrAnSection ( $\phi :: \phi s$ )
```

def nilEquiv[\[source\]](#)

For the empty list of states there is only one CrAnSection. Corresponding to the empty list of CrAnStates.

```
def nilEquiv : CrAnSection ( $\mathcal{F} := \mathcal{F}$ ) []  $\approx$  Unit where
```

def singletonEquiv[\[source\]](#)

The creation and annihilation sections for a singleton list is given by a choice of $\mathcal{F}.statesToCreateAnnihilateType$ ϕ . If ϕ is a asymptotic state there is no choice here, else there are two choices.

```
def singletonEquiv { $\phi$  :  $\mathcal{F}.States$ } : CrAnSection [ $\phi$ ]  $\approx$ 
   $\mathcal{F}.statesToCrAnType$   $\phi$  where
```

def consEquiv[\[source\]](#)

An equivalence seperating the head of a creation and annihilation section from the tail.

```
def consEquiv { $\phi$  :  $\mathcal{F}.States$ } { $\phi s$  : List  $\mathcal{F}.States$ } : CrAnSection ( $\phi :: \phi s$ )  $\approx$ 
   $\mathcal{F}.statesToCrAnType$   $\phi \times$  CrAnSection  $\phi s$  where
```

instance fintype[\[source\]](#)

The instance of a finite type on CrAnSections defined recursively through consEquiv.

```
instance fintype : ( $\phi s$  : List  $\mathcal{F}.States$ )  $\rightarrow$  Fintype (CrAnSection  $\phi s$ )
| []  $\Rightarrow$  Fintype.ofEquiv _ nilEquiv.symm
| _ ::  $\phi s \Rightarrow$ 
```

```
haveI : Fintype (CrAnSection  $\phi$ s)
```

lemma card_nil_eq

[\[source\]](#)

```
@[simp]
```

```
lemma card_nil_eq : Fintype.card (CrAnSection ( $\mathcal{F} := \mathcal{F}$ ) []) = 1
```

lemma card_cons_eq

[\[source\]](#)

```
lemma card_cons_eq { $\phi$  :  $\mathcal{F}$ .States} { $\phi$ s : List  $\mathcal{F}$ .States} :  
  Fintype.card (CrAnSection ( $\phi :: \phi$ s)) = Fintype.card ( $\mathcal{F}$ .statesToCrAnType  $\phi$ ) *  
  Fintype.card (CrAnSection  $\phi$ s)
```

lemma card_eq_mul

[\[source\]](#)

```
lemma card_eq_mul : { $\phi$ s : List  $\mathcal{F}$ .States} → Fintype.card (CrAnSection  $\phi$ s) =  
  2 ^ (List.countP  $\mathcal{F}$ .statesIsPosition  $\phi$ s)  
| [] => by  
  simp  
| States.position _ ::  $\phi$ s => by  
  simp only [statesIsPosition, List.countP_cons_of_pos]  
  rw [card_cons_eq]  
  rw [card_eq_mul]  
  simp only [statesToCrAnType, CreateAnnihilate.CreateAnnihilate_card_eq_two]  
  ring  
| States.inAsymp x_ ::  $\phi$ s => by  
  simp only [statesIsPosition, Bool.false_eq_true, not_false_eq_true, List.  
    countP_cons_of_neg]  
  rw [card_cons_eq]  
  rw [card_eq_mul]  
  simp [statesToCrAnType]  
| States.outAsymp _ ::  $\phi$ s => by  
  simp only [statesIsPosition, Bool.false_eq_true, not_false_eq_true, List.  
    countP_cons_of_neg]  
  rw [card_cons_eq]  
  rw [card_eq_mul]  
  simp [statesToCrAnType]
```

lemma card_perm_eq

[\[source\]](#)

```
lemma card_perm_eq { $\phi$ s  $\phi$ s' : List  $\mathcal{F}$ .States} (h :  $\phi$ s.Perm  $\phi$ s') :  
  Fintype.card (CrAnSection  $\phi$ s) = Fintype.card (CrAnSection  $\phi$ s')
```

lemma sum_nil

[\[source\]](#)

```
@[simp]
```

```
lemma sum_nil (f : CrAnSection ( $\mathcal{F} := \mathcal{F}$ ) [] → M) [AddCommMonoid M] :  
   $\sum (s : CrAnSection []), f s = f []$ , rfl
```

lemma sum_cons

[\[source\]](#)

```
lemma sum_cons (f : CrAnSection ( $\phi :: \phi$ s) → M) [AddCommMonoid M] :  
   $\sum (s : CrAnSection (\phi :: \phi$ s)), f s =  $\sum (a : \mathcal{F}$ .statesToCrAnType  $\phi$ ),
```

$$\sum (s : \text{CrAnSection } \phi s), f (\text{cons } a \ s)$$
lemma sum_over_length[\[source\]](#)

```
lemma sum_over_length {s : CrAnSection  $\phi$ s} (f : Fin s.l.length  $\rightarrow$  M)
  [AddCommMonoid M] :  $\sum (n : \text{Fin } s.l.length), f \ n =$ 
 $\sum (n : \text{Fin } \phi s.length), f (\text{Fin.cast } (\text{length\_eq } s).symm \ n)$ 
```

def congr[\[source\]](#)

The equivalence between CrAnSection ϕs and CrAnSection $\phi s'$ induced by an equality $\phi s = \phi s'$.

```
def congr : { $\phi s \ \phi s' : \text{List } \mathcal{F}.\text{States}$ }  $\rightarrow$  (h :  $\phi s = \phi s'$ )  $\rightarrow$ 
  CrAnSection  $\phi s \approx$  CrAnSection  $\phi s'$ 
| _, _, rfl => Equiv.refl _
```

lemma congr_fst[\[source\]](#)

```
@[simp]
lemma congr_fst { $\phi s \ \phi s' : \text{List } \mathcal{F}.\text{States}$ } (h :  $\phi s = \phi s'$ ) ( $\psi s : \text{CrAnSection } \phi s$ ) :
  (congr h  $\psi s$ ).1 =  $\psi s$ .1
```

lemma congr_symm[\[source\]](#)

```
@[simp]
lemma congr_symm { $\phi s \ \phi s' : \text{List } \mathcal{F}.\text{States}$ } (h :  $\phi s = \phi s'$ ) :
  (congr h).symm = congr h.symm
```

lemma congr_trans_apply[\[source\]](#)

```
@[simp]
lemma congr_trans_apply { $\phi s \ \phi s' \ \phi s'' : \text{List } \mathcal{F}.\text{States}$ } (h1 :  $\phi s = \phi s'$ ) (h2 :  $\phi s' = \phi s''$ )
  ( $\psi s : \text{CrAnSection } \phi s$ ) :
  (congr h2 (congr h1  $\psi s$ )) = congr (by rw [h1, h2])  $\psi s$ 
```

def take[\[source\]](#)

Returns the first n elements of a section and its underlying list.

```
def take (n :  $\mathbb{N}$ ) ( $\psi s : \text{CrAnSection } \phi s$ ) : CrAnSection ( $\phi s$ .take n)
```

lemma take_congr[\[source\]](#)

```
@[simp]
lemma take_congr { $\phi s \ \phi s' : \text{List } \mathcal{F}.\text{States}$ } (h :  $\phi s = \phi s'$ ) (n :  $\mathbb{N}$ )
  ( $\psi s : \text{CrAnSection } \phi s$ ) :
  (take n (congr h  $\psi s$ )) = congr (by rw [h]) (take n  $\psi s$ )
```

def drop[\[source\]](#)

Removes the first n elements of a section and its underlying list.

```
def drop (n : ℕ) (ψs : CrAnSection φs) : CrAnSection (φs.drop n)
```

lemma drop_congr

[\[source\]](#)

```
@[simp]
lemma drop_congr {φs φs' : List ℱ.States} (h : φs = φs') (n : ℕ)
  (ψs : CrAnSection φs) :
  (drop n (congr h ψs)) = congr (by rw [h]) (drop n ψs)
```

def append

[\[source\]](#)

Appends two sections and their underlying lists.

```
def append {φs φs' : List ℱ.States} (ψs : CrAnSection φs)
  (ψs' : CrAnSection φs') : CrAnSection (φs ++ φs')
```

lemma append_assoc

[\[source\]](#)

```
lemma append_assoc {φs φs' φs'' : List ℱ.States} (ψs : CrAnSection φs)
  (ψs' : CrAnSection φs') (ψs'' : CrAnSection φs'') :
  append ψs (append ψs' ψs'') = congr (by simp) (append (append ψs ψs') ψs'')
```

lemma append_assoc'

[\[source\]](#)

```
lemma append_assoc' {φs φs' φs'' : List ℱ.States} (ψs : CrAnSection φs)
  (ψs' : CrAnSection φs') (ψs'' : CrAnSection φs'') :
  (append (append ψs ψs') ψs'') = congr (by simp) (append ψs (append ψs' ψs''))
```

lemma singletonEquiv_append_eq_cons

[\[source\]](#)

```
lemma singletonEquiv_append_eq_cons {φs : List ℱ.States} {φ : ℱ.States}
  (ψs : CrAnSection φs) (ψ : ℱ.statesToCrAnType φ) :
  append (singletonEquiv.symm ψ) ψs = cons ψ ψs
```

lemma take_append_drop

[\[source\]](#)

```
@[simp]
lemma take_append_drop {n : ℕ} (ψs : CrAnSection φs) :
  append (take n ψs) (drop n ψs) = congr (List.take_append_drop n φs).symm ψs
```

lemma congr_append

[\[source\]](#)

```
lemma congr_append {φs1 φs1' φs2 φs2' : List ℱ.States} (h1 : φs1 = φs1') (h2 : φs2 = φs2')
  (ψs1 : CrAnSection φs1) (ψs2 : CrAnSection φs2) :
  (append (congr h1 ψs1) (congr h2 ψs2)) = congr (by rw [h1, h2]) (append ψs1 ψs2)
```

lemma congr_fst_append

[\[source\]](#)

```
@[simp]
lemma congr_fst_append {φs1 φs1' φs2 : List ℱ.States} (h1 : φs1 = φs1')
  (ψs1 : CrAnSection φs1) (ψs2 : CrAnSection φs2) :
  (append (congr h1 ψs1) (ψs2)) = congr (by rw [h1]) (append ψs1 ψs2)
```


lemma congr_snd_append[\[source\]](#)

```
@[simp]
lemma congr_snd_append {φs1 φs2 φs2' : List ℱ.States} (h2 : φs2 = φs2')
  (ψs1 : CrAnSection φs1) (ψs2 : CrAnSection φs2) :
  (append ψs1 (congr h2 ψs2)) = congr (by rw [h2]) (append ψs1 ψs2)
```

lemma take_left[\[source\]](#)

```
@[simp]
lemma take_left {φs φs' : List ℱ.States} (ψs : CrAnSection φs)
  (ψs' : CrAnSection φs') :
  take φs.length (ψs.append ψs') = congr (by simp) ψs
```

lemma drop_left[\[source\]](#)

```
@[simp]
lemma drop_left {φs φs' : List ℱ.States} (ψs : CrAnSection φs)
  (ψs' : CrAnSection φs') :
  drop φs.length (ψs.append ψs') = congr (by simp) ψs'
```

def appendEquiv[\[source\]](#)

*The equivalence between $\text{CrAnSection } (\phi s ++ \phi s')$ and $\text{CrAnSection } \phi s \times \text{CrAnSection } \phi s'$ formed by *append*, *take* and *drop* and their interrelationship.*

```
def appendEquiv {φs φs' : List ℱ.States} : CrAnSection (φs ++ φs') ≈
  CrAnSection φs × CrAnSection φs' where
```

lemma map_eraseIdx[\[source\]](#)

```
@[simp]
lemma _root_.List.map_eraseIdx {α β : Type} (f : α → β) : (l : List α) → (n : ℕ) →
  List.map f (l.eraseIdx n) = (List.map f l).eraseIdx n
| [], _ => rfl
| a :: l, 0 => rfl
| a :: l, n+1 => by
  simp only [List.eraseIdx, List.map_cons, List.cons.injEq, true_and]
  exact List.map_eraseIdx f l n
```

def eraseIdx[\[source\]](#)

Erasing an element from a section and it's underlying list.

```
def eraseIdx (n : ℕ) (ψs : CrAnSection φs) : CrAnSection (φs.eraseIdx n)
```

def eraseIdxEquiv[\[source\]](#)

The equivalence formed by extracting an element from a section.

```
def eraseIdxEquiv (n : ℕ) (φs : List ℱ.States) (hn : n < φs.length) :
  CrAnSection φs ≈ ℱ.statesToCrAnType φs[n] ×
  CrAnSection (φs.eraseIdx n)
```

lemma eraseIdxEquiv_apply_snd[\[source\]](#)

```
@[simp]
lemma eraseIdxEquiv_apply_snd {n : ℕ} (ψs : CrAnSection ψs) (hn : n < ψs.length) :
  (eraseIdxEquiv n ψs hn ψs).snd = eraseIdx n ψs
```

lemma eraseIdxEquiv_symm_eq_take_cons_drop[\[source\]](#)

```
lemma eraseIdxEquiv_symm_eq_take_cons_drop {n : ℕ} (ψs : List ℱ.States) (hn : n < ψs.length)
  (a : ℱ.statesToCrAnType ψs[n]) (s : CrAnSection (ψs.eraseIdx n)) :
  (eraseIdxEquiv n ψs hn).symm {a, s} =
  congr (by
    rw [HepLean.List.take_eraseIdx_same, HepLean.List.drop_eraseIdx_succ]
    conv_rhs => rw [← List.take_append_drop n ψs]) (append (take n s) (cons a (drop n s)))
```

lemma eraseIdxEquiv_symm_getElem[\[source\]](#)

```
@[simp]
lemma eraseIdxEquiv_symm_getElem {n : ℕ} (ψs : List ℱ.States) (hn : n < ψs.length)
  (a : ℱ.statesToCrAnType ψs[n]) (s : CrAnSection (ψs.eraseIdx n)) :
  getElem ((eraseIdxEquiv n ψs hn).symm {a, s}).1 n
  (by rw [length_eq; exact hn] = {ψs[n], a})
```

lemma eraseIdxEquiv_symm_eraseIdx[\[source\]](#)

```
@[simp]
lemma eraseIdxEquiv_symm_eraseIdx {n : ℕ} (ψs : List ℱ.States) (hn : n < ψs.length)
  (a : ℱ.statesToCrAnType ψs[n]) (s : CrAnSection (ψs.eraseIdx n)) :
  ((eraseIdxEquiv n ψs hn).symm {a, s}).1.eraseIdx n = s.1
```

lemma sum_eraseIdxEquiv[\[source\]](#)

```
lemma sum_eraseIdxEquiv (n : ℕ) (ψs : List ℱ.States) (hn : n < ψs.length)
  (f : CrAnSection ψs → M) [AddCommMonoid M] : ∑ (s : CrAnSection ψs), f s =
  ∑ (a : ℱ.statesToCrAnType ψs[n]), ∑ (s : CrAnSection (ψs.eraseIdx n)),
  f ((eraseIdxEquiv n ψs hn).symm {a, s})
```

2.26 HepLean.PerturbationTheory.FieldSpecification.CrAnStates

[HepLean/PerturbationTheory/FieldSpecification/CrAnStates.lean](#) on GitHub.

def statesToCrAnType[\[source\]](#)

To each state the specifcication of the type of creation and annihilation parts. For asymptotic staes there is only one allowed part, whilst for position states there is two.

```
def statesToCrAnType : ℱ.States → Type
| States.inAsymp _ => Unit
| States.position _ => CreateAnnihilate
| States.outAsymp _ => Unit
```

instance $\forall$ i , Fintype ($\mathcal{F}.\text{statesToCrAnType } i$) [\[source\]](#)

The instance of a finite type on $\mathcal{F}.\text{statesToCreateAnnihilateType } i$.

```
instance :  $\forall$   $i$ , Fintype ( $\mathcal{F}.\text{statesToCrAnType } i$ )
```

instance $\forall$ i , DecidableEq ($\mathcal{F}.\text{statesToCrAnType } i$) [\[source\]](#)

The instance of a decidable equality on $\mathcal{F}.\text{statesToCreateAnnihilateType } i$.

```
instance :  $\forall$   $i$ , DecidableEq ( $\mathcal{F}.\text{statesToCrAnType } i$ )
```

def statesToCreateAnnihilateTypeCongr [\[source\]](#)

The equivalence between $\mathcal{F}.\text{statesToCreateAnnihilateType } i$ and $\mathcal{F}.\text{statesToCreateAnnihilateType } j$ from an equality $i = j$.

```
def statesToCreateAnnihilateTypeCongr : {i j :  $\mathcal{F}.\text{States}$ }  $\rightarrow$   $i = j \rightarrow$   
   $\mathcal{F}.\text{statesToCrAnType } i = \mathcal{F}.\text{statesToCrAnType } j$   
  | _, _, rfl => Equiv.refl _
```

def CrAnStates [\[source\]](#)

A creation and annihilation state is a state plus an valid specification of the creation or annihilation part of that state. (For asymptotic states there is only one valid choice).

```
def CrAnStates : Type
```

def crAnStatesToStates [\[source\]](#)

The map from creation and annihilation states to their underlying states.

```
def crAnStatesToStates :  $\mathcal{F}.\text{CrAnStates} \rightarrow \mathcal{F}.\text{States}$ 
```

lemma crAnStatesToStates_prod [\[source\]](#)

```
@[simp]  
lemma crAnStatesToStates_prod (s :  $\mathcal{F}.\text{States}$ ) (t :  $\mathcal{F}.\text{statesToCrAnType } s$ ) :  
   $\mathcal{F}.\text{crAnStatesToStates } (s, t) = s$ 
```

def crAnStatesToCreateAnnihilate [\[source\]](#)

The map from creation and annihilation states to the type CreateAnnihilate specifying if a state is a creation or an annihilation state.

```
def crAnStatesToCreateAnnihilate :  $\mathcal{F}.\text{CrAnStates} \rightarrow \text{CreateAnnihilate}$   
  | (States.inAsymp _, _) => CreateAnnihilate.create  
  | (States.position _, CreateAnnihilate.create) => CreateAnnihilate.create  
  | (States.position _, CreateAnnihilate.annihilate) => CreateAnnihilate.annihilate  
  | (States.outAsymp _, _) => CreateAnnihilate.annihilate
```

def crAnStatistics[\[source\]](#)

Takes a CrAnStates state to its corresponding fields statistic (bosonic or fermionic).

```
def crAnStatistics :  $\mathcal{F}$ .CrAnStates → FieldStatistic
```

2.27 HepLean.PerturbationTheory.FieldSpecification.Examples

[HepLean/PerturbationTheory/FieldSpecification/Examples.lean](#) on GitHub.

def singleBoson[\[source\]](#)

The Field specification corresponding to a single bosonic field. The type of fields is the unit type, and the statistic of the unique element of the unit type is bosonic.

```
def singleBoson : FieldSpecification where
```

def singleFermion[\[source\]](#)

The Field specification corresponding to a single fermionic field. The type of fields is the unit type, and the statistic of the unique element of the unit type is fermionic.

```
def singleFermion : FieldSpecification where
```

def doubleBosonDoubleFermion[\[source\]](#)

The Field specification corresponding to two bosonic fields and two fermionic fields. The type of fields is $\text{Fin } 2 \oplus \text{Fin } 2$, and the statistic of the two .inl (left) elements is bosonic and the statistic of the two .inr (right) elements is fermionic.

```
def doubleBosonDoubleFermion : FieldSpecification where
```

2.28 HepLean.PerturbationTheory.FieldSpecification.Filters

[HepLean/PerturbationTheory/FieldSpecification/Filters.lean](#) on GitHub.

def createFilter[\[source\]](#)

Given a list of creation and annihilation states, the filtered list only containing the creation states. As a schematic example, for the list: - [ϕ_{1c} , ϕ_{1a} , ϕ_{2c} , ϕ_{2a}] this will return [ϕ_{1c} , ϕ_{2c}].

```
def createFilter ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) : List  $\mathcal{F}$ .CrAnStates
```

lemma createFilter_cons_create[\[source\]](#)

```
lemma createFilter_cons_create { $\phi$  :  $\mathcal{F}$ .CrAnStates}
  (h $\phi$  :  $\mathcal{F}$  |>c  $\phi$  = CreateAnnihilate.create) ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) :
  createFilter ( $\phi$  ::  $\phi$ s) =  $\phi$  :: createFilter  $\phi$ s
```

lemma createFilter_cons_annihilate[\[source\]](#)

```
lemma createFilter_cons_annihilate {φ :  $\mathcal{F}$ .CrAnStates}
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.annihilate}$ ) (φs : List  $\mathcal{F}$ .CrAnStates) :
  createFilter (φ :: φs) = createFilter φs
```

lemma createFilter_append[\[source\]](#)

```
lemma createFilter_append (φs φs' : List  $\mathcal{F}$ .CrAnStates) :
  createFilter (φs ++ φs') = createFilter φs ++ createFilter φs'
```

lemma createFilter_singleton_create[\[source\]](#)

```
lemma createFilter_singleton_create (φ :  $\mathcal{F}$ .CrAnStates)
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.create}$ ) :
  createFilter [φ] = [φ]
```

lemma createFilter_singleton_annihilate[\[source\]](#)

```
lemma createFilter_singleton_annihilate (φ :  $\mathcal{F}$ .CrAnStates)
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.annihilate}$ ) : createFilter [φ] = []
```

def annihilateFilter[\[source\]](#)

Given a list of creation and annihilation states, the filtered list only containing the annihilation states. As a schematic example, for the list: - [φ_{1c} , φ_{1a} , φ_{2c} , φ_{2a}] this will return [φ_{1a} , φ_{2a}].

```
def annihilateFilter (φs : List  $\mathcal{F}$ .CrAnStates) : List  $\mathcal{F}$ .CrAnStates
```

lemma annihilateFilter_cons_create[\[source\]](#)

```
lemma annihilateFilter_cons_create {φ :  $\mathcal{F}$ .CrAnStates}
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.create}$ ) (φs : List  $\mathcal{F}$ .CrAnStates) :
  annihilateFilter (φ :: φs) = annihilateFilter φs
```

lemma annihilateFilter_cons_annihilate[\[source\]](#)

```
lemma annihilateFilter_cons_annihilate {φ :  $\mathcal{F}$ .CrAnStates}
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.annihilate}$ ) (φs : List  $\mathcal{F}$ .CrAnStates) :
  annihilateFilter (φ :: φs) = φ :: annihilateFilter φs
```

lemma annihilateFilter_append[\[source\]](#)

```
lemma annihilateFilter_append (φs φs' : List  $\mathcal{F}$ .CrAnStates) :
  annihilateFilter (φs ++ φs') = annihilateFilter φs ++ annihilateFilter φs'
```

lemma annihilateFilter_singleton_create[\[source\]](#)

```
lemma annihilateFilter_singleton_create (φ :  $\mathcal{F}$ .CrAnStates)
  (hφ :  $\mathcal{F} \mid >^c \varphi = \text{CreateAnnihilate.create}$ ) :
  annihilateFilter [φ] = []
```

lemma annihilateFilter_singleton_annihilate[\[source\]](#)

```

lemma annihilateFilter_singleton_annihilate ( $\phi$  :  $\mathcal{F}.\text{CrAnStates}$ )
  ( $h\phi$  :  $\mathcal{F} \mid >^c \phi = \text{CreateAnnihilate.annihilate}$ ) :
  annihilateFilter  $[\phi] = [\phi]$ 

```

2.29 HepLean.PerturbationTheory.FieldSpecification.NormalOrder

[HepLean/PerturbationTheory/FieldSpecification/NormalOrder.lean](#) on GitHub.

def normalOrderRel[\[source\]](#)

The normal ordering relation on creation and annihilation operators. For a list of creation and annihilation states, this relation is designed to move all creation states to the left, and all annihilation operators to the right. We have that $\text{normalOrderRel } \phi_1 \phi_2$ is true if - ϕ_1 is a creation operator or - ϕ_2 is an annihilate operator.

```

def normalOrderRel :  $\mathcal{F}.\text{CrAnStates} \rightarrow \mathcal{F}.\text{CrAnStates} \rightarrow \text{Prop}$ 

```

instance IsTotal $\mathcal{F}.\text{CrAnStates}$ $\mathcal{F}.\text{normalOrderRel}$ [\[source\]](#)

Normal ordering is total.

```

instance : IsTotal  $\mathcal{F}.\text{CrAnStates}$   $\mathcal{F}.\text{normalOrderRel}$  where

```

instance IsTrans $\mathcal{F}.\text{CrAnStates}$ $\mathcal{F}.\text{normalOrderRel}$ [\[source\]](#)

Normal ordering is transitive.

```

instance : IsTrans  $\mathcal{F}.\text{CrAnStates}$   $\mathcal{F}.\text{normalOrderRel}$  where

```

instance Decidable (normalOrderRel $\phi \phi'$)[\[source\]](#)

A decidable instance on the normal ordering relation.

```

instance ( $\phi \phi'$  :  $\mathcal{F}.\text{CrAnStates}$ ) : Decidable (normalOrderRel  $\phi \phi'$ )

```

def normalOrderSign[\[source\]](#)

The sign associated with putting a list of creation and annihilation states into normal order (with the creation operators on the left). We pick up a minus sign for every fermion paired crossed.

```

def normalOrderSign ( $\phi s$  : List  $\mathcal{F}.\text{CrAnStates}$ ) :  $\mathbb{C}$ 

```

lemma normalOrderSign_mul_self[\[source\]](#)

```

@[simp]
lemma normalOrderSign_mul_self ( $\phi s$  : List  $\mathcal{F}.\text{CrAnStates}$ ) :
  normalOrderSign  $\phi s$  * normalOrderSign  $\phi s$  = 1

```

lemma koszulSignInsert_create[\[source\]](#)

```

lemma koszulSignInsert_create ( $\phi$  :  $\mathcal{F}.\text{CrAnStates}$ )
  ( $h\phi$  :  $\mathcal{F} \mid^c \phi = \text{CreateAnnihilate.create}$ ) : ( $\phi s$  : List  $\mathcal{F}.\text{CrAnStates}$ ) →
  Wick.koszulSignInsert  $\mathcal{F}.\text{crAnStatistics}$  normalOrderRel  $\phi$   $\phi s$  = 1
| [] => rfl
|  $\phi' :: \phi s$  => by
  dsimp only [Wick.koszulSignInsert]
  rw [if_pos]
  · exact koszulSignInsert_create  $\phi$   $h\phi$   $\phi s$ 
  · dsimp only [normalOrderRel]
    rw [ $h\phi$ ]
    dsimp [CreateAnnihilate.normalOrder]

```

lemma normalOrderSign_cons_create[\[source\]](#)

```

lemma normalOrderSign_cons_create ( $\phi$  :  $\mathcal{F}.\text{CrAnStates}$ )
  ( $h\phi$  :  $\mathcal{F} \mid^c \phi = \text{CreateAnnihilate.create}$ ) ( $\phi s$  : List  $\mathcal{F}.\text{CrAnStates}$ ) :
  normalOrderSign ( $\phi :: \phi s$ ) = normalOrderSign  $\phi s$ 

```

lemma normalOrderSign_singleton[\[source\]](#)

```

@[simp]
lemma normalOrderSign_singleton ( $\phi$  :  $\mathcal{F}.\text{CrAnStates}$ ) : normalOrderSign [ $\phi$ ] = 1

```

lemma normalOrderSign_nil[\[source\]](#)

```

@[simp]
lemma normalOrderSign_nil : normalOrderSign ( $\mathcal{F} := \mathcal{F}$ ) [] = 1

```

lemma koszulSignInsert_append_annihilate[\[source\]](#)

```

lemma koszulSignInsert_append_annihilate ( $\phi' \phi$  :  $\mathcal{F}.\text{CrAnStates}$ )
  ( $h\phi$  :  $\mathcal{F} \mid^c \phi = \text{CreateAnnihilate.annihilate}$ ) :
  ( $\phi s$  : List  $\mathcal{F}.\text{CrAnStates}$ ) →
  Wick.koszulSignInsert  $\mathcal{F}.\text{crAnStatistics}$  normalOrderRel  $\phi'$  ( $\phi s ++ [\phi]$ ) =
  Wick.koszulSignInsert  $\mathcal{F}.\text{crAnStatistics}$  normalOrderRel  $\phi'$   $\phi s$ 
| [] => by
  simp only [Wick.koszulSignInsert, normalOrderRel,  $h\phi$ , ite_eq_left_iff,
    CreateAnnihilate.not_normalOrder_annihilate_iff_false, ite_eq_right_iff, and_imp,
    IsEmpty.forall_iff]
|  $\phi' :: \phi s$  => by
  dsimp only [List.cons_append, Wick.koszulSignInsert]
  rw [koszulSignInsert_append_annihilate  $\phi' \phi$   $h\phi$   $\phi s$ ]

```

lemma normalOrderSign_append_annihilate[\[source\]](#)

```

lemma normalOrderSign_append_annihilate ( $\phi$  :  $\mathcal{F}.\text{CrAnStates}$ )
  ( $h\phi$  :  $\mathcal{F} \mid^c \phi = \text{CreateAnnihilate.annihilate}$ ) :
  ( $\phi s$  : List  $\mathcal{F}.\text{CrAnStates}$ ) →
  normalOrderSign ( $\phi s ++ [\phi]$ ) = normalOrderSign  $\phi s$ 
| [] => by simp
|  $\phi' :: \phi s$  => by
  dsimp only [List.cons_append, normalOrderSign, Wick.koszulSign]
  have hi

```

lemma koszulSignInsert_annihilate_cons_create[\[source\]](#)

```

lemma koszulSignInsert_annihilate_cons_create (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi c = \text{CreateAnnihilate.create}$ )
  (hφa :  $\mathcal{F} \mid^{>c} \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (φs : List  $\mathcal{F}$ .CrAnStates) :
  Wick.koszulSignInsert  $\mathcal{F}$ .crAnStatistics normalOrderRel φa (φc :: φs)
  = FieldStatistic.exchangeSign ( $\mathcal{F}$ .crAnStatistics φc) ( $\mathcal{F}$ .crAnStatistics φa) *
  Wick.koszulSignInsert  $\mathcal{F}$ .crAnStatistics normalOrderRel φa φs

```

lemma normalOrderSign_swap_create_annihlate_fst[\[source\]](#)

```

lemma normalOrderSign_swap_create_annihlate_fst (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi c = \text{CreateAnnihilate.create}$ )
  (hφa :  $\mathcal{F} \mid^{>c} \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (φs : List  $\mathcal{F}$ .CrAnStates) :
  normalOrderSign (φc :: φa :: φs) =
  FieldStatistic.exchangeSign ( $\mathcal{F}$ .crAnStatistics φc) ( $\mathcal{F}$ .crAnStatistics φa) *
  normalOrderSign (φa :: φc :: φs)

```

lemma koszulSignInsert_swap[\[source\]](#)

```

lemma koszulSignInsert_swap (φ φc φa :  $\mathcal{F}$ .CrAnStates) (φs φs' : List  $\mathcal{F}$ .CrAnStates) :
  Wick.koszulSignInsert  $\mathcal{F}$ .crAnStatistics normalOrderRel φ (φs ++ φa :: φc :: φs')
  = Wick.koszulSignInsert  $\mathcal{F}$ .crAnStatistics normalOrderRel φ (φs ++ φc :: φa :: φs')

```

lemma normalOrderSign_swap_create_annihlate[\[source\]](#)

```

lemma normalOrderSign_swap_create_annihlate (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi c = \text{CreateAnnihilate.create}$ ) (hφa :  $\mathcal{F} \mid^{>c} \varphi a = \text{CreateAnnihilate.annihilate}$ )
  :
  (φs φs' : List  $\mathcal{F}$ .CrAnStates) → normalOrderSign (φs ++ φc :: φa :: φs') =
  FieldStatistic.exchangeSign ( $\mathcal{F}$ .crAnStatistics φc) ( $\mathcal{F}$ .crAnStatistics φa) *
  normalOrderSign (φs ++ φa :: φc :: φs')
| [], φs' => normalOrderSign_swap_create_annihlate_fst φc φa hφc hφa φs'
| φ :: φs, φs' => by
  rw [normalOrderSign]
  dsimp only [List.cons_append, Wick.koszulSign, FieldStatistic.instCommGroup.eq_1]
  rw [← normalOrderSign, normalOrderSign_swap_create_annihlate φc φa hφc hφa φs φs']
  rw [← mul_assoc, mul_comm _ (FieldStatistic.exchangeSign _ _), mul_assoc]
  simp only [FieldStatistic.instCommGroup.eq_1, mul_eq_mul_left_iff]
  apply Or.inl
  conv_rhs => rw [normalOrderSign, Wick.koszulSign, ← normalOrderSign]
  simp only [mul_eq_mul_right_iff]
  left
  rw [koszulSignInsert_swap]

```

lemma normalOrderSign_swap_create_create_fst[\[source\]](#)

```

lemma normalOrderSign_swap_create_create_fst (φc φc' :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid^{>c} \varphi c = \text{CreateAnnihilate.create}$ ) (hφc' :  $\mathcal{F} \mid^{>c} \varphi c' = \text{CreateAnnihilate.create}$ )
  (φs : List  $\mathcal{F}$ .CrAnStates) :
  normalOrderSign (φc :: φc' :: φs) = normalOrderSign (φc' :: φc :: φs)

```


lemma normalOrderSign_swap_create_create[\[source\]](#)

```

lemma normalOrderSign_swap_create_create (φc φc' :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid >^c \varphi c = \text{CreateAnnihilate.create}$ ) (hφc' :  $\mathcal{F} \mid >^c \varphi c' = \text{CreateAnnihilate.create}$ ) :
  (φs φs' : List  $\mathcal{F}$ .CrAnStates) →
  normalOrderSign (φs ++ φc :: φc' :: φs') = normalOrderSign (φs ++ φc' :: φc :: φs')
| [], φs' => by
  exact normalOrderSign_swap_create_create_fst φc φc' hφc hφc' φs'
| φ :: φs, φs' => by
  rw [normalOrderSign]
  dsimp only [List.cons_append, Wick.koszulSign]
  rw [← normalOrderSign, normalOrderSign_swap_create_create φc φc' hφc hφc']
  dsimp only [normalOrderSign, Wick.koszulSign]
  rw [← normalOrderSign]
  simp only [mul_eq_mul_right_iff]
  apply Or.inl (Wick.koszulSignInsert_eq_perm _ _ _ _ _)
  exact List.Perm.append_left φs (List.Perm.swap φc' φc φs')

```

lemma normalOrderSign_swap_annihilate_annihilate_fst[\[source\]](#)

```

lemma normalOrderSign_swap_annihilate_annihilate_fst (φa φa' :  $\mathcal{F}$ .CrAnStates)
  (hφa :  $\mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (hφa' :  $\mathcal{F} \mid >^c \varphi a' = \text{CreateAnnihilate.annihilate}$ )
  (φs : List  $\mathcal{F}$ .CrAnStates) :
  normalOrderSign (φa :: φa' :: φs) =
  normalOrderSign (φa' :: φa :: φs)

```

lemma normalOrderSign_swap_annihilate_annihilate[\[source\]](#)

```

lemma normalOrderSign_swap_annihilate_annihilate (φa φa' :  $\mathcal{F}$ .CrAnStates)
  (hφa :  $\mathcal{F} \mid >^c \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (hφa' :  $\mathcal{F} \mid >^c \varphi a' = \text{CreateAnnihilate.annihilate}$ ) : (φs φs' : List  $\mathcal{F}$ .CrAnStates) →
  normalOrderSign (φs ++ φa :: φa' :: φs') = normalOrderSign (φs ++ φa' :: φa :: φs')
| [], φs' => by
  exact normalOrderSign_swap_annihilate_annihilate_fst φa φa' hφa hφa' φs'
| φ :: φs, φs' => by
  rw [normalOrderSign]
  dsimp only [List.cons_append, Wick.koszulSign]
  rw [← normalOrderSign]
  rw [normalOrderSign_swap_annihilate_annihilate φa φa' hφa hφa']
  dsimp only [normalOrderSign, Wick.koszulSign]
  rw [← normalOrderSign]
  simp only [mul_eq_mul_right_iff]
  apply Or.inl
  apply Wick.koszulSignInsert_eq_perm
  refine List.Perm.append_left φs ?h.h.a
  exact List.Perm.swap φa' φa φs'
open FieldStatistic

```

def normalOrderList[\[source\]](#)

The normal ordering of a list of creation and annihilation states. To give some schematic. For example: - normalOrderList [φ1c, φ1a, φ2c, φ2a] = [φ1c, φ2c, φ1a, φ2a]

```

def normalOrderList (φs : List  $\mathcal{F}$ .CrAnStates) : List  $\mathcal{F}$ .CrAnStates

```

lemma normalOrderList_nil[\[source\]](#)

```
@[simp]
lemma normalOrderList_nil : normalOrderList ( $\mathcal{F} := \mathcal{F}$ ) [] = []
```

lemma normalOrderList_statistics[\[source\]](#)

```
@[simp]
lemma normalOrderList_statistics ( $\varphi$ s : List  $\mathcal{F}$ .CrAnStates) :
  ( $\mathcal{F} \mid>_s$  (normalOrderList  $\varphi$ s)) =  $\mathcal{F} \mid>_s$   $\varphi$ s
```

lemma orderedInsert_create[\[source\]](#)

```
lemma orderedInsert_create ( $\varphi$  :  $\mathcal{F}$ .CrAnStates)
  ( $h\varphi$  :  $\mathcal{F} \mid>^c \varphi$  = CreateAnnihilate.create) :
  ( $\varphi$ s : List  $\mathcal{F}$ .CrAnStates) → List.orderedInsert normalOrderRel  $\varphi$   $\varphi$ s =  $\varphi$  ::  $\varphi$ s
| [] => rfl
|  $\varphi'$  ::  $\varphi$ s => by
  dsimp only [List.orderedInsert.eq_2]
  rw [if_pos]
  dsimp only [normalOrderRel]
  rw [h $\varphi$ ]
  dsimp [CreateAnnihilate.normalOrder]
```

lemma normalOrderList_cons_create[\[source\]](#)

```
lemma normalOrderList_cons_create ( $\varphi$  :  $\mathcal{F}$ .CrAnStates)
  ( $h\varphi$  :  $\mathcal{F} \mid>^c \varphi$  = CreateAnnihilate.create) ( $\varphi$ s : List  $\mathcal{F}$ .CrAnStates) :
  normalOrderList ( $\varphi$  ::  $\varphi$ s) =  $\varphi$  :: normalOrderList  $\varphi$ s
```

lemma orderedInsert_append_annihilate[\[source\]](#)

```
lemma orderedInsert_append_annihilate ( $\varphi'$   $\varphi$  :  $\mathcal{F}$ .CrAnStates)
  ( $h\varphi$  :  $\mathcal{F} \mid>^c \varphi$  = CreateAnnihilate.annihilate) :
  ( $\varphi$ s : List  $\mathcal{F}$ .CrAnStates) → List.orderedInsert normalOrderRel  $\varphi'$  ( $\varphi$ s ++ [ $\varphi$ ]) =
  List.orderedInsert normalOrderRel  $\varphi'$   $\varphi$ s ++ [ $\varphi$ ]
| [] => by
  simp [Wick.koszulSignInsert, normalOrderRel, h $\varphi$ ]
|  $\varphi''$  ::  $\varphi$ s => by
  dsimp only [List.cons_append, List.orderedInsert.eq_2]
  have hi
```

lemma normalOrderList_append_annihilate[\[source\]](#)

```
lemma normalOrderList_append_annihilate ( $\varphi$  :  $\mathcal{F}$ .CrAnStates)
  ( $h\varphi$  :  $\mathcal{F} \mid>^c \varphi$  = CreateAnnihilate.annihilate) :
  ( $\varphi$ s : List  $\mathcal{F}$ .CrAnStates) →
  normalOrderList ( $\varphi$ s ++ [ $\varphi$ ]) = normalOrderList  $\varphi$ s ++ [ $\varphi$ ]
| [] => by simp [normalOrderList]
|  $\varphi'$  ::  $\varphi$ s => by
  simp only [normalOrderList, List.insertionSort, List.append_eq]
  have hi
```

lemma normalOrder_swap_create_annihlate_fst[\[source\]](#)

```

lemma normalOrder_swap_create_annihlate_fst (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid^{>^c} \varphi c = \text{CreateAnnihilate.create}$ )
  (hφa :  $\mathcal{F} \mid^{>^c} \varphi a = \text{CreateAnnihilate.annihilate}$ )
  (φs : List  $\mathcal{F}$ .CrAnStates) :
  normalOrderList (φc :: φa :: φs) = normalOrderList (φa :: φc :: φs)

```

lemma normalOrderList_swap_create_annihlate[\[source\]](#)

```

lemma normalOrderList_swap_create_annihlate (φc φa :  $\mathcal{F}$ .CrAnStates)
  (hφc :  $\mathcal{F} \mid^{>^c} \varphi c = \text{CreateAnnihilate.create}$ )
  (hφa :  $\mathcal{F} \mid^{>^c} \varphi a = \text{CreateAnnihilate.annihilate}$ ) :
  (φs φs' : List  $\mathcal{F}$ .CrAnStates) →
  normalOrderList (φs ++ φc :: φa :: φs') = normalOrderList (φs ++ φa :: φc :: φs')
| [], φs' => normalOrder_swap_create_annihlate_fst φc φa hφc hφa φs'
| φ :: φs, φs' => by
  dsimp only [List.cons_append, normalOrderList, List.insertionSort]
  have hi

```

def normalOrderEquiv[\[source\]](#)

For a list of creation and annihilation states, the equivalence between $\text{Fin } \varphi s.\text{length}$ and $\text{Fin } (\text{normalOrderList } \varphi s).\text{length}$ taking each position in φs to it's corresponding position in the normal ordered list. This assumes that we are using the insertion sort method. For example: - For $[\varphi 1c, \varphi 1a, \varphi 2c, \varphi 2a]$ this equivalence sends $0 \mapsto 0, 1 \mapsto 2, 2 \mapsto 1, 3 \mapsto 3$.

```

def normalOrderEquiv {φs : List  $\mathcal{F}$ .CrAnStates} : Fin φs.length ≈ Fin (normalOrderList φs).length

```

lemma sum_normalOrderList_length[\[source\]](#)

```

lemma sum_normalOrderList_length {M : Type} [AddCommMonoid M]
  (φs : List  $\mathcal{F}$ .CrAnStates) (f : Fin (normalOrderList φs).length → M) :
  ∑ (n : Fin (normalOrderList φs).length), f n = ∑ (n : Fin φs.length), f (normalOrderEquiv n)

```

lemma normalOrderList_get_normalOrderEquiv[\[source\]](#)

```

@[simp]
lemma normalOrderList_get_normalOrderEquiv {φs : List  $\mathcal{F}$ .CrAnStates} (n : Fin φs.length) :
  (normalOrderList φs)[(normalOrderEquiv n).val] = φs[n.val]

```

lemma normalOrderList_eraseIdx_normalOrderEquiv[\[source\]](#)

```

@[simp]
lemma normalOrderList_eraseIdx_normalOrderEquiv {φs : List  $\mathcal{F}$ .CrAnStates} (n : Fin φs.length) :
  (normalOrderList φs).eraseIdx (normalOrderEquiv n).val =
  normalOrderList (φs.eraseIdx n.val)

```

lemma normalOrderSign_eraseIdx[\[source\]](#)

```

lemma normalOrderSign_eraseIdx (φs : List  $\mathcal{F}$ .CrAnStates) (n : Fin φs.length) :
  normalOrderSign (φs.eraseIdx n) = normalOrderSign φs *
   $\mathcal{S}(\mathcal{F} \mid_{>s} (\phi s.get\ n), \mathcal{F} \mid_{>s} (\phi s.take\ n)) *$ 
   $\mathcal{S}(\mathcal{F} \mid_{>s} (\phi s.get\ n), \mathcal{F} \mid_{>s} ((normalOrderList\ \phi s).take\ (normalOrderEquiv\ n)))$ 

```

lemma orderedInsert_createFilter_append_annihilate

[\[source\]](#)

```

lemma orderedInsert_createFilter_append_annihilate (φ :  $\mathcal{F}$ .CrAnStates)
  (hφ :  $\mathcal{F} \mid_{>^c} \phi = CreateAnnihilate.annihilate$ ) : (φs φs' : List  $\mathcal{F}$ .CrAnStates) →
  List.orderedInsert normalOrderRel φ (createFilter φs ++ φs') =
  createFilter φs ++ List.orderedInsert normalOrderRel φ φs'
| [], φs' => by simp [createFilter]
| φ' :: φs, φs' => by
  rcases CreateAnnihilate.eq_create_or_annihilate ( $\mathcal{F} \mid_{>^c} \phi'$ ) with hφ' | hφ'
  · rw [createFilter_cons_create hφ']
    dsimp only [List.cons_append, List.orderedInsert.eq_2]
    rw [if_neg, orderedInsert_createFilter_append_annihilate φ hφ φs φs']
    simp [normalOrderRel, hφ, hφ', CreateAnnihilate.normalOrder]
  · rw [createFilter_cons_annihilate hφ', orderedInsert_createFilter_append_annihilate φ hφ
    φs]

```

lemma orderedInsert_annihilateFilter

[\[source\]](#)

```

lemma orderedInsert_annihilateFilter (φ :  $\mathcal{F}$ .CrAnStates) : (φs : List  $\mathcal{F}$ .CrAnStates) →
  List.orderedInsert normalOrderRel φ (annihilateFilter φs) =
  φ :: annihilateFilter φs
| [] => by simp [annihilateFilter]
| φ' :: φs => by
  rcases CreateAnnihilate.eq_create_or_annihilate ( $\mathcal{F} \mid_{>^c} \phi'$ ) with hφ' | hφ'
  · rw [annihilateFilter_cons_create hφ', orderedInsert_annihilateFilter φ φs]
  · rw [annihilateFilter_cons_annihilate hφ']
    dsimp only [List.orderedInsert.eq_2]
    rw [if_pos]
    dsimp only [normalOrderRel]
    rw [hφ']
    rcases CreateAnnihilate.eq_create_or_annihilate ( $\mathcal{F} \mid_{>^c} \phi$ ) with hφ | hφ
    · rw [hφ]
      simp only [CreateAnnihilate.normalOrder]
    · rw [hφ]
      simp [CreateAnnihilate.normalOrder]

```

lemma orderedInsert_createFilter_append_annihilateFilter_annihilate

[\[source\]](#)

```

lemma orderedInsert_createFilter_append_annihilateFilter_annihilate (φ :  $\mathcal{F}$ .CrAnStates)
  (hφ :  $\mathcal{F} \mid_{>^c} \phi = CreateAnnihilate.annihilate$ ) (φs : List  $\mathcal{F}$ .CrAnStates) :
  List.orderedInsert normalOrderRel φ (createFilter φs ++ annihilateFilter φs) =
  createFilter φs ++ φ :: annihilateFilter φs

```

lemma normalOrderList_eq_createFilter_append_annihilateFilter

[\[source\]](#)

```

lemma normalOrderList_eq_createFilter_append_annihilateFilter : (φs : List  $\mathcal{F}$ .CrAnStates) →
  normalOrderList φs = createFilter φs ++ annihilateFilter φs
| [] => by simp [normalOrderList, createFilter, annihilateFilter]
| φ :: φs => by

```

```

by_cases hφ :  $\mathcal{F}$  |>c φ = CreateAnnihilate.create
· rw [normalOrderList_cons_create φ hφ φs]
  dsimp only [createFilter]
  rw [List.filter_cons_of_pos]
  swap
  simp only [hφ, decide_true]
  dsimp only [annihilateFilter, List.cons_append]
  rw [List.filter_cons_of_neg]
  swap
  simp only [hφ, reduceCtorEq, decide_false, Bool.false_eq_true, not_false_eq_true]
  rw [normalOrderList_eq_createFilter_append_annihilateFilter φs]
  rfl
· dsimp only [normalOrderList, List.insertionSort]
  rw [← normalOrderList]
  have hφ' :  $\mathcal{F}$  |>c φ = CreateAnnihilate.annihilate

```

2.30 HepLean.PerturbationTheory.FieldSpecification.TimeOrder

[HepLean/PerturbationTheory/FieldSpecification/TimeOrder.lean](#) on GitHub.

def timeOrderRel

[\[source\]](#)

The time ordering relation on states. We have that `timeOrderRel φ0 φ1` is true if and only if `φ1` has a time less-than or equal to `φ0`, or `φ1` is a negative asymptotic state, or `φ0` is a positive asymptotic state.

```

def timeOrderRel :  $\mathcal{F}$ .States →  $\mathcal{F}$ .States → Prop
| States.outAsymp _, _ => True
| States.position φ0, States.position φ1 => φ1.2 0 ≤ φ0.2 0
| States.position _, States.inAsymp _ => True
| States.position _, States.outAsymp _ => False
| States.inAsymp _, States.outAsymp _ => False
| States.inAsymp _, States.position _ => False
| States.inAsymp _, States.inAsymp _ => True

```

instance (φ φ' : $\mathcal{F}$.States) → Decidable (timeOrderRel φ φ') | States.outAsymp _,... [\[source\]](#)

The relation `timeOrderRel` is decidable, but not computably so due to `Real.decidableLE`.

```

noncomputable instance : (φ φ' :  $\mathcal{F}$ .States) → Decidable (timeOrderRel φ φ')
| States.outAsymp _, _ => isTrue True.intro
| States.position φ0, States.position φ1 => inferInstanceAs (Decidable (φ1.2 0 ≤ φ0.2 0))
| States.position _, States.inAsymp _ => isTrue True.intro
| States.position _, States.outAsymp _ => isFalse (fun a => a)
| States.inAsymp _, States.outAsymp _ => isFalse (fun a => a)
| States.inAsymp _, States.position _ => isFalse (fun a => a)
| States.inAsymp _, States.inAsymp _ => isTrue True.intro

```

instance IsTotal $\mathcal{F}$.States $\mathcal{F}$.timeOrderRel

[\[source\]](#)

Time ordering is total.

instance : IsTotal $\mathcal{F}$.States $\mathcal{F}$.timeOrderRel **where**

instance IsTrans $\mathcal{F}$.States $\mathcal{F}$.timeOrderRel

[\[source\]](#)

Time ordering is transitive.

instance : IsTrans $\mathcal{F}$.States $\mathcal{F}$.timeOrderRel **where**

def maxTimeFieldPos

[\[source\]](#)

Given a list $\phi :: \phi s$ of states, the (zero-based) position of the state which is of maximum time. For example - for the list $[\phi 1(t = 4), \phi 2(t = 5), \phi 3(t = 3), \phi 4(t = 5)]$ this would return 1. This is defined for a list $\phi :: \phi s$ instead of ϕs to ensure that such a position exists.

def maxTimeFieldPos ($\phi : \mathcal{F}$.States) ($\phi s : \text{List } \mathcal{F}$.States) : $\mathbb{N}$

lemma maxTimeFieldPos_lt_length

[\[source\]](#)

lemma maxTimeFieldPos_lt_length ($\phi : \mathcal{F}$.States) ($\phi s : \text{List } \mathcal{F}$.States) :
maxTimeFieldPos ϕ $\phi s < (\phi :: \phi s).$ length

def maxTimeField

[\[source\]](#)

Given a list $\phi :: \phi s$ of states, the left-most state of maximum time, if there are more. As an example: - for the list $[\phi 1(t = 4), \phi 2(t = 5), \phi 3(t = 3), \phi 4(t = 5)]$ this would return $\phi 2(t = 5)$. It is the state at the position maxTimeFieldPos ϕ ϕs .

def maxTimeField ($\phi : \mathcal{F}$.States) ($\phi s : \text{List } \mathcal{F}$.States) : $\mathcal{F}$.States

def eraseMaxTimeField

[\[source\]](#)

Given a list $\phi :: \phi s$ of states, the list with the left-most state of maximum time removed. As an example: - for the list $[\phi 1(t = 4), \phi 2(t = 5), \phi 3(t = 3), \phi 4(t = 5)]$ this would return $[\phi 1(t = 4), \phi 3(t = 3), \phi 4(t = 5)]$.

def eraseMaxTimeField ($\phi : \mathcal{F}$.States) ($\phi s : \text{List } \mathcal{F}$.States) : $\text{List } \mathcal{F}$.States

lemma eraseMaxTimeField_length

[\[source\]](#)

@[simp]

lemma eraseMaxTimeField_length ($\phi : \mathcal{F}$.States) ($\phi s : \text{List } \mathcal{F}$.States) :
(eraseMaxTimeField ϕ ϕs).length = ϕs .length

lemma maxTimeFieldPos_lt_eraseMaxTimeField_length_succ

[\[source\]](#)

lemma maxTimeFieldPos_lt_eraseMaxTimeField_length_succ ($\phi : \mathcal{F}$.States) ($\phi s : \text{List } \mathcal{F}$.States) :
maxTimeFieldPos ϕ $\phi s < (\text{eraseMaxTimeField } \phi \phi s).$ length.succ

def maxTimeFieldPosFin

[\[source\]](#)

Given a list $\phi :: \phi s$ of states, the position of the left-most state of maximum time as an element of Fin (eraseMaxTimeField ϕ ϕs).length.succ. As an example: - for the list $[\phi 1(t = 4), \phi 2(t = 5), \phi 3(t = 3), \phi 4(t = 5)]$ this would return $\langle 1, \dots \rangle$.

```
def maxTimeFieldPosFin (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States) :
  Fin (eraseMaxTimeField φ φs).length.succ
```

lemma lt_maxTimeFieldPosFin_not_timeOrder

[\[source\]](#)

```
lemma lt_maxTimeFieldPosFin_not_timeOrder (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (i : Fin (eraseMaxTimeField φ φs).length)
  (hi : (maxTimeFieldPosFin φ φs).succAbove i < maxTimeFieldPosFin φ φs) :
  ¬ timeOrderRel ((eraseMaxTimeField φ φs)[i.val]) (maxTimeField φ φs)
```

lemma timeOrder_maxTimeField

[\[source\]](#)

```
lemma timeOrder_maxTimeField (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (i : Fin (eraseMaxTimeField φ φs).length) :
  timeOrderRel (maxTimeField φ φs) ((eraseMaxTimeField φ φs)[i.val])
```

def timeOrderSign

[\[source\]](#)

The sign associated with putting a list of states into time order (with the state of greatest time to the left). We pick up a minus sign for every fermion paired crossed.

```
def timeOrderSign (φs : List  $\mathcal{F}$ .States) :  $\mathbb{C}$ 
```

lemma timeOrderSign_nil

[\[source\]](#)

```
@[simp]
lemma timeOrderSign_nil : timeOrderSign ( $\mathcal{F}$  :=  $\mathcal{F}$ ) [] = 1
```

lemma timeOrderSign_pair_ordered

[\[source\]](#)

```
lemma timeOrderSign_pair_ordered {φ ψ :  $\mathcal{F}$ .States} (h : timeOrderRel φ ψ) :
  timeOrderSign [φ, ψ] = 1
```

lemma timeOrderSign_pair_not_ordered

[\[source\]](#)

```
lemma timeOrderSign_pair_not_ordered {φ ψ :  $\mathcal{F}$ .States} (h : ¬ timeOrderRel φ ψ) :
  timeOrderSign [φ, ψ] =  $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \psi)$ 
```

lemma timerOrderSign_of_eraseMaxTimeField

[\[source\]](#)

```
lemma timerOrderSign_of_eraseMaxTimeField (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States) :
  timeOrderSign (eraseMaxTimeField φ φs) = timeOrderSign (φ :: φs) *
   $S(\mathcal{F} \mid_{>s} \text{maxTimeField } \phi \ \phi s, \mathcal{F} \mid_{>s} (\phi :: \phi s).take (\text{maxTimeFieldPos } \phi \ \phi s))$ 
```

def timeOrderList

[\[source\]](#)

*The time ordering of a list of states. A schematic example is: - normalOrderList
[φ1(t = 4), φ2(t = 5), φ3(t = 3), φ4(t = 5)] is equal to [φ2(t = 5), φ4(t = 5),
φ1(t = 4), φ3(t = 3)]*

```
def timeOrderList (φs : List  $\mathcal{F}$ .States) : List  $\mathcal{F}$ .States
```

lemma timeOrderList_pair_ordered[\[source\]](#)

```
lemma timeOrderList_pair_ordered {φ ψ : F.States} (h : timeOrderRel φ ψ) :
  timeOrderList [φ, ψ] = [φ, ψ]
```

lemma timeOrderList_pair_not_ordered[\[source\]](#)

```
lemma timeOrderList_pair_not_ordered {φ ψ : F.States} (h : ¬ timeOrderRel φ ψ) :
  timeOrderList [φ, ψ] = [ψ, φ]
```

lemma timeOrderList_nil[\[source\]](#)

```
@[simp]
lemma timeOrderList_nil : timeOrderList (F := F) [] = []
```

lemma timeOrderList_eq_maxTimeField_timeOrderList[\[source\]](#)

```
lemma timeOrderList_eq_maxTimeField_timeOrderList (φ : F.States) (φs : List F.States) :
  timeOrderList (φ :: φs) = maxTimeField φ φs :: timeOrderList (eraseMaxTimeField φ φs)
```

def crAnTimeOrderRel[\[source\]](#)

The time ordering relation on CrAnStates.

```
def crAnTimeOrderRel (a b : F.CrAnStates) : Prop
```

instance Decidable (crAnTimeOrderRel φ φ')[\[source\]](#)

The relation crAnTimeOrderRel is decidable, but not computably so due to Real.decidableLE.

```
noncomputable instance (φ φ' : F.CrAnStates) : Decidable (crAnTimeOrderRel φ φ')
```

instance IsTotal F.CrAnStates F.crAnTimeOrderRel[\[source\]](#)

Time ordering of CrAnStates is total.

```
instance : IsTotal F.CrAnStates F.crAnTimeOrderRel where
```

instance IsTrans F.CrAnStates F.crAnTimeOrderRel[\[source\]](#)

Time ordering of CrAnStates is transitive.

```
instance : IsTrans F.CrAnStates F.crAnTimeOrderRel where
```

lemma crAnTimeOrderRel_refl[\[source\]](#)

```
@[simp]
lemma crAnTimeOrderRel_refl (φ : F.CrAnStates) : crAnTimeOrderRel φ φ
```


def crAnTimeOrderSign[\[source\]](#)

The sign associated with putting a list of `CrAnStates` into time order (with the state of greatest time to the left). We pick up a minus sign for every fermion paired crossed.

```
def crAnTimeOrderSign (φs : List  $\mathcal{F}$ .CrAnStates) :  $\mathbb{C}$ 
```

lemma crAnTimeOrderSign_nil[\[source\]](#)

```
@[simp]
```

```
lemma crAnTimeOrderSign_nil : crAnTimeOrderSign ( $\mathcal{F}$  :=  $\mathcal{F}$ ) [] = 1
```

lemma crAnTimeOrderSign_pair_ordered[\[source\]](#)

```
lemma crAnTimeOrderSign_pair_ordered {φ ψ :  $\mathcal{F}$ .CrAnStates} (h : crAnTimeOrderRel φ ψ) :  
  crAnTimeOrderSign [φ, ψ] = 1
```

lemma crAnTimeOrderSign_pair_not_ordered[\[source\]](#)

```
lemma crAnTimeOrderSign_pair_not_ordered {φ ψ :  $\mathcal{F}$ .CrAnStates} (h : ¬ crAnTimeOrderRel φ ψ) :  
  crAnTimeOrderSign [φ, ψ] =  $S(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \psi)$ 
```

lemma crAnTimeOrderSign_swap_eq_time[\[source\]](#)

```
lemma crAnTimeOrderSign_swap_eq_time {φ ψ :  $\mathcal{F}$ .CrAnStates}  
  (h1 : crAnTimeOrderRel φ ψ) (h2 : crAnTimeOrderRel ψ φ) (φs φs' : List  $\mathcal{F}$ .CrAnStates) :  
  crAnTimeOrderSign (φs ++ φ :: ψ :: φs') = crAnTimeOrderSign (φs ++ ψ :: φ :: φs')
```

def crAnTimeOrderList[\[source\]](#)

Sort a list of `CrAnStates` based on `crAnTimeOrderRel`.

```
def crAnTimeOrderList (φs : List  $\mathcal{F}$ .CrAnStates) : List  $\mathcal{F}$ .CrAnStates
```

lemma crAnTimeOrderList_nil[\[source\]](#)

```
@[simp]
```

```
lemma crAnTimeOrderList_nil : crAnTimeOrderList ( $\mathcal{F}$  :=  $\mathcal{F}$ ) [] = []
```

lemma crAnTimeOrderList_pair_ordered[\[source\]](#)

```
lemma crAnTimeOrderList_pair_ordered {φ ψ :  $\mathcal{F}$ .CrAnStates} (h : crAnTimeOrderRel φ ψ) :  
  crAnTimeOrderList [φ, ψ] = [φ, ψ]
```

lemma crAnTimeOrderList_pair_not_ordered[\[source\]](#)

```
lemma crAnTimeOrderList_pair_not_ordered {φ ψ :  $\mathcal{F}$ .CrAnStates} (h : ¬ crAnTimeOrderRel φ ψ) :  
  crAnTimeOrderList [φ, ψ] = [ψ, φ]
```

lemma orderedInsert_swap_eq_time[\[source\]](#)

```

lemma orderedInsert_swap_eq_time { $\phi$   $\psi$  :  $\mathcal{F}$ .CrAnStates}
  (h1 : crAnTimeOrderRel  $\phi$   $\psi$ ) (h2 : crAnTimeOrderRel  $\psi$   $\phi$ ) ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) :
  List.orderedInsert crAnTimeOrderRel  $\phi$  (List.orderedInsert crAnTimeOrderRel  $\psi$   $\phi$ s) =
  List.takeWhile (fun b =>  $\neg$  crAnTimeOrderRel  $\psi$  b)  $\phi$ s ++  $\phi$  ::  $\psi$  ::
  List.dropWhile (fun b =>  $\neg$  crAnTimeOrderRel  $\psi$  b)  $\phi$ s

```

lemma orderedInsert_in_swap_eq_time[\[source\]](#)

```

lemma orderedInsert_in_swap_eq_time { $\phi$   $\psi$   $\phi'$  :  $\mathcal{F}$ .CrAnStates} (h1 : crAnTimeOrderRel  $\phi$   $\psi$ )
  (h2 : crAnTimeOrderRel  $\psi$   $\phi$ ) : ( $\phi$ s  $\phi$ s' : List  $\mathcal{F}$ .CrAnStates)  $\rightarrow$   $\exists$  l1 l2,
  List.orderedInsert crAnTimeOrderRel  $\phi'$  ( $\phi$ s ++  $\phi$  ::  $\psi$  ::  $\phi$ s') = l1 ++  $\phi$  ::  $\psi$  :: l2  $\wedge$ 
  List.orderedInsert crAnTimeOrderRel  $\phi'$  ( $\phi$ s ++  $\psi$  ::  $\phi$  ::  $\phi$ s') = l1 ++  $\psi$  ::  $\phi$  :: l2
| [],  $\phi$ s' => by
  have h1 (b :  $\mathcal{F}$ .CrAnStates) : (crAnTimeOrderRel b  $\phi$ )  $\leftrightarrow$  (crAnTimeOrderRel b  $\psi$ )

```

lemma crAnTimeOrderList_swap_eq_time[\[source\]](#)

```

lemma crAnTimeOrderList_swap_eq_time { $\phi$   $\psi$  :  $\mathcal{F}$ .CrAnStates}
  (h1 : crAnTimeOrderRel  $\phi$   $\psi$ ) (h2 : crAnTimeOrderRel  $\psi$   $\phi$ ) :
  ( $\phi$ s  $\phi$ s' : List  $\mathcal{F}$ .CrAnStates)  $\rightarrow$ 
   $\exists$  (l1 l2 : List  $\mathcal{F}$ .CrAnStates),
    crAnTimeOrderList ( $\phi$ s ++  $\phi$  ::  $\psi$  ::  $\phi$ s') = l1 ++  $\phi$  ::  $\psi$  :: l2  $\wedge$ 
    crAnTimeOrderList ( $\phi$ s ++  $\psi$  ::  $\phi$  ::  $\phi$ s') = l1 ++  $\psi$  ::  $\phi$  :: l2
| [],  $\phi$ s' => by
  simp only [crAnTimeOrderList]
  simp only [List.insertionSort]
  use List.takeWhile (fun b =>  $\neg$  crAnTimeOrderRel  $\psi$  b) (List.insertionSort crAnTimeOrderRel
 $\phi$ s'),
    List.dropWhile (fun b =>  $\neg$  crAnTimeOrderRel  $\psi$  b) (List.insertionSort crAnTimeOrderRel  $\phi$ s
  ')
  apply And.intro
  · exact orderedInsert_swap_eq_time h1 h2 _
  · have h1' (b :  $\mathcal{F}$ .CrAnStates) : (crAnTimeOrderRel  $\phi$  b)  $\leftrightarrow$  (crAnTimeOrderRel  $\psi$  b)

```

lemma koszulSignInsert_crAnTimeOrderRel_crAnSection[\[source\]](#)

```

lemma koszulSignInsert_crAnTimeOrderRel_crAnSection { $\phi$  :  $\mathcal{F}$ .States} { $\psi$  :  $\mathcal{F}$ .CrAnStates}
  (h :  $\psi$ .1 =  $\phi$ ) : { $\phi$ s : List  $\mathcal{F}$ .States}  $\rightarrow$  ( $\psi$ s : CrAnSection  $\phi$ s)  $\rightarrow$ 
  Wick.koszulSignInsert  $\mathcal{F}$ .crAnStatistics  $\mathcal{F}$ .crAnTimeOrderRel  $\psi$   $\psi$ s.1 =
  Wick.koszulSignInsert  $\mathcal{F}$ .statesStatistic  $\mathcal{F}$ .timeOrderRel  $\phi$   $\phi$ s
| [], ( $\phi$ s, h) => by
  simp [Wick.koszulSignInsert]
|  $\phi'$  ::  $\phi$ s, ( $\psi'$  ::  $\psi$ s, h1) => by
  simp only [Wick.koszulSignInsert, crAnTimeOrderRel, h]
  simp only [List.map_cons, List.cons.injEq] at h1
  have hi

```

lemma crAnTimeOrderSign_crAnSection[\[source\]](#)

```

@[simp]
lemma crAnTimeOrderSign_crAnSection : { $\phi$ s : List  $\mathcal{F}$ .States}  $\rightarrow$  ( $\psi$ s : CrAnSection  $\phi$ s)  $\rightarrow$ 
  crAnTimeOrderSign  $\psi$ s.1 = timeOrderSign  $\phi$ s
| [], ( $\phi$ s, h) => by

```

```

simp
|  $\phi :: \phi s, \langle \psi :: \psi s, h \rangle \Rightarrow$  by
  simp only [crAnTimeOrderSign, Wick.koszulSign, timeOrderSign]
  simp only [List.map_cons, List.cons.injEq] at h
  congr 1
  · rw [koszulSignInsert_crAnTimeOrderRel_crAnSection h.1  $\langle \psi s, h.2 \rangle$ ]
  · exact crAnTimeOrderSign_crAnSection  $\langle \psi s, h.2 \rangle$ 

```

lemma orderedInsert_crAnTimeOrderRel_crAnSection[\[source\]](#)

```

lemma orderedInsert_crAnTimeOrderRel_crAnSection  $\{\mathcal{F} : \mathcal{F}.States\} \{\psi : \mathcal{F}.CrAnStates\}$ 
   $(h : \psi.1 = \phi) : \{\phi s : List \mathcal{F}.States\} \rightarrow (\psi s : CrAnSection \phi s) \rightarrow$ 
   $(List.orderedInsert \mathcal{F}.crAnTimeOrderRel \psi \psi s.1).map \mathcal{F}.crAnStatesToStates =$ 
   $List.orderedInsert \mathcal{F}.timeOrderRel \phi \phi s$ 
| [],  $\langle [], \_ \rangle \Rightarrow$  by
  simp only [List.orderedInsert, List.map_cons, List.map_nil, List.cons.injEq, and_true]
  exact h
|  $\phi' :: \phi s, \langle \psi' :: \psi s, h1 \rangle \Rightarrow$  by
  simp only [List.orderedInsert, crAnTimeOrderRel, h]
  simp only [List.map_cons, List.cons.injEq] at h1
  by_cases hr : timeOrderRel  $\phi \phi'$ 
  · simp only [hr,  $\downarrow$ reduceIte]
    rw [ $\leftarrow$  h1.1] at hr
    simp only [crAnStatesToStates] at hr
    simp only [hr,  $\downarrow$ reduceIte, List.map_cons, List.cons.injEq]
    exact And.intro h (And.intro h1.1 h1.2)
  · simp only [hr,  $\downarrow$ reduceIte]
    rw [ $\leftarrow$  h1.1] at hr
    simp only [crAnStatesToStates] at hr
    simp only [hr,  $\downarrow$ reduceIte, List.map_cons, List.cons.injEq]
    apply And.intro h1.1
    exact orderedInsert_crAnTimeOrderRel_crAnSection h  $\langle \psi s, h1.2 \rangle$ 

```

lemma crAnTimeOrderList_crAnSection_is_crAnSection[\[source\]](#)

```

lemma crAnTimeOrderList_crAnSection_is_crAnSection :  $\{\phi s : List \mathcal{F}.States\} \rightarrow (\psi s : CrAnSection$ 
   $\phi s) \rightarrow$ 
   $(crAnTimeOrderList \psi s.1).map \mathcal{F}.crAnStatesToStates = timeOrderList \phi s$ 
| [],  $\langle [], h \rangle \Rightarrow$  by
  simp
|  $\phi :: \phi s, \langle \psi :: \psi s, h \rangle \Rightarrow$  by
  simp only [crAnTimeOrderList, List.insertionSort, timeOrderList]
  simp only [List.map_cons, List.cons.injEq] at h
  exact orderedInsert_crAnTimeOrderRel_crAnSection h.1  $\langle (List.insertionSort crAnTimeOrderRel$ 
   $\psi s),$ 
   $crAnTimeOrderList_crAnSection_is_crAnSection \langle \psi s, h.2 \rangle \rangle$ 

```

def crAnSectionTimeOrder[\[source\]](#)

Time ordering of sections of a list of states.

```

def crAnSectionTimeOrder  $(\phi s : List \mathcal{F}.States) (\psi s : CrAnSection \phi s) :$ 
   $CrAnSection (timeOrderList \phi s)$ 

```

lemma orderedInsert_crAnTimeOrderRel_injective[\[source\]](#)

```

lemma orderedInsert_crAnTimeOrderRel_injective {ψ ψ' :  $\mathcal{F}$ .CrAnStates} (h : ψ.1 = ψ'.1) :
  {φs : List  $\mathcal{F}$ .States} → (ψs ψs' :  $\mathcal{F}$ .CrAnSection φs) →
  (ho : List.orderedInsert crAnTimeOrderRel ψ ψs.1 =
    List.orderedInsert crAnTimeOrderRel ψ' ψs'.1) → ψ = ψ' ∧ ψs = ψs'
| [], [], _, [], _ => by
  simp only [List.orderedInsert, List.cons.injEq, and_true] at h
  simp using h
| φ :: φs, (ψ1 :: ψs, h1), (ψ1' :: ψs', h1'), ho => by
  simp only [List.map_cons, List.cons.injEq] at h1 h1'
have ih

```

lemma crAnSectionTimeOrder_injective

[\[source\]](#)

```

lemma crAnSectionTimeOrder_injective : {φs : List  $\mathcal{F}$ .States} →
  Function.Injective ( $\mathcal{F}$ .crAnSectionTimeOrder φs)
| [], [], _, [], _ => by
  simp
| φ :: φs, (ψ :: ψs, h), (ψ' :: ψs', h') => by
  intro h1
  apply Subtype.ext
  simp only [List.cons.injEq]
  simp only [crAnSectionTimeOrder] at h1
  rw [Subtype.eq_iff] at h1
  simp only [crAnTimeOrderList, List.insertionSort] at h1
  simp only [List.map_cons, List.cons.injEq] at h h'
  rw [crAnStatesToStates] at h h'
have hin

```

lemma crAnSectionTimeOrder_bijective

[\[source\]](#)

```

lemma crAnSectionTimeOrder_bijective (φs : List  $\mathcal{F}$ .States) :
  Function.Bijective ( $\mathcal{F}$ .crAnSectionTimeOrder φs)

```

lemma sum_crAnSections_timeOrder

[\[source\]](#)

```

lemma sum_crAnSections_timeOrder {φs : List  $\mathcal{F}$ .States} [AddCommMonoid M]
  (f : CrAnSection (timeOrderList φs) → M) :  $\sum s, f s = \sum s, f (\mathcal{F}.crAnSectionTimeOrder \phi s s)$ 

```

def normTimeOrderRel

[\[source\]](#)

The time ordering relation on CrAnStates such that if two CrAnStates have the same time, we normal order them.

```

def normTimeOrderRel (a b :  $\mathcal{F}$ .CrAnStates) : Prop

```

instance Decidable (normTimeOrderRel φ φ')

[\[source\]](#)

The relation normTimeOrderRel is decidable, but not computably so due to Real.decidableLE.

```

noncomputable instance (φ φ' :  $\mathcal{F}$ .CrAnStates) : Decidable (normTimeOrderRel φ φ')

```

instance IsTotal $\mathcal{F}$.CrAnStates $\mathcal{F}$.normTimeOrderRel

[\[source\]](#)

Norm-Time ordering of CrAnStates is total.

```
instance : IsTotal  $\mathcal{F}$ .CrAnStates  $\mathcal{F}$ .normTimeOrderRel where
```

```
instance IsTrans  $\mathcal{F}$ .CrAnStates  $\mathcal{F}$ .normTimeOrderRel
```

[\[source\]](#)

Norm-Time ordering of CrAnStates is transitive.

```
instance : IsTrans  $\mathcal{F}$ .CrAnStates  $\mathcal{F}$ .normTimeOrderRel where
```

```
def normTimeOrderSign
```

[\[source\]](#)

The sign associated with putting a list of CrAnStates into normal-time order (with the state of greatest time to the left). We pick up a minus sign for every fermion paired crossed.

```
def normTimeOrderSign ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) :  $\mathbb{C}$ 
```

```
def normTimeOrderList
```

[\[source\]](#)

Sort a list of CrAnStates based on normTimeOrderRel.

```
def normTimeOrderList ( $\phi$ s : List  $\mathcal{F}$ .CrAnStates) : List  $\mathcal{F}$ .CrAnStates
```

2.31 HepLean.PerturbationTheory.FieldStatistics.Basic

[HepLean/PerturbationTheory/FieldStatistics/Basic.lean](#) on GitHub.

```
instance CommGroup FieldStatistic
```

[\[source\]](#)

Field statistics form a commuative group isomorphic to $\mathbb{Z}_2$.

```
@[simp]
```

```
instance : CommGroup FieldStatistic where
```

```
lemma bosonic_mul_bosonic
```

[\[source\]](#)

```
@[simp]
```

```
lemma bosonic_mul_bosonic : bosonic * bosonic = bosonic
```

```
lemma bosonic_mul_fermionic
```

[\[source\]](#)

```
@[simp]
```

```
lemma bosonic_mul_fermionic : bosonic * fermionic = fermionic
```

```
lemma fermionic_mul_bosonic
```

[\[source\]](#)

```
@[simp]
```

```
lemma fermionic_mul_bosonic : fermionic * bosonic = fermionic
```

lemma fermionic_mul_fermionic[\[source\]](#)

```
@[simp]
lemma fermionic_mul_fermionic : fermionic * fermionic = bosonic
```

lemma mul_bosonic[\[source\]](#)

```
@[simp]
lemma mul_bosonic (a : FieldStatistic) : a * bosonic = a
```

lemma mul_self[\[source\]](#)

```
@[simp]
lemma mul_self (a : FieldStatistic) : a * a = 1
```

lemma neq_bosonic_iff_eq_fermionic[\[source\]](#)

```
@[simp]
lemma neq_bosonic_iff_eq_fermionic (a : FieldStatistic) : ¬ a = bosonic ↔ a = fermionic
```

lemma mul_eq_one_iff[\[source\]](#)

```
lemma mul_eq_one_iff (a b : FieldStatistic) : a * b = 1 ↔ a = b
```

lemma one_eq_mul_iff[\[source\]](#)

```
lemma one_eq_mul_iff (a b : FieldStatistic) : 1 = a * b ↔ a = b
```

lemma mul_eq_iff_eq_mul[\[source\]](#)

```
lemma mul_eq_iff_eq_mul (a b c : FieldStatistic) : a * b = c ↔ a = b * c
```

lemma mul_eq_iff_eq_mul'[\[source\]](#)

```
lemma mul_eq_iff_eq_mul' (a b c : FieldStatistic) : a * b = c ↔ b = a * c
```

lemma ofList_cons_eq_mul[\[source\]](#)

```
lemma ofList_cons_eq_mul (s :  $\mathcal{F}$  → FieldStatistic) (φ :  $\mathcal{F}$ ) (φs : List  $\mathcal{F}$ ) :
  ofList s (φ :: φs) = s φ * ofList s φs
```

lemma ofList_eq_prod[\[source\]](#)

```
lemma ofList_eq_prod (s :  $\mathcal{F}$  → FieldStatistic) : (φs : List  $\mathcal{F}$ ) →
  ofList s φs = (List.map s φs).prod
| [] => rfl
| φ :: φs => by
  rw [ofList_cons_eq_mul, List.map_cons, List.prod_cons, ofList_eq_prod]
```

lemma ofList_append_eq_mul[\[source\]](#)

```

lemma ofList_append_eq_mul (s :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) ( $\phi$ s  $\phi$ s' : List  $\mathcal{F}$ ) :
  ofList s ( $\phi$ s ++  $\phi$ s') = ofList s  $\phi$ s * ofList s  $\phi$ s'

```

lemma ofList_map_eq_finset_prod[\[source\]](#)

```

lemma ofList_map_eq_finset_prod (s :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) :
  ( $\phi$ s : List  $\mathcal{F}$ ) → (l : List (Fin  $\phi$ s.length)) → (hl : l.Nodup) →
  ofList s (l.map  $\phi$ s.get) =  $\prod$  (i : Fin  $\phi$ s.length), if i ∈ l then s  $\phi$ s[i] else 1
| [], [], _ => rfl
| [], i :: l, hl => Fin.elim0 i
|  $\phi$  ::  $\phi$ s, [], hl => by
  simp only [List.length_cons, List.map_nil, ofList_empty, List.not_mem_nil,  $\lambda$ reduceIte,
    Finset.prod_const_one]
  rfl
|  $\phi$  ::  $\phi$ s, i :: l, hl => by
  simp only [List.length_cons, List.map_cons, List.get_eq_getElem, List.mem_cons,
    instCommGroup,
    Fin.getElem_fin]
  rw [ofList_cons_eq_mul]
  rw [ofList_map_eq_finset_prod s ( $\phi$  ::  $\phi$ s) l]
have hl : s ( $\phi$  ::  $\phi$ s)[i] =  $\prod$  (j : Fin ( $\phi$  ::  $\phi$ s).length),
  if j = i then s ( $\phi$  ::  $\phi$ s)[i] else 1

```

lemma ofList_pair[\[source\]](#)

```

lemma ofList_pair (s :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) ( $\phi$ 1  $\phi$ 2 :  $\mathcal{F}$ ) :
  ofList s [ $\phi$ 1,  $\phi$ 2] = s  $\phi$ 1 * s  $\phi$ 2

```

lemma ofList_take_insert[\[source\]](#)

```

lemma ofList_take_insert (n :  $\mathbb{N}$ ) ( $\phi$  :  $\mathcal{F}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) :
  ofList q (List.take n  $\phi$ s) = ofList q (List.take n (List.insertIdx n  $\phi$   $\phi$ s))

```

lemma ofList_take_eraseIdx[\[source\]](#)

```

lemma ofList_take_eraseIdx (n :  $\mathbb{N}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) :
  ofList q (List.take n ( $\phi$ s.eraseIdx n)) = ofList q (List.take n  $\phi$ s)

```

lemma ofList_take_zero[\[source\]](#)

```

lemma ofList_take_zero ( $\phi$ s : List  $\mathcal{F}$ ) :
  ofList q (List.take 0  $\phi$ s) = 1

```

lemma ofList_take_succ_cons[\[source\]](#)

```

lemma ofList_take_succ_cons (n :  $\mathbb{N}$ ) ( $\phi$ 1 :  $\mathcal{F}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) :
  ofList q (( $\phi$ 1 ::  $\phi$ s).take (n + 1)) = q  $\phi$ 1 * ofList q ( $\phi$ s.take n)

```

lemma ofList_take_insertIdx_gt[\[source\]](#)

```
lemma ofList_take_insertIdx_gt (n m : ℕ) (ϕ1 :  $\mathcal{F}$ ) (ϕs : List  $\mathcal{F}$ ) (hn : n < m) :
  ofList q ((List.insertIdx m ϕ1 ϕs).take n) = ofList q (ϕs.take n)
```

lemma ofList_insert_lt_eq

[\[source\]](#)

```
lemma ofList_insert_lt_eq (n m : ℕ) (ϕ1 :  $\mathcal{F}$ ) (ϕs : List  $\mathcal{F}$ ) (hn : m ≤ n)
  (hm : m ≤ ϕs.length) :
  ofList q ((List.insertIdx m ϕ1 ϕs).take (n + 1)) =
  ofList q ((ϕ1 :: ϕs).take (n + 1))
```

lemma ofList_take_insertIdx_le

[\[source\]](#)

```
lemma ofList_take_insertIdx_le (n m : ℕ) (ϕ1 :  $\mathcal{F}$ ) (ϕs : List  $\mathcal{F}$ ) (hn : m ≤ n) (hm : m ≤ ϕs.
  length) :
  ofList q ((List.insertIdx m ϕ1 ϕs).take (n + 1)) = q ϕ1 * ofList q (ϕs.take n)
```

instance AddMonoid FieldStatistic

[\[source\]](#)

The instance of an additive monoid on FieldStatistic.

```
instance : AddMonoid FieldStatistic where
```

lemma add_eq_mul

[\[source\]](#)

```
@[simp]
lemma add_eq_mul (a b : FieldStatistic) : a + b = a * b
```

2.32 HepLean.PerturbationTheory.FieldStatistics.ExchangeSign

[HepLean/PerturbationTheory/FieldStatistics/ExchangeSign.lean](#) on GitHub.

def exchangeSign

[\[source\]](#)

The exchange sign of two field statistics is defined to be -1 if both field statistics are fermionic and 1 otherwise. It is a group homomorphism from FieldStatistic to the group of homomorphisms from FieldStatistic to $\mathbb{C}$.

```
def exchangeSign : FieldStatistic →* FieldStatistic →*  $\mathbb{C}$  where
```

lemma exchangeSign_symm

[\[source\]](#)

The exchange sign is symmetric.

```
lemma exchangeSign_symm (a b : FieldStatistic) :  $\mathcal{S}(a, b) = \mathcal{S}(b, a)$ 
```

lemma exchangeSign_bosonic

[\[source\]](#)

```
@[simp]
lemma exchangeSign_bosonic (a : FieldStatistic) :  $\mathcal{S}(a, \text{bosonic}) = 1$ 
```


lemma bosonic_exchangeSign[\[source\]](#)

@[simp]

lemma bosonic_exchangeSign (a : FieldStatistic) : $\mathcal{S}(\text{bosonic}, a) = 1$ **lemma fermionic_exchangeSign_fermionic**[\[source\]](#)

@[simp]

lemma fermionic_exchangeSign_fermionic : $\mathcal{S}(\text{fermionic}, \text{fermionic}) = -1$ **lemma exchangeSign_eq_if**[\[source\]](#)**lemma** exchangeSign_eq_if (a b : FieldStatistic) : $\mathcal{S}(a, b) = \text{if } a = \text{fermionic} \wedge b = \text{fermionic} \text{ then } -1 \text{ else } 1$ **lemma exchangeSign_mul_self**[\[source\]](#)

@[simp]

lemma exchangeSign_mul_self (a b : FieldStatistic) : $\mathcal{S}(a, b) * \mathcal{S}(a, b) = 1$ **lemma exchangeSign_mul_self_swap**[\[source\]](#)

@[simp]

lemma exchangeSign_mul_self_swap (a b : FieldStatistic) : $\mathcal{S}(a, b) * \mathcal{S}(b, a) = 1$ **lemma exchangeSign_ofList_cons**[\[source\]](#)**lemma** exchangeSign_ofList_cons (a : FieldStatistic)(s : $\mathcal{F} \rightarrow \text{FieldStatistic}$) ($\phi : \mathcal{F}$) ($\phi s : \text{List } \mathcal{F}$) : $\mathcal{S}(a, \text{ofList } s (\phi :: \phi s)) = \mathcal{S}(a, s \ \phi) * \mathcal{S}(a, \text{ofList } s \ \phi s)$ **lemma exchangeSign_cocycle**[\[source\]](#)*The exchange sign is a cocycle.***lemma** exchangeSign_cocycle (a b c : FieldStatistic) : $\mathcal{S}(a, b * c) * \mathcal{S}(b, c) = \mathcal{S}(a, b) * \mathcal{S}(a * b, c)$ **2.33 HepLean.PerturbationTheory.FieldStatistics.OfFinset**[HepLean/PerturbationTheory/FieldStatistics/OfFinset.lean](#) on GitHub.**def ofFinset**[\[source\]](#)*The field statistic associated with a map $f : \text{Fin } n \rightarrow \mathcal{F}$ (usually `.get` of a list) and a finite set of elements of $\text{Fin } n$.***def** ofFinset {n : $\mathbb{N}$ } (q : $\mathcal{F} \rightarrow \text{FieldStatistic}$) (f : $\text{Fin } n \rightarrow \mathcal{F}$) (a : $\text{Finset } (\text{Fin } n)$) :
FieldStatistic

lemma offFinset_empty[\[source\]](#)

@[simp]

```
lemma offFinset_empty (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) (f :  $\text{Fin } n \rightarrow \mathcal{F}$ ) :
  offFinset q f  $\emptyset$  = 1
```

lemma offFinset_singleton[\[source\]](#)

```
lemma offFinset_singleton {n :  $\mathbb{N}$ } (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) (f :  $\text{Fin } n \rightarrow \mathcal{F}$ ) (i :  $\text{Fin } n$ ) :
  offFinset q f {i} = q (f i)
```

lemma offFinset_finset_map[\[source\]](#)

```
lemma offFinset_finset_map {n m :  $\mathbb{N}$ }
  (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) (i :  $\text{Fin } m \rightarrow \text{Fin } n$ ) (hi : Function.Injective i)
  (f :  $\text{Fin } n \rightarrow \mathcal{F}$ ) (a : Finset (Fin m)) :
  offFinset q (f  $\circ$  i) a = offFinset q f (a.map (i, hi))
```

lemma offFinset_insert[\[source\]](#)

```
lemma offFinset_insert (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) (a : Finset (Fin  $\phi$ s.length))
  (i : Fin  $\phi$ s.length) (h : i  $\notin$  a) :
  offFinset q  $\phi$ s.get (Insert.insert i a) = (q  $\phi$ s[i]) * offFinset q  $\phi$ s.get a
```

lemma offFinset_erase[\[source\]](#)

```
lemma offFinset_erase (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) (a : Finset (Fin  $\phi$ s.length))
  (i : Fin  $\phi$ s.length) (h : i  $\in$  a) :
  offFinset q  $\phi$ s.get (a.erase i) = (q  $\phi$ s[i]) * offFinset q  $\phi$ s.get a
```

lemma offFinset_eq_prod[\[source\]](#)

```
lemma offFinset_eq_prod (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) (a : Finset (Fin  $\phi$ s.length)) :
  offFinset q  $\phi$ s.get a =  $\prod$  (i : Fin  $\phi$ s.length), if i  $\in$  a then (q  $\phi$ s[i]) else 1
```

lemma offFinset_union[\[source\]](#)

```
lemma offFinset_union (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) (a b : Finset (Fin  $\phi$ s.length)) :
  offFinset q  $\phi$ s.get a * offFinset q  $\phi$ s.get b = offFinset q  $\phi$ s.get ((a  $\cup$  b) \ (a  $\cap$  b))
```

lemma offFinset_union_disjoint[\[source\]](#)

```
lemma offFinset_union_disjoint (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) ( $\phi$ s : List  $\mathcal{F}$ ) (a b : Finset (Fin  $\phi$ s.length))
  (h : Disjoint a b) :
  offFinset q  $\phi$ s.get a * offFinset q  $\phi$ s.get b = offFinset q  $\phi$ s.get (a  $\cup$  b)
```

2.34 HepLean.PerturbationTheory.Koszul.KoszulSign

[HepLean/PerturbationTheory/Koszul/KoszulSign.lean](#) on GitHub.

lemma koszulSign_singleton[\[source\]](#)

```
@[simp]
lemma koszulSign_singleton (q :  $\mathcal{F} \rightarrow \text{FieldStatistic}$ ) (le :  $\mathcal{F} \rightarrow \mathcal{F} \rightarrow \text{Prop}$ ) [DecidableRel le] ( $\phi$  :  $\mathcal{F}$ ) :
  koszulSign q le [ $\phi$ ] = 1
```

lemma koszulSign_eraseIdx[\[source\]](#)

```
lemma koszulSign_eraseIdx [IsTotal  $\mathcal{F}$  le] [IsTrans  $\mathcal{F}$  le] ( $\phi s$  : List  $\mathcal{F}$ ) (n : Fin  $\phi s.\text{length}$ ) :
  koszulSign q le ( $\phi s.\text{eraseIdx } n$ ) = koszulSign q le  $\phi s$  *  $\mathcal{S}(q (\phi s.\text{get } n), \text{ofList } q (\phi s.\text{take } n))$ 
  *
   $\mathcal{S}(q (\phi s.\text{get } n), \text{ofList } q (\text{List.take } (\uparrow(\text{insertionSortEquiv le } \phi s \ n))$ 
    ( $\text{List.insertionSort le } \phi s$ )))
```

lemma koszulSign_eraseIdx_insertionSortMinPos[\[source\]](#)

```
lemma koszulSign_eraseIdx_insertionSortMinPos [IsTotal  $\mathcal{F}$  le] [IsTrans  $\mathcal{F}$  le] ( $\phi$  :  $\mathcal{F}$ ) ( $\phi s$  : List  $\mathcal{F}$ ) :
  koszulSign q le (( $\phi :: \phi s$ ).eraseIdx (insertionSortMinPos le  $\phi$   $\phi s$ )) = koszulSign q le ( $\phi :: \phi s$ )
  *  $\mathcal{S}(q (\text{insertionSortMin le } \phi \ \phi s), \text{ofList } q ((\phi :: \phi s).\text{take } (\text{insertionSortMinPos le } \phi \ \phi s)))$ 
```

lemma koszulSign_swap_eq_rel_cons[\[source\]](#)

```
lemma koszulSign_swap_eq_rel_cons { $\psi \ \phi$  :  $\mathcal{F}$ }
  (h1 : le  $\phi \ \psi$ ) (h2 : le  $\psi \ \phi$ ) ( $\phi s'$  : List  $\mathcal{F}$ ) :
  koszulSign q le ( $\phi :: \psi :: \phi s'$ ) = koszulSign q le ( $\psi :: \phi :: \phi s'$ )
```

lemma koszulSign_swap_eq_rel[\[source\]](#)

```
lemma koszulSign_swap_eq_rel { $\psi \ \phi$  :  $\mathcal{F}$ } (h1 : le  $\phi \ \psi$ ) (h2 : le  $\psi \ \phi$ ) : ( $\phi s \ \phi s'$  : List  $\mathcal{F}$ ) →
  koszulSign q le ( $\phi s ++ \phi :: \psi :: \phi s'$ ) = koszulSign q le ( $\phi s ++ \psi :: \phi :: \phi s'$ )
| [],  $\phi s'$  => by
  simp only [List.nil_append]
  exact koszulSign_swap_eq_rel_cons q le h1 h2  $\phi s'$ 
|  $\phi'' :: \phi s, \phi s'$  => by
  simp only [Wick.koszulSign, List.append_eq]
  rw [koszulSign_swap_eq_rel h1 h2]
  congr 1
  apply Wick.koszulSignInsert_eq_perm
  exact List.Perm.append_left  $\phi s$  (List.Perm.swap  $\psi \ \phi \ \phi s'$ )
```

lemma koszulSign_eq_rel_eq_stat_append[\[source\]](#)

```
lemma koszulSign_eq_rel_eq_stat_append { $\psi \ \phi$  :  $\mathcal{F}$ } [IsTrans  $\mathcal{F}$  le]
  (h1 : le  $\phi \ \psi$ ) (h2 : le  $\psi \ \phi$ ) (hq :  $q \ \psi = q \ \phi$ ) : ( $\phi s$  : List  $\mathcal{F}$ ) →
  koszulSign q le ( $\phi :: \psi :: \phi s$ ) = koszulSign q le  $\phi s$ 
```

lemma koszulSign_eq_rel_eq_stat[\[source\]](#)

```
lemma koszulSign_eq_rel_eq_stat { $\psi \ \phi$  :  $\mathcal{F}$ } [IsTrans  $\mathcal{F}$  le]
  (h1 : le  $\phi \ \psi$ ) (h2 : le  $\psi \ \phi$ ) (hq :  $q \ \psi = q \ \phi$ ) : ( $\phi s' \ \phi s$  : List  $\mathcal{F}$ ) →
  koszulSign q le ( $\phi s' ++ \phi :: \psi :: \phi s$ ) = koszulSign q le ( $\phi s' ++ \phi s$ )
```

```

| [],  $\phi s \Rightarrow$  by
  simp only [List.nil_append]
  exact koszulSign_eq_rel_eq_stat_append q le h1 h2 hq  $\phi s$ 
|  $\phi'' :: \phi s'$ ,  $\phi s \Rightarrow$  by
  simp only [koszulSign, List.append_eq]
  rw [koszulSign_eq_rel_eq_stat h1 h2 hq  $\phi s'$   $\phi s$ ]
  simp only [mul_eq_mul_right_iff]
  left
  trans koszulSignInsert q le  $\phi''$  ( $\phi :: \psi :: (\phi s' ++ \phi s)$ )
  apply koszulSignInsert_eq_perm
  refine List.Perm.symm (List.perm_cons_append_cons  $\phi$  ?_)
  exact List.Perm.symm List.perm_middle
  rw [koszulSignInsert_eq_remove_same_stat_append q le]
  exact h1
  exact h2
  exact hq

```

lemma koszulSign_of_sorted[\[source\]](#)

```

lemma koszulSign_of_sorted : ( $\phi s$  : List  $\mathcal{F}$ )
   $\rightarrow$  ( $hs$  : List.Sorted le  $\phi s$ )  $\rightarrow$  koszulSign q le  $\phi s$  = 1
| [], _  $\Rightarrow$  by
  simp [koszulSign]
|  $\phi :: \phi s$ , h  $\Rightarrow$  by
  simp only [koszulSign]
  simp only [List.sorted_cons] at h
  rw [koszulSign_of_sorted  $\phi s$  h.2]
  simp only [mul_one]
  exact koszulSignInsert_of_le_mem _ _ _ _ h.1

```

lemma koszulSign_of_insertionSort[\[source\]](#)

```

@[simp]
lemma koszulSign_of_insertionSort [IsTotal  $\mathcal{F}$  le] [IsTrans  $\mathcal{F}$  le] ( $\phi s$  : List  $\mathcal{F}$ ) :
  koszulSign q le (List.insertionSort le  $\phi s$ ) = 1

```

lemma koszulSign_of_append_eq_insertionSort_left[\[source\]](#)

```

lemma koszulSign_of_append_eq_insertionSort_left [IsTotal  $\mathcal{F}$  le] [IsTrans  $\mathcal{F}$  le] :
  ( $\phi s \phi s'$  : List  $\mathcal{F}$ )  $\rightarrow$  koszulSign q le ( $\phi s ++ \phi s'$ ) =
    koszulSign q le (List.insertionSort le  $\phi s ++ \phi s'$ ) * koszulSign q le  $\phi s$ 
|  $\phi s$ , []  $\Rightarrow$  by
  simp
|  $\phi s$ ,  $\phi :: \phi s'$   $\Rightarrow$  by
  have h1 : ( $\phi s ++ \phi :: \phi s'$ ) = List.insertIdx  $\phi s$ .length  $\phi$  ( $\phi s ++ \phi s'$ )

```

lemma koszulSign_of_append_eq_insertionSort[\[source\]](#)

```

lemma koszulSign_of_append_eq_insertionSort [IsTotal  $\mathcal{F}$  le] [IsTrans  $\mathcal{F}$  le] : ( $\phi s'' \phi s \phi s'$  :
  List  $\mathcal{F}$ )  $\rightarrow$ 
  koszulSign q le ( $\phi s'' ++ \phi s ++ \phi s'$ ) =
    koszulSign q le ( $\phi s'' ++$  List.insertionSort le  $\phi s ++ \phi s'$ ) * koszulSign q le  $\phi s$ 
| [],  $\phi s$ ,  $\phi s'$   $\Rightarrow$  by
  simp only [List.nil_append]
  exact koszulSign_of_append_eq_insertionSort_left q le  $\phi s \phi s'$ 

```

```

|  $\phi'' :: \phi s''$ ,  $\phi s$ ,  $\phi s' \Rightarrow$  by
  simp only [koszulSign, List.append_eq]
  rw [koszulSign_of_append_eq_insertionSort  $\phi s''$   $\phi s$   $\phi s'$ ,  $\leftarrow$  mul_assoc]
  congr 2
  apply koszulSignInsert_eq_perm
  refine (List.perm_append_right_iff  $\phi s'$ ).mpr ?_
  refine List.Perm.append_left  $\phi s''$  ?_
  exact List.Perm.symm (List.perm_insertionSort le  $\phi s$ )

```

lemma koszulSign_perm_eq_append[\[source\]](#)

```

lemma koszulSign_perm_eq_append [IsTrans  $\mathcal{F}$  le] ( $\phi : \mathcal{F}$ ) ( $\phi s \phi s' \phi s2 : \text{List } \mathcal{F}$ )
  ( $hp : \phi s.\text{Perm } \phi s'$ ) : ( $h : \forall \phi' \in \phi s, \text{le } \phi \phi' \wedge \text{le } \phi' \phi$ )  $\rightarrow$ 
  koszulSign q le ( $\phi s ++ \phi s2$ ) = koszulSign q le ( $\phi s' ++ \phi s2$ )

```

lemma koszulSign_perm_eq[\[source\]](#)

```

lemma koszulSign_perm_eq [IsTrans  $\mathcal{F}$  le] ( $\phi : \mathcal{F}$ ) : ( $\phi s1 \phi s \phi s' \phi s2 : \text{List } \mathcal{F}$ )  $\rightarrow$ 
  ( $h : \forall \phi' \in \phi s, \text{le } \phi \phi' \wedge \text{le } \phi' \phi$ )  $\rightarrow$  ( $hp : \phi s.\text{Perm } \phi s'$ )  $\rightarrow$ 
  koszulSign q le ( $\phi s1 ++ \phi s ++ \phi s2$ ) = koszulSign q le ( $\phi s1 ++ \phi s' ++ \phi s2$ )
| [],  $\phi s$ ,  $\phi s'$ ,  $\phi s2$ ,  $h$ ,  $hp \Rightarrow$  by
  simp only [List.nil_append]
  exact koszulSign_perm_eq_append q le  $\phi \phi s \phi s' \phi s2$   $hp$   $h$ 
|  $\phi1 :: \phi s1$ ,  $\phi s$ ,  $\phi s'$ ,  $\phi s2$ ,  $h$ ,  $hp \Rightarrow$  by
  simp only [koszulSign, List.append_eq]
  have ih

```

2.35 HepLean.PerturbationTheory.Koszul.KoszulSignInsert

[HepLean/PerturbationTheory/Koszul/KoszulSignInsert.lean](#) on GitHub.

lemma koszulSignInsert_eq_exchangeSign_take[\[source\]](#)

```

lemma koszulSignInsert_eq_exchangeSign_take [IsTotal  $\mathcal{F}$  le] [IsTrans  $\mathcal{F}$  le] ( $\phi : \mathcal{F}$ ) ( $\phi s : \text{List } \mathcal{F}$ ) :
  koszulSignInsert q le  $\phi \phi s = \mathcal{S}(q \phi, \text{ofList } q$ 
    ( $(\text{List.insertionSort le } \phi s).\text{take } (\text{orderedInsertPos le } (\text{List.insertionSort le } \phi s) \phi)$ ))

```

lemma koszulSignCons_eq_exchangeSign[\[source\]](#)

```

lemma koszulSignCons_eq_exchangeSign ( $\phi0 \phi1 : \mathcal{F}$ ) : koszulSignCons q le  $\phi0 \phi1 =$ 
  if le  $\phi0 \phi1$  then 1 else  $\mathcal{S}(q \phi0, q \phi1)$ 

```

lemma koszulSignInsert_of_le_mem[\[source\]](#)

```

lemma koszulSignInsert_of_le_mem ( $\phi0 : \mathcal{F}$ ) : ( $\phi s : \text{List } \mathcal{F}$ )  $\rightarrow$  ( $h : \forall b \in \phi s, \text{le } \phi0 b$ )  $\rightarrow$ 
  koszulSignInsert q le  $\phi0 \phi s = 1$ 
| [], _  $\Rightarrow$  by
  simp [koszulSignInsert]
|  $\phi1 :: \phi s$ ,  $h \Rightarrow$  by
  simp only [koszulSignInsert]
  rw [if_pos]
  · apply koszulSignInsert_of_le_mem
  · intro b hb

```

```

    exact h b (List.mem_cons_of_mem _ hb)
  · exact h φ1 (List.mem_cons_self _ _)

```

lemma koszulSignInsert_eq_rel_eq_stat

[\[source\]](#)

```

lemma koszulSignInsert_eq_rel_eq_stat {ψ φ :  $\mathcal{F}$ } [IsTrans  $\mathcal{F}$  le]
  (h1 : le φ ψ) (h2 : le ψ φ) (hq : q ψ = q φ) : (φs : List  $\mathcal{F}$ ) →
  koszulSignInsert q le φ φs = koszulSignInsert q le ψ φs
| [] => by
  simp [koszulSignInsert]
| φ' :: φs => by
  simp only [koszulSignInsert]
  simp_all only
  by_cases hr : le φ φ'
  · simp only [hr, ↓reduceIte]
    have h1' : le ψ φ'

```

lemma koszulSignInsert_eq_remove_same_stat_append

[\[source\]](#)

```

lemma koszulSignInsert_eq_remove_same_stat_append {ψ φ φ' :  $\mathcal{F}$ } [IsTrans  $\mathcal{F}$  le]
  (h1 : le φ ψ) (h2 : le ψ φ) (hq : q ψ = q φ) : (φs : List  $\mathcal{F}$ ) →
  koszulSignInsert q le φ' (φ :: ψ :: φs) = koszulSignInsert q le φ' φs

```

2.36 HepLean.PerturbationTheory.WickContraction.Basic

[HepLean/PerturbationTheory/WickContraction/Basic.lean](#) on GitHub.

def WickContraction

[\[source\]](#)

Given a natural number n corresponding to the number of fields, a Wick contraction is a finite set of pairs of $\text{Fin } n$, such that no element of $\text{Fin } n$ occurs in more than one pair. For example for $n = 3$ there are 4 Wick contractions: - $\emptyset$, corresponding to the case where no fields are contracted. - $\{\{0, 1\}\}$, corresponding to the case where the field at position 0 and 1 are contracted. - $\{\{0, 2\}\}$, corresponding to the case where the field at position 0 and 2 are contracted. - $\{\{1, 2\}\}$, corresponding to the case where the field at position 1 and 2 are contracted. For $n=4$ some possible Wick contractions are - $\emptyset$, corresponding to the case where no fields are contracted. - $\{\{0, 1\}, \{2, 3\}\}$, corresponding to the case where the field at position 0 and 1 are contracted and the field at position 2 and 3 are contracted. - $\{\{0, 2\}, \{1, 3\}\}$, corresponding to the case where the field at position 0 and 2 are contracted and the field at position 1 and 3 are contracted. etc.

```

def WickContraction (n :  $\mathbb{N}$ ) : Type

```

def empty

[\[source\]](#)

The contraction consisting of no contracted pairs.

```

def empty : WickContraction n

```

def congr

[\[source\]](#)

The equivalence between WickContraction n and WickContraction m derived from a propositional equality of n and m .

```
def congr : {n m : ℕ} → (h : n = m) → WickContraction n ≈ WickContraction m
| n, .(n), rfl => Equiv.refl _
```

lemma congr_refl

[\[source\]](#)

```
@[simp]
lemma congr_refl : c.congr rfl = c
```

lemma congr_contractions

[\[source\]](#)

```
lemma congr_contractions {n m : ℕ} (h : n = m) (c : WickContraction n) :
((congr h) c).1 = Finset.map (Finset.mapEmbedding (finCongr h)).toEmbedding c.1
```

lemma congr_trans

[\[source\]](#)

```
@[simp]
lemma congr_trans {n m o : ℕ} (h1 : n = m) (h2 : m = o) :
(congr h1).trans (congr h2) = congr (h1.trans h2)
```

lemma congr_trans_apply

[\[source\]](#)

```
@[simp]
lemma congr_trans_apply {n m o : ℕ} (h1 : n = m) (h2 : m = o) (c : WickContraction n) :
(congr h2) ((congr h1) c) = congr (h1.trans h2) c
```

def congrLift

[\[source\]](#)

Given a contracted pair in $c : \text{WickContraction } n$ the contracted pair in $\text{congr } h \ c$.

```
def congrLift {n m : ℕ} (h : n = m) {c : WickContraction n} (a : c.1) : (congr h c).1
```

lemma congrLift_rfl

[\[source\]](#)

```
@[simp]
lemma congrLift_rfl {n : ℕ} {c : WickContraction n} :
c.congrLift rfl = id
```

lemma congrLift_injective

[\[source\]](#)

```
lemma congrLift_injective {n m : ℕ} {c : WickContraction n} (h : n = m) :
Function.Injective (c.congrLift h)
```

lemma congrLift_surjective

[\[source\]](#)

```
lemma congrLift_surjective {n m : ℕ} {c : WickContraction n} (h : n = m) :
Function.Surjective (c.congrLift h)
```

lemma congrLift_bijective[\[source\]](#)

```
lemma congrLift_bijective {n m : ℕ} {c : WickContraction n} (h : n = m) :
  Function.Bijective (c.congrLift h)
```

lemma eq_filter_mem_self[\[source\]](#)

```
lemma eq_filter_mem_self : c.1 = Finset.filter (fun x => x ∈ c.1) Finset.univ
```

def getDual?[\[source\]](#)

For a contraction $c : \text{WickContraction } n$ and $i : \text{Fin } n$ the j such that $\{i, j\}$ is a contracted pair in c . If such an j does not exist, this returns none.

```
def getDual? (i : Fin n) : Option (Fin n)
```

lemma getDual?_congr[\[source\]](#)

```
lemma getDual?_congr {n m : ℕ} (h : n = m) (c : WickContraction n) (i : Fin m) :
  (congr h c).getDual? i = Option.map (finCongr h) (c.getDual? (finCongr h.symm i))
```

lemma getDual?_congr_get[\[source\]](#)

```
lemma getDual?_congr_get {n m : ℕ} (h : n = m) (c : WickContraction n) (i : Fin m)
  (hg : ((congr h c).getDual? i).isSome) :
  ((congr h c).getDual? i).get hg =
  (finCongr h ((c.getDual? (finCongr h.symm i)).get (by simp [getDual?_congr] using hg)))
```

lemma getDual?_eq_some_iff_mem[\[source\]](#)

```
lemma getDual?_eq_some_iff_mem (i j : Fin n) :
  c.getDual? i = some j ↔ {i, j} ∈ c.1
```

lemma getDual?_one_eq_none[\[source\]](#)

```
@[simp]
lemma getDual?_one_eq_none (c : WickContraction 1) (i : Fin 1) : c.getDual? i = none
```

lemma getDual?_get_self_mem[\[source\]](#)

```
@[simp]
lemma getDual?_get_self_mem (i : Fin n) (h : (c.getDual? i).isSome) :
  {(c.getDual? i).get h, i} ∈ c.1
```

lemma self_getDual?_get_mem[\[source\]](#)

```
@[simp]
lemma self_getDual?_get_mem (i : Fin n) (h : (c.getDual? i).isSome) :
  {i, (c.getDual? i).get h} ∈ c.1
```


lemma getDual?_eq_some_neq[\[source\]](#)

```
lemma getDual?_eq_some_neq (i j : Fin n) (h : c.getDual? i = some j) :
  ¬ i = j
```

lemma self_neq_getDual?_get[\[source\]](#)

```
@[simp]
lemma self_neq_getDual?_get (i : Fin n) (h : (c.getDual? i).isSome) :
  ¬ i = (c.getDual? i).get h
```

lemma getDual?_get_self_neq[\[source\]](#)

```
@[simp]
lemma getDual?_get_self_neq (i : Fin n) (h : (c.getDual? i).isSome) :
  ¬ (c.getDual? i).get h = i
```

lemma getDual?_isSome_iff[\[source\]](#)

```
lemma getDual?_isSome_iff (i : Fin n) : (c.getDual? i).isSome ↔ ∃ (a : c.1), i ∈ a.1
```

lemma getDual?_isSome_of_mem[\[source\]](#)

```
lemma getDual?_isSome_of_mem (a : c.1) (i : a.1) : (c.getDual? i).isSome
```

lemma getDual?_getDual?_get_get[\[source\]](#)

```
@[simp]
lemma getDual?_getDual?_get_get (i : Fin n) (h : (c.getDual? i).isSome) :
  c.getDual? ((c.getDual? i).get h) = some i
```

lemma getDual?_getDual?_get_isSome[\[source\]](#)

```
lemma getDual?_getDual?_get_isSome (i : Fin n) (h : (c.getDual? i).isSome) :
  (c.getDual? ((c.getDual? i).get h)).isSome
```

lemma getDual?_getDual?_get_not_none[\[source\]](#)

```
lemma getDual?_getDual?_get_not_none (i : Fin n) (h : (c.getDual? i).isSome) :
  ¬ (c.getDual? ((c.getDual? i).get h)) = none
```

def fstFieldOfContract[\[source\]](#)

The smallest of the two positions in a contracted pair given a Wick contraction.

```
def fstFieldOfContract (c : WickContraction n) (a : c.1) : Fin n
```

lemma fstFieldOfContract_congr[\[source\]](#)

```
@[simp]
lemma fstFieldOfContract_congr {n m : ℕ} (h : n = m) (c : WickContraction n) (a : c.1) :
  (congr h c).fstFieldOfContract (c.congrLift h a) = (finCongr h) (c.fstFieldOfContract a)
```

def sndFieldOfContract[\[source\]](#)

The largest of the two positions in a contracted pair given a Wick contraction.

```
def sndFieldOfContract (c : WickContraction n) (a : c.1) : Fin n
```

lemma sndFieldOfContract_congr[\[source\]](#)

```
@[simp]
```

```
lemma sndFieldOfContract_congr {n m : ℕ} (h : n = m) (c : WickContraction n) (a : c.1) :
  (congr h c).sndFieldOfContract (c.congrLift h a) = (finCongr h) (c.sndFieldOfContract a)
```

lemma finset_eq_fstFieldOfContract_sndFieldOfContract[\[source\]](#)

```
lemma finset_eq_fstFieldOfContract_sndFieldOfContract (c : WickContraction n) (a : c.1) :
  a.1 = {c.fstFieldOfContract a, c.sndFieldOfContract a}
```

lemma fstFieldOfContract_neq_sndFieldOfContract[\[source\]](#)

```
lemma fstFieldOfContract_neq_sndFieldOfContract (c : WickContraction n) (a : c.1) :
  c.fstFieldOfContract a ≠ c.sndFieldOfContract a
```

lemma fstFieldOfContract_le_sndFieldOfContract[\[source\]](#)

```
lemma fstFieldOfContract_le_sndFieldOfContract (c : WickContraction n) (a : c.1) :
  c.fstFieldOfContract a ≤ c.sndFieldOfContract a
```

lemma fstFieldOfContract_lt_sndFieldOfContract[\[source\]](#)

```
lemma fstFieldOfContract_lt_sndFieldOfContract (c : WickContraction n) (a : c.1) :
  c.fstFieldOfContract a < c.sndFieldOfContract a
```

lemma fstFieldOfContract_mem[\[source\]](#)

```
@[simp]
```

```
lemma fstFieldOfContract_mem (c : WickContraction n) (a : c.1) :
  c.fstFieldOfContract a ∈ a.1
```

lemma fstFieldOfContract_getDual?_isSome[\[source\]](#)

```
lemma fstFieldOfContract_getDual?_isSome (c : WickContraction n) (a : c.1) :
  (c.getDual? (c.fstFieldOfContract a)).isSome
```

lemma fstFieldOfContract_getDual?[\[source\]](#)

```
@[simp]
```

```
lemma fstFieldOfContract_getDual? (c : WickContraction n) (a : c.1) :
  c.getDual? (c.fstFieldOfContract a) = some (c.sndFieldOfContract a)
```

lemma sndFieldOfContract_mem[\[source\]](#)

```
@[simp]
lemma sndFieldOfContract_mem (c : WickContraction n) (a : c.1) :
  c.sndFieldOfContract a ∈ a.1
```

lemma sndFieldOfContract_getDual?_isSome[\[source\]](#)

```
lemma sndFieldOfContract_getDual?_isSome (c : WickContraction n) (a : c.1) :
  (c.getDual? (c.sndFieldOfContract a)).isSome
```

lemma sndFieldOfContract_getDual?[\[source\]](#)

```
@[simp]
lemma sndFieldOfContract_getDual? (c : WickContraction n) (a : c.1) :
  c.getDual? (c.sndFieldOfContract a) = some (c.fstFieldOfContract a)
```

lemma eq_fstFieldOfContract_of_mem[\[source\]](#)

```
lemma eq_fstFieldOfContract_of_mem (c : WickContraction n) (a : c.1) (i j : Fin n)
  (hi : i ∈ a.1) (hj : j ∈ a.1) (hij : i < j) :
  c.fstFieldOfContract a = i
```

lemma eq_sndFieldOfContract_of_mem[\[source\]](#)

```
lemma eq_sndFieldOfContract_of_mem (c : WickContraction n) (a : c.1) (i j : Fin n)
  (hi : i ∈ a.1) (hj : j ∈ a.1) (hij : i < j) :
  c.sndFieldOfContract a = j
```

def contractEquivFinTwo[\[source\]](#)

As a type, any pair of contractions is equivalent to $\text{Fin } 2$ with 0 being associated with $c.\text{fstFieldOfContract } a$ and 1 being associated with $c.\text{sndFieldOfContract}$.

```
def contractEquivFinTwo (c : WickContraction n) (a : c.1) :
  a ≈ Fin 2 where
```

lemma prod_finset_eq_mul_fst_snd[\[source\]](#)

```
lemma prod_finset_eq_mul_fst_snd (c : WickContraction n) (a : c.1)
  (f : a.1 → M) [CommMonoid M] :
  [] (x : a), f x = f ((c.fstFieldOfContract a, fstFieldOfContract_mem c a))
  * f ((c.sndFieldOfContract a, sndFieldOfContract_mem c a))
```

def GradingCompliant[\[source\]](#)

*A Wick contraction associated with a list of states is said to be **GradingCompliant** if in any contracted pair of states they are either both fermionic or both bosonic.*

```
def GradingCompliant (ϕs : List  $\mathcal{F}$ .States) (ϕsΛ : WickContraction ϕs.length)
```

def sigmaContractedEquiv[\[source\]](#)

An equivalence from the sigma type $(a : c.1) \times a$ to the subtype of $\text{Fin } n$ consisting of those positions which are contracted.

```
def sigmaContractedEquiv : (a : c.1) × a ≈ {x : Fin n // (c.getDual? x).isSome} where
```

2.37 HepLean.PerturbationTheory.WickContraction.Erase

[HepLean/PerturbationTheory/WickContraction/Erase.lean](#) on GitHub.

def erase[\[source\]](#)

Given a Wick contraction $\text{WickContraction } n.\text{succ}$ and a $i : \text{Fin } n.\text{succ}$ the Wick contraction associated with n obtained by removing i . If i is contracted with j in the new Wick contraction j will be uncontracted.

```
def erase (c : WickContraction n.succ) (i : Fin n.succ) : WickContraction n
```

lemma mem_erase_uncontracted_iff[\[source\]](#)

```
lemma mem_erase_uncontracted_iff (c : WickContraction n.succ) (i : Fin n.succ) (j : Fin n) :
  j ∈ (c.erase i).uncontracted ↔
  i.succAbove j ∈ c.uncontracted ∨ c.getDual? (i.succAbove j) = some i
```

lemma mem_not_eq_erase_of_isSome[\[source\]](#)

```
lemma mem_not_eq_erase_of_isSome (c : WickContraction n.succ) (i : Fin n.succ)
  (h : (c.getDual? i).isSome) (ha : a ∈ c.1) (ha2 : a ≠ {i, (c.getDual? i).get h}) :
  ∃ a', a' ∈ (c.erase i).1 ∧ a = Finset.map i.succAboveEmb a'
```

lemma mem_not_eq_erase_of_isNone[\[source\]](#)

```
lemma mem_not_eq_erase_of_isNone (c : WickContraction n.succ) (i : Fin n.succ)
  (h : (c.getDual? i).isNone) (ha : a ∈ c.1) :
  ∃ a', a' ∈ (c.erase i).1 ∧ a = Finset.map i.succAboveEmb a'
```

def getDualErase[\[source\]](#)

Given a Wick contraction $c : \text{WickContraction } n.\text{succ}$ and a $i : \text{Fin } n.\text{succ}$ the (optional) element of $(\text{erase } c \ i).\text{uncontracted}$ which comes from the element in c contracted with i .

```
def getDualErase {n : ℕ} (c : WickContraction n.succ) (i : Fin n.succ) :
  Option ((erase c i).uncontracted)
```

lemma getDualErase_isSome_iff_getDual?_isSome[\[source\]](#)

```
@[simp]
lemma getDualErase_isSome_iff_getDual?_isSome (c : WickContraction n.succ) (i : Fin n.succ) :
  (c.getDualErase i).isSome ↔ (c.getDual? i).isSome
```

lemma getDualErase_one[\[source\]](#)

```
@[simp]
lemma getDualErase_one (c : WickContraction 1) (i : Fin 1) :
  c.getDualErase i = none
```

2.38 HepLean.PerturbationTheory.WickContraction.ExtractEquiv

[HepLean/PerturbationTheory/WickContraction/ExtractEquiv.lean](#) on GitHub.

lemma extractEquiv_equiv[\[source\]](#)

```
lemma extractEquiv_equiv {c1 c2 : (c : WickContraction n) × Option c.uncontracted}
  (h : c1.1 = c2.1) (ho : c1.2 = uncontractedCongr (by rw [h]) c2.2) : c1 = c2
```

def extractEquiv[\[source\]](#)

The equivalence between $WickContraction\ n.succ$ and the sigma type $(c : WickContraction\ n) \times Option\ c.uncontracted$ formed by inserting and erasing elements from a contraction.

```
def extractEquiv (i : Fin n.succ) : WickContraction n.succ =
  (c : WickContraction n) × Option c.uncontracted where
```

lemma extractEquiv_symm_none_uncontracted[\[source\]](#)

```
lemma extractEquiv_symm_none_uncontracted (i : Fin n.succ) (c : WickContraction n) :
  ((extractEquiv i).symm {c, none}).uncontracted =
  (Insert.insert i (c.uncontracted.map i.succAboveEmb))
```

lemma extractEquiv_apply_congr_symm_apply[\[source\]](#)

```
@[simp]
lemma extractEquiv_apply_congr_symm_apply {n m : ℕ} (k : ℕ)
  (hnk : k < n.succ) (hkm : k < m.succ) (hnm : n = m) (c : WickContraction n)
  (i : c.uncontracted) : congr (by rw [hnm]) ((extractEquiv {k, hkm}
    (congr (by rw [hnm]) ((extractEquiv {k, hnk}).symm {c, i}))))).1 = c
```

instance fintype_zero[\[source\]](#)

The fintype instance of $WickContraction\ 0$ defined through its single element empty.

```
instance fintype_zero : Fintype (WickContraction 0) where
```

lemma sum_WickContraction_nil[\[source\]](#)

```
lemma sum_WickContraction_nil (f : WickContraction 0 → M) [AddCommMonoid M] :
  ∑ c, f c = f empty
```

instance fintype_succ[\[source\]](#)

The fintype instance of $WickContraction\ n$, for $n.succ$ this is defined through the equivalence `extractEquiv`.

```

instance fintype_succ : (n : ℕ) → Fintype (WickContraction n)
| 0 => fintype_zero
| Nat.succ n => by
  letI

```

lemma sum_extractEquiv_congr

[\[source\]](#)

```

lemma sum_extractEquiv_congr [AddCommMonoid M] {n m : ℕ} (i : Fin n) (f : WickContraction n → M)
  (h : n = m.succ) :
  ∑ c, f c = ∑ (c : WickContraction m), ∑ (k : Option c.uncontracted),
  f (congr h.symm ((extractEquiv (finCongr h i)).symm (c, k)))

```

2.39 HepLean.PerturbationTheory.WickContraction.InsertAndContract

[HepLean/PerturbationTheory/WickContraction/InsertAndContract.lean](#) on GitHub.

def insertAndContract

[\[source\]](#)

Given a Wick contraction c associated to a list ϕs , a position $i : \text{Fin } n.\text{succ}$, an element ϕ , and an optional uncontracted element $j : \text{Option } (c.\text{uncontracted})$ of c . The Wick contraction associated with $(\phi s.\text{insertIdx } i \ \phi).\text{length}$ formed by 'inserting' ϕ into ϕs after the first i elements and contracting it optionally with j .

```

def insertAndContract {ϕs : List ℱ.States} (ϕ : ℱ.States) (ϕsΛ : WickContraction ϕs.length)
  (i : Fin ϕs.length.succ) (j : Option ϕsΛ.uncontracted) :
  WickContraction (ϕs.insertIdx i ϕ).length

```

lemma insertAndContract_fstFieldOfContract

[\[source\]](#)

```

@[simp]
lemma insertAndContract_fstFieldOfContract (ϕ : ℱ.States) (ϕs : List ℱ.States)
  (ϕsΛ : WickContraction ϕs.length) (i : Fin ϕs.length.succ) (j : Option ϕsΛ.uncontracted)
  (a : ϕsΛ.1) : (ϕsΛ ↦Λ ϕ i j).fstFieldOfContract
  (congrLift (insertIdx_length_fin ϕ ϕs i).symm (insertLift i j a)) =
  finCongr (insertIdx_length_fin ϕ ϕs i).symm (i.succAbove (ϕsΛ.fstFieldOfContract a))

```

lemma insertAndContract_sndFieldOfContract

[\[source\]](#)

```

@[simp]
lemma insertAndContract_sndFieldOfContract (ϕ : ℱ.States) (ϕs : List ℱ.States)
  (ϕsΛ : WickContraction ϕs.length) (i : Fin ϕs.length.succ) (j : Option (ϕsΛ.uncontracted))
  (a : ϕsΛ.1) : (ϕsΛ ↦Λ ϕ i j).sndFieldOfContract
  (congrLift (insertIdx_length_fin ϕ ϕs i).symm (insertLift i j a)) =
  finCongr (insertIdx_length_fin ϕ ϕs i).symm (i.succAbove (ϕsΛ.sndFieldOfContract a))

```

lemma insertAndContract_fstFieldOfContract_some_incl

[\[source\]](#)

```

@[simp]
lemma insertAndContract_fstFieldOfContract_some_incl (ϕ : ℱ.States) (ϕs : List ℱ.States)
  (ϕsΛ : WickContraction ϕs.length) (i : Fin ϕs.length.succ) (j : ϕsΛ.uncontracted) :
  (insertAndContract ϕ ϕsΛ i (some j)).fstFieldOfContract
  (congrLift (insertIdx_length_fin ϕ ϕs i).symm {i, i.succAbove j}, by

```

```

    simp [insertAndContractNat])) =
  if i < i.succAbove j.1 then
    finCongr (insertIdx_length_fin  $\phi$   $\phi$ s i).symm i else
    finCongr (insertIdx_length_fin  $\phi$   $\phi$ s i).symm (i.succAbove j.1)

```

lemma insertAndContract_none_getDual?_self[\[source\]](#)

```

@[simp]
lemma insertAndContract_none_getDual?_self ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) :
  ( $\phi$ s $\Lambda$   $\Leftarrow$   $\phi$  i none).getDual? (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm i) = none

```

lemma insertAndContract_isSome_getDual?_self[\[source\]](#)

```

lemma insertAndContract_isSome_getDual?_self ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) (j :  $\phi$ s $\Lambda$ .uncontracted) :
  (( $\phi$ s $\Lambda$   $\Leftarrow$   $\phi$  i (some j)).getDual?
   (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm i)).isSome

```

lemma insertAndContract_some_getDual?_self_not_none[\[source\]](#)

```

lemma insertAndContract_some_getDual?_self_not_none ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) (j :  $\phi$ s $\Lambda$ .uncontracted) :
   $\neg$  (( $\phi$ s $\Lambda$   $\Leftarrow$   $\phi$  i (some j)).getDual?
    (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm i)) = none

```

lemma insertAndContract_some_getDual?_self_eq[\[source\]](#)

```

@[simp]
lemma insertAndContract_some_getDual?_self_eq ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) (j :  $\phi$ s $\Lambda$ .uncontracted) :
  (( $\phi$ s $\Lambda$   $\Leftarrow$   $\phi$  i (some j)).getDual? (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm i))
  = some (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm (i.succAbove j))

```

lemma insertAndContract_some_getDual?_some_eq[\[source\]](#)

```

@[simp]
lemma insertAndContract_some_getDual?_some_eq ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) (j :  $\phi$ s $\Lambda$ .uncontracted) :
  (( $\phi$ s $\Lambda$   $\Leftarrow$   $\phi$  i (some j)).getDual?
   (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm (i.succAbove j)))
  = some (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm i)

```

lemma insertAndContract_none_succAbove_getDual?_eq_none_iff[\[source\]](#)

```

@[simp]
lemma insertAndContract_none_succAbove_getDual?_eq_none_iff ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) (j : Fin  $\phi$ s.length) :
  ( $\phi$ s $\Lambda$   $\Leftarrow$   $\phi$  i none).getDual? (Fin.cast (insertIdx_length_fin  $\phi$   $\phi$ s i).symm
    (i.succAbove j)) = none  $\leftrightarrow$   $\phi$ s $\Lambda$ .getDual? j = none

```

lemma insertAndContract_some_succAbove_getDual?_eq_option[\[source\]](#)

```
@[simp]
lemma insertAndContract_some_succAbove_getDual?_eq_option (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : Fin φs.length)
  (k : φsΛ.uncontracted) (hkj : j ≠ k.1) :
  (φsΛ  $\hookrightarrow$ Λ φ i (some k)).getDual? (Fin.cast (insertIdx_length_fin φ φs i).symm
    (i.succAbove j)) = Option.map (Fin.cast (insertIdx_length_fin φ φs i).symm ∘ i.succAbove)
    (φsΛ.getDual? j)
```

lemma insertAndContract_none_succAbove_getDual?_isSome_iff[\[source\]](#)

```
@[simp]
lemma insertAndContract_none_succAbove_getDual?_isSome_iff (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : Fin φs.length) :
  ((φsΛ  $\hookrightarrow$ Λ φ i none).getDual? (Fin.cast (insertIdx_length_fin φ φs i).symm
    (i.succAbove j))).isSome  $\leftrightarrow$  (φsΛ.getDual? j).isSome
```

lemma insertAndContract_none_getDual?_get_eq[\[source\]](#)

```
@[simp]
lemma insertAndContract_none_getDual?_get_eq (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : Fin φs.length)
  (h : ((φsΛ  $\hookrightarrow$ Λ φ i none).getDual? (Fin.cast (insertIdx_length_fin φ φs i).symm
    (i.succAbove j))).isSome) :
  ((φsΛ  $\hookrightarrow$ Λ φ i none).getDual? (Fin.cast (insertIdx_length_fin φ φs i).symm
    (i.succAbove j))).get h = Fin.cast (insertIdx_length_fin φ φs i).symm
    (i.succAbove ((φsΛ.getDual? j).get (by simp using h)))
```

lemma insertAndContract_sndFieldOfContract_some_incl[\[source\]](#)

```
@[simp]
lemma insertAndContract_sndFieldOfContract_some_incl (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : φsΛ.uncontracted) :
  (φsΛ  $\hookrightarrow$ Λ φ i (some j)).sndFieldOfContract
  (congrLift (insertIdx_length_fin φ φs i).symm ({i, i.succAbove j}, by
    simp [insertAndContractNat])) =
  if i < i.succAbove j.1 then
  finCongr (insertIdx_length_fin φ φs i).symm (i.succAbove j.1) else
  finCongr (insertIdx_length_fin φ φs i).symm i
```

lemma insertAndContract_none_prod_contractions[\[source\]](#)

```
lemma insertAndContract_none_prod_contractions (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ)
  (f : (φsΛ  $\hookrightarrow$ Λ φ i none).1  $\rightarrow$  M) [CommMonoid M] :
  [] a, f a = [] (a : φsΛ.1), f (congrLift (insertIdx_length_fin φ φs i).symm
    (insertLift i none a))
```

lemma insertAndContract_some_prod_contractions[\[source\]](#)

```
lemma insertAndContract_some_prod_contractions (φ :  $\mathcal{F}$ .States) (φs : List  $\mathcal{F}$ .States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : φsΛ.uncontracted)
  (f : (φsΛ  $\hookrightarrow$ Λ φ i (some j)).1  $\rightarrow$  M) [CommMonoid M] :
  [] a, f a = f (congrLift (insertIdx_length_fin φ φs i).symm
```



```
{i, i.succAbove j}, by simp [insertAndContractNat])) *
[] (a :  $\varphi s \Lambda.1$ ), f (congrLift (insertIdx_length_fin  $\varphi$   $\varphi s$  i).symm (insertLift i (some j) a))
```

def insertAndContractLiftFinset

[\[source\]](#)

Given a finite set of $\text{Fin } \varphi s.\text{length}$ the finite set of $(\varphi s.\text{insertIdx } i \ \varphi).\text{length}$ formed by mapping elements using $i.\text{succAboveEmb}$ and finCongr .

```
def insertAndContractLiftFinset ( $\varphi : \mathcal{F}.\text{States}$ ) { $\varphi s : \text{List } \mathcal{F}.\text{States}$ }
  (i :  $\text{Fin } \varphi s.\text{length}.\text{succ}$ ) (a :  $\text{Finset } (\text{Fin } \varphi s.\text{length})$ ) :
   $\text{Finset } (\text{Fin } (\varphi s.\text{insertIdx } i \ \varphi).\text{length})$ 
```

lemma self_not_mem_insertAndContractLiftFinset

[\[source\]](#)

```
@[simp]
lemma self_not_mem_insertAndContractLiftFinset ( $\varphi : \mathcal{F}.\text{States}$ ) { $\varphi s : \text{List } \mathcal{F}.\text{States}$ }
  (i :  $\text{Fin } \varphi s.\text{length}.\text{succ}$ ) (a :  $\text{Finset } (\text{Fin } \varphi s.\text{length})$ ) :
   $\text{Fin.cast } (\text{insertIdx\_length\_fin } \varphi \ \varphi s \ i).\text{symm } i \notin \text{insertAndContractLiftFinset } \varphi \ i \ a$ 
```

lemma succAbove_mem_insertAndContractLiftFinset

[\[source\]](#)

```
lemma succAbove_mem_insertAndContractLiftFinset ( $\varphi : \mathcal{F}.\text{States}$ ) { $\varphi s : \text{List } \mathcal{F}.\text{States}$ }
  (i :  $\text{Fin } \varphi s.\text{length}.\text{succ}$ ) (a :  $\text{Finset } (\text{Fin } \varphi s.\text{length})$ ) (j :  $\text{Fin } \varphi s.\text{length}$ ) :
   $\text{Fin.cast } (\text{insertIdx\_length\_fin } \varphi \ \varphi s \ i).\text{symm } (i.\text{succAbove } j) \in \text{insertAndContractLiftFinset } \varphi \ i \ a \leftrightarrow j \in a$ 
```

lemma insert_fin_eq_self

[\[source\]](#)

```
lemma insert_fin_eq_self ( $\varphi : \mathcal{F}.\text{States}$ ) { $\varphi s : \text{List } \mathcal{F}.\text{States}$ }
  (i :  $\text{Fin } \varphi s.\text{length}.\text{succ}$ ) (j :  $\text{Fin } (\text{List.insertIdx } i \ \varphi \ \varphi s).\text{length}$ ) :
  j =  $\text{Fin.cast } (\text{insertIdx\_length\_fin } \varphi \ \varphi s \ i).\text{symm } i$ 
   $\vee \exists k, j = \text{Fin.cast } (\text{insertIdx\_length\_fin } \varphi \ \varphi s \ i).\text{symm } (i.\text{succAbove } k)$ 
```

lemma insertLift_sum

[\[source\]](#)

```
lemma insertLift_sum ( $\varphi : \mathcal{F}.\text{States}$ ) { $\varphi s : \text{List } \mathcal{F}.\text{States}$ }
  (i :  $\text{Fin } \varphi s.\text{length}.\text{succ}$ ) [AddCommMonoid M] (f :  $\text{WickContraction } (\varphi s.\text{insertIdx } i \ \varphi).\text{length} \rightarrow M$ ) :
   $\sum c, f \ c =$ 
   $\sum (\varphi s \Lambda : \text{WickContraction } \varphi s.\text{length}), \sum (k : \text{Option } \varphi s \Lambda.\text{uncontracted}), f \ (\varphi s \Lambda \hookleftarrow \Lambda \ \varphi \ i \ k)$ 
```

lemma insertAndContract_uncontractedList_none_map

[\[source\]](#)

```
lemma insertAndContract_uncontractedList_none_map ( $\varphi : \mathcal{F}.\text{States}$ ) { $\varphi s : \text{List } \mathcal{F}.\text{States}$ }
  ( $\varphi s \Lambda : \text{WickContraction } \varphi s.\text{length}$ ) (i :  $\text{Fin } \varphi s.\text{length}.\text{succ}$ ) :
   $[\varphi s \Lambda \hookleftarrow \Lambda \ \varphi \ i \ \text{none}]^{uc} = \text{List.insertIdx } (\varphi s \Lambda.\text{uncontractedListOrderPos } i) \ \varphi \ [\varphi s \Lambda]^{uc}$ 
```

lemma insertAndContract_uncontractedList_none_zero

[\[source\]](#)

```
@[simp]
lemma insertAndContract_uncontractedList_none_zero ( $\varphi : \mathcal{F}.\text{States}$ ) { $\varphi s : \text{List } \mathcal{F}.\text{States}$ }
  ( $\varphi s \Lambda : \text{WickContraction } \varphi s.\text{length}$ ) :
   $[\varphi s \Lambda \hookleftarrow \Lambda \ \varphi \ 0 \ \text{none}]^{uc} = \varphi :: [\varphi s \Lambda]^{uc}$ 
```

2.40 HepLean.PerturbationTheory.WickContraction.InsertAndContractNat

[HepLean/PerturbationTheory/WickContraction/InsertAndContractNat.lean](#) on GitHub.

def insertAndContractNat

[\[source\]](#)

Given a Wick contraction c for n , a position $i : \text{Fin } n.\text{succ}$ and an optional uncontracted element $j : \text{Option } (c.\text{uncontracted})$ of c . The Wick contraction for $n.\text{succ}$ formed by 'inserting' i into $\text{Fin } n$ and contracting it optionally with j .

```
def insertAndContractNat (c : WickContraction n) (i : Fin n.succ) (j : Option (c.uncontracted))
  :
  WickContraction n.succ
```

lemma insertAndContractNat_of_isSome

[\[source\]](#)

```
lemma insertAndContractNat_of_isSome (c : WickContraction n) (i : Fin n.succ)
  (j : Option c.uncontracted) (hj : j.isSome) :
  (insertAndContractNat c i j).1 = Insert.insert {i, i.succAbove (j.get hj)}
  (Finset.map (Finset.mapEmbedding i.succAboveEmb).toEmbedding c.1)
```

lemma self_mem_uncontracted_of_insertAndContractNat_none

[\[source\]](#)

```
@[simp]
lemma self_mem_uncontracted_of_insertAndContractNat_none (c : WickContraction n) (i : Fin n.
  succ) :
  i ∈ (insertAndContractNat c i none).uncontracted
```

lemma self_not_mem_uncontracted_of_insertAndContractNat_some

[\[source\]](#)

```
@[simp]
lemma self_not_mem_uncontracted_of_insertAndContractNat_some (c : WickContraction n)
  (i : Fin n.succ) (j : c.uncontracted) :
  i ∉ (insertAndContractNat c i (some j)).uncontracted
```

lemma insertAndContractNat_succAbove_mem_uncontracted_iff

[\[source\]](#)

```
lemma insertAndContractNat_succAbove_mem_uncontracted_iff (c : WickContraction n) (i : Fin n.
  succ)
  (j : Fin n) :
  (i.succAbove j) ∈ (insertAndContractNat c i none).uncontracted ↔ j ∈ c.uncontracted
```

lemma mem_uncontracted_insertAndContractNat_none_iff

[\[source\]](#)

```
@[simp]
lemma mem_uncontracted_insertAndContractNat_none_iff (c : WickContraction n) (i : Fin n.succ)
  (k : Fin n.succ) : k ∈ (insertAndContractNat c i none).uncontracted ↔
  k = i ∨ ∃ j, k = i.succAbove j ∧ j ∈ c.uncontracted
```

lemma insertAndContractNat_none_uncontracted

[\[source\]](#)

```
lemma insertAndContractNat_none_uncontracted (c : WickContraction n) (i : Fin n.succ) :
  (insertAndContractNat c i none).uncontracted =
  Insert.insert i (c.uncontracted.map i.succAboveEmb)
```

lemma mem_uncontracted_insertAndContractNat_some_iff[\[source\]](#)

```
@[simp]
lemma mem_uncontracted_insertAndContractNat_some_iff (c : WickContraction n) (i : Fin n.succ)
  (k : Fin n.succ) (j : c.uncontracted) :
  k ∈ (insertAndContractNat c i (some j)).uncontracted ↔
  ∃ z, k = i.succAbove z ∧ z ∈ c.uncontracted ∧ z ≠ j
```

lemma insertAndContractNat_some_uncontracted[\[source\]](#)

```
lemma insertAndContractNat_some_uncontracted (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) :
  (insertAndContractNat c i (some j)).uncontracted =
  (c.uncontracted.erase j).map i.succAboveEmb
```

lemma insertAndContractNat_none_getDual?_isNone[\[source\]](#)

```
lemma insertAndContractNat_none_getDual?_isNone (c : WickContraction n) (i : Fin n.succ) :
  ((insertAndContractNat c i none).getDual? i).isNone
```

lemma insertAndContractNat_succAbove_getDual?_eq_none_iff[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_succAbove_getDual?_eq_none_iff (c : WickContraction n) (i : Fin n.succ)
  (j : Fin n) :
  (insertAndContractNat c i none).getDual? (i.succAbove j) = none ↔ c.getDual? j = none
```

lemma insertAndContractNat_succAbove_getDual?_isSome_iff[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_succAbove_getDual?_isSome_iff (c : WickContraction n) (i : Fin n.succ)
  (j : Fin n) :
  ((insertAndContractNat c i none).getDual? (i.succAbove j)).isSome ↔ (c.getDual? j).isSome
```

lemma insertAndContractNat_succAbove_getDual?_get[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_succAbove_getDual?_get (c : WickContraction n) (i : Fin n.succ)
  (j : Fin n) (h : ((insertAndContractNat c i none).getDual? (i.succAbove j)).isSome) :
  ((insertAndContractNat c i none).getDual? (i.succAbove j)).get h =
  i.succAbove ((c.getDual? j).get (by simp using h))
```

lemma insertAndContractNat_some_getDual?_eq[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_some_getDual?_eq (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) :
  (insertAndContractNat c i (some j)).getDual? i = some (i.succAbove j)
```

lemma insertAndContractNat_some_getDual?_neq_none[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_some_getDual?_neq_none (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) (k : Fin n) (hkj : k ≠ j.1) :
  (insertAndContractNat c i (some j)).getDual? (i.succAbove k) = none ↔ c.getDual? k = none
```

lemma insertAndContractNat_some_getDual?_neq_isSome[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_some_getDual?_neq_isSome (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) (k : Fin n) (hkj : k ≠ j.1) :
  ((insertAndContractNat c i (some j)).getDual? (i.succAbove k)).isSome ↔
  (c.getDual? k).isSome
```

lemma insertAndContractNat_some_getDual?_neq_isSome_get[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_some_getDual?_neq_isSome_get (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) (k : Fin n) (hkj : k ≠ j.1)
  (h : ((insertAndContractNat c i (some j)).getDual? (i.succAbove k)).isSome) :
  ((insertAndContractNat c i (some j)).getDual? (i.succAbove k)).get h =
  i.succAbove ((c.getDual? k).get (by simp [hkj] using h))
```

lemma insertAndContractNat_some_getDual?_of_neq[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_some_getDual?_of_neq (c : WickContraction n) (i : Fin n.succ)
  (j : c.uncontracted) (k : Fin n) (hkj : k ≠ j.1) :
  (insertAndContractNat c i (some j)).getDual? (i.succAbove k) =
  Option.map i.succAbove (c.getDual? k)
```

lemma insertAndContractNat_erase[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_erase (c : WickContraction n) (i : Fin n.succ)
  (j : Option c.uncontracted) : erase (insertAndContractNat c i j) i = c
```

lemma insertAndContractNat_getDualErase[\[source\]](#)

```
lemma insertAndContractNat_getDualErase (c : WickContraction n) (i : Fin n.succ)
  (j : Option c.uncontracted) : (insertAndContractNat c i j).getDualErase i =
  uncontractedCongr (c := c) (c' := (c.insertAndContractNat i j).erase i) (by simp) j
```

lemma erase_insert[\[source\]](#)

```
@[simp]
lemma erase_insert (c : WickContraction n.succ) (i : Fin n.succ) :
  insertAndContractNat (erase c i) i (getDualErase c i) = c
```

def insertLift[\[source\]](#)

Lifts a contraction in c to a contraction in $(c.insert\ i\ j)$.

```
def insertLift {c : WickContraction n} (i : Fin n.succ) (j : Option (c.uncontracted))
  (a : c.1) : (c.insertAndContractNat i j).1
```

lemma insertLift_injective[\[source\]](#)

```
lemma insertLift_injective {c : WickContraction n} (i : Fin n.succ) (j : Option (c.uncontracted
)) :
  Function.Injective (insertLift i j)
```

lemma insertLift_none_surjective[\[source\]](#)

```
lemma insertLift_none_surjective {c : WickContraction n} (i : Fin n.succ) :
  Function.Surjective (c.insertLift i none)
```

lemma insertLift_none_bijective[\[source\]](#)

```
lemma insertLift_none_bijective {c : WickContraction n} (i : Fin n.succ) :
  Function.Bijective (c.insertLift i none)
```

lemma insertAndContractNat_fstFieldOfContract[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_fstFieldOfContract (c : WickContraction n) (i : Fin n.succ)
  (j : Option (c.uncontracted)) (a : c.1) :
  (c.insertAndContractNat i j).fstFieldOfContract (insertLift i j a) =
    i.succAbove (c.fstFieldOfContract a)
```

lemma insertAndContractNat_sndFieldOfContract[\[source\]](#)

```
@[simp]
lemma insertAndContractNat_sndFieldOfContract (c : WickContraction n) (i : Fin n.succ)
  (j : Option (c.uncontracted)) (a : c.1) :
  (c.insertAndContractNat i j).sndFieldOfContract (insertLift i j a) =
    i.succAbove (c.sndFieldOfContract a)
```

def insertLiftSome[\[source\]](#)

Given a contracted pair for a Wick contraction $WickContraction\ n$, the corresponding contracted pair of a wick contraction $(c.insert\ i\ (some\ j))$ formed by inserting an element i into the contraction.

```
def insertLiftSome {c : WickContraction n} (i : Fin n.succ) (j : c.uncontracted)
  (a : Unit  $\oplus$  c.1) : (c.insertAndContractNat i (some j)).1
```

lemma insertLiftSome_injective[\[source\]](#)

```
lemma insertLiftSome_injective {c : WickContraction n} (i : Fin n.succ) (j : c.uncontracted) :
  Function.Injective (insertLiftSome i j)
```

lemma insertLiftSome_surjective[\[source\]](#)

```
lemma insertLiftSome_surjective {c : WickContraction n} (i : Fin n.succ) (j : c.uncontracted) :
  Function.Surjective (insertLiftSome i j)
```

lemma insertLiftSome_bijective[\[source\]](#)

```
lemma insertLiftSome_bijective {c : WickContraction n} (i : Fin n.succ) (j : c.uncontracted) :
  Function.Bijective (insertLiftSome i j)
```

2.41 HepLean.PerturbationTheory.WickContraction.Involutions

[HepLean/PerturbationTheory/WickContraction/Involutions.lean](#) on GitHub.

def toInvolution[\[source\]](#)

The involution of $\text{Fin } n$ associated with a Wick contraction $c : \text{WickContraction } n$ as follows. If $i : \text{Fin } n$ is contracted in c then it is taken to its dual, otherwise it is taken to itself.

```
def toInvolution : {f : Fin n → Fin n // Function.Involutive f}
```

def fromInvolution[\[source\]](#)

The Wick contraction formed by an involution f of $\text{Fin } n$ by taking as the contracted sets of the contraction the orbits of f of cardinality 2.

```
def fromInvolution (f : {f : Fin n → Fin n // Function.Involutive f}) : WickContraction n
```

lemma fromInvolution_getDual?_isSome[\[source\]](#)

```
@[simp]
```

```
lemma fromInvolution_getDual?_isSome (f : {f : Fin n → Fin n // Function.Involutive f})
  (i : Fin n) : ((fromInvolution f).getDual? i).isSome ↔ f.1 i ≠ i
```

lemma fromInvolution_getDual?_eq_some[\[source\]](#)

```
lemma fromInvolution_getDual?_eq_some (f : {f : Fin n → Fin n // Function.Involutive f}) (i :
  Fin n)
  (h : ((fromInvolution f).getDual? i).isSome) :
  ((fromInvolution f).getDual? i) = some (f.1 i)
```

lemma fromInvolution_getDual?_get[\[source\]](#)

```
@[simp]
```

```
lemma fromInvolution_getDual?_get (f : {f : Fin n → Fin n // Function.Involutive f}) (i : Fin n)
  (h : ((fromInvolution f).getDual? i).isSome) :
  ((fromInvolution f).getDual? i).get h = (f.1 i)
```

lemma toInvolution_fromInvolution[\[source\]](#)

```
lemma toInvolution_fromInvolution : fromInvolution c.toInvolution = c
```

lemma fromInvolution_toInvolution[\[source\]](#)

```
lemma fromInvolution_toInvolution (f : {f : Fin n → Fin n // Function.Involutive f}) :
  (fromInvolution f).toInvolution = f
```

def equivInvolution[\[source\]](#)

The equivalence between Wick contractions for n and involutions of $\text{Fin } n$. The involution of $\text{Fin } n$ associated with a Wick contraction $c : \text{WickContraction } n$ as follows. If $i : \text{Fin } n$ is contracted in c then it is taken to its dual, otherwise it is taken to itself.

```
def equivInvolution : WickContraction n ≈ {f : Fin n → Fin n // Function.Involutive f} where
```

2.42 HepLean.PerturbationTheory.WickContraction.IsFull

[HepLean/PerturbationTheory/WickContraction/IsFull.lean](#) on GitHub.

def IsFull[\[source\]](#)

A contraction is full if there are no uncontracted fields, i.e. the finite set of uncontracted fields is empty.

```
def IsFull : Prop
```

instance Decidable (IsFull c)[\[source\]](#)

The condition on whether or not a contraction is full is decidable.

```
instance : Decidable (IsFull c)
```

lemma isFull_iff_equivInvolution_no_fixed_point[\[source\]](#)

```
lemma isFull_iff_equivInvolution_no_fixed_point :
  IsFull c ↔ ∀ (i : Fin n), (equivInvolution c).1 i ≠ i
```

def isFullInvolutionEquiv[\[source\]](#)

The equivalence between full contractions and fixed-point free involutions.

```
def isFullInvolutionEquiv : {c : WickContraction n // IsFull c} ≈
  {f : Fin n → Fin n // Function.Involutive f ∧ (∀ i, f i ≠ i)} where
```

theorem card_of_isfull_even[\[source\]](#)

If n is even then the number of full contractions is $(n-1)!!$.

```
theorem card_of_isfull_even (he : Even n) :
  Fintype.card {c : WickContraction n // IsFull c} = (n - 1)!!
```

theorem card_of_isfull_odd[\[source\]](#)

If n is odd then there are no full contractions. This is because there will always be at least one element unpaired.

```
theorem card_of_isfull_odd (ho : Odd n) :
  Fintype.card {c : WickContraction n // IsFull c} = 0
```

2.43 HepLean.PerturbationTheory.WickContraction.Sign

[HepLean/PerturbationTheory/WickContraction/Sign.lean](#) on GitHub.

def signFinset[\[source\]](#)

Given a Wick contraction $c : \text{WickContraction } n$ and $i1\ i2 : \text{Fin } n$ the finite set of elements of $\text{Fin } n$ between $i1$ and $i2$ which are either uncontracted or are contracted but are contracted with an element occurring after $i1$. One should assume $i1 < i2$ otherwise this finite set is empty.

```
def signFinset (c : WickContraction n) (i1 i2 : Fin n) : Finset (Fin n)
```

lemma signFinset_insertAndContract_none[\[source\]](#)

```
lemma signFinset_insertAndContract_none (φ : F.States) (φs : List F.States)
  (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (i1 i2 : Fin φs.length) :
  (φsΛ ↪Λ φ i none).signFinset (finCongr (insertIdx_length_fin φ φs i).symm
    (i.succAbove i1)) (finCongr (insertIdx_length_fin φ φs i).symm (i.succAbove i2)) =
  if i.succAbove i1 < i ∧ i < i.succAbove i2 then
    Insert.insert (finCongr (insertIdx_length_fin φ φs i).symm i)
      (insertAndContractLiftFinset φ i (φsΛ.signFinset i1 i2))
  else
    (insertAndContractLiftFinset φ i (φsΛ.signFinset i1 i2))
```

lemma stat_ofFinset_of_insertAndContractLiftFinset[\[source\]](#)

```
lemma stat_ofFinset_of_insertAndContractLiftFinset (φ : F.States) (φs : List F.States)
  (i : Fin φs.length.succ) (a : Finset (Fin φs.length)) :
  (F |>_s ((φs.insertIdx i φ).get, insertAndContractLiftFinset φ i a)) = F |>_s (φs.get, a)
```

lemma stat_ofFinset_eq_one_of_gradingCompliant[\[source\]](#)

```
lemma stat_ofFinset_eq_one_of_gradingCompliant (φs : List F.States)
  (a : Finset (Fin φs.length)) (φsΛ : WickContraction φs.length) (hg : GradingCompliant φs φs
    Λ)
  (hnon : ∀ i, φsΛ.getDual? i = none → i ∉ a)
  (hsom : ∀ i, (h : (φsΛ.getDual? i).isSome) → i ∈ a → (φsΛ.getDual? i).get h ∈ a) :
  (F |>_s (φs.get, a)) = 1
```

lemma signFinset_insertAndContract_some[\[source\]](#)

```
lemma signFinset_insertAndContract_some (φ : F.States) (φs : List F.States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (i1 i2 : Fin φs.length)
  (j : φsΛ.uncontracted) :
  (φsΛ ↪Λ φ i (some j)).signFinset (finCongr (insertIdx_length_fin φ φs i).symm
```



```

(i.succAbove i1)) (finCongr (insertIdx_length_fin  $\phi$   $\phi$ s i).symm (i.succAbove i2)) =
if i.succAbove i1 < i  $\wedge$  i < i.succAbove i2  $\wedge$  (i1 < j) then
  Insert.insert (finCongr (insertIdx_length_fin  $\phi$   $\phi$ s i).symm i)
  (insertAndContractLiftFinset  $\phi$  i ( $\phi$ s $\wedge$ .signFinset i1 i2))
else
  if i1 < j  $\wedge$  j < i2  $\wedge$   $\neg$  i.succAbove i1 < i then
    (insertAndContractLiftFinset  $\phi$  i ( $\phi$ s $\wedge$ .signFinset i1 i2)).erase
    (finCongr (insertIdx_length_fin  $\phi$   $\phi$ s i).symm (i.succAbove j))
  else
    (insertAndContractLiftFinset  $\phi$  i ( $\phi$ s $\wedge$ .signFinset i1 i2))

```

def sign[\[source\]](#)

Given a Wick contraction c associated with a list of states ϕ s the sign associated with c is sign corresponding to the number of fermionic-fermionic exchanges one must do to put elements in contracted pairs of c next to each other. It is important to note that this sign does not depend on any ordering placed on ϕ s other than the order of the list itself.

```
def sign ( $\phi$ s : List  $\mathcal{F}$ .States) ( $\phi$ s $\wedge$  : WickContraction  $\phi$ s.length) :  $\mathbb{C}$ 
```

def signInsertNone[\[source\]](#)

Given a Wick contraction c associated with a list of states ϕ s and an $i : \text{Fin } \phi$ s.length.succ, the change in sign of the contraction associated with inserting ϕ into ϕ s at position i without contracting it.

```
def signInsertNone ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States) ( $\phi$ s $\wedge$  : WickContraction  $\phi$ s.length)
(i : Fin  $\phi$ s.length.succ) :  $\mathbb{C}$ 
```

lemma sign_insert_none[\[source\]](#)

```
lemma sign_insert_none ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States) ( $\phi$ s $\wedge$  : WickContraction  $\phi$ s.length)
(i : Fin  $\phi$ s.length.succ) :
( $\phi$ s $\wedge$   $\hookrightarrow$   $\phi$  i none).sign = ( $\phi$ s $\wedge$ .signInsertNone  $\phi$   $\phi$ s i) *  $\phi$ s $\wedge$ .sign
```

lemma signInsertNone_eq_mul_fst_snd[\[source\]](#)

```
lemma signInsertNone_eq_mul_fst_snd ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
( $\phi$ s $\wedge$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) :
 $\phi$ s $\wedge$ .signInsertNone  $\phi$   $\phi$ s i =  $\prod$  (a :  $\phi$ s $\wedge$ .1),
  (if i.succAbove ( $\phi$ s $\wedge$ .fstFieldOfContract a) < i then
     $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi$ s[ $\phi$ s $\wedge$ .sndFieldOfContract a])
  else 1) *
  (if i.succAbove ( $\phi$ s $\wedge$ .sndFieldOfContract a) < i then
     $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi$ s[ $\phi$ s $\wedge$ .sndFieldOfContract a])
  else 1)
```

lemma signInsertNone_eq_prod_prod[\[source\]](#)

```
lemma signInsertNone_eq_prod_prod ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi$ s : List  $\mathcal{F}$ .States)
( $\phi$ s $\wedge$  : WickContraction  $\phi$ s.length) (i : Fin  $\phi$ s.length.succ) (hG : GradingCompliant  $\phi$ s  $\phi$ s $\wedge$ ) :
 $\phi$ s $\wedge$ .signInsertNone  $\phi$   $\phi$ s i =  $\prod$  (a :  $\phi$ s $\wedge$ .1),  $\prod$  (x : a),
  (if i.succAbove x < i then  $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi$ s[x.1]) else 1)
```

lemma sign_insert_none_zero[\[source\]](#)

```

lemma sign_insert_none_zero ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi s$  : List  $\mathcal{F}$ .States) ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) :
  ( $\phi s \Lambda \hookrightarrow \Lambda \phi \ 0$  none).sign =  $\phi s \Lambda$ .sign

```

lemma signInsertNone_eq_prod_getDual?_Some[\[source\]](#)

```

lemma signInsertNone_eq_prod_getDual?_Some ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi s$  : List  $\mathcal{F}$ .States)
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) ( $i$  : Fin  $\phi s$ .length.succ) ( $hG$  : GradingCompliant  $\phi s \ \phi s \Lambda$ ) :
   $\phi s \Lambda$ .signInsertNone  $\phi \ \phi s \ i$  =  $\prod (x : \text{Fin } \phi s$ .length),
    if ( $\phi s \Lambda$ .getDual?  $x$ ).isSome then
      (if  $i$ .succAbove  $x < i$  then  $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi s[x.1])$  else 1)
    else 1

```

lemma signInsertNone_eq_filter_map[\[source\]](#)

```

lemma signInsertNone_eq_filter_map ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi s$  : List  $\mathcal{F}$ .States)
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length) ( $i$  : Fin  $\phi s$ .length.succ) ( $hG$  : GradingCompliant  $\phi s \ \phi s \Lambda$ ) :
   $\phi s \Lambda$ .signInsertNone  $\phi \ \phi s \ i$  =
   $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} ((\text{List.filter } (\text{fun } x \Rightarrow (\phi s \Lambda$ .getDual?  $x$ ).isSome  $\wedge i$ .succAbove  $x < i$ )
    (List.finRange  $\phi s$ .length)).map  $\phi s$ .get))

```

lemma signInsertNone_eq_filterset[\[source\]](#)

```

lemma signInsertNone_eq_filterset ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi s$  : List  $\mathcal{F}$ .States)
  ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $i$  : Fin  $\phi s$ .length.succ) ( $hG$  : GradingCompliant  $\phi s \ \phi s \Lambda$ ) :
   $\phi s \Lambda$ .signInsertNone  $\phi \ \phi s \ i$  =  $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \langle \phi s$ .get, Finset.univ.filter
    (fun  $x \Rightarrow (\phi s \Lambda$ .getDual?  $x$ ).isSome  $\wedge i$ .succAbove  $x < i$ ))

```

def signInsertSomeProd[\[source\]](#)

Given a Wick contraction c associated with a list of states ϕs and an $i : \text{Fin } \phi s$.length.succ, the change in sign of the contraction associated with inserting ϕ into ϕs at position i and contracting it with $j : c$.uncontracted coming from contractions other than the i and j contraction but which are effected by this new contraction.

```

def signInsertSomeProd ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi s$  : List  $\mathcal{F}$ .States) ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $i$  : Fin  $\phi s$ .length.succ) ( $j$  :  $\phi s \Lambda$ .uncontracted) :  $\mathbb{C}$ 

```

def signInsertSomeCoef[\[source\]](#)

Given a Wick contraction c associated with a list of states ϕs and an $i : \text{Fin } \phi s$.length.succ, the change in sign of the contraction associated with inserting ϕ into ϕs at position i and contracting it with $j : c$.uncontracted coming from putting i next to j .

```

def signInsertSomeCoef ( $\phi$  :  $\mathcal{F}$ .States) ( $\phi s$  : List  $\mathcal{F}$ .States) ( $\phi s \Lambda$  : WickContraction  $\phi s$ .length)
  ( $i$  : Fin  $\phi s$ .length.succ) ( $j$  :  $\phi s \Lambda$ .uncontracted) :  $\mathbb{C}$ 

```

def signInsertSome[\[source\]](#)

Given a Wick contraction c associated with a list of states ϕs and an $i : \text{Fin } \phi s.\text{length}.\text{succ}$, the change in sign of the contraction associated with inserting ϕ into ϕs at position i and contracting it with $j : c.\text{uncontracted}$.

```
def signInsertSome (φ : F.States) (φs : List F.States) (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) : ℂ
```

lemma sign_insert_some[\[source\]](#)

```
lemma sign_insert_some (φ : F.States) (φs : List F.States) (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) :
  (φsΛ ↪Λ φ i (some j)).sign = (φsΛ.signInsertSome φ φs i j) * φsΛ.sign
```

lemma signInsertSomeProd_eq_one_if[\[source\]](#)

```
lemma signInsertSomeProd_eq_one_if (φ : F.States) (φs : List F.States)
  (φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (j : φsΛ.uncontracted)
  (hφj : (F |>_s φ) = (F |>_s φs[j.1])) :
  φsΛ.signInsertSomeProd φ φs i j =
  [] (a : φsΛ.1),
  if (φsΛ.fstFieldOfContract a) < j
    ∧ (i.succAbove (φsΛ.fstFieldOfContract a) < i ∧ i < i.succAbove (φsΛ.sndFieldOfContract a)
    )
    ∨ j < (φsΛ.sndFieldOfContract a) ∧ ¬ i.succAbove (φsΛ.fstFieldOfContract a) < i)
  then
    S(F |>_s φ, F |>_s φs[φsΛ.sndFieldOfContract a])
  else
    1
```

lemma signInsertSomeProd_eq_prod_prod[\[source\]](#)

```
lemma signInsertSomeProd_eq_prod_prod (φ : F.States) (φs : List F.States)
  (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) (hφj : (F |>_s φ) = (F |>_s φs[j.1]))
  (hg : GradingCompliant φs φsΛ) :
  φsΛ.signInsertSomeProd φ φs i j =
  [] (a : φsΛ.1), [] (x : a.1), if x.1 < j
    ∧ (i.succAbove x.1 < i ∧
    i < i.succAbove ((φsΛ.getDual? x.1).get (getDual?_isSome_of_mem φsΛ a x))
    ∨ j < ((φsΛ.getDual? x.1).get (getDual?_isSome_of_mem φsΛ a x)) ∧ ¬ i.succAbove x < i)
  then
    S(F |>_s φ, F |>_s φs[x.1])
  else
    1
```

lemma signInsertSomeProd_eq_prod_fin[\[source\]](#)

```
lemma signInsertSomeProd_eq_prod_fin (φ : F.States) (φs : List F.States)
  (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) (hφj : (F |>_s φ) = (F |>_s φs[j.1]))
  (hg : GradingCompliant φs φsΛ) :
  φsΛ.signInsertSomeProd φ φs i j =
  [] (x : Fin φs.length),
```

```

if h : (φsΛ.getDual? x).isSome then
  if x < j ∧ (i.succAbove x < i ∧ i < i.succAbove ((φsΛ.getDual? x).get h)
    ∨ j < ((φsΛ.getDual? x).get h) ∧ ¬ i.succAbove x < i)
  then S(ℱ |>s φ, ℱ |>s φs[x.1])
  else 1
else 1

```

lemma signInsertSomeProd_eq_list[\[source\]](#)

```

lemma signInsertSomeProd_eq_list (φ : ℱ.States) (φs : List ℱ.States)
  (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) (hφj : (ℱ |>s φ) = (ℱ |>s φs[j.1]))
  (hg : GradingCompliant φs φsΛ) :
  φsΛ.signInsertSomeProd φ φs i j =
  S(ℱ |>s φ, ℱ |>s (List.filter (fun x => (φsΛ.getDual? x).isSome ∧
    ∀ (h : (φsΛ.getDual? x).isSome), x < j ∧ (i.succAbove x < i ∧
    i < i.succAbove ((φsΛ.getDual? x).get h)
    ∨ j < ((φsΛ.getDual? x).get h) ∧ ¬ i.succAbove x < i))
    (List.finRange φs.length)).map φs.get)

```

lemma signInsertSomeProd_eq_finset[\[source\]](#)

```

lemma signInsertSomeProd_eq_finset (φ : ℱ.States) (φs : List ℱ.States)
  (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) (hφj : (ℱ |>s φ) = (ℱ |>s φs[j.1]))
  (hg : GradingCompliant φs φsΛ) :
  φsΛ.signInsertSomeProd φ φs i j =
  S(ℱ |>s φ, ℱ |>s (φs.get, (Finset.univ.filter (fun x => (φsΛ.getDual? x).isSome ∧
    ∀ (h : (φsΛ.getDual? x).isSome),
    x < j ∧ (i.succAbove x < i ∧ i < i.succAbove ((φsΛ.getDual? x).get h)
    ∨ j < ((φsΛ.getDual? x).get h) ∧ ¬ i.succAbove x < i))))))

```

lemma signInsertSomeCoef_if[\[source\]](#)

```

lemma signInsertSomeCoef_if (φ : ℱ.States) (φs : List ℱ.States) (φsΛ : WickContraction φs.
  length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) (hφj : (ℱ |>s φ) = (ℱ |>s φs[j.1])) :
  φsΛ.signInsertSomeCoef φ φs i j =
  if i < i.succAbove j then
    S(ℱ |>s φ, ℱ |>s ((φs.insertIdx i φ).get,
      (signFinset (φsΛ ↔ φ i (some j)) (finCongr (insertIdx_length_fin φ φs i).symm i)
      (finCongr (insertIdx_length_fin φ φs i).symm (i.succAbove j)))))
  else
    S(ℱ |>s φ, ℱ |>s ((φs.insertIdx i φ).get,
      signFinset (φsΛ ↔ φ i (some j))
      (finCongr (insertIdx_length_fin φ φs i).symm (i.succAbove j))
      (finCongr (insertIdx_length_fin φ φs i).symm i))))

```

lemma stat_signFinset_insert_some_self_fst[\[source\]](#)

```

lemma stat_signFinset_insert_some_self_fst
  (φ : ℱ.States) (φs : List ℱ.States) (φsΛ : WickContraction φs.length)
  (i : Fin φs.length.succ) (j : φsΛ.uncontracted) :
  (ℱ |>s ((φs.insertIdx i φ).get,
    (signFinset (φsΛ ↔ φ i (some j)) (finCongr (insertIdx_length_fin φ φs i).symm i)

```

```

    (finCongr (insertIdx_length_fin  $\phi$   $\phi s$  i).symm (i.succAbove j)))) =
 $\mathcal{F} \mid_{>s} \langle \phi s.get,$ 
    (Finset.univ.filter (fun x => i < i.succAbove x  $\wedge$  x < j  $\wedge$  (( $\phi s\Lambda.getDual?$  x = none)  $\vee$ 
       $\forall$  (h : ( $\phi s\Lambda.getDual?$  x).isSome), i < i.succAbove (( $\phi s\Lambda.getDual?$  x).get h))))))

```

lemma stat_signFinset_insert_some_self_snd

[\[source\]](#)

```

lemma stat_signFinset_insert_some_self_snd ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s\Lambda$  : WickContraction  $\phi s.length$ ) (i : Fin  $\phi s.length.succ$ ) (j :  $\phi s\Lambda.uncontracted$ ) :
  ( $\mathcal{F} \mid_{>s} \langle (\phi s.insertIdx i \phi).get,$ 
    (signFinset ( $\phi s\Lambda \leftarrow \phi$  i (some j))
      (finCongr (insertIdx_length_fin  $\phi$   $\phi s$  i).symm (i.succAbove j))
      (finCongr (insertIdx_length_fin  $\phi$   $\phi s$  i).symm i)))) =
 $\mathcal{F} \mid_{>s} \langle \phi s.get,$ 
  (Finset.univ.filter (fun x => j < x  $\wedge$  i.succAbove x < i  $\wedge$  (( $\phi s\Lambda.getDual?$  x = none)  $\vee$ 
     $\forall$  (h : ( $\phi s\Lambda.getDual?$  x).isSome), j < (( $\phi s\Lambda.getDual?$  x).get h))))))

```

lemma signInsertSomeCoef_eq_finset

[\[source\]](#)

```

lemma signInsertSomeCoef_eq_finset ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s\Lambda$  : WickContraction  $\phi s.length$ ) (i : Fin  $\phi s.length.succ$ ) (j :  $\phi s\Lambda.uncontracted$ )
  (h $\phi j$  : ( $\mathcal{F} \mid_{>s} \phi$ ) = ( $\mathcal{F} \mid_{>s} \phi s[j.1]$ )) :  $\phi s\Lambda.signInsertSomeCoef \phi \phi s i j$  =
  if i < i.succAbove j then
     $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \langle \phi s.get,$ 
      (Finset.univ.filter (fun x => i < i.succAbove x  $\wedge$  x < j  $\wedge$  (( $\phi s\Lambda.getDual?$  x = none)  $\vee$ 
         $\forall$  (h : ( $\phi s\Lambda.getDual?$  x).isSome), i < i.succAbove (( $\phi s\Lambda.getDual?$  x).get h))))))
  else
     $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \langle \phi s.get,$ 
      (Finset.univ.filter (fun x => j < x  $\wedge$  i.succAbove x < i  $\wedge$  (( $\phi s\Lambda.getDual?$  x = none)  $\vee$ 
         $\forall$  (h : ( $\phi s\Lambda.getDual?$  x).isSome), j < (( $\phi s\Lambda.getDual?$  x).get h))))))

```

lemma signInsertSome_mul_filter_contracted_of_lt

[\[source\]](#)

```

lemma signInsertSome_mul_filter_contracted_of_lt ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s\Lambda$  : WickContraction  $\phi s.length$ ) (i : Fin  $\phi s.length.succ$ ) (k :  $\phi s\Lambda.uncontracted$ )
  (hk : i.succAbove k < i) (hg : GradingCompliant  $\phi s \phi s\Lambda \wedge (\mathcal{F} \mid_{>s} \phi) = \mathcal{F} \mid_{>s} \phi s[k.1]$ ) :
  signInsertSome  $\phi \phi s \phi s\Lambda i k$  *
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \langle \phi s.get, \phi s\Lambda.uncontracted.filter (\text{fun } x \Rightarrow x \leq \uparrow k))$ 
  =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \langle \phi s.get, Finset.univ.filter (\text{fun } x \Rightarrow i.succAbove x < i))$ 

```

lemma signInsertSome_mul_filter_contracted_of_not_lt

[\[source\]](#)

```

lemma signInsertSome_mul_filter_contracted_of_not_lt ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s\Lambda$  : WickContraction  $\phi s.length$ ) (i : Fin  $\phi s.length.succ$ ) (k :  $\phi s\Lambda.uncontracted$ )
  (hk :  $\neg i.succAbove k < i$ ) (hg : GradingCompliant  $\phi s \phi s\Lambda \wedge (\mathcal{F} \mid_{>s} \phi) = \mathcal{F} \mid_{>s} \phi s[k.1]$ ) :
  signInsertSome  $\phi \phi s \phi s\Lambda i k$  *
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \langle \phi s.get, \phi s\Lambda.uncontracted.filter (\text{fun } x \Rightarrow x < \uparrow k))$ 
  =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \langle \phi s.get, Finset.univ.filter (\text{fun } x \Rightarrow i.succAbove x < i))$ 

```

2.44 HepLean.PerturbationTheory.WickContraction.StaticContract

[HepLean/PerturbationTheory/WickContraction/StaticContract.lean](#) on GitHub.

def staticContract[\[source\]](#)

Given a Wick contraction $\phi s \Lambda$ associated with a list ϕs , the product of all time-contractions of pairs of contracted elements in ϕs , as a member of the center of $\mathcal{O}.A$.

```
noncomputable def staticContract { $\phi s$  : List  $\mathcal{F}.States$ }
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) :
  Subalgebra.center  $\mathbb{C} \mathcal{F}.FieldOpAlgebra$ 
```

lemma staticContract_insertAndContract_none[\[source\]](#)

```
@[simp]
lemma staticContract_insertAndContract_none ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $i$  : Fin  $\phi s.length.succ$ ) :
  ( $\phi s \Lambda \hookrightarrow \phi i$  none).staticContract =  $\phi s \Lambda.staticContract$ 
```

lemma staticContract_insertAndContract_some[\[source\]](#)

For $\phi s \Lambda$ a Wick contraction for $\phi s = \phi_0 \dots \phi_n$, the time contraction $(\phi s \Lambda \hookrightarrow \phi i$ (some j)).timeContract $\mathcal{O}$ is equal to the multiple of - the time contraction of ϕ with ϕ_j if $i < i.succAbove j$ else ϕ_j with ϕ . - $\phi s \Lambda.timeContract \mathcal{O}$. This follows from the fact that $(\phi s \Lambda \hookrightarrow \phi i$ (some j)) has one more contracted pair than $\phi s \Lambda$, corresponding to ϕ contracted with ϕ_j . The order depends on whether we insert ϕ before or after ϕ_j .

```
lemma staticContract_insertAndContract_some
  ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $i$  : Fin  $\phi s.length.succ$ ) ( $j$  :  $\phi s \Lambda.uncontracted$ ) :
  ( $\phi s \Lambda \hookrightarrow \phi i$  (some  $j$ )).staticContract =
  (if  $i < i.succAbove j$  then
    ([anPart  $\phi$ , ofFieldOp  $\phi s[j.1]]_s$ , superCommute_anPart_ofFieldOp_mem_center _ _)
  else ([anPart  $\phi s[j.1]$ , ofFieldOp  $\phi$ ]s, superCommute_anPart_ofFieldOp_mem_center _ _)) *
   $\phi s \Lambda.staticContract$ 
```

lemma staticContract_insertAndContract_some_eq_mul_contractStateAtIndex_lt[\[source\]](#)

```
lemma staticContract_insertAndContract_some_eq_mul_contractStateAtIndex_lt
  ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $i$  : Fin  $\phi s.length.succ$ ) ( $k$  :  $\phi s \Lambda.uncontracted$ )
  ( $hik$  :  $i < i.succAbove k$ ) :
  ( $\phi s \Lambda \hookrightarrow \phi i$  (some  $k$ )).staticContract =
   $S(\mathcal{F} \mid_s \phi, \mathcal{F} \mid_s (\phi s.get, (\phi s \Lambda.uncontracted.filter (\text{fun } x \Rightarrow x < k))))$ 
  • ( $contractStateAtIndex \phi [\phi s \Lambda]^{uc} ((uncontractedStatesEquiv \phi s \phi s \Lambda) (some k))$  *
     $\phi s \Lambda.staticContract$ )
```

lemma staticContract_of_not_gradingCompliant[\[source\]](#)

```
lemma staticContract_of_not_gradingCompliant ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $h$  :  $\neg$  GradingCompliant  $\phi s \phi s \Lambda$ ) :
   $\phi s \Lambda.staticContract = 0$ 
```

2.45 HepLean.PerturbationTheory.WickContraction.TimeContract

[HepLean/PerturbationTheory/WickContraction/TimeContract.lean](#) on GitHub.

def timeContract[\[source\]](#)

Given a Wick contraction $\phi s \Lambda$ associated with a list ϕs , the product of all time-contractions of pairs of contracted elements in ϕs , as a member of the center of $\mathcal{O}.A$.

```
noncomputable def timeContract { $\phi s$  : List  $\mathcal{F}.States$ }
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) :
  Subalgebra.center  $\mathbb{C} \mathcal{F}.FieldOpAlgebra$ 
```

lemma timeContract_insertAndContract_none[\[source\]](#)

For $\phi s \Lambda$ a Wick contraction for ϕs , the time contraction $(\phi s \Lambda \hookrightarrow \Lambda \phi \ i \ none).timeContract$ $\mathcal{O}$ is equal to $\phi s \Lambda.timeContract \mathcal{O}$.

This result follows from the fact that `timeContract` only depends on contracted pairs, and $(\phi s \Lambda \hookrightarrow \Lambda \phi \ i \ none)$ has the 'same' contracted pairs as $\phi s \Lambda$.

```
@[simp]
lemma timeContract_insertAndContract_none
  ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $i$  : Fin  $\phi s.length.succ$ ) :
  ( $\phi s \Lambda \hookrightarrow \Lambda \phi \ i \ none$ ).timeContract =  $\phi s \Lambda.timeContract$ 
```

lemma timeContract_insertAndContract_some[\[source\]](#)

For $\phi s \Lambda$ a Wick contraction for $\phi s = \phi_0 \dots \phi_n$, the time contraction $(\phi s \Lambda \hookrightarrow \Lambda \phi \ i \ (some \ j)).timeContract$ $\mathcal{O}$ is equal to the multiple of - the time contraction of ϕ with ϕ_j if $i < i.succAbove \ j$ else ϕ_j with ϕ . - $\phi s \Lambda.timeContract \mathcal{O}$. This follows from the fact that $(\phi s \Lambda \hookrightarrow \Lambda \phi \ i \ (some \ j))$ has one more contracted pair than $\phi s \Lambda$, corresponding to ϕ contracted with ϕ_j . The order depends on whether we insert ϕ before or after ϕ_j .

```
lemma timeContract_insertAndContract_some
  ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $i$  : Fin  $\phi s.length.succ$ ) ( $j$  :  $\phi s \Lambda.uncontracted$ ) :
  ( $\phi s \Lambda \hookrightarrow \Lambda \phi \ i \ (some \ j)$ ).timeContract =
  (if  $i < i.succAbove \ j$  then
    (FieldOpAlgebra.timeContract  $\phi \ \phi s[j.1]$ , timeContract_mem_center _ _)
  else (FieldOpAlgebra.timeContract  $\phi s[j.1] \ \phi$ , timeContract_mem_center _ _)) *
   $\phi s \Lambda.timeContract$ 
```

lemma timeContract_insertAndContract_some_eq_mul_contractStateAtIndex_lt[\[source\]](#)

```
lemma timeContract_insertAndContract_some_eq_mul_contractStateAtIndex_lt
  ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
  ( $\phi s \Lambda$  : WickContraction  $\phi s.length$ ) ( $i$  : Fin  $\phi s.length.succ$ ) ( $k$  :  $\phi s \Lambda.uncontracted$ )
  ( $ht$  :  $\mathcal{F}.timeOrderRel \ \phi \ \phi s[k.1]$ ) ( $hik$  :  $i < i.succAbove \ k$ ) :
  ( $\phi s \Lambda \hookrightarrow \Lambda \phi \ i \ (some \ k)$ ).timeContract =
   $\mathcal{S}(\mathcal{F} \mid_{>_s} \phi, \mathcal{F} \mid_{>_s} \langle \phi s.get, (\phi s \Lambda.uncontracted.filter \ (\text{fun } x \Rightarrow x < k)) \rangle)$ 
  • (contractStateAtIndex  $\phi \ [\phi s \Lambda]^{uc} ((uncontractedStatesEquiv \ \phi s \ \phi s \Lambda) (some \ k))$ ) *
   $\phi s \Lambda.timeContract$ 
```

lemma timeContract_insertAndContract_some_eq_mul_contractStateAtIndex_not_lt[\[source\]](#)

```
lemma timeContract_insertAndContract_some_eq_mul_contractStateAtIndex_not_lt
  ( $\phi$  :  $\mathcal{F}.States$ ) ( $\phi s$  : List  $\mathcal{F}.States$ )
```

```

(φsΛ : WickContraction φs.length) (i : Fin φs.length.succ) (k : φsΛ.uncontracted)
(ht : ¬ F.timeOrderRel φs[k.1] φ) (hik : ¬ i < i.succAbove k) :
(φsΛ ↔Λ φ i (some k)).timeContract =
S(F |>_s φ, F |>_s (φs.get, (φsΛ.uncontracted.filter (fun x => x ≤ k))))
• (contractStateAtIndex φ [φsΛ]uc
  ((uncontractedStatesEquiv φs φsΛ) (some k)) * φsΛ.timeContract)

```

lemma timeContract_of_not_gradingCompliant

[\[source\]](#)

```

lemma timeContract_of_not_gradingCompliant (φs : List F.States)
(φsΛ : WickContraction φs.length) (h : ¬ GradingCompliant φs φsΛ) :
φsΛ.timeContract = 0

```

2.46 HepLean.PerturbationTheory.WickContraction.Uncontracted

[HepLean/PerturbationTheory/WickContraction/Uncontracted.lean](#) on GitHub.

def uncontracted

[\[source\]](#)

Given a Wick contraction, the finset of elements of Fin n which are not contracted.

```
def uncontracted : Finset (Fin n)
```

lemma congr_uncontracted

[\[source\]](#)

```

lemma congr_uncontracted {n m : ℕ} (c : WickContraction n) (h : n = m) :
(c.congr h).uncontracted = Finset.map (finCongr h).toEmbedding c.uncontracted

```

def uncontractedCongr

[\[source\]](#)

The equivalence of Option c.uncontracted for two propositionally equal Wick contractions.

```

def uncontractedCongr {c c' : WickContraction n} (h : c = c') :
Option c.uncontracted ≈ Option c'.uncontracted

```

lemma uncontractedCongr_none

[\[source\]](#)

```

@[simp]
lemma uncontractedCongr_none {c c' : WickContraction n} (h : c = c') :
(uncontractedCongr h) none = none

```

lemma uncontractedCongr_some

[\[source\]](#)

```

@[simp]
lemma uncontractedCongr_some {c c' : WickContraction n} (h : c = c') (i : c.uncontracted) :
(uncontractedCongr h) (some i) = some (Equiv.subtypeEquivRight (by rw [h]; simp) i)

```

lemma mem_uncontracted_iff_not_contracted

[\[source\]](#)

```

lemma mem_uncontracted_iff_not_contracted (i : Fin n) :
i ∈ c.uncontracted ↔ ∀ p ∈ c.1, i ∉ p

```


2.47 HepLean.PerturbationTheory.WickContraction.UncontractedList

[HepLean/PerturbationTheory/WickContraction/UncontractedList.lean](#) on GitHub.

lemma fin_list_sorted_monotone_sorted

[\[source\]](#)

```
lemma fin_list_sorted_monotone_sorted {n m : ℕ} (l : List (Fin n)) (hl : l.Sorted (· ≤ ·))
  (f : Fin n → Fin m) (hf : StrictMono f) : ((List.map f l)).Sorted (· ≤ ·)
```

lemma fin_list_sorted_succAboveEmb_sorted

[\[source\]](#)

```
lemma fin_list_sorted_succAboveEmb_sorted (l : List (Fin n)) (hl : l.Sorted (· ≤ ·))
  (i : Fin n.succ) : ((List.map i.succAboveEmb l)).Sorted (· ≤ ·)
```

lemma fin_list_sorted_split

[\[source\]](#)

```
lemma fin_list_sorted_split :
  (l : List (Fin n)) → (hl : l.Sorted (· ≤ ·)) → (i : ℕ) →
  l = l.filter (fun x => x.1 < i) ++ l.filter (fun x => i ≤ x.1)
| [], _, _ => by simp
| a :: l, hl, i => by
  simp only [List.sorted_cons] at hl
  by_cases ha : a < i
  · conv_lhs => rw [fin_list_sorted_split l hl.2 i]
    rw [← List.cons_append]
    rw [List.filter_cons_of_pos, List.filter_cons_of_neg]
    simp only [decide_eq_true_eq, not_le, ha]
    simp [ha]
  · have hx : List.filter (fun x => decide (x.1 < i)) (a :: l) = []
```

lemma fin_list_sorted_indexOf_filter_le_mem

[\[source\]](#)

```
lemma fin_list_sorted_indexOf_filter_le_mem :
  (l : List (Fin n)) → (hl : l.Sorted (· ≤ ·)) → (i : Fin n) →
  (hl : i ∈ l) →
  List.indexOf i (List.filter (fun x => decide (↑i ≤ ↑x)) l) = 0
| [], _, _, _ => by simp
| a :: l, hl, i, hi => by
  simp only [List.sorted_cons] at hl
  by_cases ha : i ≤ a
  · simp only [ha, decide_true, List.filter_cons_of_pos]
    have ha : a = i
```

lemma fin_list_sorted_indexOf_mem

[\[source\]](#)

```
lemma fin_list_sorted_indexOf_mem :
  (l : List (Fin n)) → (hl : l.Sorted (· ≤ ·)) → (i : Fin n) →
  (hi : i ∈ l) →
  l.indexOf i = (l.filter (fun x => x.1 < i.1)).length
```

lemma orderedInsert_of_fin_list_sorted

[\[source\]](#)

```
lemma orderedInsert_of_fin_list_sorted :
  (l : List (Fin n)) → (hl : l.Sorted (· ≤ ·)) → (i : Fin n) →
  List.orderedInsert (· ≤ ·) i l = l.filter (fun x => x.1 < i.1) ++
```

```

i :: l.filter (fun x => i.l ≤ x.l)
| [], _, _ => by simp
| a :: l, hl, i => by
  simp only [List.sorted_cons] at hl
  by_cases ha : i ≤ a
  · simp only [List.orderedInsert, ha, ↪reduceIte, Fin.val_fin_lt, decide_eq_true_eq, not_lt,
    List.filter_cons_of_neg, Fin.val_fin_le, decide_true, List.filter_cons_of_pos]
    have hl : List.filter (fun x => decide (↑x < ↑i)) l = []

```

lemma orderedInsert_eq_insertIdx_of_fin_list_sorted[\[source\]](#)

```

lemma orderedInsert_eq_insertIdx_of_fin_list_sorted (l : List (Fin n)) (hl : l.Sorted (· ≤ ·))
  (i : Fin n) :
  List.orderedInsert (· ≤ ·) i l = l.insertIdx (l.filter (fun x => x.l < i.l)).length i

```

def uncontractedList[\[source\]](#)

Given a Wick contraction c , the ordered list of elements of $\text{Fin } n$ which are not contracted, i.e. do not appera anywhere in $c.1$.

```

def uncontractedList : List (Fin n)

```

lemma uncontractedList_mem_iff[\[source\]](#)

```

lemma uncontractedList_mem_iff (i : Fin n) :
  i ∈ c.uncontractedList ↔ i ∈ c.uncontracted

```

lemma nil_zero_uncontractedList[\[source\]](#)

```

@[simp]
lemma nil_zero_uncontractedList : (empty (n := 0)).uncontractedList = []

```

lemma congr_uncontractedList[\[source\]](#)

```

lemma congr_uncontractedList {n m : ℕ} (h : n = m) (c : WickContraction n) :
  ((congr h) c).uncontractedList = List.map (finCongr h) c.uncontractedList

```

lemma uncontractedList_get_mem_uncontracted[\[source\]](#)

```

lemma uncontractedList_get_mem_uncontracted (i : Fin c.uncontractedList.length) :
  c.uncontractedList.get i ∈ c.uncontracted

```

lemma uncontractedList_sorted[\[source\]](#)

```

lemma uncontractedList_sorted : List.Sorted (· ≤ ·) c.uncontractedList

```

lemma uncontractedList_nodup[\[source\]](#)

```

lemma uncontractedList_nodup : c.uncontractedList.Nodup

```

lemma uncontractedList_toFinset[\[source\]](#)

```
lemma uncontractedList_toFinset (c : WickContraction n) :
  c.uncontractedList.toFinset = c.uncontracted
```

lemma uncontractedList_eq_sort[\[source\]](#)

```
lemma uncontractedList_eq_sort (c : WickContraction n) :
  c.uncontractedList = c.uncontracted.sort (· ≤ ·)
```

lemma uncontractedList_length_eq_card[\[source\]](#)

```
lemma uncontractedList_length_eq_card (c : WickContraction n) :
  c.uncontractedList.length = c.uncontracted.card
```

lemma filter_uncontractedList[\[source\]](#)

```
lemma filter_uncontractedList (c : WickContraction n) (p : Fin n → Prop) [DecidablePred p] :
  (c.uncontractedList.filter p) = (c.uncontracted.filter p).sort (· ≤ ·)
```

def uncontractedIndexEquiv[\[source\]](#)

The equivalence between the positions of $c.uncontractedList$ i.e. elements of $Fin (c.uncontractedList).length$ and the finite set $c.uncontracted$ considered as a finite type.

```
def uncontractedIndexEquiv (c : WickContraction n) :
  Fin (c.uncontractedList.length) ≃ c.uncontracted where
```

lemma uncontractedList_getElem_uncontractedIndexEquiv_symm[\[source\]](#)

```
@[simp]
lemma uncontractedList_getElem_uncontractedIndexEquiv_symm (k : c.uncontracted) :
  c.uncontractedList[(c.uncontractedIndexEquiv.symm k).val] = k
```

lemma uncontractedIndexEquiv_symm_eq_filter_length[\[source\]](#)

```
lemma uncontractedIndexEquiv_symm_eq_filter_length (k : c.uncontracted) :
  (c.uncontractedIndexEquiv.symm k).val =
  (List.filter (fun i => i < k.val) c.uncontractedList).length
```

lemma take_uncontractedIndexEquiv_symm[\[source\]](#)

```
lemma take_uncontractedIndexEquiv_symm (k : c.uncontracted) :
  c.uncontractedList.take (c.uncontractedIndexEquiv.symm k).val =
  c.uncontractedList.filter (fun i => i < k.val)
```

def uncontractedListGet[\[source\]](#)

Given a Wick Contraction $\phi s \Lambda$ for a list of states ϕs . The list of uncontracted states in ϕs .

```
def uncontractedListGet { $\phi$ s : List  $\mathcal{F}$ .States} ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) :
  List  $\mathcal{F}$ .States
```

def uncontractedStatesEquiv

[\[source\]](#)

The equivalence between the type `Option c.uncontracted` for `WickContraction  $\phi$ s.length` and `Option (Fin (c.uncontractedList.map  $\phi$ s.get).length)`, that is optional positions of `c.uncontractedList.map  $\phi$ s.get` induced by `uncontractedIndexEquiv`.

```
def uncontractedStatesEquiv ( $\phi$ s : List  $\mathcal{F}$ .States) ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) :
  Option  $\phi$ s $\Lambda$ .uncontracted = Option (Fin [ $\phi$ s $\Lambda$ ] $u^c$ .length)
```

lemma uncontractedStatesEquiv_none

[\[source\]](#)

@[simp]

```
lemma uncontractedStatesEquiv_none ( $\phi$ s : List  $\mathcal{F}$ .States) ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) :
  (uncontractedStatesEquiv  $\phi$ s  $\phi$ s $\Lambda$ ).toFun none = none
```

lemma uncontractedStatesEquiv_list_sum

[\[source\]](#)

```
lemma uncontractedStatesEquiv_list_sum [AddCommMonoid  $\alpha$ ] ( $\phi$ s : List  $\mathcal{F}$ .States)
  ( $\phi$ s $\Lambda$  : WickContraction  $\phi$ s.length) (f : Option (Fin [ $\phi$ s $\Lambda$ ] $u^c$ .length)  $\rightarrow$   $\alpha$ ) :
   $\sum$  (i : Option (Fin [ $\phi$ s $\Lambda$ ] $u^c$ .length)), f i =
   $\sum$  (i : Option  $\phi$ s $\Lambda$ .uncontracted), f ( $\phi$ s $\Lambda$ .uncontractedStatesEquiv  $\phi$ s i)
```

lemma uncontractedList_succAboveEmb_sorted

[\[source\]](#)

```
lemma uncontractedList_succAboveEmb_sorted (c : WickContraction n) (i : Fin n.succ) :
  ((List.map i.succAboveEmb c.uncontractedList)).Sorted ( $\cdot \leq \cdot$ )
```

lemma uncontractedList_succAboveEmb_nodup

[\[source\]](#)

```
lemma uncontractedList_succAboveEmb_nodup (c : WickContraction n) (i : Fin n.succ) :
  ((List.map i.succAboveEmb c.uncontractedList)).Nodup
```

lemma uncontractedList_succAbove_orderedInsert_nodup

[\[source\]](#)

```
lemma uncontractedList_succAbove_orderedInsert_nodup (c : WickContraction n) (i : Fin n.succ) :
  (List.orderedInsert ( $\cdot \leq \cdot$ ) i (List.map i.succAboveEmb c.uncontractedList)).Nodup
```

lemma uncontractedList_succAbove_orderedInsert_sorted

[\[source\]](#)

```
lemma uncontractedList_succAbove_orderedInsert_sorted (c : WickContraction n) (i : Fin n.succ) :
  :
  (List.orderedInsert ( $\cdot \leq \cdot$ ) i
    (List.map i.succAboveEmb c.uncontractedList)).Sorted ( $\cdot \leq \cdot$ )
```

lemma uncontractedList_succAbove_orderedInsert_toFinset

[\[source\]](#)

```
lemma uncontractedList_succAbove_orderedInsert_toFinset (c : WickContraction n) (i : Fin n.succ) :
  :
```

```
(List.orderedInsert (· ≤ ·) i (List.map i.succAboveEmb c.uncontractedList)).toFinset =
  (Insert.insert i (Finset.map i.succAboveEmb c.uncontracted))
```

lemma uncontractedList_succAbove_orderedInsert_eq_sort[\[source\]](#)

```
lemma uncontractedList_succAbove_orderedInsert_eq_sort (c : WickContraction n) (i : Fin n.succ) :
  :
  (List.orderedInsert (· ≤ ·) i (List.map i.succAboveEmb c.uncontractedList)) =
  (Insert.insert i (Finset.map i.succAboveEmb c.uncontracted)).sort (· ≤ ·)
```

lemma uncontractedList_extractEquiv_symm_none[\[source\]](#)

```
lemma uncontractedList_extractEquiv_symm_none (c : WickContraction n) (i : Fin n.succ) :
  ((extractEquiv i).symm (c, none)).uncontractedList =
  List.orderedInsert (· ≤ ·) i (List.map i.succAboveEmb c.uncontractedList)
```

lemma uncontractedList_succAboveEmb_eraseIdx_toFinset[\[source\]](#)

```
lemma uncontractedList_succAboveEmb_eraseIdx_toFinset (c : WickContraction n) (i : Fin n.succ)
  (k : ℕ) (hk : k < c.uncontractedList.length) :
  ((List.map i.succAboveEmb c.uncontractedList).eraseIdx k).toFinset =
  (c.uncontracted.map i.succAboveEmb).erase (i.succAboveEmb c.uncontractedList[k])
```

lemma uncontractedList_succAboveEmb_eraseIdx_sorted[\[source\]](#)

```
lemma uncontractedList_succAboveEmb_eraseIdx_sorted (c : WickContraction n) (i : Fin n.succ)
  (k : ℕ) : ((List.map i.succAboveEmb c.uncontractedList).eraseIdx k).Sorted (· ≤ ·)
```

lemma uncontractedList_succAboveEmb_eraseIdx_nodup[\[source\]](#)

```
lemma uncontractedList_succAboveEmb_eraseIdx_nodup (c : WickContraction n) (i : Fin n.succ) (k :
  ℕ) :
  ((List.map i.succAboveEmb c.uncontractedList).eraseIdx k).Nodup
```

lemma uncontractedList_succAboveEmb_eraseIdx_eq_sort[\[source\]](#)

```
lemma uncontractedList_succAboveEmb_eraseIdx_eq_sort (c : WickContraction n) (i : Fin n.succ)
  (k : ℕ) (hk : k < c.uncontractedList.length) :
  ((List.map i.succAboveEmb c.uncontractedList).eraseIdx k) =
  ((c.uncontracted.map i.succAboveEmb).erase
  (i.succAboveEmb c.uncontractedList[k])).sort (· ≤ ·)
```

lemma uncontractedList_extractEquiv_symm_some[\[source\]](#)

```
lemma uncontractedList_extractEquiv_symm_some (c : WickContraction n) (i : Fin n.succ)
  (k : c.uncontracted) : ((extractEquiv i).symm (c, some k)).uncontractedList =
  ((c.uncontracted).map i.succAboveEmb).eraseIdx (c.uncontractedIndexEquiv.symm k)
```

lemma uncontractedList_succAboveEmb_toFinset[\[source\]](#)

```
lemma uncontractedList_succAboveEmb_toFinset (c : WickContraction n) (i : Fin n.succ) :
  (List.map i.succAboveEmb c.uncontractedList).toFinset =
```

```
(Finset.map i.succAboveEmb c.uncontracted)
```

def uncontractedListOrderPos

[\[source\]](#)

Given a Wick contraction $c : \text{WickContraction } n$ and a $\text{Fin } n.\text{succ}$, the number of elements of $c.\text{uncontractedList}$ which are less than i . Suppose we want to insert into c at position i , then this is the position we would need to insert into $c.\text{uncontractedList}$.

```
def uncontractedListOrderPos (c : WickContraction n) (i : Fin n.succ) : ℕ
```

lemma uncontractedListOrderPos_lt_length_add_one

[\[source\]](#)

```
@[simp]
```

```
lemma uncontractedListOrderPos_lt_length_add_one (c : WickContraction n) (i : Fin n.succ) :  
  c.uncontractedListOrderPos i < c.uncontractedList.length + 1
```

lemma take_uncontractedListOrderPos_eq_filter

[\[source\]](#)

```
lemma take_uncontractedListOrderPos_eq_filter (c : WickContraction n) (i : Fin n.succ) :  
  (c.uncontractedList.take (c.uncontractedListOrderPos i)) =  
  c.uncontractedList.filter (fun x => x.1 < i.1)
```

lemma take_uncontractedListOrderPos_eq_filter_sort

[\[source\]](#)

```
lemma take_uncontractedListOrderPos_eq_filter_sort (c : WickContraction n) (i : Fin n.succ) :  
  (c.uncontractedList.take (c.uncontractedListOrderPos i)) =  
  (c.uncontracted.filter (fun x => x.1 < i.1)).sort (· ≤ ·)
```

lemma orderedInsert_succAboveEmb_uncontractedList_eq_insertIdx

[\[source\]](#)

```
lemma orderedInsert_succAboveEmb_uncontractedList_eq_insertIdx (c : WickContraction n)  
  (i : Fin n.succ) :  
  (List.orderedInsert (· ≤ ·) i (List.map i.succAboveEmb c.uncontractedList)) =  
  (List.map i.succAboveEmb c.uncontractedList).insertIdx (uncontractedListOrderPos c i) i
```

2.48 HepLean.PerturbationTheory.WicksTheorem

[HepLean/PerturbationTheory/WicksTheorem.lean](#) on GitHub.

lemma normalOrder_uncontracted_none

[\[source\]](#)

Let c be a Wick Contraction for $\varphi_S := \varphi_0 \varphi_1 \dots \varphi_n$. We have (roughly) $\mathcal{N}^f([\varphi_S \wedge \wedge \varphi \text{ i none}]^{uc}) = s \cdot \mathcal{N}^f(\varphi :: [\varphi_S]^{uc})$ Where s is the exchange sign for φ and the uncontracted fields in $\varphi_0 \varphi_1 \dots \varphi_{i-1}$.

```
lemma normalOrder_uncontracted_none (φ : ℱ.States) (φs : List ℱ.States)  
  (i : Fin φs.length.succ) (φsΛ : WickContraction φs.length) :  
  ℳ(ofFieldOpList [φsΛ ↦ φ i none]^{uc})  
  = ℳ(ℱ |>_s φ, ℱ |>_s (φs.get, φsΛ.uncontracted.filter (fun x => i.succAbove x < i))) •  
  ℳ(ofFieldOpList (φ :: [φsΛ]^{uc}))
```

lemma normalOrder_uncontracted_some[\[source\]](#)

Let c be a Wick Contraction for $\varphi_0\varphi_1\ldots\varphi_n$. We have (roughly) $N(c \hookrightarrow \Lambda \varphi \ i \ k).uncontractedList$ is equal to $N((c.uncontractedList).eraseIdx \ k')$ where k' is the position in $c.uncontractedList$ corresponding to k .

```
lemma normalOrder_uncontracted_some (φ : F.States) (ψs : List F.States)
  (i : Fin ψs.length.succ) (ψsΛ : WickContraction ψs.length) (k : ψsΛ.uncontracted) :
  N(ofFieldOpList [ψsΛ ↪Λ φ i (some k)]uc)
  = N(ofFieldOpList (optionEraseZ [ψsΛ]uc φ ((uncontractedStatesEquiv ψs ψsΛ) k)))
```

lemma wick_term_none_eq_wick_term_cons[\[source\]](#)

Let $\varphi s \Lambda$ be a Wick Contraction for $\varphi s = \varphi_0\varphi_1\ldots\varphi_n$. Then the wick-term of $(\varphi s \Lambda \hookrightarrow \Lambda \varphi \ i \ none)$

$\backslash (\varphi s \Lambda \hookrightarrow \Lambda \varphi \ i \ none).sign \cdot (\varphi s \Lambda \hookrightarrow \Lambda \varphi \ i \ none).timeContract \mathcal{O} * \mathcal{O}.crAnF \mathcal{N}^f([\varphi s \Lambda \hookrightarrow \Lambda \varphi \ i \ none]^{uc}) \backslash$

is equal to

$s \cdot (\varphi s \Lambda.sign \cdot \varphi s \Lambda.timeContract \mathcal{O} * \mathcal{O}.crAnF \mathcal{N}^f(\varphi :: [\varphi s \Lambda]^{uc}))$

where s is the exchange sign of φ and the uncontracted fields in $\varphi_0\varphi_1\ldots\varphi_{i-1}$.

The proof of this result relies primarily on: - `normalOrderF_uncontracted_none` which replaces $\mathcal{N}^f([\varphi s \Lambda \hookrightarrow \Lambda \varphi \ i \ none]^{uc})$ with $\mathcal{N}^f(\varphi :: [\varphi s \Lambda]^{uc})$ up to a sign. - `timeContract_insertAndContract` which replaces $(\varphi s \Lambda \hookrightarrow \Lambda \varphi \ i \ none).timeContract \mathcal{O}$ with $\varphi s \Lambda.timeContract \mathcal{O}$. - `sign_insert_none` and `signInsertNone_eq_filterset` which are used to take account of signs.

```
lemma wick_term_none_eq_wick_term_cons (φ : F.States) (ψs : List F.States)
  (i : Fin ψs.length.succ) (ψsΛ : WickContraction ψs.length) :
  (ψsΛ ↪Λ φ i none).sign • (ψsΛ ↪Λ φ i none).timeContract
  * N(ofFieldOpList [ψsΛ ↪Λ φ i none]uc) =
  S(F |>_s φ, F |>_s (φs.get, (Finset.univ.filter (fun k => i.succAbove k < i))))
  • (ψsΛ.sign • ψsΛ.timeContract * N(ofFieldOpList (φ :: [ψsΛ]uc)))
```

lemma wick_term_some_eq_wick_term_optionEraseZ[\[source\]](#)

Let c be a Wick Contraction for $\varphi_0\varphi_1\ldots\varphi_n$. This lemma states that $(c.sign \cdot c.timeContract \mathcal{O}) * N(c.uncontracted)$ for c equal to $c \hookrightarrow \Lambda \varphi \ i \ (some \ k)$ is equal to that for just c multiplied by the exchange sign of φ and $\varphi_0\varphi_1\ldots\varphi_{i-1}$.

```
lemma wick_term_some_eq_wick_term_optionEraseZ (φ : F.States) (ψs : List F.States)
  (i : Fin ψs.length.succ) (ψsΛ : WickContraction ψs.length) (k : ψsΛ.uncontracted)
  (hlt : ∀ (k : Fin ψs.length), timeOrderRel φ ψs[k])
  (hn : ∀ (k : Fin ψs.length), i.succAbove k < i → ¬ timeOrderRel φ ψs[k] φ) :
  (ψsΛ ↪Λ φ i (some k)).sign • (ψsΛ ↪Λ φ i (some k)).timeContract
  * N(ofFieldOpList [ψsΛ ↪Λ φ i (some k)]uc) =
  S(F |>_s φ, F |>_s (φs.get, (Finset.univ.filter (fun x => i.succAbove x < i))))
  • (ψsΛ.sign • (contractStateAtIndex φ [ψsΛ]uc
    ((uncontractedStatesEquiv ψs ψsΛ) (some k)) * ψsΛ.timeContract)
  * N(ofFieldOpList (optionEraseZ [ψsΛ]uc φ (uncontractedStatesEquiv ψs ψsΛ k))))
```

lemma wick_term_cons_eq_sum_wick_term[\[source\]](#)

Given a Wick contraction $\varphi\Lambda$ of $\varphi\mathbf{s} = \varphi_0\varphi_1\ldots\varphi_n$ and an i , we have that $(\varphi\Lambda.\text{sign} \cdot \varphi\Lambda.\text{timeContract } \mathcal{O}) * \mathcal{O}.\text{crAnF } (\varphi * \mathcal{N}^f([\varphi\Lambda]^{uc}))$ is equal to the product of - the exchange sign of φ and $\varphi_0\varphi_1\ldots\varphi_{i-1}$, - the sum of $((\varphi\Lambda \Leftarrow \varphi \ i \ k).\text{sign} \cdot (\varphi\Lambda \Leftarrow \varphi \ i \ k).\text{timeContract } \mathcal{O}) * \mathcal{O}.\text{crAnF } \mathcal{N}^f([\varphi\Lambda \Leftarrow \varphi \ i \ k]^{uc})$ over all k in $\text{Option } \varphi\Lambda.\text{uncontracted}$.

The proof of this result primarily depends on - `crAnF_ofState_mul_normalOrderF_ofStatesList_eq_sum` to rewrite $\mathcal{O}.\text{crAnF } (\varphi * \mathcal{N}^f([\varphi\Lambda]^{uc}))$ - `wick_term_none_eq_wick_term_cons` - `wick_term_some_eq_wick_te`

```
lemma wick_term_cons_eq_sum_wick_term (φ : F.States) (φs : List F.States) (i : Fin φs.length.
succ)
(φsΛ : WickContraction φs.length) (hlt : ∀ (k : Fin φs.length), timeOrderRel φ φs[k])
(hn : ∀ (k : Fin φs.length), i.succAbove k < i → ¬timeOrderRel φs[k] φ) :
(φsΛ.sign • φsΛ.timeContract) * ((ofFieldOp φ) * N(ofFieldOpList [φsΛ]uc)) =
S(F |>s φ, F |>s (φs.get, (Finset.univ.filter (fun x => i.succAbove x < i)))) •
∑ (k : Option φsΛ.uncontracted), ((φsΛ ⇐ φ i k).sign •
(φsΛ ⇐ φ i k).timeContract * (N(ofFieldOpList [φsΛ ⇐ φ i k]uc)))
```

lemma wicks_theorem_nil

[\[source\]](#)

Wick's theorem for the empty list.

```
lemma wicks_theorem_nil :
T(ofFieldOpList (F := F) []) = ∑ (nilΛ : WickContraction [].length),
(nilΛ.sign (F := F) • nilΛ.timeContract) * N(ofFieldOpList [nilΛ]uc)
```

lemma wicks_theorem_congr

[\[source\]](#)

```
lemma wicks_theorem_congr {φs φs' : List F.States} (h : φs = φs') :
∑ (φsΛ : WickContraction φs.length), (φsΛ.sign • φsΛ.timeContract) * N(ofFieldOpList [φsΛ]uc)
= ∑ (φs'Λ : WickContraction φs'.length), (φs'Λ.sign • φs'Λ.timeContract) *
N(ofFieldOpList [φs'Λ]uc)
```

theorem wicks_theorem

[\[source\]](#)

Wick's theorem for time-ordered products of bosonic and fermionic fields. The time ordered product $T(\varphi_0\varphi_1\ldots\varphi_n)$ is equal to the sum of terms, for all possible Wick contractions c of the list of fields $\varphi\mathbf{s} := \varphi_0\varphi_1\ldots\varphi_n$, given by the multiple of: - The sign corresponding to the number of fermionic-fermionic exchanges one must do to put elements in contracted pairs of c next to each other. - The product of time-contractions of the contracted pairs of c . - The normal-ordering of the uncontracted fields in c .

```
theorem wicks_theorem : (φs : List F.States) → T(ofFieldOpList φs) =
∑ (φsΛ : WickContraction φs.length), (φsΛ.sign • φsΛ.timeContract) * N(ofFieldOpList [φsΛ]uc)
| [] => wicks_theorem_nil
| φ :: φs => by
have ih
```

2.49 HepLean.Tensors.Tree.NodeIdentities.ProdContr

[HepLean/Tensors/Tree/NodeIdentities/ProdContr.lean](#) on GitHub.

lemma contrMap_prod_tprod_aux_2[\[source\]](#)

```

lemma contrMap_prod_tprod_aux_2 (p : (i : (1 Type).obj (OverColor.mk c).left) →
  CoeSort.coe (S.FD.obj { as := (OverColor.mk c).hom i }))
  (a : Fin n.succ.succ) (b : Fin (n + 1 + 1) * Fin n1)
  (h : b = Sum.inl a) : p a = (S.FD.map (Discrete.eqToHom (by rw [h]; simp))).hom
  ((lift.discreteSumEquiv' S.FD b)
  (HepLean.PiTensorProduct.elimPureTensor p q' b))

```

2.50 scripts.check_file_imports

[scripts/check_file_imports.lean](#) on GitHub.

def listDif[\[source\]](#)

```

def listDif : (a : List String) → (b : List String) → (List String × List String)
| [], [] => ([], [])
| a :: as, [] => (a :: as, [])
| [], b :: bs => ([], b :: bs)
| a :: as, b :: bs =>
  if a = b then listDif as bs
  else (a :: (listDif as bs).1, b :: (listDif as bs).2)

```

2.51 scripts.MetaPrograms.free_simps

[scripts/MetaPrograms/free_simps.lean](#) on GitHub.

def processFileArray[\[source\]](#)

```

unsafe def processFileArray (files : Array FilePath) : IO Unit

```

def argToArrayFilePath[\[source\]](#)

```

unsafe def argToArrayFilePath (args : List String) : IO (Array FilePath)

```

2.52 scripts.MetaPrograms.notes

[scripts/MetaPrograms/notes.lean](#) on GitHub.

inductive NotePart[\[source\]](#)

```

inductive NotePart
| h1 : String → NotePart
| h2 : String → NotePart
| p : String → NotePart
| name : Name → NotePart

```

structure DeclInfo[\[source\]](#)

```

structure DeclInfo where
  line : Nat
  fileName : Name
  name : Name
  declString : String

```

```
docString : String
```

def ofName [\[source\]](#)

```
def DeclInfo.ofName (n : Name) : MetaM DeclInfo
```

def DeclInfo.toYML [\[source\]](#)

```
def DeclInfo.toYML (d : DeclInfo) : MetaM String
```

def toYMLM [\[source\]](#)

```
def NotePart.toYMLM : ((List String) × Nat × Nat) → NotePart → MetaM ((List String) × Nat × Nat)
| x, NotePart.h1 s =>
  let newString
```

structure Note [\[source\]](#)

```
structure Note where
  title : String
  curators : List String
  parts : List NotePart
```

def Note.toYML [\[source\]](#)

```
def Note.toYML : Note → MetaM String
| (title, curators, parts) => do
  let parts ← parts.foldlM NotePart.toYMLM ([], (0, 0))
  return s!"
title: \"{title}\"
curators: {String.intercalate ", " curators}
parts:
  {String.intercalate "\n" parts.1}"
```

def perturbationTheory [\[source\]](#)

```
def perturbationTheory : Note where
```

2.53 scripts.MetaPrograms.TODO_to_yaml

[scripts/MetaPrograms/TOD0_to_yaml.lean](#) on GitHub.

def getTodoInfo [\[source\]](#)

```
unsafe def getTodoInfo : MetaM (List todoInfo)
```

def todoToYAML [\[source\]](#)

```
def todoToYAML (todo : todoInfo) : MetaM String
```

def todosToYAML[\[source\]](#)

```
unsafe def todosToYAML : MetaM String
```